O'REILLY

Прикладная линейная алгебра для исследователей данных

с примерами и упражнениями на Python



Майк Икс Коэн

ДМК Пресс
издательство

Майк Икс Коэн

Прикладная линейная алгебра для исследователей данных

Майк Икс Коэн

Прикладная линейная алгебра для исследователей данных

От ключевых концепций до приложений с использованием Python

УДК 512.64
ББК 22.143
К76

Коэн М. И.
К76 Прикладная линейная алгебра для исследователей данных / пер. с англ. А. В. Логунова. – М.: ДМК Пресс, 2023. – 328 с.: ил.

ISBN 978-6-01798-945-3

В этой книге рассказывается о ключевых концепциях линейной алгебры, реализованных на Python, и о том, как их использовать в науке о данных, машинном и глубоком обучении и вычислительном моделировании. Рассматриваются интерпретации и приложения векторов и матриц, матричная арифметика, важные разложения, используемые в прикладной линейной алгебре, и пр. Прочитав книгу, вы научитесь внедрять и адаптировать под свои задачи целый ряд современных методов анализа и алгоритмов.

Издание адресовано специалистам по обработке данных, а также будет полезно студентам и широкому кругу разработчиков ПО.

УДК 512.64
ББК 22.143

Authorized Russian translation of the English edition of Practical Linear Algebra for Data Science ISBN 9781098120610 © 2022 Syncxpress BV. This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-1-098-12061-0 (англ.)
ISBN 978-6-01798-945-3 (каз.)

© 2022 Syncxpress BV
© Перевод, оформление, издание, Books.kz, 2023

Содержание

От издательства 12

Об авторе 13

Колофон 14

Предисловие 15

Глава 1. Введение 17

Что такое линейная алгебра и зачем ее изучать? 17
Об этой книге 18
Предварительные требования 19
  Математика 19
  Отношение 19
  Программирование 20
Математические доказательства в противовес интуитивному пониманию на основе программирования 20
Рабочий код в книге и предназначенный для скачивания онлайн 22
Упражнения по программированию 22
Как пользоваться этой книгой (для учителей и самообучающихся) 23

Глава 2. Векторы. Часть 1 24

Создание и визуализация векторов в NumPy 24
  Геометрия векторов 27
Операции на векторах 28
  Сложение двух векторов 28
  Вычитание двух векторов 29
  Геометрия сложения и вычитания векторов 30
  Умножение вектора на скаляр 31
  Сложение скаляра с вектором 32
  Геометрия умножения вектора на скаляр 32

Транспонирование 33
Транслирование векторов в Python 34
Модуль вектора и единичные векторы 35
Точечное произведение векторов 36
Точечное произведение является дистрибутивным 38
Геометрия точечного произведения 39
Другие умножения векторов 40
Адамарово умножение 40
Внешнее произведение 41
Перекрестное и тройное произведения 42
Ортогональное разложение векторов 42
Резюме 46
Упражнения по программированию 46

Глава 3. Векторы. Часть 2 49

Множества векторов 49
Линейно-взвешенная комбинация 50
Линейная независимость 51
Математика линейной независимости 53
Независимость и вектор нулей 54
Подпространство и охват 54
Базис 57
Определение базиса 60
Резюме 61
Упражнения по программированию 62

Глава 4. Применения векторов 64

Корреляция и косинусное сходство 64
Фильтрация временных рядов и обнаружение признаков 67
Кластеризация методом ***k***-средних 68
Упражнения по программированию 71
Упражнения по корреляции 71
Упражнения по фильтрации и обнаружению признаков 73
Упражнения по алгоритму ***k***-средних 75

Глава 5. Матрицы. Часть 1 76

Создание и визуализация матриц в NumPy 76
Визуализация, индексация и нарезка матриц 76
Специальные матрицы 78
Матричная математика: сложение, умножение на скаляр, адамарово умножение 80
Сложение и вычитание 80
«Сдвиг» матрицы 81

Умножение на скаляр и адамарово умножение ... 82
Стандартное умножение матриц ... 82
Правила допустимости умножения матриц ... 83
Умножение матриц ... 84
Умножение матрицы на вектор ... 85
Линейно-взвешенные комбинации ... 86
Результаты геометрических преобразований ... 86
Матричные операции: транспонирование ... 88
Обозначение точечного и внешнего произведений ... 88
Матричные операции: LIVE EVIL (порядок следования операций) ... 89
Симметричные матрицы ... 89
Создание симметричных матриц из несимметричных ... 90
Резюме ... 91
Упражнения по программированию ... 92

Глава 6. Матрицы. Часть 2 ... 97

Нормы матриц ... 97
След матрицы и норма Фробениуса ... 99
Пространства матрицы (столбцовое, строчное, нуль-пространство) ... 100
Столбцовое пространство ... 100
Строчное пространство ... 104
Нуль-пространства ... 104
Ранг ... 108
Ранги специальных матриц ... 110
Ранг сложенных и умноженных матриц ... 112
Ранг сдвинутых матриц ... 113
Теория и практика ... 113
Применения ранга ... 114
В столбцовом пространстве ... 115
Линейная независимость множества векторов ... 116
Определитель ... 117
Вычисление определителя ... 117
Определитель с линейными зависимостями ... 119
Характеристический многочлен ... 119
Резюме ... 121
Упражнения по программированию ... 123

Глава 7. Применения матриц ... 128

Матрицы ковариаций многопеременных данных ... 128
Геометрические преобразования посредством умножения матриц на векторы ... 131
Обнаружение признаков изображения ... 135
Резюме ... 138
Упражнения по программированию ... 138

Упражнения по матрицам ковариаций и корреляций 138
Упражнения по геометрическим преобразованиям 140
Упражнения по обнаружению признаков изображения 142

Глава 8. Обратные матрицы 144

Обратная матрица 144
Типы обратных матриц и условия обратимости 145
Вычисление обратной матрицы 146
Обратная матрица матрицы 2×2 146
Обратная матрица диагональной матрицы 148
Инвертирование любой квадратной полноранговой матрицы 149
Односторонние обратные матрицы 151
Уникальность обратной матрицы 153
Псевдообратная матрица Мура–Пенроуза 154
Численная стабильность обратной матрицы 155
Геометрическая интерпретация обратной матрицы 156
Резюме 158
Упражнения по программированию 158

Глава 9. Ортогональные матрицы и QR-разложение 162

Ортогональные матрицы 162
Процедура Грама–Шмидта 164
QR-разложение 165
Размеры матриц Q и R 166
Почему матрица R является верхнетреугольной 168
QR и обратные матрицы 169
Резюме 169
Упражнения по программированию 170

Глава 10. Приведение строк и LU-разложение 174

Системы уравнений 174
Конвертирование уравнений в матрицы 175
Работа с матричными уравнениями 176
Приведение строк 178
Метод устранения по Гауссу 180
Метод устранения по Гауссу–Жордану 181
Обратная матрица посредством метода устранения по Гауссу–Жордану 182
LU-разложение 183
Взаимообмен строками посредством матриц перестановок 185
Резюме 186
Упражнения по программированию 186

Глава 11. Общие линейные модели и наименьшие квадраты 189

Общие линейные модели 190
Терминология 190
Настройка общей линейной модели 190
Решение общих линейных моделей 192
Является ли решение точным? 193
Геометрическая перспектива наименьших квадратов 194
В чем причина работы метода наименьших квадратов? 195
Общая линейная модель на простом примере 197
Наименьшие квадраты посредством QR-разложения 201
Резюме 202
Упражнения по программированию 203

Глава 12. Применения метода наименьших квадратов 207

Предсказывание количеств велопрокатов на основе погоды 207
Регрессионная таблица с использованием библиотеки statsmodels 212
Мультиколлинеарность 213
Регуляризация 213
Полиномиальная регрессия 215
Поиск в параметрической решетке для отыскания модельных параметров 218
Резюме 220
Упражнения по программированию 221
Упражнения по аренде велосипедов 221
Упражнения по мультиколлинеарности 222
Упражнения по регуляризации 223
Упражнение по полиномиальной регрессии 224
Упражнения по поиску в параметрической решетке 225

Глава 13. Собственное разложение 227

Интерпретации собственных чисел и собственных векторов 228
Геометрия 228
Статистика (анализ главных компонент) 229
Подавление шума 230
Уменьшение размерности (сжатие данных) 231
Отыскание собственных чисел 231
Отыскание собственных векторов 234
Неопределенность собственных векторов по знаку и шкале 235
Диагонализация квадратной матрицы 236
Особая удивительность симметричных матриц 238
Ортогональные собственные векторы 238
Действительно-значные собственные числа 240

Собственное разложение сингулярных матриц 241
Квадратичная форма, определенность и собственные числа 243
Квадратичная форма матрицы 243
Определенность 245
A^TA является положительной (полу)определенной 245
Обобщенное собственное разложение 246
Резюме 248
Упражнения по программированию 249

Глава 14. Сингулярное разложение 254

Общая картина сингулярного разложения 254
Сингулярные числа и ранг матрицы 256
Сингулярное разложение на Python 256
Сингулярное разложение и одноранговые «слои» матрицы 257
Сингулярное разложение из собственного разложения 259
Сингулярное разложение матрицы A^TA 260
Конвертация сингулярных чисел в дисперсию: объяснение 260
Кондиционное число 261
Сингулярное разложение и псевдообратная матрица Мура–Пенроуза 262
Резюме 263
Упражнения по программированию 264

Глава 15. Применения собственного и сингулярного разложений 268

Анализ главных компонент с использованием собственного и сингулярного разложений 268
Математика анализа главных компонент 269
Шаги выполнения PCA 271
PCA посредством сингулярного разложения 272
Линейный дискриминантный анализ 273
Низкоранговая аппроксимация посредством сингулярного разложения 275
Сингулярное разложение для шумоподавления 276
Резюме 276
Упражнения 277
Анализ главных компонент (PCA) 277
Линейный дискриминантный анализ (LDA) 281
Сингулярное разложение для низкоранговых аппроксимаций 285
Сингулярное разложение для шумоподавления в изображениях 287

Глава 16. Краткое руководство по языку Python 291

Почему Python и какие есть альтернативы? 291
Интерактивные среды разработки 292
Использование Python локально и онлайн 292

Работа с файлами исходного кода в Google Colab ... 293
Переменные ... 294
Типы данных ... 296
Индексация ... 297
Функции ... 297
Методы в качестве функций ... 299
Написание своих собственных функций ... 299
Библиотеки ... 301
NumPy ... 301
Индексация и нарезка в NumPy ... 302
Визуализация ... 303
Переложение формул в исходный код ... 305
Форматирование печати и F-строки ... 308
Поток управления ... 309
Компараторы ... 309
Инструкции if ... 310
Инструкции elif и else ... 310
Несколько условий ... 311
Циклы for ... 312
Вложенные инструкции управления ... 312
Измерение времени вычислений ... 313
Получение помощи и приобретение новых знаний ... 313
Что делать, когда дела идут наперекосяк ... 314
Резюме ... 314

Дополнение А. Теорема о ранге и нульности ... 315

Тематический указатель ... 317

От издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг, мы будем очень благодарны, если вы сообщите о ней главному редактору по адресу dmkpress@gmail.com. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Об авторе

Майк Икс Коэн – адъюнкт-профессор неврологии[1] в Институте Дондерса (Медицинский центр Университета Радбуда) в Нидерландах. Имеет более чем 20-летний опыт преподавания научного программирования, анализа данных, статистики и смежных тем, а также является автором нескольких онлайновых курсов[2] и учебников. У него подозрительно сухое чувство юмора, и ему нравится все фиолетовое.

[1] См. https://oreil.ly/Ee23F.

[2] См. https://oreil.ly/BurUH.

Колофон

Животное на обложке книги «Прикладная линейная алгебра для исследователей данных» – это антилопа ньяла, также именуемая низменной ньялой либо просто ньялой (Tragelaphus angasii). Самки и молодые ньялы обычно имеют светло-красновато-коричневый окрас шерсти, в то время как взрослые самцы имеют темно-коричневую или даже сероватую шерсть. Как у самцов, так и у самок есть белые полосы вдоль тела и белые пятна на боках. У самцов спиралевидные рога, которые вырастают до 33 дюймов в длину, а их шерсть намного более лохматая, с длинной бахромой, свисающей от горла до задних конечностей, и гривой густых черных волос вдоль позвоночника. Самки весят около 130 фунтов, тогда как самцы могут весить до 275 фунтов.

Ньялы обитают в лесах Юго-Восточной Африки, ареал которых включает Малави, Мозамбик, Южную Африку, Эсватини, Замбию и Зимбабве. Они пугливы и предпочитают пастись ранним утром, ближе к вечеру либо ночью, проводя бóльшую часть жаркой части дня, отдыхая в укрытии. Ньялы образуют свободные стада численностью до десяти особей, хотя самцы постарше ведут одиночный образ жизни. Они не являются территориальными животными, хотя самцы будут бороться за доминирование во время спаривания.

Ньялы считаются видом, вызывающим наименьшее беспокойство, хотя выпас скота, сельское хозяйство и потеря среды обитания представляют для них угрозу. Многие животные на обложках издательства O'Reilly находятся под угрозой исчезновения, и все они важны для нашего мира.

Иллюстрация на обложке выполнена Карен Монтгомери по мотивам старинной линейной гравюры из Histoire Naturelle.

Предисловие

Условные обозначения в книге

В книге используются следующие типографические условные обозначения:

курсивный шрифт
: обозначает новые термины, URL-адреса, адреса электронной почты, имена файлов и расширения файлов.

моноширинный шрифт
: используется для листингов программ, а также внутри абзацев для ссылки на элементы программ, такие как переменные или имена функций, базы данных, типы данных, переменные среды, инструкции и ключевые слова.

Данный элемент обозначает общее замечание.

Данный элемент обозначает предупреждение или предостережение.

Использование примеров исходного кода

Дополнительные материалы (примеры исходного кода, упражнения и т. д.) доступны для скачивания по адресу https://github.com/mikexcohen/LinAlg4DataScience.

Если у вас есть технический вопрос или проблема с использованием примеров исходного кода, то, пожалуйста, отправьте электронное письмо по адресу bookquestions@oreilly.com.

Благодарности

Должен признаться, я действительно не люблю писать разделы с признаниями. И это не потому, что мне не хватает благодарности или я считаю, что мне некого благодарить, – совсем наоборот: у меня слишком много людей, которых нужно поблагодарить, и я не знаю, с чего начать, кого перечислить по имени, а кого пропустить. Должен ли я поблагодарить своих родителей за их роль в формировании меня таким человеком, который написал эту книгу? Возможно, их родителей за то, что они сформировали моих родителей? Помню, как моя учительница в четвертом классе говорила мне, что я, должно быть, стану писателем, когда вырасту. (Я не помню ее имени и не уверен, когда я вырасту, но, возможно, она оказала некоторое влияние на эту книгу.) Я написал бóльшую часть этой книги во время поездок на Канарские острова с работой на удалении; возможно, мне следует поблагодарить пилотов, кото-

рые доставили меня туда? Или электриков, которые устанавливали проводку в коворкингах? Вероятно, я должен быть благодарен Оздемиру Паше за его роль в популяризации кофе, который одновременно облегчал и отвлекал меня от писательства. И давайте не будем забывать фермеров, которые выращивали вкусную еду, которая меня поддерживала и делала счастливым.

Видите, к чему это ведет: мои пальцы печатали, но потребовалась вся история человеческой цивилизации, чтобы создать меня и среду, которая позволила мне написать эту книгу – и которая позволила вам прочитать эту книгу. Таким образом, спасибо человечеству!

Ну да ладно, я также могу посвятить один абзац более традиционному разделу благодарностей. Самое главное, я благодарен всем моим студентам на моих курсах в университете и летней школе с живым преподаванием, а также на моих онлайновых курсах Udemy за то, что они доверили мне преподавание у них и мотивировали меня продолжать совершенствовать свои объяснения прикладной математики и других технических тем. Я также благодарен Джесс Хаберману, редактору отдела закупок в O'Reilly, которая установила «первый контакт», чтобы спросить, не буду ли я заинтересован в написании этой книги. Шира Эванс (редактор разработки), Джонатан Оуэн (производственный редактор), Элизабет Оливер (выпускающий редактор), Кристен Браун (менеджер по контентным услугам) и два эксперта – технических рецензента сыграли непосредственную роль в трансформации моих нажатий клавиш в книгу, которую вы сейчас читаете. Уверен, что этот список неполон, потому что другие люди, которые помогли опубликовать эту книгу, мне неизвестны либо потому, что я забыл их из-за потери памяти в моем преклонном возрасте[1]. Благодарности всем читающим этот текст, кто чувствует, что внес хотя бы ничтожный вклад в эту книгу.

[1] ЛОЛ, когда я написал эту книгу, мне было 42 года.

Глава 1

Введение

Что такое линейная алгебра и зачем ее изучать?

Линейная алгебра имеет интересную историю в математике, восходящую к XVII веку на Западе и гораздо раньше в Китае. Матрицы – развернутые таблицы чисел, лежащие в основе линейной алгебры, – использовались для обеспечения компактной системы счисления с целью хранения наборов чисел, таких как геометрические координаты (это было первоначальное применение матриц Декартом), и систем уравнений (впервые введенных Гауссом). В XX веке матрицы и векторы использовались для многопеременной математики, включая математическое исчисление, дифференциальные уравнения, физику и экономику.

Но до недавнего времени большинству людей не нужно было заботиться о матрицах. Однако, как оказалось, компьютеры чрезвычайно эффективны в работе с матрицами. И поэтому современные вычисления породили современную линейную алгебру. Современная линейная алгебра является вычислительной, в то время как традиционная линейная алгебра является абстрактной. Современную линейную алгебру лучше всего изучать на исходном коде и приложениях в области графики, статистики, науки о данных, искусственного интеллекта и численного моделирования, в то время как традиционная линейная алгебра изучается на доказательствах и размышлениях о бесконечномерных пространствах векторов. Современная линейная алгебра предоставляет структурные элементы, которые поддерживают почти каждый алгоритм, реализованный на компьютерах, в то время как традиционная линейная алгебра зачастую является интеллектуальной пищей для продвинутых студентов математических университетов.

Добро пожаловать в современную линейную алгебру.

Стоит ли вам изучать линейную алгебру? Это зависит от того, хотите ли вы понимать алгоритмы и процедуры или же просто применять методы, разработанные другими. Я не хочу принижать последнее – в сущности, нет ничего

плохого в использовании инструментов, которые вы не понимаете (я пишу это на ноутбуке, который я могу использовать, но не смог создать его с нуля). Но, учитывая, что вы читаете книгу с таким названием в коллекции книг O'Reilly, я предполагаю, что вы (1) хотите узнать принцип работы алгоритмов либо (2) хотите разрабатывать или адаптировать вычислительные методы. Так что да, вы должны изучать линейную алгебру, и вы должны изучить ее современную версию.

Об этой книге

Предназначение этой книги – научить вас современной линейной алгебре. Но речь идет не о том, чтобы запомнить несколько ключевых уравнений и пройтись по абстрактным доказательствам; цель в том, чтобы научить вас думать о матрицах, векторах и операциях на них. Вы разовьете геометрическое понимание на уровне интуиции относительно того, почему линейная алгебра такова, какая она есть. И вы поймете, как реализовывать концепции линейной алгебры в рабочем коде Python, уделяя особое внимание приложениям в области машинного обучения и науки о данных.

Во многих традиционных учебниках по линейной алгебре избегаются числовые примеры в интересах обобщений, ожидается, что вы самостоятельно получите сложные доказательства, и преподается множество концепций, которые имеют мало отношения либо вообще не имеют отношения к применению или реализации на компьютерах. Я пишу эти строки не как критику – абстрактная линейная алгебра прекрасна и элегантна. Но если ваша цель – использовать линейную алгебру (и математику в целом) в качестве инструмента для понимания данных, статистики, глубокого обучения, обработки изображений и т. д., то традиционные учебники по линейной алгебре, возможно, покажутся досадной тратой времени, которая поставит вас в замешательство и обеспокоит, взяв под сомнение ваши потенциальные возможности в технической области.

Эта книга написана с учетом того, что учащиеся занимаются самообучением. Возможно, у вас есть степень в области математики, инженерии или физики, но вам нужно научиться реализовывать линейные алгоритмы в рабочем коде. Или же, вполне вероятно, вы не изучали математику в университете и теперь осознаете важность линейной алгебры для вашей учебы или работы. В любом случае, эта книга является самостоятельным ресурсом; она не представляет собой исключительно дополнение к лекционному курсу (хотя ее можно было бы использовать для этой цели).

Если, читая последние три абзаца, вы кивнули головой в знак согласия, то эта книга определенно для вас.

Если вы хотите погрузиться в линейную алгебру поглубже, с большим количеством доказательств и разведывательной работы, то существует несколько отличных учебников, о прочтении которых вы можете подумать,

включая мой учебник «Линейная алгебра: теория, интуиция, исходный код (Sincxpress BV)[1].

Предварительные требования

Я пытался написать эту книгу для увлеченных учеников с минимальной формальной подготовкой. И тем не менее ничего никогда не изучается по-настоящему с нуля.

Математика

Вы должны чувствовать себя комфортно в математике уровня средней школы. Просто основы алгебры и геометрии, и ничего особенного.

Для этой книги требуется абсолютно нулевой уровень в исчислении (хотя дифференциальное исчисление имеет большую важность для приложений, в которых линейная алгебра используется часто, таких как глубокое обучение и оптимизация).

Но самое главное – вам нужно чувствовать себя комфортно, думая о математике, рассматривая уравнения и графики и не боясь противостоять интеллектуальным трудностям, которые возникают при изучении математики.

Отношение

Линейная алгебра – это раздел математики, и, следовательно, данная книга посвящена математике. Изучение математики, в особенности во взрослом возрасте, требует определенного терпения, целеустремленности и напористости. Выпейте чашку кофе, сделайте глубокий вдох, уберите свой телефон в другую комнату и погрузитесь в работу.

В глубине вашей головы будет звучать голос, говорящий о том, что вы слишком стары или слишком глупы, чтобы изучать продвинутую математику. Иногда этот голос будет звучать громче, а иногда мягче, но он всегда будет присутствовать. И это не только у вас – он есть у всех. Этот голос невозможно подавить или уничтожить; даже не пытайтесь. Просто примите, что некоторая неуверенность в себе – это часть человеческого бытия. Всякий раз, когда этот голос заговаривает, вам приходится доказывать, что он неправ.

---

[1] Приношу извинения за бесстыдную саморекламу; обещаю, что в данной книге это единственный случай, когда я позволяю себе подобное послабление.

Программирование

Данная книга посвящена линейно-алгебраическим приложениям в рабочем коде. Я написал эту книгу, ориентируясь на Python, потому что Python в настоящее время является наиболее широко используемым языком в науке о данных, машинном обучении и смежных областях. Если вы предпочитаете другие языки, такие как MATLAB, R, C или Julia, то я надеюсь, что вам будет легко переложить рабочий код Python на эти языки.

Я постарался сделать код Python как можно проще, оставляя его при этом релевантным для приложений. Глава 16 содержит базовое введение в программирование на Python. Стоит ли вам просматривать эту главу? Все зависит от вашего уровня владения языком Python:

Средний/продвинутый уровень (опыт программирования > 1 года)
: Полностью пропустите главу 16 либо, по возможности, пролистайте ее, чтобы получить представление о типе рабочего кода, который появится в остальной части книги.

Некоторые знания (опыт < 1 года)
: Рекомендую проработать эту главу на случай, если в ней есть новый материал или вам нужно его освежить. Но вы должны быть в состоянии пройти ее довольно быстро.

Полный новичок
: Подробно ознакомьтесь с этой главой. Поймите, что данная книга не является полным руководством по Python, поэтому если вы обнаружите, что вам трудно разобраться с рабочим кодом в главах книги, то, возможно, вы захотите отложить эту книгу, поработать со специальным курсом или книгой по Python, а затем вернуться к этой книге.

Математические доказательства в противовес интуитивному пониманию на основе программирования

Цель изучения математики в общем и целом состоит в том, чтобы понять математику. Как понять математику? Давайте посчитаем способы:

Строгие доказательства
: Доказательство в математике – это последовательность утверждений, демонстрирующая то, что набор допущений приводит к логическому заключению. Доказательства, несомненно, имеют большую важность в чистой математике.

Визуализации и примеры

Четко написанные объяснения, диаграммы и численные примеры помогут приобретать понимание концепций и операций в линейной алгебре на уровне интуиции. Для удобства визуализации большинство примеров выполнено в 2D либо 3D, но принципы применимы и к более высоким измерениям.

Разница между ними заключается в том, что формальные математические доказательства обеспечивают строгость, но редко понимание на уровне интуиции, в то время как визуализации и примеры обеспечивают прочное понимание на уровне интуиции благодаря практическому опыту, но рискуют приводить к неточностям, будучи основанными на конкретных примерах, которые не обобщаются.

Доказательства важных утверждений включены, но я больше концентрируюсь на упрочении интуитивного понимания посредством объяснений, визуализаций и примеров исходного кода.

И это подводит меня к пониманию математики на уровне интуиции на основе программирования (тому, что я иногда называю «мягкими доказательствами»). Вот идея: вы исходите из того, что в Python (и таких библиотеках, как NumPy и SciPy) правильно реализована низкоуровневая обработка чисел, в то время как вы концентрируетесь на принципах путем обследования многочисленных числовых примеров в исходном коде.

Краткий пример: мы «мягко докажем» принцип коммутативности умножения, который гласит, что $a \times b = b \times a$:

```
a = np.random.randn()
b = np.random.randn()
a * b - b * a
```

Приведенный выше исходный код генерирует два случайных числа и проверяет гипотезу о том, что изменение порядка умножения не влияет на результат. Если принцип коммутативности истинен, то в третьей строке будет выведено значение 0.0. Если вы выполните этот исходный код несколько раз и всегда будете получать 0.0, то, увидев один и тот же результат в многочисленных примерах с разными числами, вы приобретете понимание коммутативности на уровне интуиции.

Для ясности: понимание на уровне интуиции на основе исходного кода не заменяет строгого математического доказательства. Все дело в том, что «мягкие доказательства» позволяют понимать математические концепции, не беспокоясь о деталях абстрактного математического синтаксиса и аргументов. Это особенно выгодно для программистов, которым не хватает продвинутого математического образования.

В сухом остатке все сводится к тому, что *много математических концепций могут усваиваться при помощи небольшой порции исходного кода*.

Рабочий код в книге и предназначенный для скачивания онлайн

Данную книгу можно читать, не глядя на исходный код и не решая упражнения по программированию. Все в порядке; вы обязательно чему-нибудь научитесь. Но не расстраивайтесь, если ваши знания будут поверхностными и мимолетными. Если вы хотите действительно *понять* линейную алгебру, то вам нужно решать задачи. Вот почему эта книга снабжена демонстрациями исходного кода и упражнениями по каждой математической концепции.

Важный исходный код напечатан непосредственно в книге. Я хочу, чтобы вы читали текст и уравнения, смотрели на графики и одновременно *видели исходный код*. Это позволит вам связывать концепции и уравнения с исходным кодом.

Но листинг исходного кода в книге может занимать много места, а ручное его копирование на вашем компьютере будет утомительным. Поэтому на страницах книги печатаются только ключевые строки кода; онлайновый код содержит дополнительный исходный код, комментарии, графические украшения и т. д. Онлайновый код также содержит решения упражнений по программированию (всех упражнений, а не только выборочных задач!). Вам обязательно следует скачать исходный код и ознакомиться с ним во время работы с книгой.

Весь исходный код можно получить из репозитория на сайте GitHub https://github.com/mikexcohen/LinAlg4DataScience. Этот репозиторий можно клонировать либо просто скачать целиком в виде ZIP-файла (вам не потребуется регистрироваться, входить в систему либо платить за скачивание кода).

Я написал исходный код в среде Google Colab, используя блокнот Jupyter. Я решил использовать Jupyter, потому что это дружественная и простая в применении среда. Тем не менее призываю вас использовать любую интегрированную среду разработки на Python, которую вы предпочитаете. Для удобства онлайновый код также предоставляется в виде простых файлов .py.

Упражнения по программированию

Математика – это не зрительский вид спорта. В большинстве книг по математике есть бесчисленное множество письменных задач (и давайте будем честны: никто не решает их все). Но данная книга целиком посвящена *прикладной* линейной алгебре, и никто не применяет линейную алгебру на бумаге! Вместо этого линейная алгебра применяется в рабочем коде. Поэтому вместо ручных задач и утомительных доказательств, «оставленных читателю в качестве упражнения» (как любят писать авторы учебников по математике), в данной книге много упражнений по программированию.

Упражнения по программированию различаются по сложности. Если вы в Python и линейной алгебре – новичок, то некоторые упражнения, возмож-

но, покажутся вам действительно сложными. Если вы застряли, то вот вам совет: кратко взгляните на мое решение, чтобы вдохновиться, затем уберите его, чтобы не видеть мой исходный код, и продолжайте работать над своим исходным кодом.

Сравнивая свое решение с моим, имейте в виду, что существует много способов решения задач на Python. Важно найти правильный ответ; предпринимаемые вами шаги, чтобы его получить, нередко зависят от личного стиля программирования.

Как пользоваться этой книгой (для учителей и самообучающихся)

Данная книга полезна в трех средах:

Самообучение
: Я постарался сделать эту книгу доступной для читателей, которые хотят изучать линейную алгебру самостоятельно, вне формальной классной комнаты. Никаких дополнительных ресурсов или онлайновых лекций не требуется, хотя, конечно же, существует масса других книг, веб-сайтов, видеороликов YouTube и онлайновых курсов, которые студенты могут счесть полезными.

Первичный учебник на занятиях по науке о данных
: Данная книга может использоваться в качестве первичного учебника в курсе математики, лежащей в основе науки о данных, машинного обучения, искусственного интеллекта и смежных тем. Содержимое состоит из 14 глав (исключая это введение и дополнительный материал по Python), и от студентов ожидается, что они будут работать над одной-двумя главами в неделю. Поскольку учащиеся имеют доступ к решениям всех упражнений, преподаватели, возможно, пожелают дополнить упражнения книги дополнительными наборами задач.

Вторичный учебник по математикоориентированному курсу линейной алгебры
: Эта книга также может использовать в качестве дополнения к курсу математики с сильным акцентом на доказательствах. В этом случае лекции были бы сосредоточены на теории и строгих доказательствах, в то время как на данную книгу можно было бы ссылаться с целью переложения концепций в исходный код с прицелом на приложения в науке о данных и машинном обучении. Как я написал выше, преподаватели, возможно, пожелают предоставить дополнительные упражнения, поскольку решения ко всем упражнениям из книги доступны онлайн.

Глава 2

Векторы. Часть 1

Векторы обеспечивают основу, на которой построена вся линейная алгебра (и, следовательно, остальная часть этой книги).

К концу этой главы вы будете знать о векторах все: что они собой представляют, что они делают, как их интерпретировать и как создавать и работать с ними на Python. Вы поймете наиболее важные операции на векторах, включая векторную алгебру и точечное произведение. Наконец, вы узнаете о разложениях векторов, являющихся одной из главных целей линейной алгебры.

Создание и визуализация векторов в NumPy

В линейной алгебре *вектор* – это упорядоченный список чисел. (В абстрактной линейной алгебре векторы могут содержать другие математические объекты, включая функции; однако поскольку данная книга ориентирована на приложения, мы будем рассматривать только те векторы, которые содержат числа.)

Векторы обладают несколькими важными характеристиками. Первые две, с которых мы начнем, – это[1]:

размерность.
Число чисел в векторе;

ориентация.
Ориентирован ли вектор *вдоль столбца* (стоя в полный рост) либо *вдоль строки* (лежа ровно во всю ширь).

Размерность нередко указывается с помощью причудливо выглядящей записи $\mathbb{R}^N$, где $\mathbb{R}$ обозначает действительно-значные числа (ср. с символом

[1] В книге проводится четкое различие между созвучными понятиями dimensionality (математическая размерность, протяженность, объемность) и dimension (геометрическое измерение, мерность как в слове двухмерный). Первый термин переводится как размерность, а второй в зависимости от контекста – как измерение либо мерность. – *Прим. перев.*

$\mathbb{C}$ для комплексно-значных чисел), и N указывает на размерность. Например, считается, что вектор с двумя элементами принадлежит $\mathbb{R}^2$.

Специальный символ $\mathbb{R}$ создан с использованием кода latex, но вы также можете написать R^2, R2 или R^2.

Уравнение 2.1 показывает несколько примеров векторов; перед чтением следующего абзаца рекомендуется определить их размерность и ориентацию.

Уравнение 2.1. Примеры вектора-столбца и вектора-строки

$$\mathbf{x} = \begin{bmatrix} 1 \\ 4 \\ 5 \\ 6 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} .3 \\ -7 \end{bmatrix}, \quad \mathbf{z} = \begin{bmatrix} 1 & 4 & 5 & 6 \end{bmatrix}.$$

Вот ответы: $\mathbf{x}$ – четырехмерный вектор-столбец, $\mathbf{y}$ – двумерный вектор-столбец и $\mathbf{z}$ – четырехмерный вектор-строка. Также можно написать, например, $\mathbf{x} \in \mathbb{R}^2$, где символ $\in$ означает «принадлежит множеству».

Являются ли $\mathbf{x}$ и $\mathbf{z}$ одним и тем же вектором? Технически они разные, хотя и содержат одни и те же элементы в одном и том же порядке. Более подробное изложение этого вопроса смотрите во врезке «Имеет ли значение ориентация вектора?» на стр. 26.

Из этой книги и на протяжении всех ваших приключений, связанных с интеграцией математики и программирования, вы узнаете, что существуют различия между математикой «на меловой доске» и реализованной в рабочем коде. Некоторые расхождения незначительны и несущественны, в то время как другие вызывают путаницу и ошибки. Давайте-ка я теперь познакомлю вас с терминологическим разночтением между математикой и программированием.

Ранее я писал, что *размерность* вектора – это число элементов в этом векторе. Однако в Python размерность вектора или матрицы – это число геометрических измерений, используемых для распечатки числового объекта. Например, все показанные выше векторы считаются в Python «двумерными массивами» независимо от содержащегося в векторах числа элементов (то есть математической размерности). Список чисел без определенной ориентации в Python считается одномерным массивом независимо от числа элементов (этот массив будет распечатан в виде строки, но, как вы увидите позже, он обрабатывается иначе, чем вектор-строка). Математическая размерность – число элементов в векторе – в Python называется *длиной*, или *очертанием*[1], вектора.

Бывает, что эта несогласованная, а иногда и противоречивая терминология сбивает с толку. И действительно, терминология нередко становится сложной проблемой на пересечении различных дисциплин (в данном случае математики и вычислительной науки). Но не волнуйтесь, вы это освоите, приобретя немного опыта.

---

[1] Англ. *shape*; син. форма, контур. – *Прим. перев.*

При упоминании векторов обычно используются строчные жирные римские буквы, например **v** для обозначения «вектор v». В некоторых учебниках используется курсив (*v*) или вверху выводится стрелка в качестве диакритического знака ($\vec{v}$).

По традиции в линейной алгебре исходят из допущения, что векторы ориентированы вдоль столбца, если не указано иное. Векторы-строки записываются в виде $\mathbf{w}^T$. Надстрочная буква ^{T} указывает на *операцию транспонирования*, о которой вы узнаете подробнее чуть позже; пока же достаточно сказать, что операция транспонирования конвертирует вектор-столбец в вектор-строку.



Имеет ли значение ориентация вектора?

Действительно ли нужно беспокоиться о том, ориентированы ли векторы вдоль столбцов либо вдоль строк или же они являются неориентированными одномерными массивами? Иногда – да, а иногда – нет. При использовании векторов для хранения данных ориентация обычно не имеет значения. Но если ориентация неправильная, то некоторые операции в Python могут давать ошибки либо неожиданные результаты. Поэтому важно понимать ориентацию векторов, поскольку, тратя 30 минут на отладку кода только для того, чтобы понять, что вектор-строка должен быть вектором-столбцом, вы гарантированно получите головную боль.

Векторы в Python могут быть представлены с использованием нескольких типов данных. Тип «список», возможно, покажется самым элементарным способом представления вектора – и это для некоторых приложений. Но многие линейно-алгебраические операции не будут работать со списками Python. Поэтому в большинстве случаев лучше всего создавать векторы в виде массивов NumPy. В следующем ниже исходном коде показаны четыре способа создания вектора:

```
asList  = [1, 2, 3]
asArray = np.array([1, 2, 3])         # одномерный массив
rowVec  = np.array([ [1, 2, 3] ])     # строка
colVec  = np.array([ [1], [2], [3] ]) # столбец
```

Переменная `asArray` – это *неориентированный* массив, и, значит, это ни вектор-строка, ни вектор-столбец, а просто одномерный список чисел в NumPy. В NumPy ориентация задается скобками: самые внешние скобки группируют все числа вместе в один объект. Затем каждый дополнительный набор скобок указывает строку: вектор-строка (переменная `rowVec`) содержит все числа в одной строке, в то время как вектор-столбец (переменная `colVec`) содержит несколько строк, причем каждая строка содержит одно число.

С этими ориентациями можно познакомиться поближе, проинспектировав очертания переменных (в программировании проверка очертаний переменных нередко бывает очень полезной):

```
print(f'asList:  {np.shape(asList)}')
print(f'asArray: {asArray.shape}')
print(f'rowVec:  {rowVec.shape}')
print(f'colVec:  {colVec.shape}')
```

Вот как выглядит результат:

```
asList:  (3,)
asArray: (3,)
rowVec:  (1, 3)
colVec:  (3, 1)
```

Результат показывает, что одномерный массив `asArray` имеет размер `(3)`, тогда как наделенные ориентацией векторы являются двумерными массивами и хранятся как размер `(1,3)` либо `(3,1)` в зависимости от ориентации. Размеры всегда указываются как (строки, столбцы).

Геометрия векторов

Упорядоченный список чисел – это алгебраическая интерпретация вектора; геометрическая интерпретация вектора представляет собой прямой отрезок с определенной длиной (также именуемой *модулем* вектора[1]) и направлением (также именуемым *углом*; он вычисляется относительно положительной оси *x*). Две точки вектора называются хвостом (где он начинается) и головой (где он заканчивается); голова часто имеет наконечник стрелки, чтобы устранить неоднозначность с хвостом.

Возможно, вы подумали, что в векторе кодируется геометрическая координата, но векторы и координаты – это на самом деле разные вещи. Однако они согласуются, когда вектор начинается в начале координат. Этот случай называется *стандартным положением* и проиллюстрирован на рис. 2.1.

Концептуализация векторов в геометрическом либо алгебраическом плане облегчает интуитивное понимание в различных приложениях, но это просто две стороны одной медали. Например, геометрическая интерпретация вектора широко применяется в физике и инженерии (например, для представления физических сил), а алгебраическая интерпретация вектора – в науке о данных (например, для хранения данных о продажах во временной динамике). Нередко линейно-алгебраические концепции усваиваются геометрически на 2D-графиках, а затем расширяются до более высоких измерений с помощью алгебры.

---

[1] Англ. *magnitude*. – *Прим. перев.*

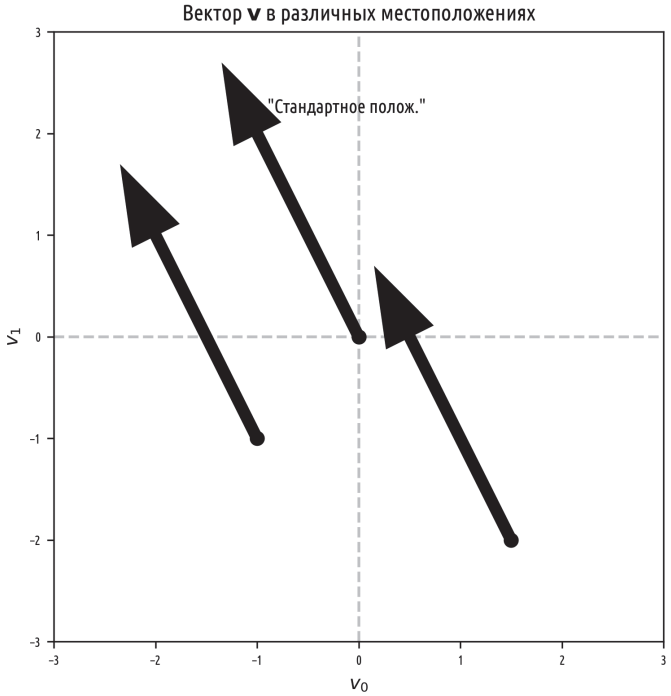


Рис. 2.1 ❖ Все стрелки выражают один и тот же вектор. Вектор в стандартном положении имеет свой хвост в начале координат и свою голову в согласованной геометрической координате

Операции на векторах

Векторы подобны существительным; они являются персонажами нашего рассказа о линейной алгебре. Веселье в линейной алгебре исходит от глаголов – действий, которые вдыхают жизнь в персонажей. Эти действия называются *операциями*.

Некоторые линейно-алгебраические операции элементарны и интуитивно понятны и работают именно так, как вы и ожидаете (например, сложение), в то время как другие более сложны и требуют объяснений в объеме целых глав (например, сингулярное разложение). Давайте начнем с элементарных операций.

Сложение двух векторов

Для того чтобы сложить два вектора, надо просто сложить каждый соответствующий элемент. Уравнение 2.2 показывает пример:

Уравнение 2.2. Сложение двух векторов

$$\begin{bmatrix}4\\5\\6\end{bmatrix}+\begin{bmatrix}10\\20\\30\end{bmatrix}=\begin{bmatrix}14\\25\\36\end{bmatrix}.$$

Как вы могли догадаться, сложение векторов определено только для двух векторов, которые имеют одинаковую размерность; например, невозможно сложить вектор в $\mathbb{R}^3$с вектором в $\mathbb{R}^5$.

Вычитание двух векторов

Вычитание векторов – это тоже то, что вы и ожидаете: вычесть второй вектор из первого поэлементно.

Уравнение 2.3 показывает пример:

Уравнение 2.3. Вычитание двух векторов

$$\begin{bmatrix}4\\5\\6\end{bmatrix}-\begin{bmatrix}10\\20\\30\end{bmatrix}=\begin{bmatrix}-6\\-15\\-24\end{bmatrix}.$$

Сложение векторов на языке Python выполняется просто:

```
v = np.array([ 4, 5, 6])
w = np.array([10,20,30])
u = np.array([ 0, 3, 6, 9])
vPlusW = v+w
uPlusW = u+w  # ошибка! измерения не совпадают!
```

Имеет ли значение ориентация вектора для сложения? Рассмотрим уравнение 2.4:

Уравнение 2.4. Можно ли сложить вектор-строку с вектором-столбцом?

$$\begin{bmatrix}4\\5\\6\end{bmatrix}+\begin{bmatrix}10 & 20 & 30\end{bmatrix}=?.$$

Возможно, вы подумаете, что между этим примером и тем, что был показан ранее, нет никакой разницы – в конце концов, оба вектора состоят из трех элементов. Давайте посмотрим, что сделает Python:

```
v = np.array([[ 4, 5, 6]])    # вектор-строка
w = np.array([[10,20,30]]).T  # вектор-столбец
v+w
```

```
>> array([[14, 15, 16],
          [24, 25, 26],
          [34, 35, 36]])
```

Возможно, результат покажется запутанным и несовместимым с приведенным ранее определением сложения векторов. На самом деле в Python реализована операция, именуемая *транслированием*. Вы узнаете о трансляции больше чуть позже в этой главе, но я призываю вас потратить немного времени на осмысление результата и подумать о том, как он возник в результате сложения вектора-строки с вектором-столбцом. Несмотря на это, данный пример показывает, что ориентация действительно важна: *два вектора можно сложить, только если они имеют одинаковую размерность и одинаковую ориентацию*.

Геометрия сложения и вычитания векторов

Для того чтобы сложить два вектора геометрически, надо расположить векторы так, чтобы хвост одного вектора находился в голове другого вектора. Суммируемый вектор проходит от хвоста первого вектора к голове второго (график A на рис. 2.2). Эту процедуру можно расширить, чтобы суммировать любое число векторов: надо просто уложить все векторы от хвоста к голове, и тогда сумма будет равна отрезку, идущему от первого хвоста к итоговой голове.

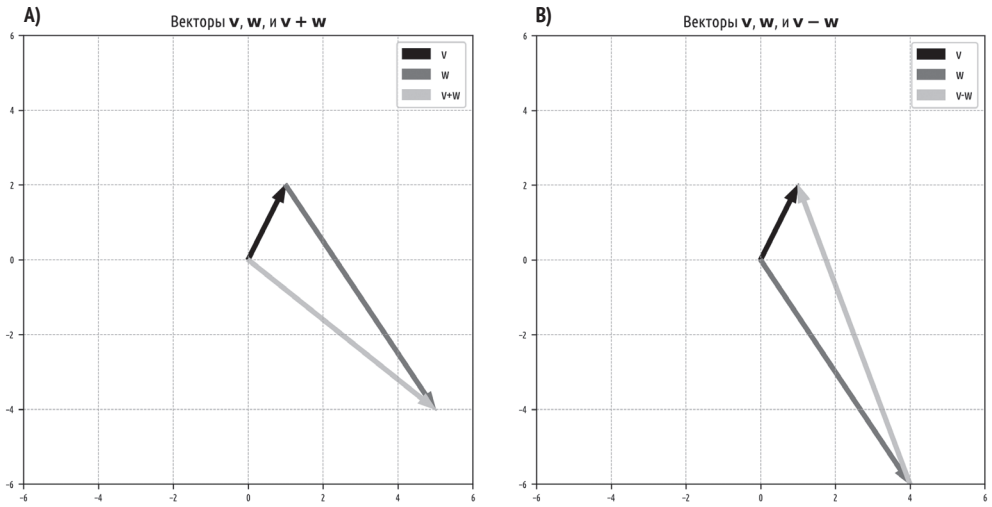


Рис. 2.2 ❖ Сумма и разность двух векторов

Геометрическое вычитание векторов немного отличается, но является одинаково элементарным: надо выровнять два вектора так, чтобы их хвосты находились в одной и той же координате (это легко достигается, если оба вектора находятся в стандартном положении); вектор разности – это отрезок,

который идет от головы «отрицательного» вектора к голове «положительного» вектора (график B на рис. 2.2).

Не стоит недооценивать важность геометрии вычитания векторов: она лежит в основе ортогонального разложения векторов, которое, в свою очередь, лежит в основе метода линейных наименьших квадратов, который является одним из наиболее важных приложений линейной алгебры в науке и технике.

Умножение вектора на скаляр

Скаляр в линейной алгебре – это число в чистом виде, не вложенное ни в вектор, ни в матрицу. Скаляры обычно обозначаются строчными греческими буквами, такими как α или λ. Поэтому умножение вектора на скаляр обозначается, например, как $\beta\mathbf{u}$.

Умножение вектора на скаляр выполняется очень просто: надо умножить каждый элемент вектора на скаляр. Для понимания будет достаточно одного численного примера (уравнение 2.5):

Уравнение 2.5. Умножение вектора на скаляр (также умножение скаляра на вектор)

$$\lambda = 4, \quad \mathbf{w} = \begin{bmatrix} 9 \\ 4 \\ 1 \end{bmatrix}, \quad \lambda\mathbf{w} = \begin{bmatrix} 36 \\ 16 \\ 4 \end{bmatrix}.$$

Вектор нулей

Вектор, состоящий из одних нулей, также именуемый *вектором нулей*, или нуль-вектором, обозначается жирным шрифтом, **0**, и в линейной алгебре является специальным вектором. Нередко использование вектора нулей для решения задачи фактически принято называть *тривиальным решением* и исключать. Линейная алгебра полна утверждений типа «найти ненулевой вектор, который может решить ...» или «найти нетривиальное решение для ...».

Ранее я писал, что тип данных хранящей вектор переменной иногда важен, а иногда и не важен. Умножение вектора на скаляр является примером случая, когда тип данных имеет значение:

```
s = 2
a = [3, 4, 5]    # в виде списка
b = np.array(a)  # в виде np-массива
print(a*s)
print(b*s)

>> [ 3, 4, 5, 3, 4, 5 ]
>> [ 6 8 10 ]
```

Приведенный выше исходный код создает скаляр (переменную `s`) и вектор в виде списка (переменную `a`), затем конвертирует их в массив NumPy (переменную `b`). В Python звездочка перегружена, то есть ее поведение зависит от типа переменной: умножение списка на скаляр повторяет список `s` раз (в данном случае два раза), что определенно *не* является линейно-алгебраической операцией умножения скаляра на вектор. Однако когда вектор хранится в виде массива NumPy, звездочка интерпретируется как поэлементное умножение. (Вот для вас небольшое упражнение: что произойдет, если задать `s = 2.0`, и почему[1]?) Обе эти операции (повторение списка и умножение вектора на скаляр) используются в реальном программировании, поэтому об указанном различии следует помнить.

Сложение скаляра с вектором

Сложение скаляра с вектором формально в линейной алгебре не определено: это два отдельных вида математических объектов, которые невозможно объединить. Однако программы числовой обработки, такие как Python, позволяют складывать скаляры с векторами, и указанная операция сравнима с умножением скаляра на вектор: скаляр прибавляется к каждому элементу вектора. Следующий ниже исходный код иллюстрирует эту идею:

```
s = 2
v = np.array([3, 6])
s+v

>> [5 8]
```

Геометрия умножения вектора на скаляр

Почему скаляры называются «скалярами»? Это вытекает из геометрической интерпретации. Скаляры шкалируют векторы, не меняя их направления. Существует четыре эффекта умножения вектора на скаляр, которые зависят от того, является ли скаляр больше 1, между 0 и 1, в точности 0 либо отрицательным. Рисунок 2.3 иллюстрирует эту идею.

Ранее я писал, что скаляры не меняют направление вектора. Но на рисунке показано, что направление вектора меняется, когда скаляр отрицательный (то есть его угол поворачивается на 180°). Возможно, это покажется противоречием, но существует интерпретация векторов, в которой они указывают вдоль бесконечно длинной линии, проходящей через начало координат и уходящей в бесконечность в обоих направлениях (в следующей главе я буду называть такую интерпретацию «одномерным подпространством»). В этом смысле «повернутый» вектор по-прежнему указывает вдоль той же самой

[1] Выражение `a*s` выдаст ошибку, потому что повторять список можно только с использованием целых чисел; невозможно повторить список 2.72 раза!

бесконечной линии, и, следовательно, отрицательный скаляр не меняет направления. Указанная интерпретация важна для пространств матриц, собственных векторов и сингулярных векторов, все из которых представлены в последующих главах.

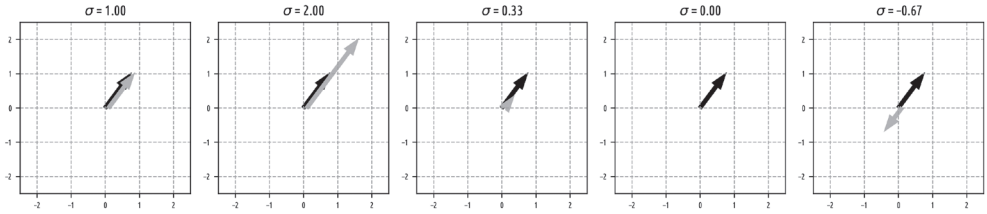


Рис. 2.3 ❖ Один и тот же вектор (черная стрелка), умноженный на разные скаляры σ (серый отрезок; для наглядности слегка сдвинут)

Умножение вектора на скаляр в сочетании со сложением векторов приводит непосредственно к *усреднению векторов*. Усреднение векторов – это то же самое, что усреднение чисел: просуммировать и поделить на количество чисел. Таким образом, для того чтобы усреднить два вектора, надо их сложить, а затем скалярно умножить на .5. В общем случае, для того чтобы усреднить N векторов, надо их просуммировать и скалярно умножить результат на $1/N$.

Транспонирование

Вы уже узнали об операции транспонирования: она конвертирует векторы-столбцы в векторы-строки и наоборот. Тут стоит дать несколько более формальное определение, которое будет обобщено на транспонирование матриц (тема в главе 5).

Матрица состоит из строк и столбцов; следовательно, каждый элемент матрицы имеет индекс в формате (строка, столбец). Операция транспонирования просто меняет местами эти индексы. Она формализуется в уравнении 2.6:

Уравнение 2.6. Операция транспонирования

$$\mathbf{m}^{\mathrm{T}}_{i,j} = \mathbf{m}_{j,i}.$$

Векторы имеют либо одну строку, либо один столбец, в зависимости от их ориентации. Например, шестимерный вектор-строка имеет $i = 1$ и индексы j от 1 до 6, тогда как шестимерный вектор-столбец имеет индексы i от 1 до 6 и $j = 1$. Таким образом, перемена мест индексов i, j меняет местами строки и столбцы.

Вот важное правило: двойное транспонирование возвращает вектор в изначальную ориентацию. Другими словами, $\mathbf{v}^{\mathrm{TT}} = \mathbf{v}$. Возможно, указанное правило покажется очевидным и тривиальным, но оно является краеугольным камнем нескольких важных доказательств в науке о данных и машин-

ном обучении, включая создание симметричных матриц ковариаций при умножении матрицы данных на ее транспонированную версию (что, в свою очередь, является причиной того, что анализ главных компонент есть ортогональный поворот пространства данных… не волнуйтесь, это предложение обретет смысл в данной книге чуть позже!).

Транслирование векторов в Python

Транслирование – это операция, которая существует только в современной компьютерной линейной алгебре; это не та процедура, которую вы бы нашли в традиционном учебнике по линейной алгебре.

Транслирование, по существу, означает многократное повторение операции между одним вектором и каждым элементом другого вектора. Рассмотрим следующую ниже серию уравнений[1]:

[1 1] + [10 20];

[2 2] + [10 20];

[3 3] + [10 20].

Обратите внимание на закономерности в векторах. Приведенный выше набор уравнений можно компактно реализовать, сжав указанные закономерности в векторы [1 2 3] и [10 20], а затем оттранслировав сложение. Вот как это выглядит на Python:

```
v = np.array([[1, 2, 3]]).T  # вектор-столбец
w = np.array([[10, 20]])     # вектор-строка
v + w  # сложение при помощи транслирования

>> array([[11, 21],
          [12, 22],
          [13, 23]])
```

Здесь вы снова заметите важность ориентации в линейно-алгебраических операциях: попробуйте выполнить приведенный выше исходный код, изменив `v` на вектор-строку и `w` – на вектор-столбец[2].

Поскольку транслирование позволяет проводить эффективные и компактные вычисления, оно встречается в числовом программировании очень часто. В данной книге вы увидите несколько примеров транслирования, в том числе в разделе, посвященном кластеризации методом *k*-средних (глава 4).

---

[1] Для ясности вот как данный термин трактуется в документации NumPy: термин «транслирование» (англ. *broadcasting*) относится к тому, как NumPy обрабатывает массивы с разным размером во время арифметических операций. С учетом определенных ограничений меньший массив «заполняется» по большему массиву, чтобы они имели совместимые очертания. – *Прим. перев.*

[2] Python по-прежнему выполнит транслирование, но в результате вместо матрицы 2×3 получится матрица 3×2.

Модуль вектора и единичные векторы

Модуль вектора, также именуемый *геометрической длиной*, или *нормой*[1], представляет собой расстояние от хвоста до головы вектора и вычисляется с использованием стандартной формулы евклидова расстояния: квадратный корень из суммы квадратов элементов вектора (см. уравнение 2.7). Модуль вектора указывается с помощью двойных вертикальных полос вокруг вектора: $||\mathbf{v}||$.

Уравнение 2.7. Норма вектора

$$||\mathbf{v}|| = \sqrt{\sum_{i=1}^{n} v_i^2}.$$

В некоторых приложениях используются квадраты модулей векторов (записываемые как $||\mathbf{v}||^2$), и в этом случае член квадратного корня в правой части выпадает.

Прежде чем показать исходный код Python, следует объяснить еще несколько терминологических разночтений между линейной алгеброй «на меловой доске» и линейной алгеброй на Python. В математике размерность вектора – это число элементов в этом векторе, тогда как длина – это геометрическое расстояние; на языке Python функция `len()` (где `len` – это сокращение от англ. *length*, длина) возвращает *размерность* массива, тогда как функция `np.norm()` возвращает геометрическую длину (модуль вектора). Во избежание путаницы в данной книге вместо термина *длина* я буду использовать термин *модуль вектора* (либо *геометрическая длина*):

```
v = np.array([1, 2, 3, 7, 8, 9])
v_dim = len(v) # математическая размерность
v_mag = np.linalg.norm(v) # матем. модуль, длина или норма вектора
```

Существуют приложения, в которых нужен вектор, геометрическая длина которого равна единице, и такой вектор называется *единичным вектором*. Примеры приложений включают ортогональные матрицы, матрицы поворота, собственные векторы и сингулярные векторы.

Единичный вектор определяется как $||\mathbf{v}|| = 1$.

Излишне говорить, что многие векторы не являются единичными векторами. (У меня возникает соблазн написать «*большинство* векторов не являются единичными векторами», но существует бесконечное число единичных векторов и неединичных векторов, хотя множество бесконечных неединичных векторов больше, чем множество бесконечных единичных векторов.) К счастью, любой неединичный вектор имеет ассоциированный с ним единичный вектор. Это означает, что мы можем создать единичный вектор в том же направлении, что и неединичный вектор. Создать ассоциированный единичный вектор несложно; надо просто умножить на скаляр модуля вектора, взаимообратный его норме (уравнение 2.8):

---

[1] Англ. *magnitude*; син. величина вектора. – *Прим. перев.*

Уравнение 2.8. Создание единичного вектора

$$\hat{\mathbf{v}} = \frac{1}{||\mathbf{v}||}\mathbf{v}.$$

На рис. 2.4 показано общепринятое правило обозначения единичных векторов ($\hat{\mathbf{v}}$) в том же направлении, что и их родительский вектор $\mathbf{v}$. Данный рисунок иллюстрирует эти случаи.

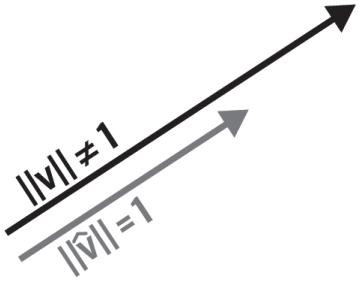


Рис. 2.4 ❖ Единичный вектор (серая стрелка) можно создать из неединичного вектора (черная стрелка); оба вектора имеют одинаковый угол, но разные модули

На самом деле утверждение о том, что «*любой* неединичный вектор имеет ассоциированный единичный вектор», не совсем верно. Существует вектор, который имеет неединичную длину и в то же время не имеет ассоциированного единичного вектора. Сможете ли вы угадать, какой это вектор[1]?

Здесь я не показываю исходный код Python создания единичных векторов, потому что это будет одним из упражнений в конце данной главы.

Точечное произведение векторов

Точечное произведение (также иногда именуемое *внутренним произведением*) является одной из наиболее важных операций во всей линейной алгебре. Это базовый вычислительный строительный блок, на основе которого строятся многие операции и алгоритмы, включая свертку, корреляцию, результаты преобразования Фурье, матричное умножение, извлечение линейных признаков, фильтрацию сигналов и т. д.

Точечное произведение между двумя векторами можно указать несколькими способами. Я буду использовать в основном общепринятую нотацию $\mathbf{a}^T\mathbf{b}$ по причинам, которые станут ясны после изучения матричного умножения. В других контекстах вы могли бы увидеть $\mathbf{a} \cdot \mathbf{b}$ или $\langle \mathbf{a}, \mathbf{b} \rangle$.

[1] Вектор нулей имеет длину 0, но не ассоциирован с единичным вектором, поскольку у него нет направления и поскольку невозможно прошкалировать вектор нулей так, чтобы он имел ненулевую длину.

Точечное произведение – это одно число, которое предоставляет информацию о взаимосвязи между двумя векторами. Давайте сначала сосредоточимся на алгоритме вычисления точечного произведения, а затем я расскажу, как его интерпретировать.

Для вычисления точечного произведения надо перемножить соответствующие элементы двух векторов, а затем просуммировать все отдельные произведения. Другими словами, это поэлементное умножение и сумма. В уравнении 2.9 **a** и **b** являются векторами, а a_i указывает на i-й элемент вектора **a**.

Уравнение 2.9. Формула точечного произведения

$$\delta = \sum_{i=1}^{n} \mathbf{a}_i \mathbf{b}_i.$$

По формуле можно заключить, что точечное произведение допустимо только между двумя векторами одинаковой размерности. Уравнение 2.10 показывает численный пример:

Уравнение 2.10. Пример вычисления точечного произведения

$$\begin{aligned}[1\ 2\ 3\ 4]\cdot[5\ 6\ 7\ 8] &= 1\times5 + 2\times6 + 3\times7 + 4\times8\\ &= 5 + 12 + 21 + 32\\ &= 70.\end{aligned}$$



Расстройства от индексации

Стандартная математическая система счисления и некоторые математикоориентированные программы обработки чисел, такие как MATLAB и Julia, начинают индексацию с 1 и останавливаются на *N*, тогда как некоторые языки программирования, такие как Python и Java, начинают индексацию с 0 и останавливаются на *N* – 1. Нам незачем обсуждать достоинства и ограничения каждого соглашения, хотя иногда я все-таки задумываюсь обо всех тех ошибках, которое данное несоответствие внесло в человеческую цивилизацию, но об этом различии важно помнить при переложении формул в исходный код Python.

Есть несколько способов реализации точечного произведения на Python; самый элементарный состоит в использовании функции `np.dot()`:

```
v = np.array([1, 2, 3, 4])
w = np.array([5, 6, 7, 8])
np.dot(v,w)
```



Примечание к функции np.dot()

На самом деле точечное произведение векторов в функции `np.dot()` не реализовано; в ней реализовано умножение матриц, которое представляет собой набор точечных произведений. Это будет иметь больше смысла после ознакомления с правилами и механизмами умножения матриц (глава 5). Если вы хотите обследовать их сейчас, то можете изменить приведенный выше исходный код, чтобы придать обоим векторам ориентацию (вдоль строки в противовес ориентации вдоль столбца). И вы обнаружите, что результат будет точечным произведением только тогда, когда первый аргумент на входе в функцию представляет собой вектор-строку, а второй аргумент – вектор-столбец.

Вот интересное свойство точечного произведения: умножение одного вектора на скаляр шкалирует точечное произведение на то же число. Указанное свойство можно обследовать, расширив приведенный ранее исходный код:

```
s = 10
np.dot(s*v, w)
```

Точечное произведение векторов `v` и `w` равно 70, а точечное произведение с использованием `s*v` (которое в математических обозначениях было бы записано как $\sigma\mathbf{v}^T\mathbf{w}$) равно 700. Теперь попробуйте этот пример с отрицательным скаляром, например `s = -1`. И вы увидите, что величина точечного произведения сохраняется, но знак меняется на противоположный. Разумеется, при `s = 0` точечное произведение равно нулю.

Теперь вы знаете, как вычислять точечное произведение. Каков смысл точечного произведения и как его интерпретировать?

Точечное произведение можно интерпретировать как *меру сходства*, или *соотнесенности*, между двумя векторами. Представьте, что вы собрали данные о росте и весе 20 человек и сохранили эти данные в двух векторах. Вы, конечно же, ожидаете, что эти переменные будут друг с другом связаны (более высокие люди, как правило, весят больше), и поэтому можно ожидать, что точечное произведение между этими двумя векторами будет большим. С другой стороны, величина точечного произведения зависит от шкалы данных, и, стало быть, точечное произведение между данными, измеренными в граммах и сантиметрах, будет больше, чем точечное произведение между данными, измеренными в фунтах и футах. Однако указанное произвольное шкалирование можно устранить с помощью коэффициента нормализации. Нормализованное точечное произведение между двумя переменными на самом деле называется *коэффициентом корреляции Пирсона*, и оно является одним из наиболее важных методов анализа в науке о данных. Подробнее об этом будет в главе 4!

Точечное произведение является дистрибутивным

Дистрибутивное свойство математики выглядит так: $a(b + c) = ab + ac$. В переложении на векторы и точечное произведение векторов это означает, что:

$$\mathbf{a}^T(\mathbf{b} + \mathbf{c}) = \mathbf{a}^T\mathbf{b} + \mathbf{a}^T\mathbf{c}.$$

На словах вы бы сказали, что точечное произведение суммы векторов равно сумме точечных произведений векторов.

Следующий ниже исходный код Python иллюстрирует свойство дистрибутивности:

```
a = np.array([  0, 1, 2 ])
b = np.array([  3, 5, 8 ])
```

```
c = np.array([ 13,21,34 ])

# точечное произведение дистрибутивно
res1 = np.dot( a, b+c )
res2 = np.dot( a,b ) + np.dot( a,c )
```

Два результата `res1` и `res2` одинаковы (с этими векторами ответ равен 110), иллюстрируя дистрибутивность точечного произведения. Обратите внимание, как математическая формула переводится в исходный код на языке Python; в математикоориентированном программировании переложение формул в исходный код является важным навыком.

Геометрия точечного произведения

Существует также геометрическое определение точечного произведения, которое представляет собой произведение модулей двух векторов, шкалированных на косинус угла между ними (уравнение 2.11).

Уравнение 2.11. Геометрическое определение точечного произведения векторов

$$\alpha = \cos(\theta_{\mathbf{v},\mathbf{w}})||\mathbf{v}||\,||\mathbf{w}||.$$

Уравнение 2.9 и уравнение 2.11 математически эквивалентны, но выражены в разных формах. Доказательство их эквивалентности является интересным упражнением в математическом анализе, но займет около страницы текста и основывается сперва на доказательстве других принципов, включая закон косинусов. Указанное доказательство не имеет отношения к данной книге и поэтому опущено.

Обратите внимание, что модули векторов являются строго положительными числами (за исключением вектора нулей, который имеет $||\mathbf{0}|| = 0$), тогда как косинус угла может находиться в диапазоне от –1 до +1. Это означает, что знак точечного произведения полностью определяется геометрической взаимосвязью между двумя векторами. На рис. 2.5 показаны пять случаев знака точечного произведения в зависимости от угла между двумя векторами (в 2D-визуализации, но данный принцип соблюдается для более высоких измерений).

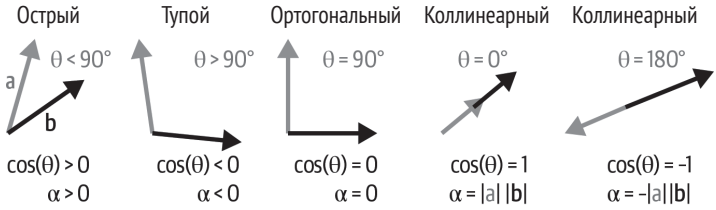


Рис. 2.5 ❖ Знак точечного произведения между двумя векторами показывает геометрическую взаимосвязь между этими векторами

Запомните: ортогональные векторы имеют нулевое точечное произведение

Некоторые учителя математики настаивают на том, что вы не должны запоминать формулы и члены, а вместо этого должны понимать процедуры и доказательства. Но давайте будем честны: запоминание – важная и неизбежная часть изучения математики. К счастью, линейная алгебра не требует чрезмерного запоминания, но есть несколько вещей, которые вам просто нужно запомнить.

Вот одна из них: ортогональные векторы имеют точечное произведение, равное нулю (это утверждение соблюдается в обоих направлениях – когда точечное произведение равно нулю, то два вектора ортогональны). Таким образом, следующие утверждения являются эквивалентными: два вектора ортогональны; два вектора имеют точечное произведение, равное нулю; два вектора пересекаются под углом 90°. Повторяйте эту эквивалентность до тех пор, пока она не отпечатается в вашем мозгу навечно.

Другие умножения векторов

Точечное произведение, пожалуй, является наиболее важным и наиболее часто используемым способом умножения векторов. Но есть и несколько других способов умножения векторов.

Адамарово умножение

Это просто причудливый термин для поэлементного умножения. Для реализации *адамарова умножения* надо перемножить каждый соответствующий элемент в двух векторах. Произведение представляет собой вектор той же размерности, что и у двух сомножителей. Например:

$$\begin{bmatrix}5\\4\\8\\2\end{bmatrix} \odot \begin{bmatrix}1\\0\\.5\\-1\end{bmatrix} = \begin{bmatrix}5\\3\\4\\-2\end{bmatrix}.$$

В Python звездочка указывает на поэлементное умножение двух векторов или матриц:

```
a = np.array([5, 4, 8, 2])
b = np.array([1, 0,.5])
a*b
```

Попробуйте выполнить приведенный выше исходный код Python и… о-хо-хо! Python выдаст сообщение об ошибке. Отыщите и исправьте ошибку. Что вы узнали об адамаровом умножении из этой ошибки? Обратитесь к сноске с ответом[1].

[1] Ошибка в том, что два вектора имеют разные размерности, указывая на то, что адамарово умножение определено только для двух векторов одинаковой размерности. Проблема устраняется за счет удаления одного числа из `a` либо добавления одного числа в `b`.

Адамарово умножение является удобным способом организовывать многочисленные умножения на скаляр. Например, представьте, что у вас есть данные о числе виджетов, продаваемых в разных магазинах, и цене за виджет в каждом магазине. Вы могли бы представить каждую переменную в виде вектора, а затем перемножать эти векторы по Адамару, чтобы вычислять доход виджета *по каждому магазину* (он отличается от суммарного дохода по *всем магазинам*, который будет вычислен как точечное произведение).

Внешнее произведение

Внешнее произведение – это способ создания матрицы из вектора-столбца и вектора-строки. Каждая строка в матрице внешнего произведения представляет собой скаляр вектора-строки, умноженный на соответствующий элемент в векторе-столбце. Можно было бы также сказать, что каждый столбец в матрице произведения является скаляром вектора-столбца, умноженным на соответствующий элемент в векторе-строке. В главе 6 я назову эту матрицу «матрицей ранга 1», а пока об этом термине не беспокойтесь и вместо этого сосредоточьтесь на закономерности, проиллюстрированной в следующем ниже примере:

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} \begin{bmatrix} d & e \end{bmatrix} = \begin{bmatrix} ad & ae \\ bd & be \\ cd & ce \end{bmatrix}.$$

> **Использование букв в линейной алгебре**
>
> На уроках алгебры в средней школе вы узнали, что использование букв в качестве абстрактных местозаполнителей вместо чисел позволяет понимать математику на более глубоком уровне, чем арифметика. Та же концепция используется в линейной алгебре: иногда, когда это облегчает понимание, внутри матриц учителя используют буквы вместо чисел. Буквы можно рассматривать как переменные.

Внешнее произведение сильно отличается от точечного произведения: вместо скаляра оно производит матрицу, а два вектора во внешнем произведении могут иметь разные размерности, тогда как два вектора в точечном произведении должны иметь одинаковую размерность.

Внешнее произведение обозначается как $\mathbf{vw}^T$ (вспомните, что мы исходим из того, что векторы ориентированы вдоль столбца; следовательно, внешнее произведение предусматривает умножение столбца на строку). Обратите внимание на тонкое, но важное различие между обозначениями точечного произведения ($\mathbf{v}^T\mathbf{w}$) и внешнего произведения ($\mathbf{vw}^T$). Возможно, сейчас это покажется странным и сбивающим с толку, но обещаю, это станет совершенно понятным после того, как вы узнаете о матричном произведении в главе 5.

Внешнее произведение похоже на *транслирование*, но это не одно и то же: *транслирование* – это общая операция программирования, которая исполь-

зуется для расширения векторов в арифметических операциях, таких как сложение, умножение и деление; *внешнее произведение* – это специфическая математическая процедура умножения двух векторов.

NumPy может вычислять внешнее произведение с помощью функции `np.outer()` либо функции `np.dot()`, при условии что два входных вектора ориентированы соответственно вдоль столбца и строки.

Перекрестное и тройное произведения

Существует несколько других способов умножения векторов, таких как перекрестное произведение и тройное произведение. Эти методы используются в геометрии и физике, но не так часто встречаются в приложениях, связанных с технологиями, чтобы тратить на них время в этой книге. Я упоминаю их здесь только для того, чтобы вы были мимолетно знакомы с названиями.

Ортогональное разложение векторов

«Разложить» вектор или матрицу означает разбить эту матрицу на несколько более элементарных частей. Разложение используется для выявления информации, которая «скрыта» в матрице, с целью облегчения работы с матрицей либо с целью сжатия данных. Не будет преуменьшением написать, что бóльшая часть линейной алгебры (абстрактной и практической) предусматривает матричные разложения. Матричные разложения – серьезная штука.

Ниже я представлю концепцию разложения, используя два простых примера со скалярами:

- число 42.01 можно разложить на две части: 42 и .01. Возможно, .01 – это шум, который следует игнорировать, либо, вероятно, цель состоит в том, чтобы сжать данные (целое число 42 требует меньше памяти, чем 42.01 с плавающей точкой[1]). Независимо от мотивации, разложение предусматривает представление одного математического объекта как суммы более элементарных объектов (42 = 42 + .01);
- число 42 можно разложить на произведение простых чисел 2, 3 и 7. Такое разложение называется *разложением на простые множители* и имеет много применений в числовой обработке и криптографии. В данном примере вместо сумм предусматривается использование произведений, но суть – та же самая: разложить один математический объект на более мелкие и более элементарные части.

В этом разделе мы начнем обследовать элементарное, но важное разложение, которое заключается в разбиении вектора на два отдельных вектора,

[1] В переводе оставлена форма записи чисел, принятая в оригинале книги, где для отделения дробной части принято использовать точку, а для отделения групп по три числа многозначного числа принято использовать запятую. – *Прим. перев.*

один из которых ортогонален опорному вектору, а другой параллелен этому опорному вектору. Ортогональное разложение векторов непосредственно приводит к процедуре Грама–Шмидта и QR-разложению, которое часто используется в решении обратных задач в статистике.

Давайте начнем с рисунка, чтобы представить цель разложения визуально. Рисунок 2.6 иллюстрирует ситуацию: даны два вектора **a** и **b** в стандартном положении, и наша цель – найти точку на векторе **a**, которая находится как можно ближе к голове вектора **b**. Указанную задачу также можно было бы выразить как задачу оптимизации: спроецировать вектор **b** на вектор **a** таким образом, чтобы проекционное расстояние было минимизировано. Разумеется, эта точка на **a** будет шкалированной версией вектора **a**; другими словами, $\beta\mathbf{a}$. Таким образом, теперь наша цель состоит в том, чтобы найти скаляр β. (Связь с ортогональным разложением векторов вскоре станет ясна.)

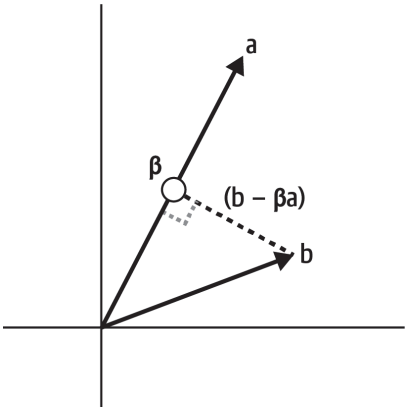


Рис. 2.6 ❖ Для того чтобы спроецировать точку в голове вектора **b** на вектор **a** с минимальным расстоянием, нужна формула вычисления β таким образом, чтобы длина проекционного вектора ($\mathbf{b} - \beta\mathbf{a}$) была минимальной

Важно отметить, что для определения отрезка от **b** до $\beta\mathbf{a}$ можно использовать вычитание векторов. Мы могли бы дать этому отрезку отдельную букву, например вектор **c**, но для отыскания решения необходимо вычитание.

Ключевой для понимания момент, который приводит к решению этой задачи, заключается в том, что точка на **a**, ближайшая к голове **b**, отыскивается путем проведения отрезка от **b**, которая пересекается с **a** под прямым углом. Интуитивное понимание здесь достигается, если вообразить треугольник, образованный началом координат, вершиной **b** и $\beta\mathbf{a}$; длина отрезка от **b** до $\beta\mathbf{a}$ становится больше по мере того, как угол $\measuredangle\beta\mathbf{a}$ становится меньше 90° или больше 90°.

Собрав все это вместе, мы пришли к выводу, что ($\mathbf{b} - \beta\mathbf{a}$) является ортогональным $\beta\mathbf{a}$, а это то же самое, что сказать, что данные векторы перпендикулярны. И стало быть, точечное произведение между ними должно быть равно нулю. Давайте трансформируем приведенные выше слова в уравнение:

$$\mathbf{a}^{\mathrm{T}}(\mathbf{b} - \beta\mathbf{a}) = 0.$$

Отсюда можно применить немного алгебры, чтобы отыскать β (обратите внимание на применение дистрибутивного свойства точечных произведений), которое показано в уравнении 2.12:

Уравнение 2.12. Решение задачи ортогональной проекции

$$\begin{aligned}\mathbf{a}^T\mathbf{b} - \beta\mathbf{a}^T\mathbf{a} &= 0;\\ \beta\mathbf{a}^T\mathbf{a} &= \mathbf{a}^T\mathbf{b};\\ \beta &= \frac{\mathbf{a}^T\mathbf{b}}{\mathbf{a}^T\mathbf{a}}.\end{aligned}$$

Получилось довольно красиво: мы начали с простой геометрической картинки, обследовали последствия геометрии, выразили эти последствия в виде формулы, а затем применили немного алгебры. И в результате обнаружили формулу проецирования точки на отрезок с минимальным расстоянием. Указанная проекция называется *ортогональной проекцией* и является основой для многих приложений в статистике и машинном обучении, включая знаменитую формулу наименьших квадратов для решения линейных моделей (вы увидите ортогональные проекции в главах 9, 10 и 11).

Могу себе представить, что вы весьма заинтригованы посмотреть, как будет выглядеть исходный код Python с реализацией этой формулы. Но вам придется написать этот код самостоятельно в упражнении 2.8 в конце данной главы. Если вы не хотите ждать конца главы, то смело решайте это упражнение сейчас, а затем продолжайте изучать ортогональное разложение.

Возможно, вам интересно, как это связано с ортогональным разложением векторов, то есть с названием данного раздела. Проекция с минимальным расстоянием была необходимой подготовительной работой, и теперь вы готовы усвоить разложение.

Как обычно, мы начнем с постановки и цели анализа. Вначале даны два вектора, которые я назову «целевым вектором» и «опорным вектором». Цель состоит в том, чтобы разложить целевой вектор на два других вектора таким образом, чтобы:

1) эти два вектора в сумме составляли целевой вектор и
2) один вектор был ортогонален опорному вектору, в то время как другой был параллелен опорному вектору.

Указанная ситуация проиллюстрирована на рис. 2.7.

Прежде чем приступать к математике, давайте проясним наши термины: я обозначу целевой вектор через **t** и опорный вектор – через **r**. Далее два вектора, сформированные из целевого вектора, будут называться *перпендикулярной компонентой*, обозначенной через $\mathbf{t}_{\perp\mathbf{r}}$, и *параллельной компонентой*, обозначенной через $\mathbf{t}_{\|\mathbf{r}}$.

Мы начнем с определения параллельной компоненты. Каким будет вектор, параллельный вектору **r**? Дело в том, что любая шкалированная версия вектора **r** очевидным образом является параллельной вектору **r**. И поэтому мы находим $\mathbf{t}_{\|\mathbf{r}}$, просто применяя формулу ортогональной проекции, которую мы только что обнаружили (уравнение 2.13):

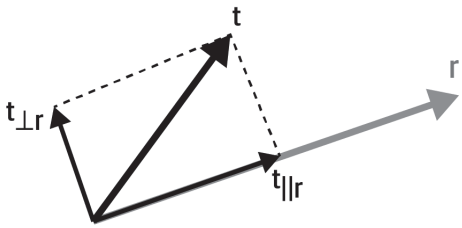


Рис. 2.7 ❖ Иллюстрация ортогонального разложения векторов: разложить вектор **t** на сумму двух других векторов, которые ортогональны и параллельны вектору **r**

Уравнение 2.13. Вычисление параллельной компоненты вектора **t** *относительно вектора* **r**

$$\mathbf{t}_{||\mathbf{r}} = \mathbf{r}\frac{\mathbf{t}^{\mathrm{T}}\mathbf{r}}{\mathbf{r}^{\mathrm{T}}\mathbf{r}}.$$

Обратите внимание на тонкое отличие от уравнения 2.12: там мы вычисляли только скаляр β; здесь же мы хотим вычислить шкалированный вектор β**r**.

Это и есть параллельная компонента. Как найти перпендикулярную компоненту? Ее найти проще, потому что мы уже знаем, что две компоненты вектора должны в сумме составлять изначальный целевой вектор. Таким образом:

$$\mathbf{t} = \mathbf{t}_{\perp\mathbf{r}} + \mathbf{t}_{||\mathbf{r}};$$

$$\mathbf{t}_{\perp\mathbf{r}} = \mathbf{t} - \mathbf{t}_{||\mathbf{r}}.$$

Другими словами, мы вычитаем параллельную компоненту из изначального вектора, и остатком будет наша перпендикулярная компонента.

Но *действительно* ли эта перпендикулярная компонента ортогональна опорному вектору? Да, это так! В целях доказательства вы показываете, что точечное произведение между перпендикулярной компонентой и опорным вектором равно нулю:

$$(\mathbf{t}_{\perp\mathbf{r}})^{\mathrm{T}}\mathbf{r} = \mathbf{0};$$

$$\left(\mathbf{t} - \mathbf{r}\frac{\mathbf{t}^{\mathrm{T}}\mathbf{r}}{\mathbf{r}^{\mathrm{T}}\mathbf{r}}\right)^{\mathrm{T}}\mathbf{r} = 0.$$

Работа с алгеброй указанного доказательства прямолинейна, но утомительна, поэтому я ее опустил. Вместо этого вы будете работать над укреплением интуитивного понимания в упражнениях, используя исходный код Python.

Надеюсь, вам понравилось изучать ортогональное разложение векторов. Еще раз обратите внимание на общий принцип: мы разбиваем один математический объект на комбинацию других объектов. Детали разложения зависят от наших ограничений (в данном случае ортогонального и параллельного

опорному вектору), означая, что разные ограничения (то есть разные цели анализа) могут приводить к разным разложениям одного и того же вектора.

Резюме

Прелесть линейной алгебры заключается в том, что даже самые сложные и вычислительно емкие операции на матрицах состоят из элементарных операций, большинство из которых можно понять на уровне интуиции с помощью геометрии. Не стоит недооценивать важность изучения элементарных операций на векторах, потому что то, что вы узнали в этой главе, ляжет в основу остальной части книги – и остальной части вашей карьеры *прикладного линейного алгебраиста* (кем вы на самом деле и являетесь, если занимаетесь наукой о данных, машинным обучением, искусственным интеллектом, глубоким обучением, обработкой изображений, компьютерным зрением, статистикой и всем таким прочим).

Вот наиболее важные выводы, которые следует вынести из этой главы.

- Вектор – это упорядоченный список чисел, который помещается в столбец либо строку. Число элементов в векторе называется его размерностью, и в геометрическом пространстве вектор можно представить в виде отрезка с числом осей, равным размерности.
- Несколько арифметических операций (сложение, вычитание и адамарово умножение) на векторах выполняется поэлементно.
- Точечное произведение – это одно число, в котором кодируется взаимосвязь между двумя векторами одинаковой размерности и которое вычисляется как поэлементное умножение и сумма.
- Точечное произведение равно нулю для ортогональных векторов, что геометрически означает, что векторы пересекаются под прямым углом.
- Ортогональное разложение векторов предусматривает разбиение вектора на сумму двух других векторов, которые ортогональны и параллельны опорному вектору. Формулу данного разложения можно снова извлечь из геометрии, но вы должны помнить фразу «отображение над модулем» как концепцию, выражаемую этой формулой.

Упражнения по программированию

Надеюсь, вы не воспринимаете эти упражнения как утомительную работу, которую вам нужно проделать. Как раз наоборот, данные упражнения дают возможность отточить ваши математические навыки и навыки программирования, а также убедиться, что вы действительно понимаете материал этой главы.

Хотелось бы также, чтобы вы рассматривали эти упражнения как трамплин, позволяющий вам продолжить обследовать линейную алгебру с использованием Python. Вносите изменения в исходный код, используя разные числа, разные размерности, разные ориентации и т. д. Пишите свой исход-

ный код тестирования других упомянутых в этой главе концепций. Самое главное: получайте удовольствие и учитесь использовать учебный опыт.

В качестве напоминания: решения ко всем упражнениям можно просмотреть либо скачать по ссылке https://github.com/mikexcohen/LA4DataScience.

Упражнение 2.1

В онлайновом репозитории исходного кода «отсутствует» код создания рис. 2.2. (На самом деле он не отсутствует – я перенес его в решение данного упражнения.) Итак, здесь ваша цель – написать свой исходный код для создания рис. 2.2.

Упражнение 2.2

Напишите алгоритм, который вычисляет норму вектора путем переложения уравнения 2.7 в исходный код. Подтвердите, используя случайные векторы с разными размерностями и ориентациями, что вы получаете тот же результат, что и `np.linalg.norm()`. Данное упражнение предназначено для того, чтобы дать вам больше опыта в индексации массивов NumPy и переложении формул в исходный код; на практике нередко проще использовать `np.linalg.norm()`.

Упражнение 2.3

Создайте функцию Python, которая на входе будет принимать вектор, а на выходе – выдавать единичный вектор в том же направлении. Что происходит при вводе вектора нулей?

Упражнение 2.4

Вы знаете, как создавать единичные векторы; что, если вы хотите создавать вектор любого произвольного модуля? Напишите функцию Python, которая на входе будет принимать вектор и желаемый модуль вектора и возвращать вектор в том же направлении, но с модулем, соответствующим второму входному аргументу.

Упражнение 2.5

Напишите цикл `for` для транспонирования вектора-строки в вектор-столбец без использования встроенной функции или метода, такого как `np.transpose()` или `v.T`. Данное упражнение поможет вам создавать и индексировать векторы с наделенной ориентацией.

Упражнение 2.6

Вот интересный факт: квадратная норма вектора может вычисляться как точечное произведение этого вектора на себя. Вернитесь к уравнению 2.8, чтобы убедиться в данной эквивалентности. Затем подтвердите ее с помощью Python.

Упражнение 2.7

Напишите исходный код, чтобы продемонстрировать, что точечное произведение является *коммутативным*. Коммутативное свойство означает $a \times b = b \times a$, что для точечного произведения векторов означает $\mathbf{a}^T\mathbf{b} = \mathbf{b}^T\mathbf{a}$. Проде-

монстрировав это в исходном коде, примените уравнение 2.9, чтобы понять, почему точечное произведение является коммутативным.

Упражнение 2.8

Напишите исходный код создания рис. 2.6. (Обратите внимание, что если присутствуют ключевые элементы, то ваше решение не обязательно должно выглядеть *в точности* так, как показано на рисунке.)

Упражнение 2.9

Реализуйте ортогональное разложение векторов. Начните с двух векторов случайных чисел **t** и **r** и воспроизведите рис. 2.8 (обратите внимание, что ваш график будет выглядеть несколько иначе из-за случайных чисел). Затем подтвердите, что сумма двух компонент равна **t** и что $\mathbf{t}_{\perp\mathbf{r}}$ и $\mathbf{t}_{||\mathbf{r}}$ ортогональны.

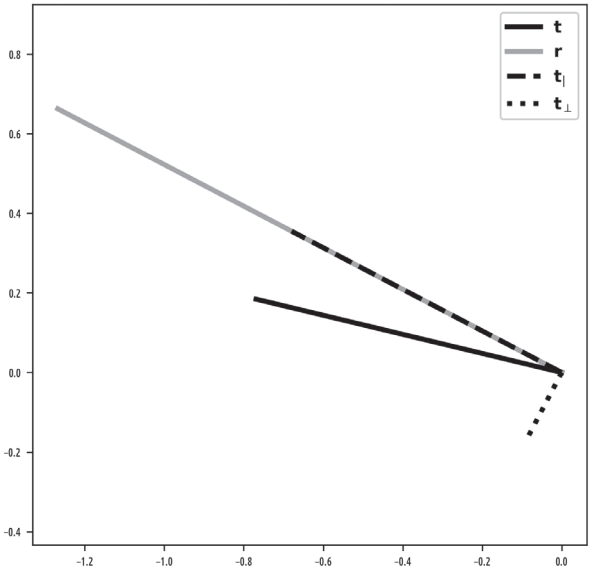


Рис. 2.8 ❖ Упражнение 9

Упражнение 2.10

Важным навыком программирования является отыскание ошибок. Допустим, в вашем исходном коде есть дефект, из-за которого знаменатель в проекционном скаляре уравнения 2.13 равен $\mathbf{t}^T\mathbf{t}$ вместо $\mathbf{r}^T\mathbf{r}$ (исходя из личного опыта написания этой главы, легко допустить ошибку!). Внедрите эту ошибку, чтобы проверить, действительно ли она уводит в сторону от точного исходного кода. Что можно сделать, чтобы проверить, является ли результат правильным либо неправильным? (В программировании подтверждение исходного кода известными результатами называется *проверкой на исправность*.)

Глава 3

Векторы. Часть 2

Предыдущая глава заложила основу для понимания векторов и базовых операций на векторах. Теперь вы расширите горизонты своих знаний в области линейной алгебры, познакомившись с набором взаимосвязанных понятий, включая линейную независимость, подпространства и базисы. Каждая из этих тем имеет решающее значение для понимания операций на матрицах.

Возможно, некоторые из представленных здесь тем покажутся абстрактными и оторванными от приложений, но между ними существует очень короткий путь, например векторные подпространства и подгонка статистических моделей к данным. Приложения в области науки о данных появятся позже, поэтому, пожалуйста, продолжайте концентрировать свое внимание на основах, чтобы продвинутые темы было легче понять.

Множества векторов

Мы начнем главу с чего-нибудь простого: коллекция векторов называется *множеством*. Вообразите, как вы кладете себе в рюкзак пучок векторов, чтобы носить его с собой. Векторные множества обозначаются курсивом и заглавными буквами, такими как S или V. Математические множества описываются следующим образом:

$$V = \{\mathbf{v}_1, \ldots, \mathbf{v}_n\}.$$

Представьте, например, набор данных о числе случаев заражения Ковидом-19, госпитализациях и смертях из ста стран; эти данные из каждой страны можно было бы сохранить в трехэлементном векторе и создать множество векторов, содержащее сто векторов.

Множество векторов может содержать конечное или бесконечное число векторов. Возможно, множества векторов с бесконечным числом векторов будет звучать как бесполезно глупая абстракция, но векторные подпространства – это бесконечные множества векторов, и они имеют большое значение для подгонки статистических моделей к данным.

Множества векторов также могут быть пустыми и обозначаются как $V = \{\}$. Вы столкнетесь с пустыми множествами векторов, когда познакомитесь с пространствами матриц.

Линейно-взвешенная комбинация

Линейно-взвешенная комбинация – это способ смешивания информации из нескольких переменных, при этом некоторые переменные вносят больший вклад, чем другие. Указанную фундаментальную операцию также иногда называют *линейной смесью*, или *взвешенной комбинацией* (часть со словом *линейная* подразумевается). Иногда вместо слова *вес* используется термин «*коэффициент*».

Линейно-взвешенная комбинация просто означает умножение векторов на скаляр и сложение: взять некоторое множество векторов, умножить каждый вектор на скаляр и сложить их, чтобы получить один вектор (уравнение 3.1).

Уравнение 3.1. Линейно-взвешенная комбинация

$$\mathbf{w} = \lambda_1\mathbf{v}_1 + \lambda_2\mathbf{v}_2 + \ldots + \lambda_n\mathbf{v}_n.$$

Подразумевается, что все векторы $\mathbf{v}_i$ имеют одинаковую размерность; в противном случае сложение недопустимо. Скаляры λ могут быть любым действительно-значным числом, включая ноль.

Технически уравнение 3.1 можно было бы переписать для вычитания векторов, но поскольку можно вычитать, задавая отрицательное значение числа λ_i, проще обсуждать линейно-взвешенные комбинации в терминах суммирования.

Уравнение 3.2 показывает пример, помогающий сделать все это конкретнее:

Уравнение 3.2. Линейно-взвешенная комбинация

$$\lambda_1 = 1,\ \ \lambda_2 = 2,\ \ \lambda_3 = -3, \qquad \mathbf{v}_1 = \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix},\ \ \mathbf{v}_2 = \begin{bmatrix} -4 \\ 0 \\ -4 \end{bmatrix},\ \ \mathbf{v}_3 = \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix};$$

$$\mathbf{w} = \lambda_1\mathbf{v}_1 + \lambda_2\mathbf{v}_2 + \lambda_3\mathbf{v}_3 = \begin{bmatrix} -7 \\ -4 \\ -13 \end{bmatrix}.$$

Линейно-взвешенные комбинации реализуются в исходном коде очень просто, как показано ниже. В Python важен тип данных; проверьте, что произойдет, когда векторы будут списками, а не массивами NumPy[1]:

[1] Как показано в главах 2 и 16, умножение списка на целое число повторяет список заданное число раз вместо его умножения на скаляр.

```
l1 = 1
l2 = 2
l3 = -3
v1 = np.array([ 4, 5, 1])
v2 = np.array([-4, 0,-4])
v3 = np.array([ 1, 3, 2])
l1*v1 + l2*v2 + l3*v3
```

Организовывать хранение каждого вектора и каждого коэффициента в виде отдельных переменных утомительно, и такой подход не масштабируется под решение более крупных задач. Поэтому на практике линейно-взвешенные комбинации реализуются с помощью компактного и масштабируемого метода умножения матриц на векторы, о котором вы узнаете в главе 5; пока же основное внимание будет сконцентрировано на концепции и реализации в исходном коде.

Линейно-взвешенные комбинации имеют несколько применений. Три из них таковы:

- предсказанные данные на выходе из статистической модели создаются путем применения линейно-взвешенной комбинации регрессоров (предсказательных переменных) и коэффициентов (скаляров), которые вычисляются с помощью алгоритма наименьших квадратов, о котором вы узнаете в главах 11 и 12;
- в процедурах уменьшения размерности, таких как анализ главных компонент, каждая компонента (иногда именуемая фактором, или модой) извлекается как линейно-взвешенная комбинация каналов данных, при этом веса (коэффициенты) отбираются таким образом, чтобы максимизировать дисперсию компоненты (наряду с некоторыми другими ограничениями, о которых вы узнаете в главе 15);
- искусственные нейронные сети (архитектура и алгоритм, приводящие в действие технологию глубокого обучения) предусматривают две операции: линейно-взвешенную комбинацию входных данных с последующим нелинейным преобразованием. Веса усваиваются алгоритмом путем минимизации стоимостной функции, которая обычно представляет собой разницу между модельным предсказанием и реальной целевой переменной.

Концепция линейно-взвешенной комбинации является механизмом создания подпространств векторов и пространств матриц и занимает центральное место в линейной независимости. И действительно, линейно-взвешенная комбинация и точечное произведение являются двумя наиболее важными элементарными строительными блоками, из которых строятся многие продвинутые линейно-алгебраические вычисления.

Линейная независимость

Множество векторов является *линейно зависимым*, если по меньшей мере один вектор в множестве можно выразить как линейно-взвешенную ком-

бинацию других векторов в этом множестве. И следовательно, множество векторов является *линейно независимым*, если ни один вектор невозможно выразить как линейно-взвешенную комбинацию других векторов в этом множестве.

Ниже приведены два множества векторов. Прежде чем читать текст, попробуйте определить, является ли каждое множество зависимым либо независимым. (Когда слово *линейная* подразумевается, термин *линейная независимость* иногда сокращается до одного слова *независимость*.)

$$V = \left\{\begin{bmatrix}1\\3\end{bmatrix}, \begin{bmatrix}2\\7\end{bmatrix}\right\}, \qquad S = \left\{\begin{bmatrix}1\\3\end{bmatrix}, \begin{bmatrix}2\\6\end{bmatrix}\right\}.$$

Векторное множество *V* является линейно независимым: невозможно выразить один вектор в множестве как линейное кратное другого вектора в множестве. То есть если обозначить векторы в множестве через $\mathbf{v}_1$ и $\mathbf{v}_2$, то не существует возможного скаляра λ, для которого $\mathbf{v}_1 = \lambda\mathbf{v}_2$.

Что скажете насчет множества *S*? Оно является зависимым, потому что мы можем применить линейно-взвешенные комбинации некоторых векторов в множестве и получить другие векторы в множестве. Существует бесконечное число таких комбинаций, две из которых – $\mathbf{s}_1 = .5^*\mathbf{s}^*_2$ и $\mathbf{s}_2 = 2^*\mathbf{s}^*_1$.

Давайте попробуем еще один пример. Опять же, вопрос заключается в том, является ли множество T линейно независимым либо линейно зависимым:

$$T = \left\{\begin{bmatrix}8\\-4\\14\\6\end{bmatrix}, \begin{bmatrix}4\\6\\0\\3\end{bmatrix}, \begin{bmatrix}14\\2\\4\\7\end{bmatrix}, \begin{bmatrix}13\\2\\9\\8\end{bmatrix}\right\}.$$

А вот тут разобраться намного сложнее, чем в двух предыдущих примерах. Оказывается, что это линейно зависимое множество (например, сумма первых трех векторов равна удвоенному четвертому вектору). Но я и не жду, что вы сможете понять это в результате визуального осмотра.

Так как же определять линейную независимость на практике? Линейная независимость определяется путем создания матрицы из множества векторов, вычисления ранга матрицы и сравнения ранга с наименьшим числом из числа строк или столбцов. Возможно, это предложение сейчас не будет иметь для вас смысла, потому что вы еще не познакомились с рангом матрицы. Поэтому сейчас сосредоточьте свое внимание на концепции, что множество векторов является линейно зависимым, если по меньшей мере один вектор в указанном множестве можно выразить как линейно-взвешенную комбинацию других векторов в данном множестве, и множество векторов является линейно независимым, если ни один вектор невозможно выразить как комбинацию других векторов.

Независимые множества

Независимость – это свойство множества векторов. То есть множество векторов может быть линейно независимым либо линейно зависимым; независимость не является свойством отдельного вектора внутри множества.

Математика линейной независимости

Теперь, когда вы понимаете эту концепцию, я хочу убедиться, что вы также понимаете формальное математическое определение линейной зависимости, которое выражено в уравнении 3.3.

Уравнение 3.3. Линейная зависимость[1]

$$\mathbf{0} = \lambda_1\mathbf{v}_1 + \lambda_2\mathbf{v}_2 + \ldots + \lambda_n\mathbf{v}_n, \quad \lambda \in \mathbb{R}.$$

Это уравнение говорит, что линейная зависимость означает существование возможности определить некоторую линейно-взвешенную комбинацию векторов в множестве, такую чтобы получить вектор нулей. Если есть возможность найти несколько скаляров λ, которые делают уравнение истинным, то множество векторов является линейно зависимым. И наоборот, если нет возможного способа линейно скомбинировать векторы так, чтобы получить векторы нулей, то множество является линейно независимым.

Вероятно, поначалу это покажется не поддающимся интуитивному пониманию. Почему нас волнует вектор нулей, когда рассматривается вопрос о возможности выразить хотя бы один вектор в множестве через взвешенную комбинацию других векторов в этом множестве? Вероятно, вы бы предпочли переписать определение линейной зависимости следующим образом:

$$\lambda_1\mathbf{v}_1 = \lambda_2\mathbf{v}_2 + \ldots + \lambda_n\mathbf{v}_n, \quad \lambda \in \mathbb{R}.$$

Почему бы не начать именно с этого уравнения, а не помещать вектор нулей в левую часть? Приравнивание уравнения к нулю помогает укрепить принцип, согласно которому *все множество целиком* является зависимым либо независимым; ни один отдельный вектор не имеет привилегированного положения в качестве «зависимого вектора» (см. «Независимые множества» на данной странице). Другими словами, когда речь заходит о независимости, множества векторов становятся исключительно эгалитарными.

Но погодите. Тщательный анализ уравнения 3.3 показывает тривиальное решение: приравнять все скаляры λ к нулю, и уравнение будет выглядеть как $\mathbf{0} = \mathbf{0}$, независимо от векторов в множестве. Однако, как я написал в главе 2, в линейной алгебре содержащие нули тривиальные решения нередко игнорируются. Таким образом, мы добавляем ограничение, что по меньшей мере один $\lambda \neq 0$.

[1] Это уравнение представляет собой применение линейно-взвешенной комбинации!

Это ограничение можно встроить в уравнение путем деления на один из скаляров; имейте в виду, что $\mathbf{v}_1$ и λ_1 могут относиться к любой векторно-скалярной паре в множестве:

$$\mathbf{0} = \mathbf{v}_1 + \ldots + \frac{\lambda_n}{\lambda_1}\mathbf{v}_n, \quad \lambda \in \mathbb{R}, \lambda_1 \neq 0.$$

Независимость и вектор нулей

Говоря по-простому, любое множество векторов, включающее вектор нулей, автоматически является линейно зависимым множеством. И вот почему: любое скалярное кратное вектору нулей по-прежнему остается вектором нулей, поэтому определение линейной зависимости всегда соблюдается. Это можно увидеть в следующем ниже уравнении:

$\lambda_0\mathbf{0} = 0\mathbf{v}_1 + 0\mathbf{v}_2 + 0\mathbf{v}_n$.

До тех пор, пока $\lambda_0 \neq 0$, мы имеем нетривиальное решение, и множество соответствует определению линейной зависимости.



Как насчет нелинейной независимости?

– Но, Майк, – я представляю, как вы протестуете, – разве жизнь, Вселенная и все остальное не *не*линейны?

Полагаю, было бы интересным упражнением подсчитать суммарное число линейных взаимодействий во Вселенной относительно числа нелинейных и посмотреть, какая сумма больше. Но линейная алгебра всецело касается, ну, вы поняли, *линейных* операций. Если один вектор можно выразить как нелинейную (а не линейную) комбинацию других векторов, то эти векторы все равно будут формировать линейно независимое множество. Причина ограничения линейности заключается в том, что мы хотим выражать преобразования как умножение матриц, которое является линейной операцией. Это говорится не для того, чтобы бросить тень на нелинейные операции – в моей воображаемой беседе вы красноречиво заявили, что чисто линейная Вселенная была бы довольно скучной и предсказуемой. Но нам вовсе не нужно объяснять всю Вселенную с помощью линейной алгебры; линейная алгебра нам нужна только для линейных частей. (Тут также стоит упомянуть, что многие нелинейные системы хорошо аппроксимируются с помощью линейных функций.)

Подпространство и охват

Когда я вводил понятие линейно-взвешенных комбинаций, я приводил примеры с конкретными числовыми значениями весов (например, $\lambda_1 = 1$, $\lambda_3 = -3$). *Подпространство* – это та же идея, но с использованием бесконечности возможных способов линейного комбинирования векторов в множестве.

То есть для некоторого (конечного) множества векторов бесконечное число способов их линейного комбинирования – с использованием одних и тех

же векторов, но разных числовых значений весов – создает *подпространство векторов*. А механизм комбинирования всех возможных линейно-взвешенных комбинаций называется *охватом* множества векторов[1]. Давайте разберем несколько примеров. Мы начнем с элементарного примера множества векторов, содержащего один вектор:

$$V = \left\{ \begin{bmatrix} 1 \\ 3 \end{bmatrix} \right\}.$$

Охватом данного множества векторов является бесконечность векторов, которые могут быть созданы как линейные комбинации векторов в множестве. Для множества с одним вектором это просто означает все возможные шкалированные версии данного вектора. На рис. 3.1 показаны вектор и подпространство, которое он охватывает. Учтите, что любой вектор в серой пунктирной линии можно сформировать как некую шкалированную версию вектора.

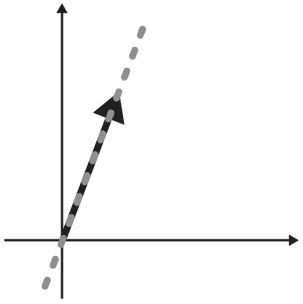


Рис. 3.1 ❖ Вектор (черным цветом) и подпространство, которое он охватывает (серым цветом)

Нашим следующим примером является множество из двух векторов в $\mathbb{R}^3$:

$$V = \left\{ \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}, \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix} \right\}.$$

Векторы находятся в $\mathbb{R}^3$, поэтому они графически представлены на трехмерной оси. Но подпространство, которое они охватывают, представляет собой двумерную плоскость в этом трехмерном пространстве (рис. 3.2). Указанная плоскость проходит через начало координат, потому что шкалирование обоих векторов на ноль дает вектор нулей.

---

[1] Также называется линейной оболочкой (англ. *linear hull*), однако используемый в книге термин охват (англ. *span*) является более релевантным по причинам, которые будут ясны далее в этой главе. – *Прим. перев.*

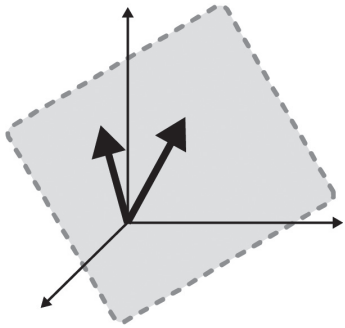


Рис. 3.2 ❖ Два вектора (черным цветом) и подпространство, которое они охватывают (серым цветом)

В первом примере был один вектор, и его охватом было одномерное подпространство, а во втором примере было два вектора, и их охватом было двумерное подпространство. Кажется, вырисовывается некая закономерность, но внешность бывает обманчивой. Рассмотрим следующий пример:

$$V = \left\{ \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} \right\}.$$

Два вектора в $\mathbb{R}^3$, но подпространство, которое они охватывают, попрежнему является всего лишь одномерным подпространством – отрезком (рис. 3.3). Почему так? А все потому, что один вектор в множестве уже находится в зоне охвата другим вектором. И поэтому, с точки зрения охвата, один из двух векторов является избыточным.

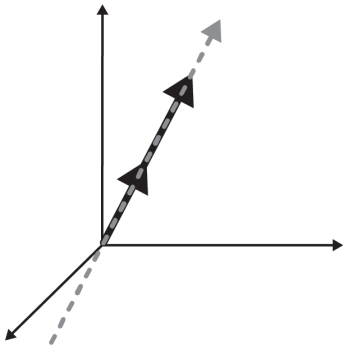


Рис. 3.3 ❖ Одномерное подпространство (серым цветом), охватываемое двумя векторами (черным цветом)

Так какова же взаимосвязь между размерностью охватываемого подпространства и числом векторов в множестве? Вероятно, вы уже догадались, что это как-то связано с линейной независимостью.

Размерность подпространства, охватываемого множеством векторов, – это наименьшее число векторов, образующих линейно независимое множество. Если множество векторов является линейно независимым, то размерность подпространства, охватываемого векторами в этом множестве, равна числу векторов в этом множестве. Если множество является зависимым, то размерность подпространства, охватываемого этими векторами, с необходимостью меньше числа векторов в этом множестве. Насколько именно меньше, это другой вопрос – для того чтобы знать взаимосвязь между числом векторов в множестве и размерностью охватываемого ими подпространства, вам нужно понимать ранг матрицы, о котором вы узнаете в главе 6.

Формальное определение подпространства векторов таково: это подмножество, которое замкнуто при сложении и умножении на скаляр и включает начало пространства[1]. Это означает, что любая линейно-взвешенная комбинация векторов в подпространстве также должна находиться в одном и том же подпространстве, включая установку всех весов равными нулю, чтобы произвести вектор нулей в начале пространства.

Пожалуйста, не теряйте сон, размышляя о том, что значит быть «замкнутым при сложении и умножении на скаляр»; просто запомните, что подпространство векторов создается из всех возможных линейных комбинаций множества векторов.

В чем разница между подпространством и охватом?

Многих студентов смущает разница между *охватом* и *подпространством*. И это понятно, потому что данные понятия тесно связаны и нередко относятся к одному и тому же. Я объясню разницу между ними, но не придавайте значения тонкостям – охват и подпространство так часто относятся к идентичным математическим объектам, что использовать эти термины взаимозаменяемо, как правило, совершенно правильно.

Я нахожу, что термин *охват* лучше использовать в глагольной форме, а термин *подпространство* – разумеется, как существительное, и это помогает понять их различие: множество векторов охватывает, и результатом охвата является подпространство. Теперь учтите, что *подпространство* может быть меньшей частью большего пространства, как вы увидели на рис. 3.3. Подытоживая все сказанное: охват – это механизм создания подпространства. (С другой стороны, при использовании охвата в качестве существительного охват и подпространство относятся к одному и тому же бесконечному множеству векторов.)

Базис

Как далеко друг от друга находятся Амстердам и Тенерифе? Примерно 2000. Что означает «2000»? Это число имеет смысл только в том случае, если добавить базисную единицу. Базис подобен линейке для измерения пространства.

В приведенном выше примере единицей измерения является *миля*. Таким образом, наше базисное измерение расстояния между Голландией и Испани-

[1] Син. начало координат пространства. – *Прим. перев.*

ей составляет 1 милю. Конечно же, мы могли бы использовать и другие единицы измерения, такие как нанометры или световые годы, но думаю, можно согласиться с тем, что миля является удобным базисом для определения расстояния по этой шкале. Как насчет длины, которую ваш ноготь отрастает за один день, – должны ли мы по-прежнему использовать мили? В техническом плане это возможно, но, думаю, можно согласиться с тем, что *миллиметр* будет более удобной базисной единицей. Для ясности: величина, на которую вырос ваш ноготь за последние 24 часа, будет одинаковой, независимо от того, в чем вы ее измеряете: в нанометрах, милях или световых годах. Но разные единицы измерения более или менее удобны для разных задач.

Вернемся к линейной алгебре: *базис* – это множество линеек, которые вы используете для описания информации в матрице (например, матрице данных). Как и в приведенных выше примерах, одни и те же данные можно описывать, используя разные линейки, но некоторые линейки более удобны для решения определенных задач, чем другие.

Наиболее распространенным базисным множеством является декартова ось: знакомая плоскость XY, которую вы используете с начальной школы. Базисные множества для двумерного и трехмерного декартовых графиков можно записать следующим образом:

$$S_2 = \left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\}, \qquad S_3 = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}.$$

Обратите внимание, что декартовы базисные множества содержат векторы, которые являются взаимно ортогональными и имеют единичную длину. Это замечательные свойства, и именно по этой причине декартовы базисные множества столь распространены (и действительно, они называются *стандартным базисным множеством*).

Но это не единственные базисные множества. Следующее ниже множество является другим базисным множеством для $\mathbb{R}^2$:

$$T = \left\{ \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \begin{bmatrix} -3 \\ 1 \end{bmatrix} \right\}.$$

Базисные множества S_2 и T оба охватывают одно и то же подпространство (все из $\mathbb{R}^2$). Почему вы бы предпочли T, а не S? Представим, что мы хотим описать точки данных p и q на рис. 3.4. Указанные точки данных можно описать как их отношение к началу координат – то есть их координаты, – используя базис S либо базис T.

В базисе S эти две координаты равны $p = (3, 1)$ и $q = (-6, 2)$. В линейной алгебре мы говорим, что точки выражаются как линейные комбинации базисных векторов. В данном случае эта комбинация равна $3\mathbf{s}_1 + 1\mathbf{s}_2$ для точки p и $-6\mathbf{s}_1 + 2\mathbf{s}_2$ для точки q.

Теперь давайте опишем эти точки в базисе T. В качестве координат мы имеем $p = (1, 0)$ и $q = (0, 2)$. И в терминах базисных векторов мы имеем $1\mathbf{t}_1 + 0\mathbf{t}_2$ для точки p и $0\mathbf{t}_1 + 2\mathbf{t}_2$ для точки q (другими словами, $p = \mathbf{t}_1$ и $q = 2\mathbf{t}_2$). Опять

же, точки данных *p* и *q* одинаковы независимо от базисного множества, но *T* обеспечил компактное и ортогональное описание.

Базисы имеют чрезвычайную важность в науке о данных и машинном обучении. Собственно говоря, многие задачи прикладной линейной алгебры концептуализируются как отыскание наилучшего множества базисных векторов с целью описания некоторого подпространства. Вы, вероятно, слышали о следующих терминах: уменьшение размерности, извлечение признаков, анализ главных компонент, анализ независимых компонент, факторный анализ, сингулярное разложение, линейный дискриминантный анализ, аппроксимация изображений, сжатие данных. Хотите верьте, хотите нет, но все эти методы анализа, по сути, являются способами определения оптимальных базисных векторов для конкретной задачи.

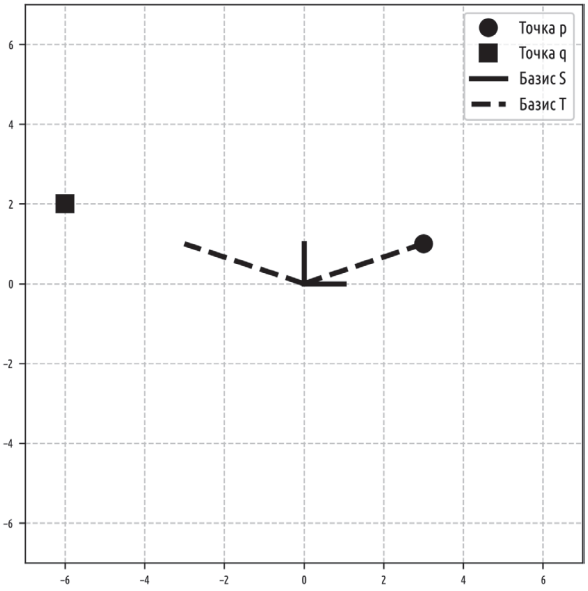


Рис. 3.4 ❖ Те же самые точки (*p* и *q*) могут быть описаны базисным множеством *S* (черные сплошные линии) либо *T* (черные пунктирные линии)

Рассмотрим рис. 3.5: это набор данных, состоящий из двух переменных (каждая точка представляет точку данных). На рисунке фактически показаны три четко различимых базиса: «стандартное базисное множество», соответствующее отрезкам $x = 0$ и $y = 0$, и базисные множества, определенные с помощью анализа главных компонент (PCA[1]; левый график) и с помощью анализа независимых компонент (ICA[2]; правый график). Какое из этих базисных множеств обеспечивает «наилучший» способ описания данных? У вас, возможно, возникнет соблазн сказать, что базисные векторы, вычисленные

[1] Англ. *Principal Components Analysis*. – *Прим. перев.*

[2] Англ. *Independent Components Analysis*. – *Прим. перев.*

с помощью ICA, будут наилучшими. Истина же будет посложнее (как это обычно и бывает): ни одно базисное множество не является, по сути, лучше или хуже; для конкретных задач разные базисные множества могут быть более или менее полезными в зависимости от целей анализа, особенностей данных, налагаемых анализом ограничений и т. д.

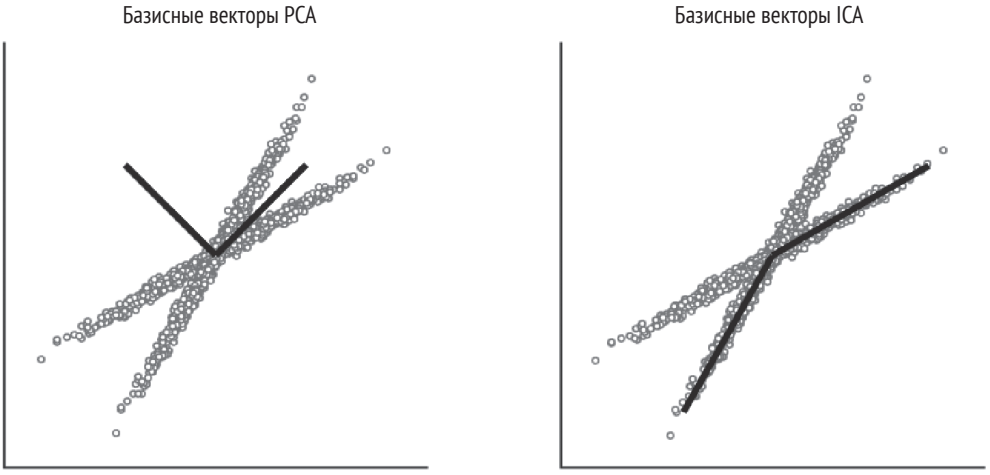


Рис. 3.5 ❖ Двумерный набор данных
с использованием разных базисных векторов (черные линии)

Определение базиса

Разобравшись со смыслом понятий базиса и базисного множества, формальное определение станет простым. В сущности, базис – это просто комбинация охвата и независимости: множество векторов может быть базисом для некоторого подпространства, если оно

1) охватывает это подпространство и
2) является независимым множеством векторов.

Базис должен охватывать подпространство, чтобы его использовать в качестве базиса для этого подпространства, потому что невозможно описать то, что невозможно измерить[1]. На рис. 3.6 показан пример точки за пределами одномерного подпространства. Базисный вектор для этого подпространства не может измерить точку *r*. Черный вектор по-прежнему является допустимым базисным вектором для подпространства, которое он охватывает, но он не формирует базиса для любого подпространства за пределами того, что он охватывает.

Таким образом, базис должен охватывать пространство, для которого он используется. Это понятно. Но почему базисное множество требует линейной независимости? Причина в том, что любой данный вектор в подпространстве должен иметь уникальную координату, используя этот базис. Давайте пред-

[1] В науке это общепризнанная азбучная истина.

ставим, что мы описываем точку *p* из рис. 3.4, используя следующее ниже множество векторов:

$$U = \left\{ \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right\}.$$

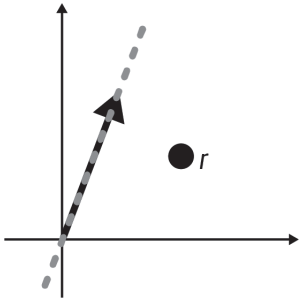


Рис. 3.6 ❖ Базисное множество может измерять только то, что содержится внутри его охвата

U – это совершенно допустимое множество векторов, но оно определенно *не* является базисным множеством. Почему[1]?

Какая линейно-взвешенная комбинация описывает точку *p* в множестве *U*? Дело в том, что коэффициентами линейно-взвешенной комбинации трех векторов в *U* могут быть (3, 0, 1) либо (0, 1,5, 1), либо … триллионы других возможностей. Такая ситуация сбивает с толку, и поэтому математики решили, что в пределах базисного множества вектор должен иметь *уникальные* координаты. А линейная независимость гарантирует такую уникальность.

Для ясности, точку *p* (или любую другую точку) можно описать с использованием бесконечного числа базисных множеств. Таким образом, результат измерения не будет уникальным с точки зрения громадного числа возможных базисных множеств. Но *в пределах* базисного множества точка определяется ровно одной линейно-взвешенной комбинацией. То же самое и с моей аналогией по поводу расстояния в начале этого раздела: расстояние от Амстердама до Тенерифе можно измерить, используя много разных единиц измерения, но это расстояние имеет только одно значение в расчете на одну единицу измерения. Расстояние не составляет одновременно 3200 миль и 2000 миль, но оно составляет одновременно 3200 *километров* и 2000 *миль*. (Примечание для ботаников: я здесь аппроксимирую, хорошо?)

Резюме

Поздравляю с завершением еще одной главы! (Ну, почти завершением: еще нужно решить несколько упражнений по программированию.) Смысл этой

[1] Потому что оно является линейно зависимым множеством.

главы состоял в том, чтобы вывести ваши фундаментальные знания о векторах на новый уровень. Ниже приведен список ключевых выводов, но, пожалуйста, помните, что в основе всех этих выводов лежит очень мало элементарных принципов, в первую очередь линейно-взвешенных комбинаций векторов.

- Множество векторов – это коллекция векторов. В множестве может быть конечное либо бесконечное число векторов.
- Линейно-взвешенная комбинация означает умножение на скаляр и сложение векторов в множестве. Линейно-взвешенная комбинация является одним из наиболее важных понятий в линейной алгебре.
- Множество векторов является линейно зависимым, если вектор в множестве можно выразить как линейно-взвешенную комбинацию других векторов в множестве. И множество является линейно независимым, если такой линейно-взвешенной комбинации не существует.
- Подпространство – это бесконечное множество всех возможных линейно-взвешенных комбинаций множества векторов.
- Базис – это линейка для измерения пространства. Множество векторов может быть базисом для подпространства, если оно
  1) охватывает это подпространство и
  2) является линейно независимым.

Одной из главнейших целей в науке о данных является отыскание наилучшего базисного множества, чтобы описывать наборы данных или решать задачи.

Упражнения по программированию

Упражнение 3.1

Перепишите исходный код линейно-взвешенной комбинации, но поместите скаляры в список, а векторы – в качестве элементов в списке (таким образом, у вас будет два списка, один из скаляров и один из массивов NumPy). Затем примените цикл `for`, чтобы реализовать операцию линейно-взвешенной комбинации. Инициализируйте выходной вектор функцией `np.zeros()`. Подтвердите, что вы получаете тот же результат, что и в приведенном ранее исходном коде.

Упражнение 3.2

Хотя метод прокручивания списков из предыдущего упражнения в цикле не так эффективен, как умножение матриц на векторы, он более масштабируем, чем без цикла `for`. Этот факт можно обследовать, добавив в качестве элементов списков дополнительные скаляры и векторы. Что произойдет, если новый добавленный вектор находится в $\mathbb{R}^4$, а не в $\mathbb{R}^3$? А что произойдет, если у вас будет больше скаляров, чем векторов?

Упражнение 3.3

В этом упражнении вы будете извлекать случайные точки в подпространствах. Благодаря этому упражнению вы укрепите идею о том, что подпространства содержат *любую* линейно-взвешенную комбинацию охватываю-

щих векторов. Определите множество векторов, содержащее один вектор [1, 3]. Затем создайте 100 чисел, выбранных случайным образом из равномерного распределения между –4 и +4. Это ваши случайные скаляры. Умножьте случайные скаляры на базисный вектор, чтобы создать 100 случайных точек в подпространстве. Нанесите эти точки на график.

Затем повторите процедуру, но используя два вектора в $\mathbb{R}^3$: [3, 5, 1] и [0, 2, 2]. Обратите внимание, что для 100 точек и двух векторов вам нужно 100×2 случайных скаляров. Результирующие случайные точки будут находиться на плоскости. На рис. 3.7 показано, как будут выглядеть результаты (по рисунку не ясно, что точки лежат на плоскости, но вы увидите это, когда будете перетаскивать график на экране).

Рекомендую для рисования точек использовать библиотеку `plotly`, чтобы иметь возможность кликать и перетаскивать трехмерную ось по всему графику. Вот подсказка, как сделать так, чтобы все работало[1]:

```
import plotly.graph_objects as go
fig = go.Figure( data=[go.Scatter3d(
                    x=points[:,0],
                    y=points[:,1],
                    z=points[:,2],
                    mode='markers' )])
fig.show()
```

Наконец, повторите случай $\mathbb{R}^3$, но задайте второй вектор как 1/2 от первого.

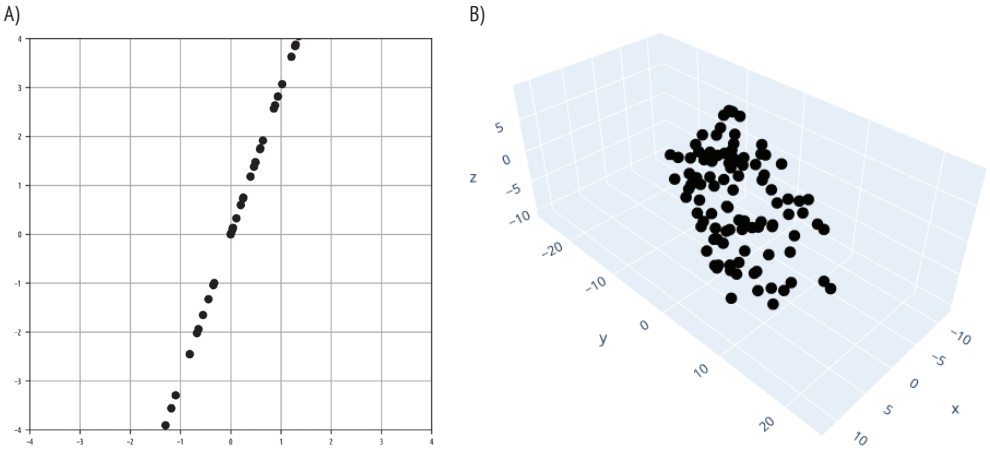


Рис. 3.7 ❖ Упражнение 3.3

[1] Указанный выше исходный код реализован в рамках среды Colab. При работе в блокноте Jupyter локально в самом начале следует добавить две следующие строки кода:

```
import plotly.io as pio
  pio.renderers.default = 'iframe'
```

– *Прим. перев.*

Глава 4

Применения векторов

Работая с предыдущими двумя главами, вы, возможно, ощущали, что часть материала была эзотерической и абстрактной. Вероятно, вы чувствовали, что задача изучения линейной алгебры не окупится пониманием приложений, реально существующих в области науки о данных и машинного обучения.

Надеюсь, что данная глава эти сомнения у вас развеет. В этой главе вы узнаете, как векторы и векторные операции используются в методах анализа в рамках науки о данных. И вы сможете расширить эти знания, выполнив упражнения.

Корреляция и косинусное сходство

Корреляция представляет собой один из наиболее фундаментальных и важных методов анализа в статистике и машинном обучении. *Коэффициент корреляции* – это одно число, которое количественно описывает линейную взаимосвязь между двумя переменными. Коэффициенты корреляции варьируются от –1 до +1, причем –1 указывает на идеальную отрицательную взаимосвязь, +1 – на идеальную положительную взаимосвязь, а 0 указывает на отсутствие линейной взаимосвязи. На рис. 4.1 показано несколько примеров пар переменных и их коэффициентов корреляции.

В главе 2 я упоминал, что точечное произведение участвует в коэффициенте корреляции и что величина точечного произведения связана с величиной числовых значений в данных (вспомните обсуждение темы использования граммов вместо фунтов для измерения веса). Следовательно, коэффициент корреляции требует некоторой нормализации, чтобы он находился в ожидаемом диапазоне от –1 до +1. Эти две нормализации таковы:

Центрировать каждую переменную по среднему значению
: Центрирование по среднему значению означает вычитание среднего значения из каждого значения данных.

Разделить точечное произведение на произведение векторных норм
: Это делящая нормализация, которая отменяет единицы измерения и шкалирует максимально возможную величину корреляции в |1|.

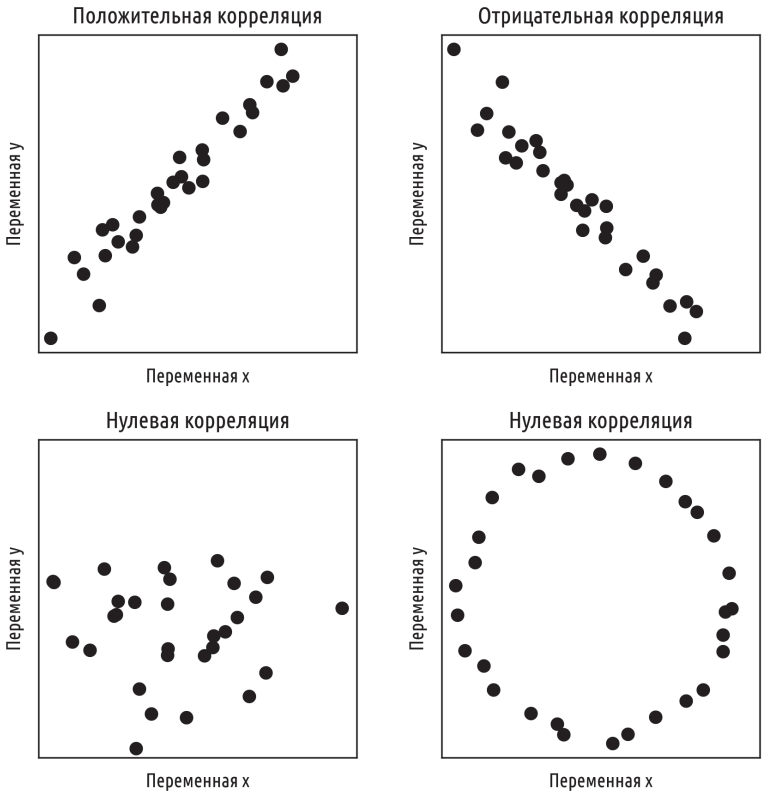


Рис. 4.1 ❖ Примеры данных, демонстрирующих положительную корреляцию, отрицательную корреляцию и нулевую корреляцию. Нижняя правая панель иллюстрирует, что корреляция является линейной мерой; нелинейные взаимосвязи между переменными могут существовать, даже если их корреляция равна нулю

Уравнение 4.1 показывает полную формулу коэффициента корреляции Пирсона.

Уравнение 4.1. Формула коэффициента корреляции Пирсона

$$\rho = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}.$$

Возможно, тут не так очевидно, что корреляция представляет собой не что иное, как три точечных произведения. Уравнение 4.2 показывает эту же формулу, переписанную с использованием обозначения линейно-алгебраического точечного произведения. В этом уравнении $\bar{\mathbf{x}}$ является среднецентрированной версией $\mathbf{x}$ (то есть переменной $\mathbf{x}$ с примененной нормализацией № 1).

Уравнение 4.2. Корреляция Пирсона, выраженная на языке линейной алгебры

$$\rho = \frac{\tilde{\mathbf{x}}^{\mathrm{T}}\tilde{\mathbf{y}}}{||\tilde{\mathbf{x}}||\,||\tilde{\mathbf{y}}||}.$$

Так что вот так: знаменитый и широко используемый коэффициент корреляции Пирсона – это просто точечное произведение между двумя переменными, нормированное векторными модулями переменных. (Кстати, по этой формуле также можно заметить, что если переменные единично нормированы таким образом, что $||\mathbf{x}|| = ||\mathbf{y}|| = 1$, то их корреляция равна их точечному произведению.

(Вспомним из упражнения 2.6, что $||\mathbf{x}|| = \sqrt{\mathbf{x}^{\mathrm{T}}\mathbf{x}}$.)

Корреляция – это не единственный способ оценивать сходство между двумя переменными. Еще один метод называется *косинусным сходством*. Формула косинусного сходства представляет собой просто геометрическую формулу точечного произведения (уравнение 2.11), решаемую для косинусного члена:

$$\cos(\theta_{x,y}) = \frac{\alpha}{||\mathbf{x}||\,||\mathbf{y}||},$$

где α – это точечное произведение между $\mathbf{x}$ и $\mathbf{y}$.

Может показаться, что корреляция и косинусное сходство – это совершенно одна и та же формула. Однако следует запомнить, что уравнение 4.1 является полной формулой, тогда как уравнение 4.2 является упрощением в рамках допущения, что переменные уже были центрированы по среднему значению. Отсюда косинусное сходство не содержит первый фактор нормализации.

Корреляция по сравнению с косинусным сходством

Что означает «связанность» двух переменных между собой? Корреляция Пирсона и косинусное сходство могут давать разные результаты для одних и тех же данных, поскольку они исходят из разных допущений. По мнению Пирсона, переменные [0, 1, 2, 3] и [100, 101, 102, 103] идеально коррелируют ($\rho = 1$), поскольку изменения в одной переменной точно отражаются в другой переменной, и не имеет значения, что одна переменная имеет более крупные числовые значения. Однако косинусное сходство между этими переменными равно .808 – они не находятся на одной числовой шкале и, следовательно, связаны не идеально. Ни одна мера не является ни неправильной, ни наилучшей, чем другая; просто разные статистические методы принимают разные допущения о данных, и эти допущения влияют на результаты – и на правильную интерпретацию. У вас будет возможность обследовать этот момент в упражнении 4.2.

Из данного раздела можно понять, почему корреляция Пирсона и косинусное сходство отражают линейную зависимость между двумя переменными: они основаны на точечном произведении, а точечное произведение является линейной операцией.

С этим разделом связаны четыре упражнения по программированию, которые приведены в конце главы. Вы можете выбрать, когда их решать: перед чтением следующего раздела либо продолжать чтение остальной части главы, а затем проработать упражнения. (Моя личная рекомендация относится к первому, но вы являетесь хозяином своей судьбы в линейной алгебре!)

Фильтрация временных рядов и обнаружение признаков

Точечное произведение также используется в фильтрации временных рядов. Фильтрация – это, по сути, метод обнаружения признаков, при котором шаблон – именуемый на языке фильтрации вычислительным *ядром* – сопоставляется с частями сигнала временного ряда, и результатом фильтрации является еще один временной ряд, который показывает, насколько характеристики сигнала соответствуют характеристикам ядра. Ядра тщательно конструируются, чтобы оптимизировать те или иные критерии, такие как плавные колебания, острые пики, определенные контуры волновых форм и т. д.

Механизм фильтрации заключается в вычислении точечного произведения между ядром и сигналом временного ряда. Но фильтрация обычно требует *локального* обнаружения признаков, и ядро обычно намного короче, чем весь временной ряд. Поэтому мы вычисляем точечное произведение между ядром и коротким фрагментом данных той же длины, что и ядро. Такая процедура создает одну временную точку в отфильтрованном сигнале (рис. 4.2),

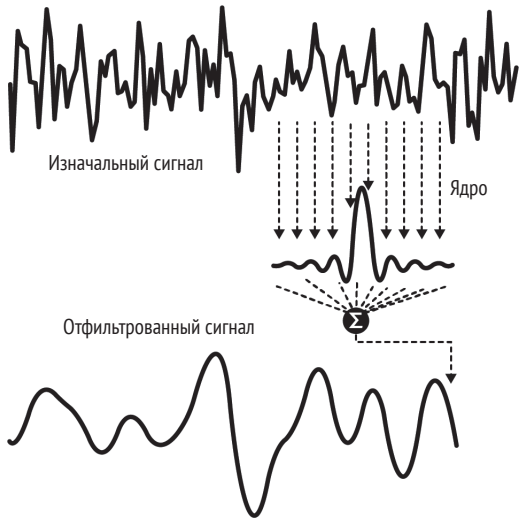


Рис. 4.2 ❖ Иллюстрация фильтрации временного ряда

а затем ядро перемещается на один временной шаг вправо, чтобы вычислить точечное произведение с другим (накладывающимся) сегментом сигнала. Формально эта процедура называется *сверткой* и предусматривает несколько дополнительных шагов, которые я опускаю, чтобы сосредоточиться на применении точечного произведения в обработке сигналов.

Темпоральная фильтрация является важной темой в науке и технике. И действительно, без темпоральной фильтрации не было бы музыки, радио, телекоммуникаций, спутников и т. д. И все же математическим сердцем, которое заставляет вашу музыку пульсировать, является точечное произведение векторов.

В упражнениях в конце главы вы узнаете, как точечные произведения используются для обнаружения признаков (резких изменений) и сглаживания данных временных рядов.

Кластеризация методом *k*-средних

Кластеризация методом *k*-средних – это неконтролируемый метод классифицирования многопеременных данных на относительно малое число групп, или категорий, основываясь на минимизации расстояния до центра группы.

Кластеризация методом *k*-средних является важным методом анализа в машинном обучении, и существуют самые изощренные варианты кластеризации методом -средних. Здесь мы реализуем простую версию алгоритма -средних с целью увидеть, как понятия о векторах (в частности, векторах, векторных нормах и транслировании) используются в алгоритме *k*-средних.

Вот краткое описание алгоритма, который мы напишем.

1. Инициализировать *k* центроидов как случайные точки в пространстве данных. Каждый центроид является классом или категорией, и на следующих шагах каждое наблюдение данных будет относиться к каждому классу. (*Центроид* – это центр, обобщенный на любое число измерений[1].)
2. Вычислить евклидово расстояние между каждым наблюдением данных и каждым центроидом[2].
3. Отнести каждое наблюдение данных к группе с ближайшим центроидом.
4. Обновить каждый центроид как среднее значение всех наблюдений данных, назначенных этому центроиду.
5. Повторять шаги 2–4 до тех пор, пока не будет удовлетворен критерий схождения либо в течение *N* итераций.

Если вам удобно программировать на Python и вы хотели бы реализовать этот алгоритм, то рекомендую это сделать сейчас, прежде чем продолжать.

---

[1] Центроид также иногда именуется центром тяжести. – *Прим. перев.*

[2] Напоминание: евклидово расстояние – это квадратный корень из суммы квадратов расстояний от точки наблюдения данных до центроида.

Далее мы проработаем математику и исходный код по каждому указанному шагу, уделяя особое внимание использованию векторов и транслированию в NumPy. Мы также протестируем указанный алгоритм, используя случайно сгенерированные двумерные данные, чтобы подтвердить правильность исходного кода.

Давайте начнем с шага 1: инициализировать центроиды k случайных кластеров. k – это параметр кластеризации методом k-средних; в реальных данных определить оптимальный параметр k довольно трудно, но здесь мы зададим $k = 3$. Инициализировать центроиды случайных кластеров можно несколькими способами; в целях упрощения задачи в качестве центроидов я случайно возьму k выборок данных. Данные содержатся в переменной `data` (эта переменная имеет размер 150×2, что соответствует 150 наблюдениям по 2 признака) и визуализируются на верхней левой панели рис. 4.3 (онлайновый исходный код показывает, как эти данные генерируются):

```
k=3
ridx = np.random.choice(range(len(data)),k,replace=False)
centroids = data[ridx,:] # матрица данных содержит образцы по признакам
```

Теперь перейдем к шагу 2: вычислить расстояние между каждым наблюдением данных и центроидом каждого кластера. Здесь мы используем линейно-алгебраические концепции, которые вы усвоили в предыдущих главах. Для одного наблюдения данных и центроида евклидово расстояние вычисляется следующим образом:

$$\delta_{i,j} = \sqrt{\left(d_i^x - c_j^x\right)^2 + \left(d_i^y - c_j^y\right)^2},$$

где $\delta_{i,j}$ указывает на расстояние от наблюдения данных i до центроида j, d_i^x – это признак x-го наблюдения данных, а c_j^x – координата по оси x для центроида j.

Возможно, вы думаете, что этот шаг должен быть реализован с использованием двойного цикла `for`: один цикл по k центроидам и второй цикл по N наблюдениям данных (вероятно, вы даже подумали о третьем цикле `for` по признакам данных). Однако тут можно применить векторы и транслирование, сделав эту операцию компактной и эффективной. Это пример того, как линейная алгебра нередко выглядит иначе в уравнениях по сравнению с исходным кодом:

```
dists = np.zeros((data.shape[0], k))
for ci in range(k):
    dists[:,ci] = np.sum((data-centroids[ci,:])**2,
                         axis=1)
```

Давайте подумаем о размерах этих переменных: `data` имеет размер 150×2 (наблюдения по числу признаков), а размер `centroids[ci,:]` равен 1×2 (кластер `ci` по числу признаков). В формальном плане вычесть эти два вектора невозможно. Однако в Python реализована операция транслирования, которая будет работать путем повтора центроидов кластеров 150 раз и, следова-

тельно, вычитая центроид из каждого наблюдения данных. Операция возведения в степень ** применяется поэлементно, и входной аргумент axis=1 говорит Python о том, что нужно суммировать по столбцам (отдельно по каждой строке). Таким образом, результатом функции np.sum() будет массив размером 150×1, в котором кодируется евклидово расстояние каждой точки до центроида ci.

Найдите минутку, чтобы сравнить этот исходный код с формулой расстояния. Действительно ли они одинаковые? На самом деле это не так: квадратный корень из евклидова расстояния в коде отсутствует. И что, значит, исходный код неправильный? Сделайте небольшую паузу и подумайте об этом; я дам подробный ответ чуть позже.

Шаг 3 состоит в отнесении каждого наблюдения данных к группе с минимальным расстоянием. Этот шаг в Python довольно компактен и реализуется одной функцией:

```
groupidx = np.argmin(dists, axis=1)
```

Обратите внимание на разницу между функцией np.min, которая возвращает минимальное значение, и функцией np.argmin, которая возвращает индекс, при котором происходит минимум.

Теперь можно вернуться к несоответствию между формулой расстояния и ее реализацией в исходном коде. В данном алгоритме *k*-средних используется расстояние, чтобы относить каждую точку данных к ближайшему к ней центроиду. Расстояние и квадрат расстояния монотонно связаны, поэтому обе метрики дают один и тот же ответ. Добавление операции квадратного корня увеличивает сложность исходного кода и время вычислений без влияния на результаты, поэтому ее можно просто опустить.

Шаг 4 заключается в перевычислении центроидов как среднего значения всех точек данных внутри класса. Здесь можно прокрутить *k* кластеров в цикле и использовать индексацию Python, чтобы найти все точки данных, отнесенные к каждому кластеру:

```
for ki in range(k):
    centroids[ki,:] = [ np.mean(data[groupidx==ki, 0]),
                        np.mean(data[groupidx==ki, 1]) ]
```

Наконец, шаг 5 заключается в размещении приведенных выше шагов в цикле, который повторяется до тех пор, пока не будет получено хорошее решение. В алгоритмах *k*-средних производственного уровня итерации продолжаются до тех пор, пока не будет достигнут критерий останова, например когда центроиды кластеров больше не перемещаются. Для простоты здесь мы выполним три итерации (данное число выбрано произвольно, чтобы сделать график визуально сбалансированным).

Четыре панели на рис. 4.3 показывают центроиды изначально случайных кластеров (итерация 0) и их обновленные местоположения после каждой из трех итераций.

Если вы изучаете алгоритмы кластеризации, то вы узнаете изощренные методы инициализации центроидов и критерии останова, а также количественные методы отбора подходящего параметра *k*. Тем не менее все методы

k-средних, по существу, являются расширениями вышеупомянутого алгоритма, и в основе их реализаций лежит линейная алгебра.

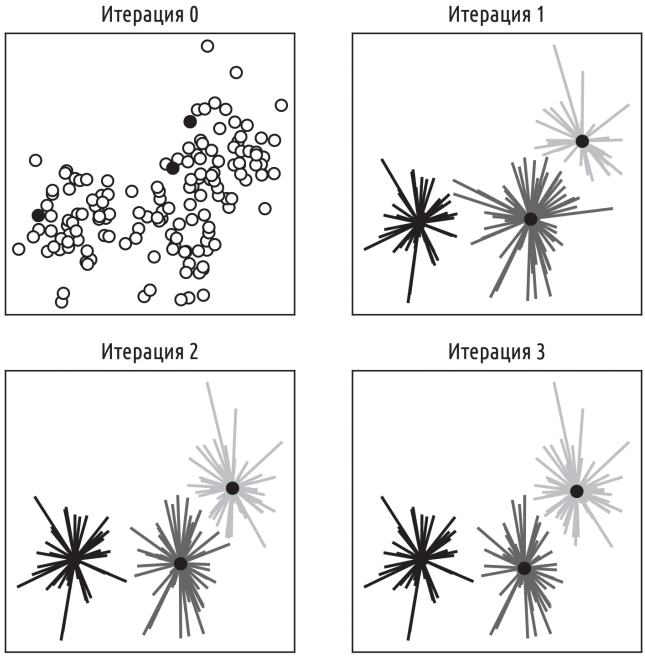


Рис. 4.3 ❖ Результат работы метода *k*-средних

Упражнения по программированию

Упражнения по корреляции

Упражнение 4.1

Напишите функцию Python, которая на входе принимает два вектора и на выходе выдает два числа: коэффициент корреляции Пирсона и значение косинусного сходства. Напишите исходный код, который следует формулам, представленным в данной главе; не используйте вызовы встроенной в NumPy функции `np.corrcoef` и встроенной в SciPy функции `spatial.distance.cosine`. Убедитесь, что два значения на выходе идентичны, когда переменные уже центрированы по среднему значению, и различны, когда переменные не центрированы по среднему значению.

Упражнение 4.2

Давайте продолжим обследовать разницу между корреляцией и косинусным сходством. Создайте переменную, содержащую целые числа от 0 до 3, и вторую переменную, равную первой переменной плюс некоторое смеще-

ние. Затем создайте симуляцию, в которой вы систематически варьируете это смещение между –50 и +50 (то есть на первой итерации симуляции вторая переменная будет равна [–50, –49, –48, –47]). В цикле `for` вычислите корреляцию и косинусное сходство между двумя переменными и сохраните эти результаты. Затем постройте линейный график, показывающий, как среднее смещение влияет на корреляцию и косинусное сходство. Вы должны суметь воспроизвести рис. 4.4.

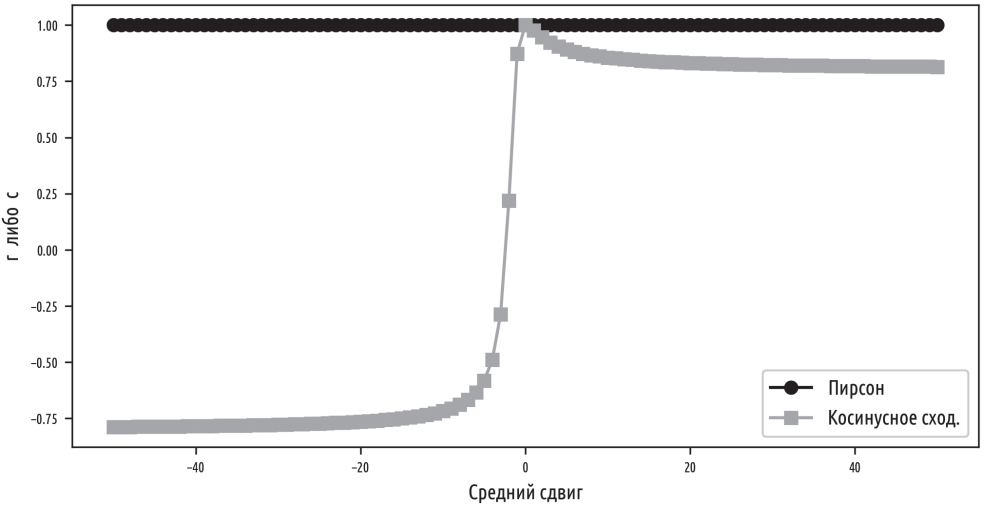


Рис. 4.4 ❖ Результаты упражнения 4.2

Упражнение 4.3

В Python есть несколько функций, которые вычисляют коэффициент корреляции Пирсона. Одна из них называется `pearsonr` и находится в модуле `stats` библиотеки SciPy. Откройте исходный код этого файла (подсказка: `??functionname`) и убедитесь, что вы понимаете, как реализация на Python соотносится с формулами, представленными в данной главе.

Упражнение 4.4

Зачем вообще нужно программировать свои конкретно-прикладные функции, если они уже существуют в Python? Отчасти причина состоит в том, что написание своих конкретно-прикладных функций имеет огромную образовательную ценность, потому что вы видите, что (в данном случае) корреляция – это простое вычисление, а не какой-то невероятно сложный черно-ящичный алгоритм, который под силу понять только кандидату в области вычислительной науки. Но еще одна причина заключается в том, что встроенные функции иногда работают медленнее из-за громадного числа проверок входных данных, работы с дополнительными входными параметрами, преобразованиями типов данных и т. п. Все это повышает удобство использования, но за счет времени вычислений.

В данном упражнении ваша цель – посмотреть, будет ли ваша сокращенная функция корреляции работать быстрее, чем функция NumPy `corrcoef`. Модифицируйте функцию из упражнения 4.2, чтобы вычислить только коэффициент корреляции. Затем, в цикле `for` по 1000 итерациям, сгенерируйте две переменные из 500 случайных чисел и вычислите корреляцию между ними. Засеките время исполнения цикла `for`. Затем повторите, но используя функцию `np.corrcoef`. В моих тестах конкретно-прикладная функция была примерно на 33 % быстрее, чем `np.corrcoef`. В этих игрушечных примерах различия измеряются в миллисекундах, но если вы выполняете миллиарды корреляций с крупными наборами данных, то эти миллисекунды складываются, давая действительно большой прирост в производительности! (Обратите внимание, что написание своих конкретно-прикладных функций без проверок входных данных сопряжено с риском ошибок на входе, которые были бы обнаружены функцией `np.corrcoef`.) (Также обратите внимание, что преимущество в скорости уменьшается с более крупными векторами. Попробуйте сами!)

Упражнения по фильтрации и обнаружению признаков

Упражнение 4.5

Давайте построим детектор резких изменений[1]. Ядро детектора резких изменений будет очень простым: [–1 +1]. Точечное произведение этого ядра на фрагмент сигнала временного ряда с постоянным значением (например, [10 10]) равно 0. Но это точечное произведение будет крупным, когда сигнал имеет резкое изменение (например, [1 10] даст точечное произведение, равное 9). Мы будем работать с сигналом, который будет представлен функцией плато. Графики A и B на рис. 4.5 показывают ядро и сигнал. На первом шаге в данном упражнении пишется исходный код, который создает эти два временных ряда.

Далее напишите цикл `for` для временных точек в сигнале. В каждый момент времени вычисляйте точечное произведение между ядром и сегментом данных временного ряда, который имеет ту же длину, что и ядро. Вы должны создать график, который выглядит как график C на рис. 4.5. (Сосредоточьтесь больше на результате, чем на эстетике.) Обратите внимание, что наш детектор резких изменений вернул 0, когда сигнал был ровным, +1, когда сигнал подскочил вверх, и –1, когда сигнал прыгнул вниз.

Смело продолжайте обследовать этот исходный код. Например, изменится ли что-нибудь, если дополнить ядро нулями ([0 –1 1 0])? А что, если перевернуть ядро так, чтобы оно было [1 –1]? Как насчет того, если ядро будет асимметричным ([–1 2])?

---

[1] В обработке временных рядов обнаружение резких изменений также называется обнаружением скачков, всплесков или отклонений от среднего уровня временного ряда или сигнала. – *Прим. перев.*

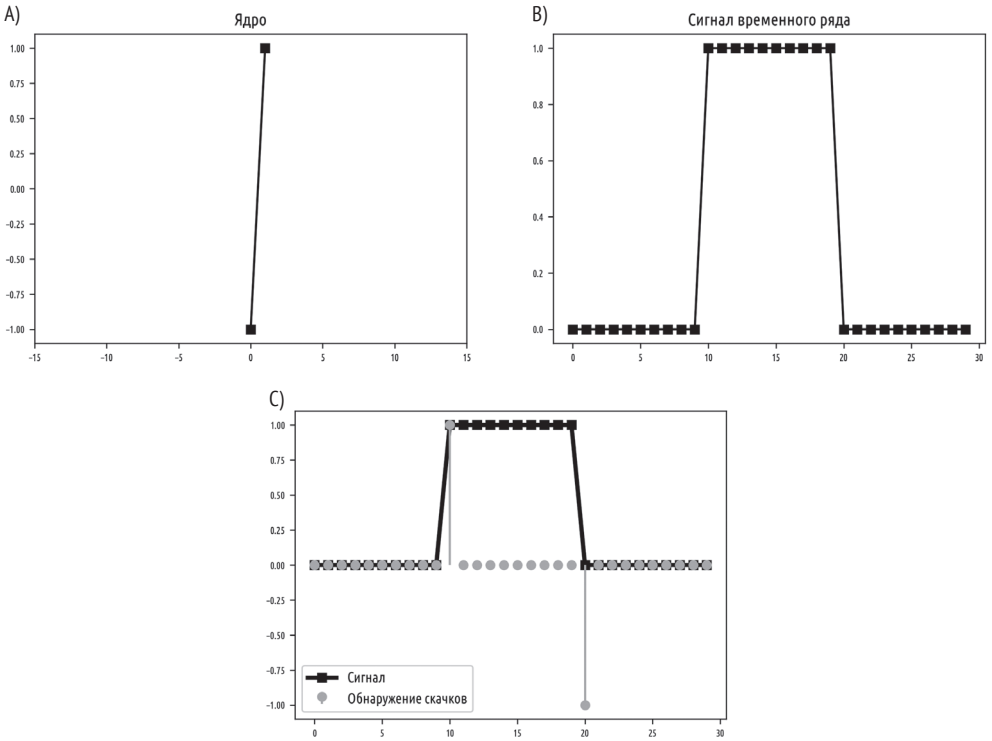


Рис. 4.5 ❖ Результаты упражнения 4.5

Упражнение 4.6

Теперь мы повторим ту же процедуру, но с другим сигналом и ядром. Цель будет состоять в том, чтобы сгладить неровный временной ряд. Временной ряд будет состоять из 100 случайных чисел, сгенерированных из гауссова распределения (также именуемого нормальным распределением). Ядро будет представлять собой функцию в форме колокола, которая аппроксимирует гауссову функцию, определенную как числа [0, .1, .3, .8, 1, .8, .3, .1, 0], но шкалированную так, чтобы сумма по ядру составляла 1. Ваше ядро должно соответствовать графику A на рис. 4.6, хотя из-за случайных чисел ваш сигнал не будет выглядеть точно так же, как график B.

Скопируйте и адаптируйте исходный код из предыдущего упражнения под вычисление скользящего временного ряда точечных произведений – сигнала, отфильтрованного гауссовым ядром. Предупреждение: будьте внимательны к индексации в цикле `for`. График C на рис. 4.6 показывает примерный результат. Хорошо видно, что отфильтрованный сигнал является сглаженной версией изначального сигнала. Такая процедура также называется низкочастотной фильтрацией.

Упражнение 4.7

Замените 1 в центре ядра на –1 и усредните центр ядра. Затем выполните исходный код фильтрации и построения графика повторно. Каким будет результат? Он действительно подчеркивает резкие признаки! По сути дела,

данное ядро теперь является высокочастотным фильтром, а значит, оно приглушает плавные (низкочастотные) признаки и выделяет быстро меняющиеся (высокочастотные) признаки.

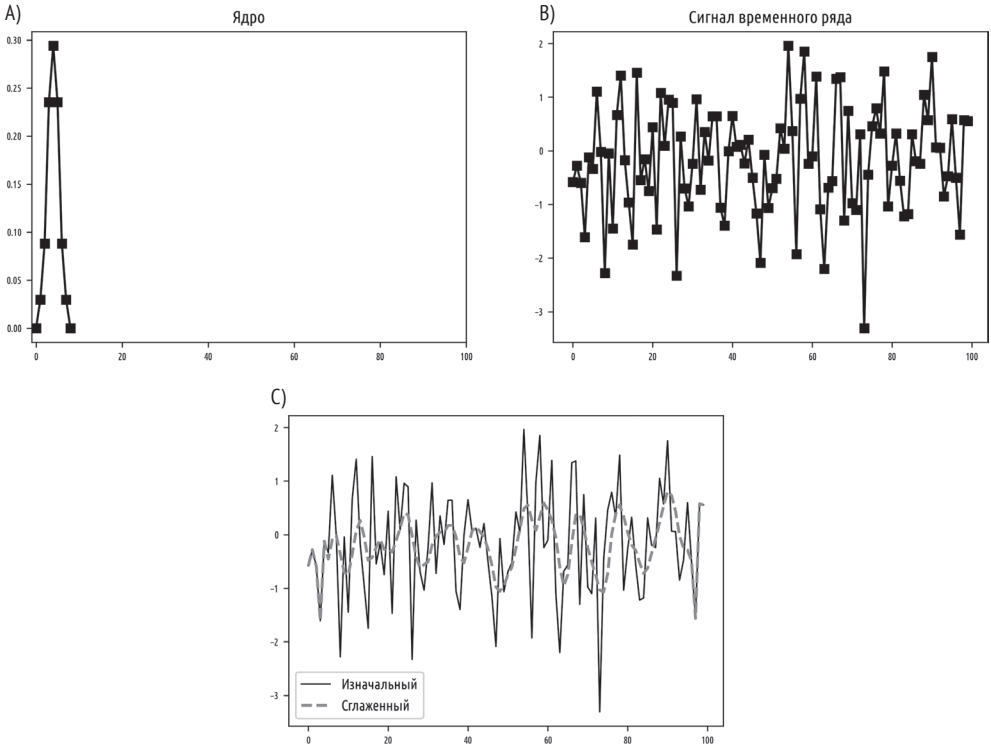


Рис. 4.6 ❖ Результаты упражнения 4.6

Упражнения по алгоритму *k*-средних

Упражнение 4.8

Оптимальное значение k можно определить путем повторения кластеризации несколько раз (всякий раз с использованием случайно инициализированных центроидов кластеров) и оценивания того, является ли итоговая кластеризация одинаковой либо другой. Не генерируя новых данных, повторите исходный код алгоритма -средних несколько раз, используя $k = 3$, чтобы увидеть похожесть/непохожесть результирующих кластеров (это качественная оценка, основанная на визуальном осмотре). Выглядят ли итоговые отнесения к кластерам в целом похожими, даже если центроиды выбраны случайным образом?

Упражнение 4.9

Повторите кластеризации несколько раз, используя $k = 2$ и $k = 4$. Что вы думаете об этих результатах?

Глава 5

Матрицы. Часть 1

Матрица – это вектор, перенесенный на следующий уровень. Матрицы как математические объекты очень разноплановы. В них могут храниться наборы уравнений, геометрические преобразования, положения частиц во времени, финансовые отчеты и громадное число других вещей. В науке о данных матрицы иногда называют таблицами данных, в которых строки соответствуют наблюдениям (например, клиентам), а столбцы – признакам (например, покупкам).

Данная и следующие две главы выведут ваши знания о линейной алгебре на новый уровень. Выпейте чашечку кофе и наденьте свою мыслительную тюбетейку. К концу главы ваш мозг станет больше.

Создание и визуализация матриц в NumPy

В зависимости от контекста матрицы концептуализируются в уме как множество векторов-столбцов, расположенных бок о бок (например, как таблицы в формате «наблюдения по признакам»), как множество уложенных стопкой векторов-строк (например, в виде мультисенсорных данных, в которых каждая строка – это временной ряд из другого канала) или как упорядоченный набор отдельных матричных элементов (например, в виде изображения, в каждом матричном элементе которого закодировано значение интенсивности пиксела).

Визуализация, индексация и нарезка матриц

Малые матрицы можно легко распечатывать полностью, как в следующих ниже примерах:

$$\begin{bmatrix} 1 & 2 \\ \pi & 4 \\ 6 & 7 \end{bmatrix}, \quad \begin{bmatrix} -6 & 1/3 \\ e^{4.3} & -1.4 \\ 6/5 & 0 \end{bmatrix}.$$

Но это не масштабируется, и матрицы, с которыми вы работаете на практике, могут быть большими, возможно, содержащими миллиарды элементов. Поэтому более крупные матрицы можно визуализировать в виде изображений. Числовое значение каждого элемента матричной карты соотносится с цветом изображения. В большинстве случаев такие карты[1] псевдоцветны, поскольку соотнесенность числового значения с цветом является произвольной. На рис. 5.1 показано несколько примеров матриц, визуализированных в виде изображений с использованием библиотеки Python `matplotlib`.



Рис. 5.1 ❖ Три матрицы, визуализированные в виде изображений

Матрицы обозначаются заглавными буквами жирным шрифтом, например матрица **A** или **M**. Размер матрицы указывается традиционным образом в формате (строка, столбец). Например, следующая ниже матрица имеет размер 3×5, поскольку в ней три строки и пять столбцов:

$$\begin{bmatrix} 1 & 3 & 5 & 7 & 9 \\ 0 & 2 & 4 & 6 & 8 \\ 1 & 4 & 7 & 8 & 9 \end{bmatrix}.$$

На конкретные элементы матрицы можно ссылаться, обращаясь по индексу строки и столбца: элемент в 3-й строке и 4-м столбце матрицы **A** обозначается как $a_{3,4}$ (в предыдущем примере матрицы $a_{3,4} = 8$). *Важное напоминание*: в математике используется индексация с отсчетом от единицы, в то время как в Python используется индексация с отсчетом от нуля. Таким образом, в Python элемент $a_{3,4}$ индексируется как `A[2,3]`.

Подмножество строк или столбцов матрицы извлекается с помощью операции нарезки. Если вы в Python новичок, то обратитесь к главе 16, чтобы ознакомиться с нарезкой списков и массивов NumPy. Для того чтобы извлечь срез из матрицы, надо указать начальные и конечные строки и столбцы, а также шаг нарезки, равный 1. Онлайновый исходный код проведет вас по всей процедуре, а следующий ниже исходный код показывает пример извлечения подматрицы из строк 2–4 и столбцов 1–5 более крупной матрицы:

```
A = np.arange(60).reshape(6, 10)
sub = A[1:4:1,0:5:1]
```

[1] Син. растровые изображения. – *Прим. перев.*

Ниже приведены полная матрица и подматрица:

```
Изначальная матрица:
[[ 0 1 2 3 4 5 6 7 8 9]
 [10 11 12 13 14 15 16 17 18 19]
 [20 21 22 23 24 25 26 27 28 29]
 [30 31 32 33 34 35 36 37 38 39]
 [40 41 42 43 44 45 46 47 48 49]
 [50 51 52 53 54 55 56 57 58 59]]

Подматрица:
[[10 11 12 13 14]
 [20 21 22 23 24]
 [30 31 32 33 34]]
```

Специальные матрицы

Существует бесконечное число матриц, потому что существует бесконечное число способов организации чисел в матрицу. Но матрицы можно описывать с использованием относительно малого числа характеристик, в результате создавая «семейства», или категории матриц. Указанные категории важно знать, потому что они появляются в определенных операциях либо обладают определенными полезными свойствами.

Некоторые категории матриц используются так часто, что для их создания есть специальные функции NumPy. Ниже приведен список нескольких распространенных специальных матриц и исходный код Python для их создания[1]; вы их увидите на рис. 5.2.

Матрица случайных чисел

Это матрица, которая содержит числа, берущиеся случайно из некоторого распределения, обычно гауссова (также именуемого нормальным). Матрицы случайных чисел отлично подходят для обследования линейной алгебры в исходном коде, потому что они быстро и легко создаются с любым размером и рангом (понятие ранга матрицы вы узнаете в главе 16).

В NumPy имеется несколько способов создания случайных матриц в зависимости от того, из какого распределения вы хотите извлекать числа. В этой книге мы будем использовать числа, в основном распределенные по Гауссу:

```
Mrows = 4 # очертание 0
Ncols = 6 # очертание 1
A = np.random.randn(Mrows, Ncols)
```

Квадратная и неквадратная

Квадратная матрица имеет такое же число строк, что и столбцов; другими словами, матрица имеет размер $\mathbb{R}^{N\times N}$. Неквадратная матрица, также

[1] Существуют и другие специальные матрицы, о которых вы узнаете в книге позже, но этого списка для начала будет достаточно.

иногда именуемая прямоугольной матрицей, имеет отличающееся число строк и столбцов. Квадратные и прямоугольные матрицы можно создавать из случайных чисел, настроив параметры очертания в приведенном выше исходном коде.

Прямоугольные матрицы называются *высокими*, если в них больше строк, чем столбцов, и *широкими*, если в них больше столбцов, чем строк.

Диагональная

Диагональ матрицы – это элементы, начинающиеся в верхнем левом углу и спускающиеся в нижний правый. *Диагональная* матрица имеет нули во всех внедиагональных элементах; диагональные элементы тоже могут содержать нули, но они являются единственными элементами, которые могут содержать ненулевые значения.

Функция NumPy `np.diag()` имеет два вида поведения в зависимости от входных данных: при вводе матрицы функция `np.diag` возвращает диагональные элементы в виде вектора; при вводе вектора функция `np.diag` возвращает матрицу с этими векторными элементами на диагонали. (Примечание: извлечение диагональных элементов матрицы не называется «диагонализацией матрицы»; это отдельная операция, представленная в главе 13.)

Треугольная

Треугольная матрица содержит одни нули выше либо ниже главной диагонали. Матрица называется *верхней треугольной*, если ненулевые элементы находятся выше диагонали, и *нижней треугольной*, если ненулевые элементы находятся ниже диагонали.

В NumPy имеются специальные функции для извлечения верхнего (`np.triu()`) либо нижнего (`np.tril()`) треугольника матрицы.

Единичная

Единичная матрица[1] является одной из наиболее важных специальных матриц. Она эквивалентна числу 1 в том смысле, что любая матрица или вектор, умноженные на единичную матрицу, будут той же самой матрицей или вектором. Единичная матрица – это квадратная диагональная матрица, все диагональные элементы которой имеют значение 1. Она обозначается буквой **I**. Иногда вместе с буквой можно увидеть индекс, указывающий на ее размер (например, $\mathbf{I}_5$ – это единичная матрица размером 5×5); если индекса нет, то ее размер можно определить из контекста (например, чтобы уравнение было совместимым).

В Python единичная матрица создается функцией `np.eye()`.

Нулей

Матрица нулей сравнима с вектором нулей: это матрица, состоящая из одних нулей. Как и вектор нулей, она обозначается отмеченным жирным шрифтом нулем: **0**. Возможно, то, что один и тот же символ обозначает как вектор, так и матрицу, будет немного сбивать с толку, но в математи-

[1] Англ. *identity matrix*; син. матрица тождественного преобразования. – *Прим. перев.*

ческой и естественно-научной нотации такого рода перегрузка довольно распространена.

Матрица нулей создается функцией `np.zeros()`.

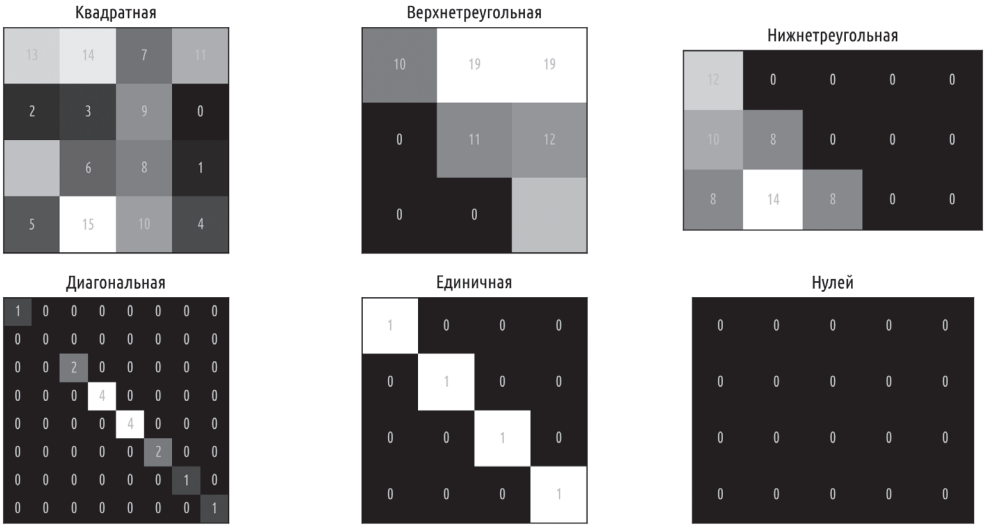


Рис. 5.2 ❖ Несколько специальных матриц. Числа и значения в оттенках серого указывают значение каждого элемента матрицы

Матричная математика: сложение, умножение на скаляр, адамарово умножение

Математические операции на матрицах делятся на две категории: понятные и непонятные на интуитивном уровне. В общем и целом интуитивно понятные операции можно выражать в виде пошаговых процедур, тогда как объяснение интуитивно непонятных операций требует больше времени, а их понимание – немного практики. Давайте начнем с интуитивно понятных операций.

Сложение и вычитание

Две матрицы складываются путем сложения соответствующих элементов матриц. Вот пример:

$$\begin{bmatrix} 2 & 3 & 4 \\ 1 & 2 & 4 \end{bmatrix} + \begin{bmatrix} 0 & 3 & 1 \\ -1 & -4 & 2 \end{bmatrix} = \begin{bmatrix} (2+0) & (3+3) & (4+1) \\ (1-1) & (2-4) & (4+2) \end{bmatrix} = \begin{bmatrix} 2 & 6 & 5 \\ 0 & -2 & 6 \end{bmatrix}.$$

Как можно было догадаться из примера, сложение матриц определяется только между двумя матрицами одинакового размера.

«Сдвиг» матрицы

Как и в случае с векторами, прибавить скаляр к матрице, как в $\lambda + \mathbf{A}$, невозможно формально. Python же такую операцию допускает (например, `3+np.eye(2)`), которая предусматривает транслирование скаляра в каждый элемент матрицы. Это удобное вычисление, но формально оно не является линейно-алгебраической операцией.

Однако существует линейно-алгебраический способ прибавления скаляра к квадратной матрице, и он называется *сдвигом* матрицы и работает путем прибавления постоянного значения к диагонали, то есть реализуется посредством прибавления умноженной на скаляр единичной матрицы:

$$\mathbf{A} + \lambda\mathbf{I}.$$

Вот численный пример:

$$\begin{bmatrix} 4 & 5 & 1 \\ 0 & 1 & 11 \\ 4 & 9 & 7 \end{bmatrix} + 6\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 10 & 5 & 1 \\ 0 & 7 & 11 \\ 4 & 9 & 13 \end{bmatrix}.$$

На языке Python сдиг выполняется просто:

```
A = np.array([ [4,5,1], [0,1,11], [4,9,7] ])
S = 6
A + s # НЕ сдвигается!
A + s*np.eye(len(A)) # сдвигается
```

Обратите внимание, что меняются только диагональные элементы; остальная часть матрицы сдвигом не искажается. На практике сдвигают на относительно малое число, чтобы в матрице сохранить как можно больше информации, при этом извлекая выгоду из эффектов сдвига, включая повышение численной стабильности матрицы (позже в книге вы узнаете, почему происходит нестабильность).

На сколько именно нужно сдвигать, зависит от текущих исследований во многих областях машинного обучения, статистики, глубокого обучения, разработки систем управления и т. д. Например, сдвиг на $\lambda = 6$ – это мало или много? Как насчет $\lambda = .001$? Очевидно, что эти числа являются «большими» либо «малыми» по отношению к числовым значениям в матрице. Поэтому на практике лямбда λ обычно задается как некоторая доля определенного матрицей элемента, такого как норма или среднее значение собственных чисел. Вы сможете обследовать эту тему в последующих главах.

«Сдвиг» матрицы имеет два первичных (чрезвычайно важных!) применения: это механизм отыскания собственных чисел матрицы и механизм регуляризации матриц при подгонке моделей к данным.

Умножение на скаляр и адамарово умножение

Указанные два типа умножения работают для матриц так же, как и для векторов, то есть поэлементно.

Умножение матрицы на скаляр означает умножение каждого элемента матрицы на один и тот же скаляр. Вот пример использования матрицы, содержащей буквы вместо чисел:

$$\gamma\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} \gamma a & \gamma b \\ \gamma c & \gamma d \end{bmatrix}.$$

Адамарово умножение аналогичным образом предусматривает поэлементное умножение двух матриц (отсюда и альтернативная терминология *поэлементное умножение*). Вот пример:

$$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \odot \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 2a & 3b \\ 4c & 5d \end{bmatrix}.$$

В NumPy адамарово умножение можно реализовать с помощью функции `np.multiply()`. Но нередко синтаксически его проще реализовать, используя звездочку между двумя матрицами: `A*B`. Возможно, это вызовет некоторую путаницу, поскольку стандартное умножение матриц (следующий раздел) обозначается символом `@`. И здесь есть тонкое, но важное различие! (Данный факт будет особенно сбивать с толку тех читателей, кто переходит на Python из MATLAB, в котором умножение матриц обозначается символом `*`.)

```
A = np.random.randn(3, 4)
B = np.random.randn(3, 4)

A*B # адамарово умножение
np.multiply(A, B) # тоже адамарово
A@B # НЕ адамарово!
```

Адамарово умножение действительно имеет несколько применений в линейной алгебре, например при вычислении обратной матрицы. Однако чаще всего оно применяется в приложениях как удобный способ хранения большого числа отдельных умножений. Это похоже на то, как нередко применяется адамарово умножение векторов, как обсуждалось в главе 2.

Стандартное умножение матриц

Теперь мы переходим к интуитивно непонятному способу умножения матриц. Для ясности стандартное умножение матриц не особенно сложное; оно просто отличается от того, что можно было бы ожидать. Вместо того чтобы оперировать поэлементно, стандартное умножение матриц оперирует по-

строчно/постолбцово. Собственно говоря, стандартное умножение матриц сводится к систематической коллекции точечных произведений между строками одной матрицы и столбцами другой матрицы. (Эта форма умножения формально называется просто *умножением матриц*; я добавил прилагательное *стандартное*, чтобы помочь устранить неоднозначность в отношении адамарова умножения и умножения на скаляр.)

Но прежде чем перейти к деталям процедуры умножения двух матриц, сначала объясню, как определять, можно умножать две матрицы или нет. Как вы узнаете, две матрицы можно умножить только в том случае, если их размеры согласуются.

Правила допустимости умножения матриц

Вы знаете, что размеры матрицы записываются как $M \times N$ – строки по столбцам. Две умножающие друг друга матрицы могут иметь разные размеры, поэтому давайте обозначим размер второй матрицы как $N \times K$. При записи двух матриц-сомножителей с их размерами внизу можно ссылаться на «внутренние» мерности N и «внешние» мерности M и K.

Вот важный момент: *умножать матрицы допустимо только тогда, когда «внутренние» мерности совпадают, а размер матрицы произведения определяется «внешними» мерностями*. Смотрите рис. 5.3.

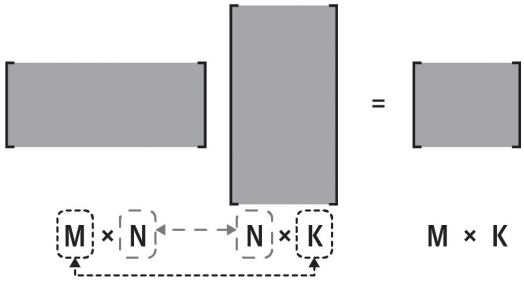


Рис. 5.3 ❖ Визуализация допустимости умножения матриц. Запомните эту картинку

Выражаясь формальнее, умножение матриц допустимо, когда число столбцов в левой матрице равно числу строк в правой матрице, а размер матрицы произведения определяется числом строк в левой матрице и числом столбцов в правой матрице. Я нахожу, что правило «внутренние/внешние» запоминается легче.

Уже можно видеть, что умножение матриц не подчиняется коммутативному закону: **AB** может быть допустимым, тогда как **BA** – недопустимым. Даже если оба умножения верны (например, если обе матрицы – квадратные), они могут давать разные результаты. То есть если $\mathbf{C} = \mathbf{AB}$ и $\mathbf{D} = \mathbf{BA}$, тогда в общем случае $\mathbf{C} \neq \mathbf{D}$ (в некоторых особых случаях они равны, но в целом допускать равенство невозможно).

Обратите внимание на обозначения: адамарово умножение обозначается кругом с точкой ($\mathbf{A} \odot \mathbf{B}$), тогда как умножение матриц обозначается как две матрицы бок о бок без какого-либо символа между ними (**AB**).

Теперь самое время узнать о механике и интерпретации умножения матриц.

Умножение матриц

Причина, по которой умножение матриц допустимо только в том случае, если число столбцов в левой матрице соответствует числу строк в правой матрице, заключается в том, что (i, j)-й элемент в матрице произведения является точечным произведением между i-й строкой левой матрицы и j-м столбцом в правой матрице.

Уравнение 5.1 показывает пример умножения матриц с использованием тех же двух матриц, которые мы использовали для адамарова умножения. Убедитесь, что вы понимаете, как каждый элемент в матрице произведения вычисляется в виде точечных произведений соответствующих строк и столбцов матриц левой части.

Уравнение 5.1. Пример умножения матриц. Круглые скобки добавлены, чтобы облегчить визуальное группирование

$$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} (2a+3c) & (2b+3d) \\ (4a+5c) & (4b+5d) \end{bmatrix}.$$

Если вы прилагаете усилия, чтобы запомнить принцип работы умножения матриц, но у вас едва получается, то на рис. 5.4 показан мнемонический трюк с выведением умножения при помощи пальцев.

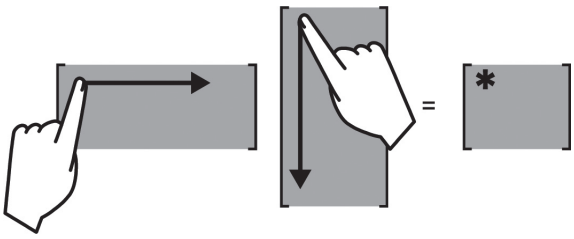


Рис. 5.4 ❖ Движения пальцев для умножения матриц

Как интерпретировать умножение матриц? Вспомните, что точечное произведение – это число, в котором кодируется взаимосвязь между двумя векторами. И таким образом, результатом умножения матриц является матрица, в которой хранятся все попарные линейные взаимосвязи между строками левой матрицы и столбцами правой матрицы. Это прекрасная вещь, и она лежит в основе вычисления матриц ковариаций и корреляций, общей ли-

нейной модели (используемой в статистическом анализе, включая модели ANOVA и регрессии), сингулярного разложения и бесчисленного количества других применений.

Умножение матрицы на вектор

В чисто механическом смысле умножение матрицы на вектор не представляет собой ничего особенного и не заслуживает отдельного подраздела: взаимное умножение матрицы и вектора – это просто умножение матрицы, в котором одна «матрица» является вектором.

Однако умножение матрицы и вектора между собой имеет много применений в науке о данных, машинном обучении и компьютерной графике, поэтому на него стоит потратить немного времени. Давайте начнем с основ.

- Матрицу можно умножить на вектор-столбец, расположенный справа, но не на вектор-строку, и ее можно умножить на вектор-строку, расположенную слева, но не на вектор-столбец. Другими словами, $\mathbf{Av}$ и $\mathbf{v}^T\mathbf{A}$ допустимы, но $\mathbf{Av}^T$ и $\mathbf{vA}$ не допустимы.

  Это ясно из осмотра размеров матрицы: матрицу $M \times N$ можно предпозиционно умножить на матрицу $1 \times M$ (также именуемую вектором-строкой) либо постпозиционно умножить на матрицу $N \times 1$ (также именуемую вектором-столбцом).
- Результатом умножения матрицы на вектор всегда является вектор, и ориентация этого вектора зависит от ориентации вектора-сомножителя: предпозиционное умножение матрицы на вектор-строку создает еще один вектор-строку, тогда как постпозиционное умножение матрицы на вектор-столбец производит еще один вектор-столбец. Опять же, это очевидно, если думать о размерах матрицы, но на это стоит указать.

Умножение матриц на векторы имеет несколько применений. В статистике предсказываемые моделью значения данных получаются путем умножения расчетной матрицы на коэффициенты регрессии, что записывается как $\mathbf{B\beta}$. В анализе главных компонент выявляется вектор весовых коэффициентов «важностей признаков», в котором максимизирована дисперсия в наборе данных $\mathbf{Y}$, и записывается как $(\mathbf{Y}^T\mathbf{Y})\mathbf{v}$ (вектор важностей признаков $\mathbf{v}$ называется собственным вектором). В многопеременной обработке сигналов размерно-редуцированная компонента получается путем применения пространственного фильтра к данным многоканального временного ряда $\mathbf{S}$ и записывается как $\mathbf{w}^T\mathbf{S}$. В геометрии и компьютерной графике множество координат изображения можно преобразовывать с использованием матрицы математического преобразования, и преобразование записывается как $\mathbf{Tp}$, где $\mathbf{T}$ – это матрица преобразования, а $\mathbf{p}$ – множество геометрических координат.

В прикладной линейной алгебре существует еще очень много примеров применения умножения матрицы на вектор, и позже в данной книге вы увидите несколько таких примеров. Умножение матрицы на вектор также является основой для пространств матриц; с этой важной темой вы познакомитесь позже в следующей главе.

А пока мне хотелось бы сосредоточиться на двух конкретных интерпретациях умножения матрицы на вектор: как средства реализации линейно-взвешенных комбинаций векторов и как механизма реализации геометрических преобразований.

Линейно-взвешенные комбинации

В предыдущей главе мы рассчитывали линейно-взвешенные комбинации, используя отдельные скаляры и векторы, а затем перемножали их по отдельности. Но теперь вы стали умнее, чем когда начинали предыдущую главу, и поэтому готовы усвоить более оптимальный, более компактный и масштабируемый метод вычисления линейно-взвешенных комбинаций: помещать отдельные векторы в матрицу, а веса – в соответствующие элементы вектора. И затем умножать. Вот численный пример:

$$4\begin{bmatrix}3\\0\\6\end{bmatrix}+3\begin{bmatrix}1\\2\\5\end{bmatrix}\Rightarrow\begin{bmatrix}3&1\\0&2\\6&5\end{bmatrix}\begin{bmatrix}4\\3\end{bmatrix}.$$

Пожалуйста, найдите минутку, чтобы проработать умножение, и убедитесь, что понимаете принцип реализации линейно-взвешенной комбинации двух векторов в виде умножения матрицы на вектор. Ключевой момент заключается в том, что каждый элемент в векторе умножает соответствующий столбец в матрице на скаляр, а затем взвешенные векторы-столбцы суммируются, чтобы получить произведение.

Данный пример предусматривает линейно-взвешенные комбинации векторов-столбцов; что бы вы изменили для вычисления линейно-взвешенных комбинаций векторов-строк[1]?

Результаты геометрических преобразований

Когда мы думаем о векторе как о геометрическом отрезке, то умножение матрицы на вектор становится способом поворота и шкалирования этого вектора (вспомните, что умножение скаляра на вектор может шкалировать, но не поворачивать).

Ради удобства визуализации давайте начнем с двумерного примера. Вот наша матрица и векторы:

```
M  = np.array([ [2,3], [2,1] ])
x  = np.array([ [1, 1.5] ]).T
Mx = M@x
```

Обратите внимание, что я создал `x` как вектор-строку, а затем транспонировал его в вектор-столбец; за счет этого сократилось число квадратных скобок при наборе исходного кода.

---

[1] Поместить коэффициенты в вектор-строку и предпозиционно умножить на этот вектор.

График A на рис. 5.5 создает визуализацию этих двух векторов. Хорошо видно, что матрица **M** одновременно повернула и растянула изначальный вектор. Давайте попробуем другой вектор с той же матрицей. На самом деле, просто ради развлечения, давайте использовать те же векторные элементы, но с переставленными позициями (то есть вектор `v = [1.5,1]`).

Теперь на графике B (рис. 5.5) происходит странная вещь: произведение матрицы и вектора больше не поворачивается в другом направлении. Матрица по-прежнему прошкалировала вектор, но его направление сохранилось. Другими словами, *умножение матрицы на вектор* действовало так, как если бы это было умножение вектора на *скаляр*. И это не случайное событие: на самом деле вектор `v` является собственным вектором матрицы `M`, а число, на которое `M` растянула `v`, является собственным числом данной матрицы[1]. Это настолько невероятно важное явление, что оно заслуживает отдельной главы (глава 13), но я просто не мог удержаться, чтобы не познакомить вас с этой концепцией сейчас.

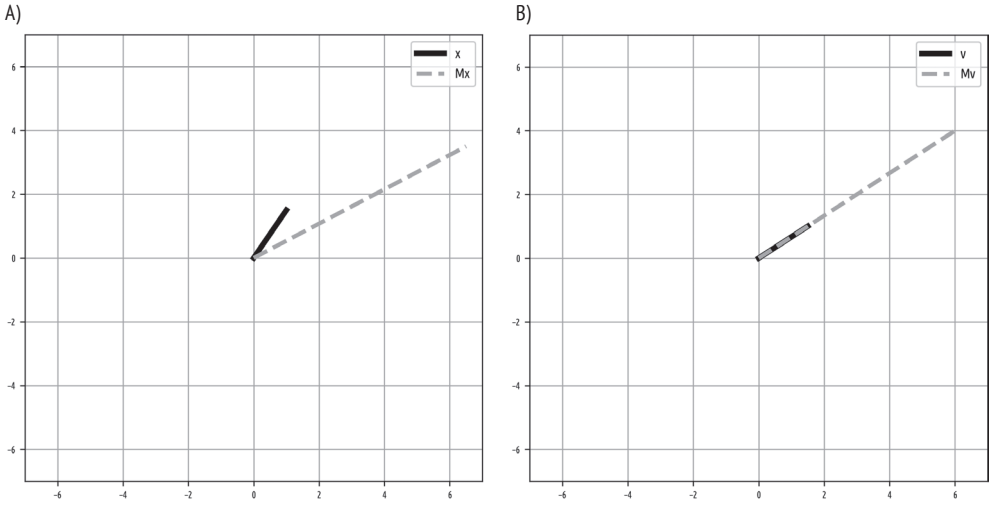


Рис. 5.5 ❖ Примеры умножения матриц на векторы

Переходя к более сложным темам, основной смысл этих демонстраций – в том, что одна из функций умножения матрицы на вектор заключается в том, что матрица содержит преобразование, которое при применении к вектору может поворачивать и растягивать этот вектор.

---

[1] Нередко используются синонимичные термины: характеристический вектор и характеристическое число. В английском языке во всех подобных терминах используется заимствованный из немецкого языка корень *eigen*, со значением «собственный», «характеристический». – *Прим. перев.*

Матричные операции: транспонирование

Вы узнали об операции транспонирования векторов в главе 2. Данный принцип соблюдается и с матрицами: поменять местами строки и столбцы. И точно так же, как с векторами, транспонирование обозначается надстрочной буквой ^{T} (таким образом, $\mathbf{C}^{\mathrm{T}}$ – это транспонированная версия матрицы $\mathbf{C}$). А двойное транспонирование матрицы возвращает изначальную матрицу ($\mathbf{C}^{\mathrm{TT}} = \mathbf{C}$).

Формальное математическое определение операции транспонирования приведено в уравнении 5.2 (и, по существу, повторяется из предыдущей главы), но, полагаю, одинаково легко запомнить, что *транспонирование меняет местами строки и столбцы*.

Уравнение 5.2. Определение операции транспонирования

$$a^{\mathrm{T}}_{i,j} = a_{j,i}.$$

Вот пример:

$$\begin{bmatrix} 3 & 0 & 4 \\ 9 & 8 & 3 \end{bmatrix}^{\mathrm{T}} = \begin{bmatrix} 3 & 9 \\ 0 & 8 \\ 4 & 3 \end{bmatrix}.$$

В Python существует несколько способов транспонирования матриц с использованием функций и методов, работающих на массивах NumPy:

```
A = np.array([ [3,4,5], [1,2,3] ])
A_T1 = A.T            # в качестве метода
A_T2 = np.transpose(A) # в качестве функции
```

В данном примере в матрице используется двумерный массив NumPy; что, по вашему мнению, произойдет, если применить метод транспонирования к вектору, запрограммированному в виде одномерного массива? Попробуйте – и узнаете[1]!

Обозначение точечного и внешнего произведений

Теперь, когда вы знакомы с операцией транспонирования и правилами допустимости умножения матриц, можно вернуться к обозначению точечного произведения векторов. Для двух векторов-столбцов $M\times1$ транспонирование первого вектора, а не второго, дает две «матрицы» размером $1\times M$ и $M\times1$. «Внутренние» мерности совпадают, а «внешние» мерности говорят о том, что

[1] Ничего. NumPy вернет тот же одномерный массив, не изменяя его и не выдавая предупреждения либо ошибки.

произведением будет 1×1, то есть скаляр. Это и есть та причина, по которой точечное произведение указывается в виде $\mathbf{a}^T\mathbf{b}$.

То же самое рассуждение будет и для внешнего произведения: умножение вектора-столбца на вектор-строку имеет размеры M×1 и 1×N. «Внутренние» мерности совпадают, и размером результата будет M×N.

Матричные операции: LIVE EVIL (порядок следования операций)

LIVE EVIL – это палиндром[1] и симпатичная мнемоника для запоминания того, как транспонирование влияет на порядок действий при умножении матриц. В сущности, правило заключается в том, что результат транспонирования умноженных матриц будет одинаков, что и транспонирование и умножение отдельных матриц, но в обратном порядке. В уравнении 5.3 **L**, **I**, **V** и **E** являются матрицами, и для того чтобы сделать умножение допустимым, исходят из того, что их размеры совпадают.

Уравнение 5.3. Пример правила LIVE EVIL

$$(\mathbf{LIVE})^T = \mathbf{E}^T\mathbf{V}^T\mathbf{I}^T\mathbf{L}^T.$$

Излишне говорить, что данное правило применимо для умножения любого числа матриц, а не только четырех, и не только с этими «случайно выбранными» буквами.

Указанное правило действительно кажется странным, но это единственный способ сделать так, чтобы транспонирование умноженных матриц работало. У вас будет возможность протестировать его самостоятельно в упражнении 5.7 в конце данной главы. Если хотите, то, перед тем как двигаться дальше, можете сразу перейти к этому упражнению.

Симметричные матрицы

Симметричные матрицы имеют целый ряд особых свойств, которые делают их удобными для работы. Кроме того, они, как правило, обладают численной стабильностью и, следовательно, удобны для вычислительных алгоритмов. Работая с данной книгой, вы узнаете об особых свойствах симметричных матриц; здесь я сосредоточусь на вопросах о том, что такое симметричные матрицы и как их создавать из несимметричных матриц.

Что значит для матрицы быть симметричной? Это означает, что соответствующие строки и столбцы равны. И это означает, что при изменении строк

[1] Палиндромом называется слово или фраза, которые пишутся одинаково в прямом и обратном порядке.

и столбцов местами с матрицей ничего не происходит. А это, в свою очередь, означает, что симметричная матрица равна ее транспонированной версии. В математических терминах матрица $\mathbf{A}$ является симметричной, если $\mathbf{A}^{\mathrm{T}} = \mathbf{A}$.

Посмотрите на симметричную матрицу в уравнении 5.4.

Уравнение 5.4. Симметричная матрица; обратите внимание, что каждая строка равна соответствующему ей столбцу

$$\begin{bmatrix} a & e & f & g \\ e & b & h & i \\ f & h & c & j \\ g & i & j & d \end{bmatrix}.$$

Может ли неквадратная матрица быть симметричной? Нет! Причина в том, что если матрица имеет размер $M \times N$, то ее транспонированная версия имеет размер $N \times M$. Эти две матрицы не могут быть равными, за исключением случая, когда $M = N$, а это означает, что матрица является квадратной.

Создание симметричных матриц из несимметричных

Возможно, поначалу это покажется удивительным, но умножение *любой* матрицы – даже неквадратной и несимметричной – на ее транспонированную версию будет приводить к получению квадратной симметричной матрицы. Другими словами, $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ является квадратной симметричной, как и $\mathbf{A}\mathbf{A}^{\mathrm{T}}$. (Если вам не хватает времени, терпения либо навыков работы с клавиатурой, чтобы форматировать надстрочную ^{T}, то можете писать AtA и AAt либо A′A и AA′.)

Прежде чем рассматривать пример, давайте строго докажем это утверждение. С одной стороны, на самом деле не требуется доказывать отдельно, что $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ является квадратной *и* симметричной, потому что из последнего вытекает первое. Но доказательство прямоугольности является простым и хорошим упражнением в линейно-алгебраических доказательствах (которые, как правило, короче и проще, чем, например, доказательства в дифференциальном исчислении).

Доказательство достигается просто путем рассмотрения размеров матрицы: если $\mathbf{A}$ имеет размер $M \times N$, то $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ имеет размеры $(N \times M)(M \times N)$ и, следовательно, матрица произведения имеет размер $N \times N$. Ту же логику можно использовать и для $\mathbf{A}\mathbf{A}^{\mathrm{T}}$.

Теперь переходим к доказательству симметрии. Вспомните определение симметричной матрицы – это матрица, равная своей транспонированной версии. Итак, давайте транспонируем $\mathbf{A}^{\mathrm{T}}\mathbf{A}$, задействуем немного алгебры и посмотрим, что получится. Внимательно проследите каждый приведенный ниже шаг; доказательство основано на правиле LIVE EVIL:

$$(\mathbf{A}^{\mathrm{T}}\mathbf{A})^{\mathrm{T}} = \mathbf{A}^{\mathrm{T}}\mathbf{A}^{\mathrm{TT}} = \mathbf{A}^{\mathrm{T}}\mathbf{A}.$$

Беря первый и последний члены, мы получаем $(\mathbf{A}^T\mathbf{A})^T = (\mathbf{A}^T\mathbf{A})$. Матрица равна ее транспонированной версии, следовательно, она является симметричной.

Теперь повторите доказательство самостоятельно, используя $\mathbf{A}\mathbf{A}^T$. Внимание, спойлер! Вы придете к тому же выводу. Но написание доказательства поможет вам усвоить концепцию до мозга костей.

Таким образом, обе матрицы, $\mathbf{A}\mathbf{A}^T$ и $\mathbf{A}^T\mathbf{A}$, являются квадратно-симметричными. Но это не одна и та же матрица! На самом деле если $\mathbf{A}$ не является квадратной, то два матричных произведения даже не имеют одинакового размера.

$\mathbf{A}^T\mathbf{A}$ называется *мультипликативным методом* создания симметричных матриц. Существует также *аддитивный метод*, который допустим, когда матрица является квадратной, но несимметричной. Этот метод обладает некоторыми интересными свойствами, но не имеет большой прикладной ценности, поэтому я не буду на нем заострять внимания. Упражнение 5.9 познакомит вас с алгоритмом; если вы готовы к испытанию, то, прежде чем приступать к выполнению упражнения, можете попробовать разработать этот алгоритм самостоятельно.

Резюме

Данная глава является первой в серии из трех глав, посвященных матрицам. Здесь вы ознакомились с основой, на которой базируются все матричные операции. Вкратце:

- Матрицы – это развернутые таблицы чисел. В различных приложениях их удобно концептуализировать в уме в виде множества векторов-столбцов, множества векторов-строк либо некоей упорядоченности отдельных значений. Как бы то ни было, визуализация матриц в виде изображений нередко бывает информативной либо, по меньшей мере, приятной на вид.
- Существует несколько категорий специальных матриц. Знакомство со свойствами типов матриц поможет вам разобраться в матричных уравнениях и продвинутых приложениях.
- Некоторые арифметические операции выполняются поэлементно, такие как сложение, умножение на скаляр и адамарово умножение.
- «Сдвиг» матрицы означает добавление константы к диагональным элементам (без изменения внедиагональных элементов). Сдвиг имеет несколько применений в машинном обучении, в первую очередь для отыскания собственных чисел и регуляризации статистических моделей.
- Умножение матриц предусматривает точечные произведения между строками левой матрицы и столбцами правой матрицы. Матрица произведения представляет собой организованный набор соотнесений между парами строк и столбцов. Запомните правило проверки правильности умножения матриц: $(M\times N)(N\times K) = (M\times K)$.

- LIVE EVIL[1,2]: транспонирование умноженных матриц равно транспонированным и умноженным отдельным матрицам с обратным порядком следования.
- Симметричные матрицы зеркально отражаются по диагонали, то есть каждая строка равна соответствующим ей столбцам, и определяются как $\mathbf{A} = \mathbf{A}^T$. Симметричные матрицы обладают многими интересными и полезными свойствами, которые делают их удобными для работы в приложениях.
- Симметричная матрица создается из любой матрицы, умножая эту матрицу на ее транспонированную версию. Результирующая матрица $\mathbf{A}^T\mathbf{A}$ занимает центральное место в статистических моделях и сингулярном разложении.

Упражнения по программированию

Упражнение 5.1

Это упражнение поможет вам ознакомиться с индексацией элементов матрицы. Создайте матрицу 3×4, используя `np.arange(12).reshape(3,4)`. Затем напишите исходный код на Python, чтобы извлечь элемент во второй строке, четвертом столбце. Используйте мягкое программирование, чтобы выбирать разные индексы строк/столбцов. Распечатайте сообщение, подобное следующему ниже:

```
Элемент матрицы по индексу (2,4) равен 7.
```

Упражнение 5.2

Это и следующее упражнения сосредоточены на нарезке матриц с целью получения подматриц. Начните с создания матрицы **C** на рис. 5.6 и примените существующую в Python операцию нарезки, чтобы извлечь подматрицу, состоящую из первых пяти строк и пяти столбцов. Давайте назовем эту матрицу $\mathbf{C}_1$. Попробуйте воспроизвести рис. 5.6, но если у вас возникли проблемы с программированием визуализации на Python, то просто сосредоточьтесь на правильном извлечении подматрицы.

Упражнение 5.3

Расширьте этот исходный код, чтобы извлечь остальные четыре блока размером 5×5. Затем создайте новую матрицу с этими блоками, которые переставлены местами в соответствии с рис. 5.7.

---

[1] LIVE EVIL – это милая мнемотехника, а не рекомендация о том, как вести себя в обществе!

[2] Для справки: LIVE EVIL (Зло в натуре, Зло живьем) – это концертный альбом хевиметал группы Black Sabbath, выпущенный в 1982 году. – *Прим. перев.*

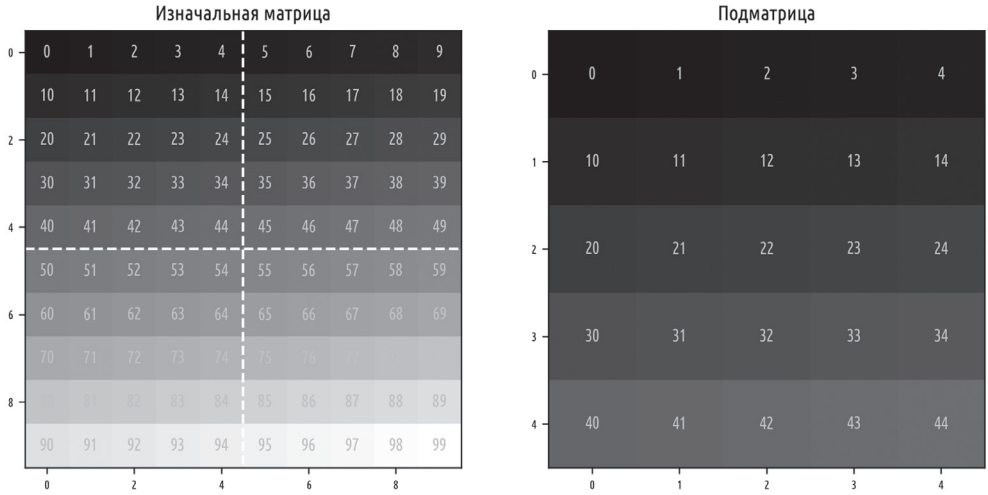


Рис. 5.6 ❖ Визуализация упражнения 5.2

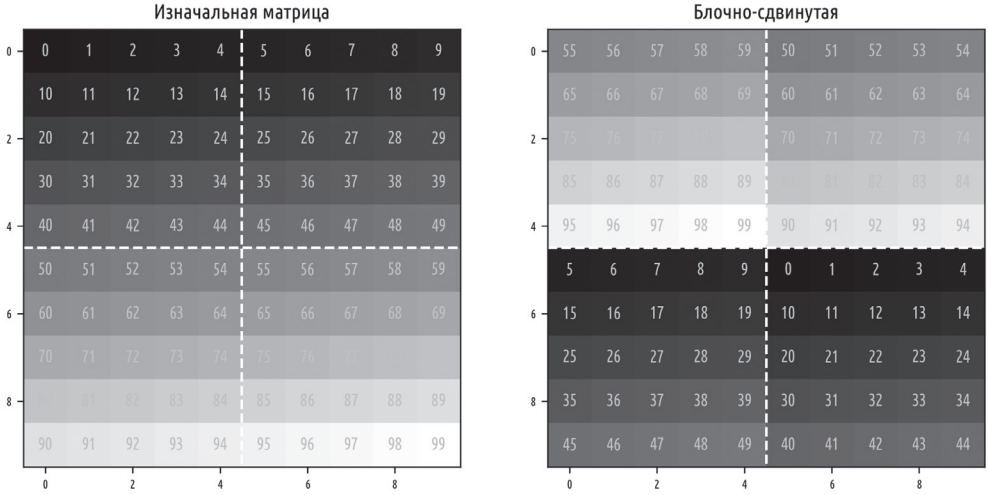


Рис. 5.7 ❖ Визуализация упражнения 5.3

Упражнение 5.4

Реализуйте поэлементное сложение матриц, используя два цикла `for` по строкам и столбцам. Что происходит, когда вы пытаетесь сложить две матрицы с несовпадающими размерами? Это упражнение поможет вам подумать о разбиении матрицы на строки, столбцы и отдельные элементы.

Упражнение 5.5

Сложение матриц и умножение матрицы на скаляр подчиняются математическим законам коммутативности и дистрибутивности. Это означает, что

следующие ниже уравнения дают одинаковые результаты (допустим, что матрицы **A** и **B** имеют одинаковый размер и что σ является неким скаляром):

$$\sigma(\mathbf{A} + \mathbf{B}) = \sigma\mathbf{A} + \sigma\mathbf{B} = \mathbf{A}\sigma + \mathbf{B}\sigma.$$

Вместо того чтобы доказывать это уравнение математически, вы продемонстрируете это с помощью программирования. Создайте на Python две матрицы случайных чисел размером 3×4 и случайный скаляр. Затем реализуйте три выражения в приведенном выше уравнении. Вам нужно будет найти способ подтвердить эквивалентность трех результатов. Имейте в виду, что крошечные ошибки вычислительной прецизионности[1] в диапазоне 10^{-15} следует игнорировать.

Упражнение 5.6

Запрограммируйте умножение матриц с использованием циклов `for`. Подтвердите свои результаты с помощью оператора `@` библиотеки NumPy. Это упражнение поможет вам укрепить ваше понимание умножения матриц, но на практике всегда лучше использовать `@` вместо написания двойного цикла `for`.

Упражнение 5.7

Подтвердите правило LIVE EVIL, выполнив следующие ниже пять шагов.

1. Создайте четыре матрицы случайных чисел, установив размеры $\mathbf{L} \in \mathbb{R}^{2\times6}$, $\mathbf{I} \in \mathbb{R}^{6\times3}$, $\mathbf{V} \in \mathbb{R}^{3\times5}$ и $\mathbf{E} \in \mathbb{R}^{5\times2}$.
2. Умножьте четыре матрицы и транспонируйте произведение.
3. Транспонируйте каждую матрицу по отдельности и умножьте их, *не меняя их порядок следования*.
4. Транспонируйте каждую матрицу по отдельности и умножьте их в обратном порядке *в соответствии с правилом LIVE EVIL*. Проверьте совпадение результата шага 2 с результатами шага 3 и шага 4.
5. Повторите приведенные выше шаги, но используя только квадратные матрицы.

Упражнение 5.8

В этом упражнении вы напишете функцию Python, которая выполняет проверку матрицы на симметричность. На входе она должна принимать матрицу и на выходе выводить булево значение `True`, если матрица является симметричной, либо `False`, если матрица является несимметричной. Имейте в виду, что малые ошибки вычислительного округления/прецизионности могут создавать впечатление, что «равные» матрицы являются неравными. Следовательно, вам нужно будет выполнять проверку на равенство с некоторой разумной терпимостью. Протестируйте функцию на симметричных и несимметричных матрицах.

---

[1] Для справки: метрика точности (accuracy) измеряет степень отклонения от целевого показателя, по сути являясь метрикой ширины отклонения, а метрика прецизионности (precision) измеряет степень глубины (резкости) измеряемой величины. – *Прим. перев.*

Упражнение 5.9

Я упоминал, что существует аддитивный метод создания симметричной матрицы из несимметричной квадратной матрицы. Указанный метод довольно прост: усреднить матрицу посредством ее транспонированной версии. Реализуйте этот алгоритм на Python и подтвердите, что результат действительно является симметричным. (Подсказка: используйте функцию, которую вы написали в предыдущем упражнении!)

Упражнение 5.10

Повторите вторую часть упражнения 3.3 (два вектора в $\mathbb{R}^3$), но используйте умножение матрицы на вектор вместо умножения вектора на скаляр. То есть вычислите $\mathbf{As}$ вместо $\boldsymbol{\sigma}_1\mathbf{v}_1 + \boldsymbol{\sigma}_2\mathbf{v}_2$.

Упражнение 5.11

Диагональные матрицы обладают многими интересными свойствами, которые делают их полезными для работы. В данном упражнении вы узнаете о двух из этих свойств:

- предпозиционное умножение диагональной матрицы на матрицу-сомножитель шкалирует строки правой матрицы на соответствующие диагональные элементы;
- постпозиционное умножение диагональной матрицы на матрицу-сомножитель шкалирует столбцы левой матрицы на соответствующие диагональные элементы.

Этот факт используется в нескольких приложениях, включая вычисление матриц корреляций (глава 7) и диагонализацию матриц (главы 13 и 14).

Давайте обследуем последствия этих свойств. Начнем с создания трех матриц 4×4: матрицы из одних единиц (подсказка: `np.ones()`); диагональной матрицы, в которой диагональные элементы равны 1, 4, 9 и 16; и диагональной матрицы, равной квадратному корню из предыдущей диагональной матрицы.

Далее распечатайте матрицу единиц, пред- и постпозиционно умноженную на первую диагональную матрицу-сомножитель. Вы получите следующие ниже результаты:

```
# Предпозиционно умножить на диагональную матрицу:
[[ 1.  1.  1.  1.]
 [ 4.  4.  4.  4.]
 [ 9.  9.  9.  9.]
 [16. 16. 16. 16.]]

# Постпозиционно умножить на диагональную матрицу:
[[ 1.  4.  9. 16.]
 [ 1.  4.  9. 16.]
 [ 1.  4.  9. 16.]
 [ 1.  4.  9. 16.]]
```

Наконец, предпозиционно *и* постпозиционно умножьте матрицу единиц на матрицу квадратных корней из диагональной матрицы. Вы получите следующее ниже:

```
# Пред- и постпозиционно умножить
# на квадратично-диагональную матрицу
[[ 1.  2.  3.  4.]
 [ 2.  4.  6.  8.]
 [ 3.  6.  9. 12.]
 [ 4.  8. 12. 16.]]
```

Обратите внимание, что строки и столбцы прошкалированы таким образом, что (i, j)-й элемент в матрице умножается на произведение i-го и j-го диагональных элементов. (На самом деле мы создали таблицу умножения!)

Упражнение 5.12

Еще один забавный факт: умножение матриц – это то же самое, что и адамарово умножение двух диагональных матриц. Подумайте, почему так, используя бумагу и карандаш с двумя диагональными матрицами 3×3, а затем проиллюстрируйте это в исходном коде на Python.

Глава 6

Матрицы. Часть 2

Умножение матриц представляет собой один из самых замечательных даров, которыми наделили нас математики. Но для того чтобы перейти от элементарной линейной алгебры к продвинутой – а затем понимать и разрабатывать алгоритмы науки о данных, – нужно делать больше, чем просто умножать матрицы.

Мы начинаем эту главу с изложения норм матриц и пространств матриц. Нормы матриц, по сути, являются расширением норм векторов, а пространства матриц, по сути, являются расширением подпространств векторов (подпространства векторов, в свою очередь, являются не чем иным, как линейно-взвешенными комбинациями). Так что у вас уже есть необходимые базовые знания для этой главы.

Такие понятия, как линейная независимость, ранг и определитель, позволят перейти от понимания элементарных понятий, таких как транспонирование и умножение, к пониманию сложных тем, таких как обратная матрица, собственные числа и сингулярные числа. И эти продвинутые темы раскрывают возможности линейной алгебры для применений в науке о данных. Таким образом, эта глава является отправной точкой в вашей трансформации из новичка в линейной алгебре в ее знатока.

Внешне матрицы выглядят очень простыми – просто развернутой таблицей чисел. Но в предыдущих главах вы уже увидели, что в матрицах кроется нечто большее, чем кажется на первый взгляд. Итак, сделайте глубокий и успокаивающий вдох и погружайтесь в тему.

Нормы матриц

Вы познакомились с нормами векторов в главе 2: норма вектора – это его евклидова геометрическая длина, которая вычисляется как квадратный корень из суммы квадратов элементов вектора.

Нормы матриц немного сложнее. Прежде всего не существует «единственной в своем роде нормы матрицы»; из матрицы можно вычислить многочисленные отличимые нормы. Нормы матриц в чем-то похожи на нормы векторов – в том смысле, что каждая норма содержит одно характеризующее

матрицу число и что норма обозначается двойными вертикальными линиями, как в норме матрицы **A**, которая обозначается как ||**A**||.

Но разные нормы матриц обладают разными смыслами. Громадное число норм матриц в общих чертах можно разделить на два семейства: поэлементные (также иногда именуемые матричными нормами «по входам»[1]) и индуцированные. Поэлементные нормы вычисляются на основе отдельных элементов матрицы, и, следовательно, эти нормы интерпретируются как отражение ими величин элементов в матрице.

Индуцированные нормы интерпретируются следующим образом: одной из функций матрицы является кодирование преобразования вектора; индуцированная норма матрицы является мерой того, насколько это преобразование масштабирует (растягивает или сжимает) этот вектор. Данная интерпретация будет иметь больше смысла в главе 7, когда вы узнаете о применении матриц для геометрических преобразований, и в главе 14, когда вы узнаете о сингулярном разложении.

В данной главе я познакомлю вас с поэлементными нормами, а начну с евклидовой нормы, которая на самом деле является прямым расширением векторной нормы на матрицы. Евклидова норма также называется нормой Фробениуса и вычисляется как квадратный корень из суммы всех элементов матрицы в квадрате (уравнение 6.1).

Уравнение 6.1. Норма Фробениуса

$$||\mathbf{A}||_{\mathrm{F}} = \sqrt{\sum_{i=1}^{M}\sum_{j=1}^{N} a_{ij}^2}.$$

Индексы *i* и *j* соответствуют *M* строкам и *N* столбцам. Также обратите внимание на подстрочную букву $_{\mathrm{F}}$, указывающую на норму Фробениуса.

Норма Фробениуса еще называется нормой $\ell 2$ (ℓ – это причудливо выглядящая буква L). А норма $\ell 2$ получила свое название от общей формулы поэлементных -норм (обратите внимание, что норму Фробениуса можно получить при $p = 2$):

$$||\mathbf{A}||_p = \left(\sum_{i=1}^{M}\sum_{j=1}^{N} |a_{ij}|^p \right)^{1/p}.$$

Нормы матриц имеют несколько применений в машинном обучении и статистическом анализе. Одним из важных применений является регуляризация, целью которой является улучшение подгонки моделей и повышение обобщаемости моделей на данные, которые в модель ранее не подавались (позже в книге вы увидите соответствующие примеры). Базовая идея регуляризации состоит в добавлении матричной нормы в качестве стоимостной функции в алгоритм минимизации. Эта норма не дает модельным параметрам становиться слишком большими (регуляризация $\ell 2$, также именуемая *гребневой регрессией*) либо не поощряет разреженные ре-

[1] Англ. *entry-wise*. – *Прим. перев.*

шения (регуляризация $\ell 1$, также именуемая *лассо-регрессией*). По сути дела, современные архитектуры глубокого обучения достигают впечатляющей результативности в решении задач компьютерного зрения, опираясь на матричные нормы.

Еще одним применением нормы Фробениуса является вычисление меры «матричного расстояния». Расстояние между матрицей и самой собой равно 0, а расстояние между двумя разными матрицами увеличивается по мере того, как числовые значения в этих матрицах становятся все более непохожими. Матричное расстояние Фробениуса вычисляется просто путем замены матрицы **A** на матрицу **C** = **A** – **B** в уравнении 6.1.

Указанное расстояние можно использовать в качестве критерия оптимизации в алгоритмах машинного обучения, например чтобы уменьшать размер хранилища изображений, минимизируя расстояние Фробениуса между уменьшенной и изначальной матрицами. Упражнение 6.2 проведет вас по простому примеру минимизации.

След матрицы и норма Фробениуса

След матрицы – это сумма ее диагональных элементов, обозначаемая как tr(**A**), и существует он только для квадратных матриц. Обе следующие ниже матрицы имеют одинаковый след (14):

$$\begin{bmatrix} 4 & 5 & 6 \\ 0 & 1 & 4 \\ 9 & 9 & 9 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 0 & 8 & 0 \\ 1 & 2 & 6 \end{bmatrix}.$$

След обладает несколькими интересными свойствами. Например, след матрицы равен сумме собственных чисел[1] матрицы и, следовательно, является мерой «объема» собственного пространства[2]. Многие свойства следа менее подходят для применения в науке о данных, но вот одно интересное исключение:

$$||A||_F = \sqrt{\sum_{i=1}^{M}\sum_{j=1}^{N} a_{ij}^2} = \sqrt{tr(\mathbf{A}^{\mathrm{T}}\mathbf{A})}.$$

Другими словами, норму Фробениуса можно вычислять как квадратный корень из следа матрицы, умноженной на ее транспонированную версию. Причина, по которой это работает, состоит в том, что каждый диагональный элемент матрицы $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ определяется точечным произведением каждой строки на саму себя.

Упражнение 6.3 поможет вам обследовать следовой метод вычисления нормы Фробениуса.

---

[1] Англ. *eigenvalues*. – *Прим. перев.*

[2] Син. характеристическое пространство. – *Прим. перев.*

Пространства матрицы (столбцовое, строчное, нуль-пространство)

Концепция *пространств матрицы* занимает центральное место во многих темах абстрактной и прикладной линейной алгебры. К счастью, пространства матрицы обладают концептуальной простотой и, по сути, представляют собой просто линейно-взвешенные комбинации разных признаков матрицы.

Столбцовое пространство

Напомню, что линейно-взвешенная комбинация векторов предусматривает умножение на скаляр и суммирование множества векторов. Две модификации этой концепции будут расширять линейно-взвешенную комбинацию до столбцового пространства матрицы. Во-первых, мы концептуализируем матрицу в виде множества векторов-столбцов. Во-вторых, вместо того чтобы работать с определенным множеством скаляров, мы рассматриваем бесконечность действительно-значных скаляров. Бесконечное число скаляров дает бесконечное число способов комбинирования множества векторов. Это результирующее бесконечное множество векторов называется *столбцовым пространством матрицы*.

Давайте конкретизируем все это с помощью нескольких числовых примеров. Мы начнем с простого – с матрицы, которая имеет только один столбец (что на самом деле то же самое, что и вектор-столбец). Ее столбцовое пространство – все возможные линейно-взвешенные комбинации этого столбца – можно выразить следующим образом:

$$C\left(\begin{bmatrix}1\\3\end{bmatrix}\right) = \lambda\begin{bmatrix}1\\3\end{bmatrix}, \quad \lambda \in \mathbb{R}.$$

$C(\mathbf{A})$ обозначает столбцовое пространство матрицы $\mathbf{A}$, а символ $\in$ обозначает принадлежность к множеству. В данном контексте это означает, что λ может быть любым возможным действительно-значным числом.

Что означает указанное математическое выражение? Оно означает, что столбцовое пространство является множеством всех возможных шкалированных версий вектора-столбца [1 3]. Давайте рассмотрим несколько конкретных случаев. Находится ли вектор [1 3] в столцовом пространстве? Да, потому что этот вектор можно выразить как матрицу, умноженную на $\lambda = 1$. Как быть с [2 –6]? Тоже да, потому что этот вектор можно выразить как матрицу, умноженную на $\lambda = -2$. А что насчет [1 4]? Здесь ответ – нет: вектор [1 4] *не* находится в столбцовом пространстве матрицы, потому что просто нет скаляра, который мог бы умножить матрицу, чтобы произвести этот вектор.

Как выглядит столбцовое пространство? Для матрицы с одним столбцом столбцовое пространство представляет собой линию, которая проходит через начало координат в направлении вектора-столбца и простирается до

бесконечности в обоих направлениях. (В техническом плане линия не растягивается до буквальной бесконечности, потому что бесконечность не является вещественным числом. Но эта линия имеет сколь угодно большую длину – намного длиннее, чем наш ограниченный человеческий разум может представить, – поэтому во всех отношениях и с любой точки зрения об этой линии можно говорить как о бесконечно длинной.) На рис. 6.1 показано изображение столбцового пространства этой матрицы.

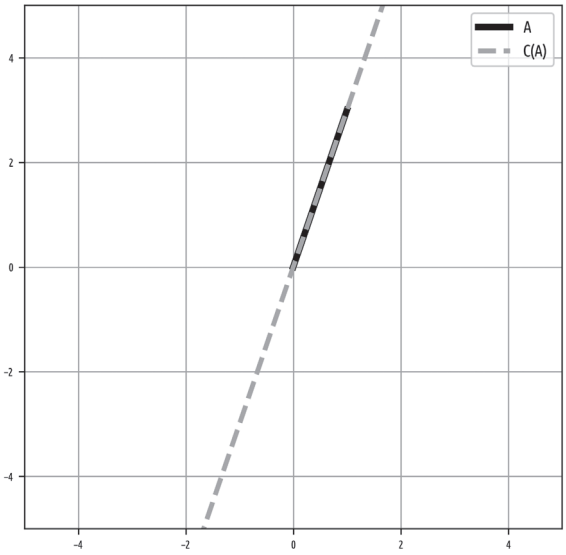


Рис. 6.1 ❖ Визуализация столбцового пространства матрицы с одним столбцом. Данное столбцовое пространство является одномерным подпространством

Теперь давайте рассмотрим матрицу с бóльшим числом столбцов. Мы оставим размерность столбца равной двум, чтобы иметь возможность визуализировать его на двумерном графике. Вот наша матрица и ее столбцовое пространство:

$$C\left(\begin{bmatrix}1 & 1\\ 3 & 2\end{bmatrix}\right) = \lambda_1\begin{bmatrix}1\\3\end{bmatrix} + \lambda_2\begin{bmatrix}1\\2\end{bmatrix}, \quad \lambda \in \mathbb{R}.$$

У нас два столбца, поэтому допускаем два отдельных λ (оба являются действительно-значными числами, но могут отличаться друг от друга). Теперь вопрос состоит в том, каким будет множество всех векторов, которое может быть достигнуто некоторой линейной комбинацией этих двух векторов-столбцов.

Ответ таков: все векторы в $\mathbb{R}^2$. Например, вектор [–4 3] можно получить путем шкалирования двух столбцов соответственно на 11 и –15. Как я додумался до этих скалярных значений? Я использовал проекцию методом наименьших квадратов, о которой вы узнаете в главе 11. А пока вы можете

сосредоточиться на концепции, согласно которой эти два столбца могут быть надлежаще взвешенны, чтобы достичь любой точки в $\mathbb{R}^2$.

График А на рис. 6.2 показывает два столбца матрицы. Я не нарисовал столбцовое пространство матрицы, потому что оно будет всей осью целиком.

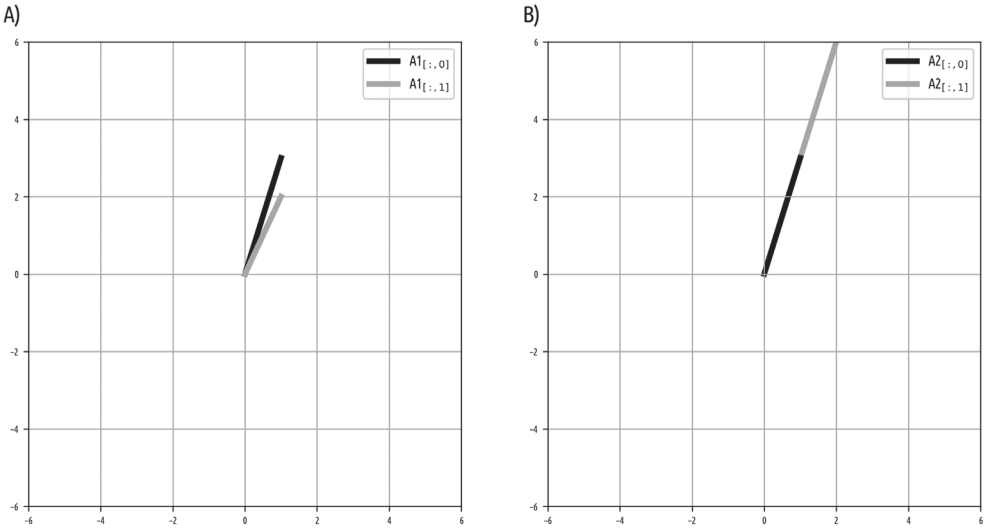


Рис. 6.2 ❖ Дополнительные примеры столбцовых пространств

Еще один пример в $\mathbb{R}^2$. Вот новая матрица, которую мы рассмотрим:

$$C\left(\begin{bmatrix}1 & 2\\ 3 & 6\end{bmatrix}\right) = \lambda_1\begin{bmatrix}1\\3\end{bmatrix} + \lambda_2\begin{bmatrix}2\\6\end{bmatrix}, \quad \lambda \in \mathbb{R}.$$

Какова размерность ее столбцового пространства? Можно ли достичь любой точки в $\mathbb{R}^2$ с помощью некоторой линейно-взвешенной комбинации двух столбцов?

Ответ на второй вопрос – нет. И если вы мне не верите, то попробуйте найти линейно-взвешенную комбинацию двух столбцов, которая создает вектор [3 5]. Это просто невозможно. Два столбца фактически являются коллинеарными (график В на рис. 6.2), потому что один уже является шкалированной версией другого. Это означает, что столбцовое пространство этой матрицы 2×2 по-прежнему остается просто линией – одномерным подпространством.

Из этого можно вынести один вывод: наличие N столбцов в матрице не гарантирует, что столбцовое пространство будет -мерным. Размерность столбцового пространства равна числу столбцов только в том случае, если столбцы образуют линейно независимое множество. (Вспомните главу 3, в которой говорилось, что линейная независимость означает множество векторов, в котором ни один вектор невозможно выразить как линейно-взвешенную комбинацию других векторов в этом множестве.)

Заключительный пример столбцовых пространств служит для того, чтобы увидеть, что происходит, когда мы переходим в три измерения.

Вот наша матрица и ее столбцовое пространство:

$$C\left(\begin{bmatrix}3 & 0\\ 5 & 2\\ 1 & 2\end{bmatrix}\right) = \lambda_1\begin{bmatrix}1\\ 5\\ 1\end{bmatrix} + \lambda_2\begin{bmatrix}0\\ 2\\ 2\end{bmatrix}, \quad \lambda \in \mathbb{R}.$$

Теперь в $\mathbb{R}^3$ два столбца. Эти два столбца линейно независимы, и, значит, выразить один как шкалированную версию другого невозможно. Таким образом, столбцовое пространство этой матрицы является двумерным, но оно является двумерной плоскостью, которая вложена в $\mathbb{R}^3$ (рис. 6.3).

Столбцовое пространство этой матрицы представляет собой бесконечную двумерную плоскость, но указанная плоскость является всего лишь бесконечно малым срезом трех измерений. Это можно представить как бесконечно тонкий лист бумаги, который охватывает Вселенную.

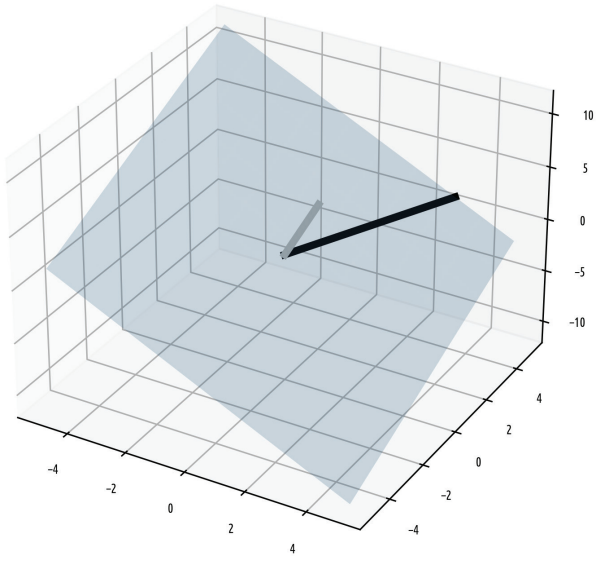


Рис. 6.3 ❖ Двумерное столбцовое пространство матрицы, встроенной в три измерения. Две толстые линии изображают два столбца матрицы

На этой плоскости есть много векторов (т. е. много векторов, которые можно получить как линейную комбинацию двух векторов-столбцов), но еще *больше* векторов, которые не находятся на плоскости. Другими словами, есть векторы в столбцовом пространстве матрицы, и есть векторы вне столбцового пространства матрицы.

Как узнать, что вектор находится в столбцовом пространстве матрицы? Это вовсе не тривиальный вопрос – на самом деле данный вопрос лежит

в основе линейного метода наименьших квадратов, и просто невозможно передать словами важность наименьших квадратов в прикладной математике и инженерии. Итак, *как* определить, что вектор находится в пространстве столбцов? В приводимых до сих пор примерах мы просто использовали некоторые догадки, арифметику и визуализацию. Смысл этого подхода состоял в том, чтобы привить интуитивное понимание, но эти методы, очевидно, не масштабируются на более высокие измерения и на более сложные задачи.

Количественные методы определения присутствия вектора в столбцовом пространстве матрицы основаны на концепции ранга матрицы, о котором вы узнаете позже в этой главе. А пока сосредоточьтесь на интуитивном понимании, что столбцы матрицы образуют векторное подпространство, которое может включать в себя все M-мерное пространство либо может быть некоторым подпространством меньшей мерности, и что важный вопрос состоит в том, не находится ли какой-либо другой вектор внутри этого подпространства (это означает, что вектор можно выразить как линейно-взвешенную комбинацию столбцов матрицы).

Строчное пространство

После того как вы разберетесь в столбцовом пространстве матрицы, понять строчное пространство будет реально легко. Строчное пространство матрицы – это, по сути, точно такая же концепция, однако вместо столбцов мы рассматриваем все возможные взвешенные комбинации строк.

Строчное пространство обозначается как $R(\mathbf{A})$. И поскольку операция транспонирования меняет местами строки и столбцы, можно также записать, что строчное пространство матрицы является столбцовым пространством транспонированной матрицы, другими словами, $R(\mathbf{A}) = C(\mathbf{A}^{\mathrm{T}})$. Между строчным и столбцовым пространствами матрицы есть несколько различий; например, строчное пространство (но не столбцовое пространство) инвариантно к операциям приведения строк. Но это выходит за рамки данной главы.

Поскольку строчное пространство равно столбцовому пространству транспонированой матрицы, эти два матричных пространства идентичны для симметричных матриц.

Нуль-пространства

Нуль-пространство незначительно, но важно отличается от столбцового пространства. Столбцовое пространство можно кратко свести к следующему ниже уравнению:

$$\mathbf{Ax} = \mathbf{b}.$$

Оно переводится на естественный так: «можно ли найти некоторый набор коэффициентов в $\mathbf{x}$ такой, что взвешенная комбинация столбцов в $\mathbf{A}$ произ-

водила бы вектор **b**?» Если ответ – да, то $\mathbf{b} \in C(\mathbf{A})$, и вектор **x** говорит о том, как взвешивать столбцы **A**, чтобы добраться до **b**.

Нуль-пространство, напротив, можно кратко свести к следующему ниже уравнению:

$$\mathbf{Ay} = \mathbf{0}.$$

Оно переводится на естественный так: «можно ли найти некоторый набор коэффициентов в **y** такой, что взвешенная комбинация столбцов в **A** давала бы вектор нулей **0**?»

Мгновенный осмотр покажет ответ, который действует для любой возможной матрицы **A**: установить **y** = **0**! Очевидно, что умножение всех столбцов на нули приведет к получению вектора нулей. Но это решение будет тривиальным, и мы его исключаем. Следовательно, возникает вопрос: «можно ли найти множество весов – не все из которых равны 0, – которое производит вектор нулей?» Любой вектор **y**, который может удовлетворять этому уравнению, находится в нуль-пространстве **A**, которое мы записываем как $N(\mathbf{A})$.

Давайте начнем с простого примера. Прежде чем читать следующий далее текст, посмотрим, сможете ли вы найти такой вектор **y**:

$$\begin{bmatrix} 1 & -1 \\ -2 & 2 \end{bmatrix}.$$

Вы придумали вектор? Мой будет таким: [7.34, 7.34]. Готов держать пари размером с Лас-Вегас, что вы не придумали тот же вектор. Возможно, вы придумали [1, 1] или [–1, –1]. А может быть, [2, 2]?

Думаю, вы видите, к чему это ведет – существует бесконечное число векторов **y**, которые удовлетворяют **Ay** = **0** для этой конкретной матрицы **A**. И все эти векторы можно выразить в виде некоторой шкалированной версии любого из этих вариантов. Таким образом, нуль-пространство данной матрицы можно выразить как

$$N(\mathbf{A}) = \lambda \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \lambda \in \mathbb{R}.$$

Вот еще один пример матрицы. Опять же, попробуйте найти набор коэффициентов такой, что взвешенная сумма столбцов давала бы вектор нулей (то есть найдите **y** в **Ay** = **0**):

$$\begin{bmatrix} 1 & -1 \\ -2 & 3 \end{bmatrix}.$$

Готов сделать еще более высокую ставку на то, что вы не сможете найти такой вектор. Но это не потому, что я не верю в ваши способности (я очень высокого мнения о своих читателях!), а потому, что в этой матрице нет нуль-пространства. Формально мы говорим, что нуль-пространство этой матрицы является пустым множеством: $N(\mathbf{A}) = \{\}$.

Взгляните на два примера матриц в этом подразделе еще раз. Вы заметите, что первая матрица содержит столбцы, которые могут быть сформированы как шкалированные версии других столбцов, тогда как вторая матрица содержит столбцы, которые образуют независимое множество. Это вовсе не совпадение: между размерностью нуль-пространства и линейной независимостью столбцов в матрице существует тесная взаимосвязь. Точная природа этой взаимосвязи задается теоремой о ранге и нульности[1], о которой вы узнаете в дополнении A к книге. Но их ключевой момент заключается в следующем: нуль-пространство является пустым, когда столбцы матрицы образуют линейно независимое множество.

Рискуя показаться избыточным, повторю вот этот важный момент: полноранговые матрицы и матрицы с полным столбцовым рангом имеют пустые нуль-пространства, тогда как рангово-пониженные матрицы имеют непустые (нетривиальные) нуль-пространства.

Библиотека SciPy в Python содержит функцию, которая вычисляет нуль-пространство матрицы. Давайте подтвердим наши результаты с помощью исходного кода:

```
A = np.array([ [1,-1], [-2,2] ])
B = np.array([ [1,-1], [-2,3] ])

print( scipy.linalg.null_space(A) )
print( scipy.linalg.null_space(B) )
Вот результат:
[[0.70710678]
 [0.70710678]]

[]
```

Второй результат ([]) – это пустое множество. Почему Python выбрал для нуль-пространства **A** числовые значения 0.70710678? Разве не было бы легче читать, если бы Python выбрал 1? Учитывая бесконечность возможных векторов, Python вернул *единичный вектор*[2] (норму этого вектора можно вычислить в уме, зная, что $\sqrt{1/2} \approx .7071$). Единичные векторы удобны в работе и обладают несколькими приятными свойствами, включая численную стабильность. Поэтому компьютерные алгоритмы часто возвращают единичные векторы в качестве базисов подпространств. Вы увидите это снова с собственными векторами и сингулярными векторами.

Как выглядит нуль-пространство? На рис. 6.4 показаны векторы-строки и нуль-пространство матрицы **A**.

Почему я вывел на график векторы-строки вместо векторов-столбцов? Оказывается, что строчное пространство расположено ортогонально нуль-

---

[1] Англ. *rank-nullity theorem*. См. https://en.wikipedia.org/wiki/Rank-nullity_theorem. – *Прим. перев.*

[2] То есть с векторной величиной, равной единице. – *Прим. перев.*

пространству. Это не по какой-то причудливой эзотерической причине; как раз наоборот, это зашито прямо в определение нуль-пространства как $\mathbf{Ay} = \mathbf{0}$. Переписывание этого уравнения для каждой строки матрицы (a_i) приводит к выражению $a_i\mathbf{y} = \mathbf{0}$; другими словами, точечное произведение между каждой строкой и нуль-пространственным вектором равно 0.

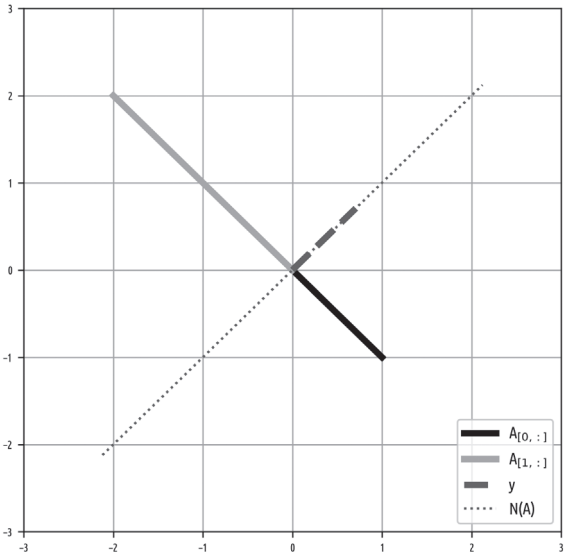


Рис. 6.4 ❖ Визуализация нуль-пространства матрицы

К чему весь этот сыр-бор по поводу нуль-пространств? Излишнее внимание векторам, которые могут умножать матрицу, чтобы получать вектор нулей, вполне может показаться странным. Однако, как вы узнаете в главе 13, нуль-пространство является краеугольным камнем в отыскании собственных и сингулярных векторов.

Заключительная мысль данного раздела такова: каждая матрица имеет четыре ассоциированных подпространства; вы узнали о трех (столбцовом, строчном, нуль-пространстве). Четвертое подпространство называется *правым нуль-пространством*[1] и представляет собой строчное нуль-пространство. Оно нередко записывается как нуль-пространство транспонированной матрицы: $N(\mathbf{A}^T)$. Традиционная учебная программа по математике теперь потратила бы несколько недель на обследование тонкостей и взаимосвязей между четырьмя подпространствами. Матричные подпространства заслуживают изучения из-за их завораживающей красоты и совершенства, но на такой уровень глубины мы погружаться не будем.

---

[1] Существует два нуль-пространства: левое и правое. Когда пишут просто «нуль-пространство», то имеют в виду левое нуль-пространство. – *Прим. перев.*

Ранг

Ранг – это число, ассоциированное с матрицей. Оно связано с размерностями матричных подпространств и имеет важные последствия для матричных операций, включая инвертирование матриц и определение числа решений системы уравнений.

Как и в случае с другими темами в этой книге, существуют богатые и подробные теории ранга матрицы, но здесь я сосредоточусь на том, что вам нужно знать для науки о данных и связанных с ней применений.

Начну с перечисления нескольких свойств ранга. Без какого-либо определенного порядка важности:

- ранг представляет собой неотрицательное целое число, поэтому матрица может иметь ранг 0, 1, 2, ..., но не –2 или 3.14;
- каждая матрица имеет один уникальный ранг; матрица не может одновременно иметь несколько отдельных рангов (Это также означает, что ранг является характеристикой матрицы, а не строк или столбцов.);
- ранг матрицы указывается с использованием $r(\mathbf{A})$ или $rank(\mathbf{A})$. Также уместно говорить, что «**A** является -ранговой матрицей»;
- максимально возможный ранг матрицы – это меньшее из числа ее строк либо столбцов. Другими словами, максимально возможный ранг равен $\min\{M, N\}$;
- матрица с максимально возможным рангом называется «полноранговой». Матрица с рангом $r < \min\{M, N\}$ по-разному называется: «рангово-пониженной», «рангово-дефицитной» или «сингулярной»;
- скалярное умножение не влияет на ранг матрицы (за исключением 0, который преобразовывает матрицу в матрицу нулей с рангом 0).

Существует несколько эквивалентных интерпретаций и определений ранга матрицы. К ним относятся:

- наибольшее число столбцов (или строк), которые образуют линейно независимое множество;
- размерность столбцового пространства (которая совпадает с размерностью строчного пространства);
- число измерений, содержащих информацию в матрице. Оно не совпадает с суммарным числом столбцов или строк в матрице из-за возможных линейных зависимостей;
- количество ненулевых сингулярных чисел матрицы.

Возможно, покажется удивительным, что определение ранга выглядит одинаково для столбцов и строк: действительно ли размерность одинакова для столбцового и строчного пространств, даже для неквадратной матрицы? Да, так и есть. Существуют различные тому доказательства, многие из которых либо достаточно сложны, либо основаны на сингулярном разложении, поэтому я не буду включать формальное доказательство в эту главу[1]. Но в ка-

[1] Если вы уже знакомы с сингулярным разложением (SVD), то короткая версия заключается в том, что SVD-разложение матриц **A** и $\mathbf{A}^T$ меняет местами строчное и столбцовое пространства, но количество ненулевых сингулярных чисел остается прежним.

честве иллюстрации покажу пример строчного и столбцового пространств неквадратной матрицы.

Вот наша матрица:

$$\begin{bmatrix} 1 & 1 & -4 \\ 2 & -2 & 2 \end{bmatrix}.$$

Столбцовое пространство матрицы находится в $\mathbb{R}^2$, тогда как строчное пространство находится в $\mathbb{R}^3$, поэтому эти два пространства необходимо выводить на разных графиках (рис. 6.5). Три столбца не образуют линейно независимого множества (любой один столбец можно описать как линейную комбинацию двух других), но они действительно охватывают все $\mathbb{R}^2$. Следовательно, столбцовое пространство матрицы – двумерное. Две строки действительно образуют линейно независимое множество, и подпространство, которое они охватывают, является двумерной плоскостью в $\mathbb{R}^3$.

Для ясности: столбцовое и строчное пространства матрицы *различны*. Но *размерность* этих пространств матрицы совпадает. И эта размерность является рангом матрицы. Таким образом, данная матрица имеет ранг 2.

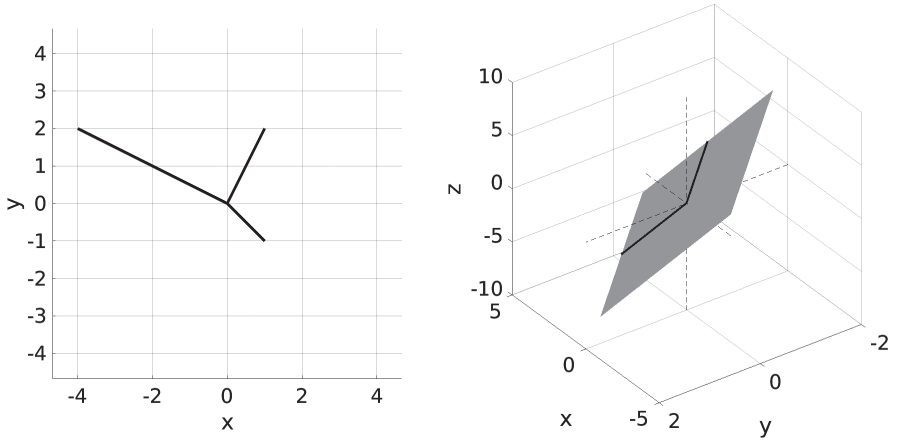


Рис. 6.5 ❖ Столбцовое и строчное пространства имеют разные охваты, но одинаковые размерности

Ниже приведено несколько матриц. Хотя я еще не научил вас вычислять ранг, попробуйте угадать ранг каждой матрицы на основе предыдущих описаний. Ответы приведены в сноске[1].

$$\mathbf{A} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 3 \\ 2 & 6 \\ 4 & 12 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 3.1 \\ 2 & 6 \\ 4 & 12 \end{bmatrix},$$

[1] $r(\mathbf{A}) = 1$, $r(\mathbf{B}) = 1$, $r(\mathbf{C}) = 2$, $r(\mathbf{D}) = 3$, $r(\mathbf{E}) = 1$, $r(\mathbf{F}) = 0$.

$$\mathbf{D}=\begin{bmatrix}1&3&2\\6&6&1\\4&2&0\end{bmatrix},\quad \mathbf{E}=\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix},\quad \mathbf{F}=\begin{bmatrix}0&0&0\\0&0&0\\0&0&0\end{bmatrix}.$$

Надеюсь, вам удалось разобраться в рангах, или, по меньшей мере, вы не были шокированы, увидев ответы в сноске.

Излишне говорить, что визуальный осмотр и некоторая интуиция не являются масштабируемым методом вычисления ранга на практике. Ранг вычисляется несколькими способами. Например, в главе 10 вы узнаете, что ранг можно вычислять построчно, приводя матрицу к ее ступенчатой форме и подсчитывая число опорных элементов. Компьютерные программы, такие как Python, вычисляют ранг путем подсчета количества ненулевых сингулярных чисел матрицы. Вы узнаете об этом в главе 14.

А пока я хотел бы, чтобы вы сосредоточились на идее, что ранг соответствует наибольшему числу столбцов, которые могут образовывать линейно независимое множество, что также соответствует размерности столбцового пространства матрицы. (Слово «столбцовый» в предыдущем предложении можно заменить на «строчный», и это будет одинаково точно.)

Ранги специальных матриц

Некоторые специальные матрицы имеют ранги, которые легко вычисляются или о которых стоит знать.

Векторы
: Все векторы имеют ранг 1. Вся причина в том, что векторы – по определению – содержат только один столбец (или строку) информации; подпространство, которое они охватывают, одномерное. Единственным исключением является вектор нулей.

Матрицы нулей
: Матрица нулей любого размера (включая вектор нулей) имеет ранг 0.

Единичные матрицы
: Ранг единичной матрицы равен числу строк (которое равно числу столбцов). Другими словами, $r(\mathbf{I}_N) = N$. На самом деле единичная матрица является просто частным случаем диагональной матрицы.

Диагональные матрицы
: Ранг диагональной матрицы равен числу ненулевых диагональных элементов. Это связано с тем, что каждая строка содержит максимум один ненулевой элемент, и невозможно создать ненулевое число посредством взвешенных комбинаций нулей. Это свойство становится полезным при решении систем уравнений и интерпретации сингулярного разложения.

Треугольные матрицы

Треугольная матрица является полноранговой только в том случае, если во всех диагональных элементах имеются ненулевые значения. Треугольная матрица с хотя бы одним нулем в диагонали будет рангово-пониженной (точный ранг будет зависеть от числовых значений в матрице).

Случайные матрицы

Ранг случайной матрицы невозможно узнать априори, поскольку он зависит от распределения чисел, из которого были взяты элементы матрицы, и от вероятности извлечения каждого числа. Например, матрица 2×2, заполненная 0 либо 1, может иметь ранг 0, если все отдельные элементы равны 0. Либо она может иметь ранг 2, например если единичная матрица отобрана в случайном порядке.

Но существует способ создания случайных матриц с гарантированным максимально возможным рангом. Это делается путем извлечения чисел с плавающей точкой в случайном порядке, например из гауссова или равномерного распределения. 64-битовый компьютер может представлять 2^{64} чисел. Извлечение нескольких десятков или сотен из этого множества, чтобы поместить в матрицу, означает, что вероятность линейных зависимостей в столбцах матрицы будет астрономически малой. На самом деле настолько маловероятной, что это могло бы привести в действие двигатель бесконечной невероятности космического корабля «Золотое сердце»[1].

Дело в том, что матрицы, созданные, например, функцией `np.random.randn()`, будут иметь максимально возможный ранг. Это удобно при использовании Python для изучения линейной алгебры, потому что можно создавать матрицу с любым произвольным рангом (с учетом обсуждавшихся ранее ограничений). Упражнение 6.5 проведет вас по всему процессу.

Одноранговые матрицы

Одноранговая матрица имеет – как вы уже догадались – ранг 1. Это означает, что на самом деле в матрице содержится информация только из одного столбца (и как вариант: информация только из одной строки), а все остальные столбцы (или строки) являются просто линейными кратными. Ранее в этой главе вы увидели несколько примеров одноранговых матриц. Вот еще несколько:

---

[1] Технология двигателя бесконечной невероятности позволяет космическому кораблю «Золотое сердце» преодолевать невозможные расстояния в космосе. Если вы не знаете, о чем я говорю, значит, вы не читали роман «Автостопом по Галактике» Дугласа Адамса (Hitchhiker's Guide to the Galaxy, Douglas Adams). И если вы не читали эту книгу, то вы реально упускаете одно из величайших, заставляющих задуматься и забавнейших интеллектуальных достижений XX века.

$$\begin{bmatrix} -2 & -4 & -4 \\ -1 & -2 & -2 \\ 0 & 0 & 0 \end{bmatrix}, \quad \begin{bmatrix} 2 & 1 \\ 0 & 0 \\ 2 & 1 \\ 4 & 2 \end{bmatrix}, \quad \begin{bmatrix} 12 & 4 & 4 & 12 & 4 \\ 6 & 2 & 2 & 6 & 2 \\ 9 & 3 & 3 & 9 & 3 \end{bmatrix}.$$

Одноранговые матрицы могут быть квадратными, высокими или широкими; независимо от размера каждый столбец является шкалированной копией первого столбца (или каждая строка является шкалированной копией первой строки).

Как создавать матрицу ранга 1? На самом деле в главе 2 вы уже узнали, как это делается (правда, я упомянул об этом лишь вскользь; так что прощаю вас, что забыли). Ответ таков: путем взятия внешнего произведения между двумя ненулевыми векторами. Например, третья показанная выше матрица является внешним произведением $[4\ 2\ 3]^T$ и [3 1 1 3 1].

Одноранговые матрицы имеют большую важность в собственном и сингулярном разложении матрицы. Вы столкнетесь со многими одноранговыми матрицами в последующих главах этой книги – и в своих приключениях по прикладной линейной алгебре.

Ранг сложенных и умноженных матриц

Если известны ранги матриц **A** и **B**, то можно ли автоматически узнать ранг **A** + **B** либо **AB**?

Ответ – нет, невозможно. Но ранги двух отдельных матриц обеспечивают верхние границы максимально возможного ранга. Вот правила:

$$rank(\mathbf{A} + \mathbf{B}) \leq rank(\mathbf{A}) + rank(\mathbf{B});$$

$$rank(\mathbf{AB}) \leq \min\{rank(\mathbf{A}), rank(\mathbf{B})\}.$$

Я не рекомендую заучивать эти правила наизусть. Но настоятельно рекомендую запомнить следующее:

- невозможно узнать точный ранг матрицы суммирования или матрицы произведения, основываясь на знании рангов отдельных матриц (за некоторыми исключениями, например матрицы нулей); вместо этого отдельные матрицы обеспечивают верхние границы ранга матрицы суммирования либо матрицы произведения;
- ранг матрицы суммирования может быть больше, чем ранги отдельных матриц;
- ранг матрицы произведения не может быть больше наибольшего ранга умножающих матриц[1].

В упражнении 6.6 у вас будет возможность проиллюстрировать эти два правила, используя случайные матрицы различных рангов.

[1] Это правило объясняет, почему внешнее произведение всегда дает матрицу ранга 1.

Ранг сдвинутых матриц

Если говорить просто, то сдвинутые матрицы имеют полный ранг. Фактически одной из первичных целей сдвига квадратной матрицы является повышение ее ранга с $r < M$ до $r = M$.

Очевидным примером является сдвиг матрицы нулей единичной матрицей. Ранг результирующей суммы $\mathbf{0} + \mathbf{I}$ является полноранговой матрицей.

Вот еще один, чуть менее очевидный пример:

$$\begin{bmatrix}1&3&2\\5&7&2\\2&2&0\end{bmatrix}+.01\begin{bmatrix}1&0&0\\0&1&0\\0&0&1\end{bmatrix}=\begin{bmatrix}1.01&3&2\\5&7.01&2\\2&2&.01\end{bmatrix}.$$

Крайняя левая матрица имеет ранг 2; обратите внимание, что третий столбец равен второму минус первый. Но ранг суммированной матрицы равен 3: третий столбец больше невозможно получить некоторой линейной комбинацией первых двух. И все же информация в матрице практически не изменилась; и действительно, корреляция Пирсона между элементами в изначальной и сдвинутой матрицах составляет $\rho = 0.999997222233796$. Этот факт имеет существенные последствия: например, матрицу ранга 2 нельзя инвертировать, тогда как сдвинутую матрицу можно. (Вы узнаете причину в главе 8.)

Теория и практика

Понимание сингулярных чисел перед пониманием сингулярных чисел

Невозможно изучать математику в чисто монотонном ключе, то есть когда вы полностью разбираетесь в концепции *a* до того, как полностью усвоите концепцию *b* и т. д. Полное понимание ранга матрицы требует знания сингулярного разложения, но сингулярное разложение не имеет смысла, пока вы не узнаете о ранге. Это своего рода уловка, из которой нет выхода из-за взаимно зависимых условий. И это часть того, что затрудняет изучение математики. Хорошие новости состоят в том, что страницы в этой книге продолжают существовать и после того, как вы их прочитали, поэтому, если следующее далее изложение будет не совсем разумным, вернитесь к нему после ознакомления с сингулярным разложением (SVD).

Если кратко, то каждая матрица $M \times N$ имеет множество $\min\{M, N\}$ неотрицательных сингулярных чисел, в которых закодирована «важность», или «пространственность», различных направлений в столбцовом и строчном пространствах матрицы. Направления с сингулярным числом 0 находятся в одном из нуль-пространств.

В абстрактной линейной алгебре ранг – незыблемое понятие. Каждая матрица имеет ровно один ранг, и на этом история заканчивается.

Однако на практике вычисление ранга матрицы влечет за собой некоторую неопределенность. Возможно, компьютеры даже не *вычисляют* ранг, а лишь рассчитывают его *оценку* с разумной степенью точности. Ранее я писал, что ранг можно вычислить как количество ненулевых сингулярных чисел. Но

это не то, что делает Python. Ниже приведены две ключевые строки, взятые из функции `np.linalg.matrix_rank()` (я опустил несколько аргументов, чтобы сосредоточиться на главном):

```
S = svd(M)
return count_nonzero(S > tol)
```

`M` – это рассматриваемая матрица, `S` – вектор сингулярных чисел, а `tol` – порог допуска. И значит, NumPy на самом деле не подсчитывает ненулевые сингулярные числа; он подсчитывает сингулярные числа, которые превышают некоторый порог. Точное пороговое значение зависит от числовых значений в матрице, но обычно оно примерно на 10^{-12} порядков меньше, чем элементы матрицы.

Это означает, что NumPy принимает решение о том, какие числа достаточно малы, чтобы считаться «фактически нулевыми». Я, конечно же, не критикую NumPy – это правильный поступок! (Другие программы числовой обработки, такие как MATLAB и Julia, вычисляют ранг таким же образом.)

Но зачем это делать? Почему бы просто не подсчитывать ненулевые сингулярные числа? Ответ в том, что допуск поглощает малые числовые неточности, которые могут возникать из-за ошибки вычислительного округления. Наличие допуска также позволяет игнорировать незначительные шумы, которыми могут, например, загрязняться датчики сбора данных. Эта идея используется для очистки данных, сжатия и уменьшения размерности. Рисунок 6.6 иллюстрирует эту концепцию.

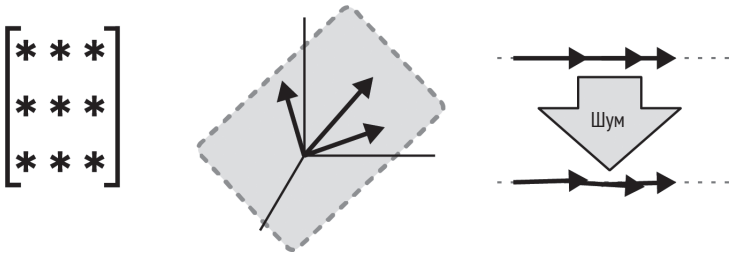


Рис. 6.6 ❖ Ранг представляющей двумерную плоскость матрицы 3×3 может считаться равным 3 при наличии малого количества шума. Крайняя правая диаграмма показывает перспективу взгляда непосредственно через поверхность плоскости

Применения ранга

Существует целый ряд применений ранга матрицы. В данном разделе я представлю два из них.

В столбцовом пространстве

В главе 4 вы узнали о столбцовом пространстве матрицы и о том, что важным вопросом в линейной алгебре является принадлежность/непринадлежность вектора к столбцовому пространству матрицы (который математически можно записать как $\mathbf{v} \in C(\mathbf{A})$?). Я также написал, что строгий и масштабируемый ответ на данный вопрос зависит от понимания ранга матрицы.

Прежде чем рассказать об алгоритме определения $\mathbf{v} \in C(\mathbf{A})$, мне нужно кратко представить процедуру, именуемую *расширением*[1] матрицы.

Расширить матрицу означает добавить дополнительные столбцы в правую часть матрицы. Вы начинаете с «базовой» матрицы $M{\times}N$ и «дополнительной» матрицы $M{\times}K$. Расширенная матрица имеет размер $M{\times}(N + K)$. Расширение двух матриц допустимо при условии, что они имеют одинаковое число строк (у них может быть разное число столбцов). Вы увидите расширенные матрицы здесь, в этом разделе, и снова в главе 10 при решении систем уравнений.

Ниже приведен пример, иллюстрирующий данную процедуру:

$$\begin{bmatrix} 4 & 5 & 6 \\ 0 & 1 & 2 \\ 9 & 9 & 4 \end{bmatrix} \sqcup \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 4 & 5 & 6 & 1 \\ 0 & 1 & 2 & 2 \\ 9 & 9 & 4 & 3 \end{bmatrix}.$$

С учетом этого обязательного отклонения от темы вот алгоритм определения принадлежности/непринадлежности вектора к столбцовому пространству матрицы:

1. **Расширить матрицу вектором**. Обозначим изначальную матрицу через $\mathbf{A}$ и раширенную матрицу – через $\tilde{\mathbf{A}}$.
2. **Вычислить ранги двух матриц**.
3. **Сравнить два ранга**. Будет один из двух возможных исходов:
   a) $rank(\mathbf{A}) = rank(\tilde{\mathbf{A}})$ Вектор $\mathbf{v}$ находится в столбцовом пространстве матрицы $\mathbf{A}$.
   b) $rank(\mathbf{A}) < rank(\tilde{\mathbf{A}})$ Вектор $\mathbf{v}$ *не* находится в столбцовом пространстве матрицы $\mathbf{A}$.

Какова логика, стоящая за этой проверкой? Если $\mathbf{v} \in C(\mathbf{A})$, то $\mathbf{v}$ можно выразить как некоторую линейно-взвешенную комбинацию столбцов матрицы $\mathbf{A}$ (столбцы расширенной матрицы $\tilde{\mathbf{A}}$ образуют линейно зависимое множество). С точки зрения охвата вектор $\mathbf{v}$ является избыточным в $\tilde{\mathbf{A}}$. Следовательно, ранг остается прежним.

И наоборот, если $\mathbf{v} \notin C(\mathbf{A})$, то $\mathbf{v}$ невозможно выразить как линейно-взвешенную комбинацию столбцов матрицы $\mathbf{A}$, и, следовательно, $\mathbf{v}$ добавил новую информацию в $\tilde{\mathbf{A}}$. А это значит, что ранг увеличится на 1.

---

[1] Англ. *augmenting*. – *Прим. перев.*

Определение принадлежности/непринадлежности вектора к столбцовому пространству матрицы – это не просто какое-то эзотерическое академическое упражнение; это часть внутренней логики линейного моделирования методом наименьших квадратов, то есть математики, лежащей в основе методов ANOVA, регрессий и общих линейных моделей. В главах 10 и 11 вы узнаете об этом больше, но главная идея заключается в том, что мы разрабатываем модель того, как устроен мир, а затем переводим эту модель в матрицу, которая называется *расчетной матрицей*[1]. Измеряемые данные из окружающего мира хранятся в векторе. Если этот вектор данных находится в столбцовом пространстве расчетной матрицы, то мы смоделировали мир идеально. Почти во всех случаях вектор данных не находится в столбцовом пространстве, и поэтому мы определяем, достаточно ли он близок, чтобы считаться статистически значимым. Очевидно, что в последующих главах я расскажу гораздо подробнее, но здесь я предвосхищаю события, чтобы поддержать ваш энтузиазм в обучении.

Линейная независимость множества векторов

Теперь, когда вы знаете о ранге матрицы, вы готовы понять алгоритм определения линейной зависимости/независимости множества векторов. Если вы хотите разработать такой алгоритм самостоятельно, то смело делайте это сейчас, перед тем как читать остальную часть данного раздела.

Алгоритм незамысловат: поместить векторы в матрицу, вычислить ранг матрицы, а затем сравнить этот ранг с максимально возможным рангом этой матрицы (вспомните, что это $\min\{M, N\}$; для удобства изложения я буду исходить из допущения, что у нас высокая матрица). Возможными результатами являются:

- $r = M$: множество векторов является линейно независимым;
- $r < M$: множество векторов является линейно зависимым.

Внутренняя логика этого алгоритма должна быть ясной: если ранг меньше, чем число столбцов, то по меньшей мере один столбец можно описать как линейную комбинацию других столбцов, то есть он является определением линейной зависимости. Если ранг равен числу столбцов, то каждый столбец вносит уникальную информацию в матрицу, и, стало быть, ни один столбец невозможно описать как линейную комбинацию других столбцов.

Здесь я хотел бы подчеркнуть более общий момент: многие операции и приложения в линейной алгебре на самом деле являются довольно простыми и разумными, но для их понимания требуется значительный объем базовых знаний. И это хорошо: чем больше вы знаете линейную алгебру, тем проще становится линейная алгебра.

С другой стороны, это утверждение не является универсально истинным: в линейной алгебре существуют операции, которые настолько умопомрачи-

[1] Англ. *design matrix*; син. модельная матрица; матрица плана эксперимента. – *Прим. перев.*

тельно утомительны и сложны, что я не могу заставить себя описать их во всех деталях в этой книге. Переходим к следующему разделу...

Определитель

Определитель[1] – это число, ассоциированное с квадратной матрицей. В абстрактном линейном алгоритме определитель является ключевым элементом в нескольких операциях, включая вычисление обратной матрицы. Однако вычисление определителя для крупных матриц на практике бывает численно нестабильным из-за проблем, связанных с потерей значимости и переполнением. Вы увидите иллюстрацию тому в упражнениях.

Тем не менее получение обратной матрицы или собственного разложения матрицы невозможно понять, не понимая определителя.

Два наиболее важных свойства определителя – и два наиболее важных вывода из этого раздела – таковы:

1) он определен только для квадратных матриц и
2) он равен нулю для сингулярных (рангово-пониженных) матриц.

Определитель обозначается как $det(\mathbf{A})$ или $|\mathbf{A}|$ (обратите внимание на одиночные вертикальные линии вместо двойных вертикальных линий, которыми обозначается норма матрицы). Греческая заглавная дельта Δ используется, когда не нужно ссылаться на конкретную матрицу.

Но что *такое* определитель? Что он означает и как его интерпретировать? Определитель имеет геометрическую интерпретацию, которая связана с тем, насколько матрица растягивает векторы во время умножения матрицы на вектор (как мы помним из предыдущей главы, умножение матрицы на вектор представляет собой механизм применения геометрических преобразований к координате, выраженной в виде вектора). Отрицательный определитель означает, что во время преобразования поворачивается одна ось.

Однако в приложениях, связанных с наукой о данных, определитель используется алгебраически; эти геометрические объяснения не очень понятны в части того, как использовать определитель для отыскания собственных чисел или инвертирования матриц ковариаций. Поэтому достаточно сказать, что определитель является решающим шагом в таких продвинутых темах, как получение обратной матрицы, собственного разложения и сингулярного разложения, и что вы можете избавить себя от стресса и бессонных ночей, просто признав, что определитель является инструментом в нашем арсенале, не слишком беспокоясь о том, что он значит.

Вычисление определителя

Вычисление определителя отнимает много времени и утомляет. Если я проживу тысячу лет и никогда не вычислю определитель матрицы 5×5 вручную,

[1] Син. детерминант. – *Прим. перев.*

то проживу богатую, полноценную и осмысленную жизнь. Тем не менее существует короткий путь вычисления определителя матрицы 2×2, который показан в уравнении 6.2.

Уравнение 6.2. Вычисление определителя матрицы 2×2

$$det\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc.$$

Из этого уравнения можно увидеть, что определитель не ограничивается целыми либо положительными числами. В зависимости от числовых значений в матрице определителем может быть −1 223 729 358 либо +.00000002, или же любое другое число. (Для действительно-значной матрицы определителем всегда будет действительное число.)

Определитель вычисляется довольно просто, не правда ли? Всего-то произведение диагонали минус произведение вне диагонали. Для многих матриц определитель можно вычислять в уме. Определитель еще проще для матрицы 1×1: это просто абсолютное значение этого числа.

Возможно, теперь вы сомневаетесь в моем утверждении о том, что определитель имеет численную нестабильность.

Дело в том, что сокращенный способ вычисления определителя матрицы 2×2 не масштабируется до крупных матриц. Есть «сокращенный способ» для матриц 3×3, но на самом деле это не сокращенный способ; это визуальная мнемоника. Я не буду ее тут показывать, но напишу главную идею:

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = aei + bfg + cdh - ceg - bdi - afh.$$

Как только вы доберетесь до матриц 4×4, вычисление определителя станет настоящей морокой, если только в матрице не будет много тщательно расставленных нулей. Но я знаю, что вам любопытно, поэтому уравнение 6.3 показывает определитель матрицы 4×4.

Уравнение 6.3. Желаю удачи с этим уравнением

$$\begin{vmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{vmatrix} = \begin{array}{c} afkp - aflo - agjp + agln + ahjo - ahkn - bekp + belo \\ + bgip - bglm - bhio + bhkm + cejp - celn - cfip + cflm. \\ + chin - chjm - dejo + dekn + dfio - dfkm - dgin + dgjm \end{array}$$

Я даже не собираюсь показывать полную процедуру вычисления определителя матрицы любого размера, потому что давайте будем честны: вы читаете эту книгу, поскольку интересуетесь прикладной линейной алгеброй; важно понимать, как использовать определитель, а не полную формулу его вычисления.

В любом случае, суть в том, что если вам когда-нибудь понадобится вычислить определитель, то следует использовать функцию `np.linalg.det()` либо функцию `scipy.linalg.det()`.

Определитель с линейными зависимостями

Второй вывод в отношении определителй заключается в том, что они равны нулю для любой рангово-пониженной матрицы. Этот факт можно обследовать с помощью матрицы 2×2. Вспомните, что любая рангово-пониженная матрица имеет по меньшей мере один столбец, который можно выразить как линейную комбинацию других столбцов:

$$\begin{vmatrix} a & \lambda a \\ c & \lambda c \end{vmatrix} = ac\lambda - a\lambda c = 0.$$

Вот определитель сингулярной матрицы 3×3:

$$\begin{vmatrix} a & b & \lambda a \\ d & e & \lambda d \\ g & h & \lambda g \end{vmatrix} = ae\lambda g + b\lambda dg + \lambda adh - \lambda aeg - bd\lambda g - a\lambda dh = 0.$$

Указанная концепция обобщается на более крупные матрицы. Отсюда следует, что все рангово-пониженные матрицы имеют определитель, равный 0. Фактический ранг не имеет значения; если $r < M$, то $\Delta = 0$. Все полноранговые матрицы имеют ненулевой определитель.

Я уже писал, что, по моему мнению, геометрические интерпретации определителя имеют ограниченную ценность для понимания причины, по которой определитель имеет важность в науке о данных. Но $\Delta = 0$ действительно имеет приятный геометрический смысл: матрица с $\Delta = 0$ – это преобразование, при котором по меньшей мере одно измерение сглаживается, чтобы получить площадь поверхности, но не объем.

Представьте себе, как вы сжимаете мяч до бесконечно тонкого диска. Вы увидите наглядный пример в следующей главе («Геометрические преобразования с помощью умножения матриц на векторы» на стр. 131).

Характеристический многочлен

Уравнение определителя матрицы 2×2 содержит пять элементов: четыре элемента в матрице и значение определителя. Их можно записать в уравнении как $ad - bc = \Delta$. Одна из замечательных особенностей уравнения заключается в том, что элементы в нем можно перемещать туда-сюда и находить решения для разных переменных. Рассмотрим уравнение 6.4; допустим, что a, b, c и Δ известны, а λ – некоторый неизвестный элемент.

Уравнение 6.4. Использование определителя для отыскания недостающего элемента матрицы

$$\begin{vmatrix} a & b \\ c & \lambda \end{vmatrix} \quad \Rightarrow \quad a\lambda - bc = \Delta.$$

Немного алгебры уровня средней школы позволит найти решение для λ в терминах других элементов. Само решение не имеет важности; главное в том, что *если определитель матрицы известен, то можно найти решение для неизвестных переменных внутри матрицы*.

Вот численный пример:

$$\begin{vmatrix} 2 & 7 \\ 4 & \lambda \end{vmatrix} = 4 \quad \Rightarrow \quad 2\lambda - 28 = 4 \quad \Rightarrow \quad 2\lambda = 32 \quad \Rightarrow \quad \lambda = 16.$$

Теперь давайте сделаем еще один шаг вперед:

$$\begin{vmatrix} \lambda & 1 \\ 3 & \lambda \end{vmatrix} = 1 \quad \Rightarrow \quad \lambda^2 - 3 = 1 \quad \Rightarrow \quad \lambda^2 = 4 \quad \Rightarrow \quad \lambda = \pm 2.$$

То же самое неизвестное λ находилось на диагонали, произведя полиномиальное уравнение второго порядка, а оно, в свою очередь, привело к двум решениям. Это вовсе не совпадение: фундаментальная теорема алгебры говорит о том, что многочлен n-го порядка имеет ровно n корней (хотя некоторые из них могут быть комплекснозначными).

Вот ключевой момент, к которому я веду: комбинирование сдвига матрицы с определителем называется *характеристическим многочленом* матрицы, как показано в уравнении 6.5.

Уравнение 6.5. Характеристический многочлен матрицы

$$det(\mathbf{A} - \lambda\mathbf{I}) = \Delta.$$

Почему он называется многочленом? Потому что сдвинутая матрица $M \times M$ имеет член λ^M и, следовательно, имеет M решений. Вот как он выглядит для матриц 2×2 и 3×3:

$$\begin{vmatrix} a-\lambda & b \\ 3 & d-\lambda \end{vmatrix} = 0 \quad \Rightarrow \quad \lambda^2 - (a+d)\lambda + (ad - bc) = 0;$$

$$\begin{vmatrix} a-\lambda & b & c \\ d & e-\lambda & f \\ g & h & i-\lambda \end{vmatrix} = 0 \quad \Rightarrow \quad \begin{matrix} -\lambda^3 + (a+e+i)\lambda^2 \\ +(-ae+bd-ai+cg-ei+fh)\lambda \\ +aei-afh-bdi+bfg+cdh-ceg=0 \end{matrix}.$$

Пожалуйста, не просите меня показать вам характеристический многочлен матрицы 4×4. Поверьте, в нем будет член λ^4.

Давайте вернемся к случаю 2×2, на этот раз используя числа вместо букв. Приведенная ниже матрица является полноранговой, то есть ее определитель не равен 0 (он равен $\lambda = -8$), но я собираюсь допустить, что она имеет определитель 0 после сдвига на некоторый скаляр λ; вопрос в том, какие значения λ сделают эту матрицу рангово-пониженной. Давайте применим характеристический многочлен, чтобы выяснить:

$$\det\left(\begin{bmatrix}1 & 3\\ 3 & 1\end{bmatrix} - \lambda\mathbf{I}\right) = 0;$$

$$\begin{vmatrix}1-\lambda & 3\\ 3 & 1-\lambda\end{vmatrix} = 0 \quad \Rightarrow \quad (1-\lambda)^2 - 9 = 0.$$

После небольших алгебраических операций (которые я рекомендую вам проработать) два решения таковы: $\lambda = -2, 4$. Что эти числа означают? Для того чтобы это выяснить, давайте включим их обратно в сдвинутую матрицу:

$$\lambda = -2 \quad \Rightarrow \quad \begin{bmatrix}3 & 3\\ 3 & 3\end{bmatrix};$$

$$\lambda = 4 \quad \Rightarrow \quad \begin{bmatrix}-3 & 3\\ 3 & -3\end{bmatrix}.$$

Совершенно ясно, что обе матрицы являются одноранговыми. Кроме того, обе матрицы имеют нетривиальные нуль-пространства, и, значит, можно найти некий ненулевой вектор $\mathbf{y}$ такой, что $(\mathbf{A} - \lambda\mathbf{I})\mathbf{y} = \mathbf{0}$. Уверен, вы сможете найти вектор для каждого из этих λ самостоятельно! Но на всякий случай, если вы хотите подтвердить свой правильный ответ, обратитесь к сноске[1].

Характеристический многочлен, выражаясь технически, суперудивителен. Прежде всего он дает замечательное представление о том, что каждую квадратную матрицу можно выразить в виде уравнения. И не просто какого-то уравнения – а уравнения, которое напрямую связывает матрицы с фундаментальной теоремой алгебры. И если это недостаточно круто, то решениями характеристического многочлена, заданного равным $\Delta = 0$, являются собственные числа матрицы (это числа λ, которые мы нашли выше). Знаю, что вам придется подождать главы 13, чтобы понять, что такое собственные числа, какова причина их важности, но сегодня ночью вы сможете спать спокойно, ибо вы раскрыли для себя важную вещь: как определять собственные числа матрицы.

[1] Любая шкалированная версия векторов [1 –1] и [1 1].

Резюме

Цель этой главы была в том, чтобы расширить ваши знания о матрицах, включив в них несколько важных понятий: нормы, пространства, ранг и определитель. Если бы это была книга об абстрактной линейной алгебре, то данная глава легко могла бы занять более ста страниц. Но я попытался сосредоточить изложение на том, что вам нужно знать для применения линейной алгебры в науке о данных и искусственном интеллекте. Вы узнаете о матричных пространствах еще больше в последующих главах (в частности, о методе наименьших квадратов в главе 11 и сингулярном разложении в главе 14). А пока вот список наиболее важных выводов, которые следует вынести из этой главы.

- Существует много видов матричных норм, которые в широком смысле можно классифицировать на *поэлементные* и *индуцированные*. Первые отражают величины[1] элементов в матрице, тогда как вторые отражают геометрически преобразующее влияние матрицы на векторы.
- Наиболее широко используемая поэлементная норма называется нормой Фробениуса (она же евклидова норма, или $\ell 2$-норма) и вычисляется как квадратный корень из суммы квадратов элементов.
- След матрицы – это сумма диагональных элементов.
- Существует четыре пространства матрицы (столбцовое, строчное, нуль-пространство, правое нуль-пространство), и они определяются как множество линейно-взвешенных комбинаций разных признаков матрицы.
- Столбцовое пространство матрицы содержит все линейно-взвешенные комбинации столбцов в матрице и записывается как $C(\mathbf{A})$.
- Принадлежность/непринадлежность некого вектора $\mathbf{b}$ столбцовому пространству матрицы является важным вопросом в линейной алгебре; если он ему принадлежит, то существует некий вектор $\mathbf{x}$ такой, что $\mathbf{Ax} = \mathbf{b}$. Ответ на этот вопрос формирует основу для многих алгоритмов подгонки статистических моделей.
- Строчное пространство матрицы – это множество линейно-взвешенных комбинаций строк матрицы и обозначается как $R(\mathbf{A})$ или $C(\mathbf{A}^T)$.
- Нуль-пространство матрицы – это множество векторов, которое линейно комбинирует столбцы, чтобы получить вектор нулей – другими словами, любой вектор $\mathbf{y}$, который находит решение уравнения $\mathbf{Ay} = \mathbf{0}$. Тривиальное решение $\mathbf{y} = \mathbf{0}$ исключается. Нуль-пространство, среди других применений, имеет большое значение для отыскания собственных векторов.
- Ранг – это ассоциированное с матрицей неотрицательное целое число. Ранг отражает наибольшее число столбцов (или строк), которые могут образовывать линейно независимое множество. Матрицы с рангом, меньшим максимально возможного, называются рангово-пониженными, или сингулярными.
- Сдвиг квадратной матрицы путем добавления константы к диагонали обеспечивает полный ранг.

---

[1] Син. магнитуды. – *Прим. перев.*

- Одно (из многих) применений ранга состоит в определении принадлежности/непринадлежности вектора столбцовому пространству матрицы, которое производтся путем сравнения ранга матрицы с рангом векторно-раширенной матрицы.
- Определитель – это ассоциированное с квадратной матрицей число (для прямоугольных матриц определителя нет). Алгоритм его вычисления весьма утомителен, но самое важное, что нужно о нем знать, сводится к тому, что он равен нулю для всех рангово-пониженных матриц и отличен от нуля для всех полноранговых матриц.
- Характеристический многочлен преобразовывает сдвинутую на λ квадратную матрицу в уравнение, приравненное к определителю. Зная определитель, можно находить решения для λ. Я лишь кратко намекнул о причине, по которой так важно знать определитель, но поверьте: это важно. (Или же не доверяйте мне и убедитесь сами в главе 13.)

Упражнения по программированию

Упражнение 6.1

Норма матрицы связана со шкалой числовых значений в матрице. В данном упражнении вы проведете эксперимент, чтобы это продемонстрировать. В каждой из 10 итераций эксперимента создавайте матрицу случайных чисел 10×10 и вычисляйте ее фробениусову норму. Затем повторите этот эксперимент 40 раз, всякий раз умножая матрицу на другой скаляр, который находится в диапазоне от 0 до 50. Результатом эксперимента станет матрица норм размером 40×10. На рис. 6.7 показаны результирующие нормы, усредненные по 10 итерациям эксперимента. Указанный эксперимент также иллюстрирует два дополнительных свойства матричных норм: они строго неотрицательны и могут быть равны 0 только для матрицы нулей.

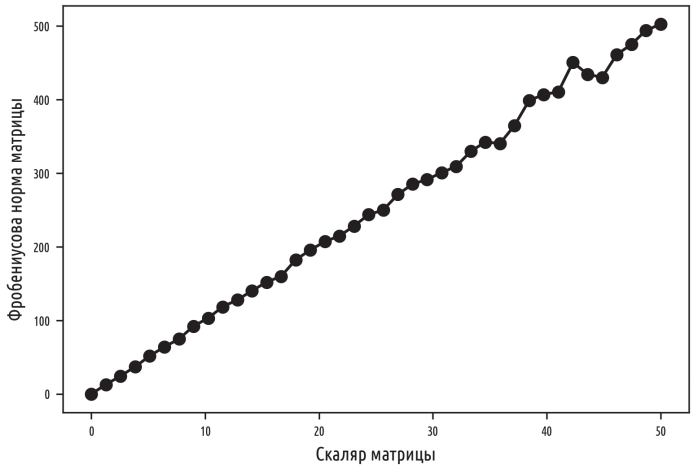


Рис. 6.7 ❖ Решение к упражнению 6.1

Упражнение 6.2

В этом упражнении вы напишете алгоритм, который находит скаляр, сводящий расстояние Фробениуса между двумя матрицами к 1. Начните с написания функции Python, которая на входе принимает две матрицы (одинакового размера) и на выходе возвращает фробениусово расстояние между ними. Затем создайте две матрицы $N \times N$ случайных чисел (в исходном коде решений я использовал $N = 7$, но вы можете использовать любой другой размер). Создайте переменную `s = 1`, которая скалярно умножает обе матрицы. Вычисляйте расстояние Фробениуса между шкалированными матрицами. До тех пор, пока это расстояние остается выше 1, устанавливайте скаляр равным ему самому, умноженному на .9, и пересчитывайте расстояние между шкалированными матрицами. Это должно быть сделано в цикле `while`. Когда фробениусово расстояние станет меньше 1, завершите цикл `while` и сообщите число итераций (что соответствует числу раз, когда скаляр `s` был умножен на .9) и скалярное значение.

Упражнение 6.3

Продемонстрируйте, что метод следа матрицы и евклидова формула дают один и тот же результат (норму Фробениуса). Работает ли формула следа только для $\mathbf{A}^T\mathbf{A}$ или же вы получаете тот же результат и для $\mathbf{A}\mathbf{A}^T$?

Упражнение 6.4

Это упражнение будет забавным[1], потому что вы сможете использовать материал из данной и предыдущих глав. Вы обследуете влияние сдвига матрицы на норму этой матрицы. Начните с создания случайной матрицы 10×10 и вычислите ее фробениусову норму. Затем запрограммируйте следующие ниже шаги внутри цикла `for`:

1) сдвиньте матрицу на долю нормы;
2) вычислите процентное изменение нормы по сравнению с изначальным;
3) вычислите фробениусово расстояние между сдвинутой и изначальной матрицами и
4) вычислите коэффициент корреляции между элементами в матрицах (подсказка: прокоррелируйте векторизованные матрицы с помощью функции `np.flatten()`).

Доля нормы, на которую вы делаете сдвиг, должна находиться в диапазоне от 0 до 1 с шагом в 30 линейно отстоящих шагов. Проверьте, чтобы на каждой итерации цикла использовалась изначальная матрица, а не сдвинутая матрица из предыдущей итерации. У вас должен получиться график, похожий на рис. 6.8.

Упражнение 6.5

Теперь я покажу вам, как создавать случайные матрицы с произвольным рангом (с учетом ограничений, касающихся размеров матриц и т. д.). Для

[1] Абсолютно все упражнения доставляют удовольствие, но некоторые больше, чем другие.

создания матрицы $M \times N$ с рангом r умножьте случайную матрицу $M \times r$ на матрицу $r \times N$. Реализуйте это на Python и подтвердите, что ранг действительно равен r. Что произойдет, если задать $r > \min\{M, \mathrm{N}\}$, и почему это происходит?

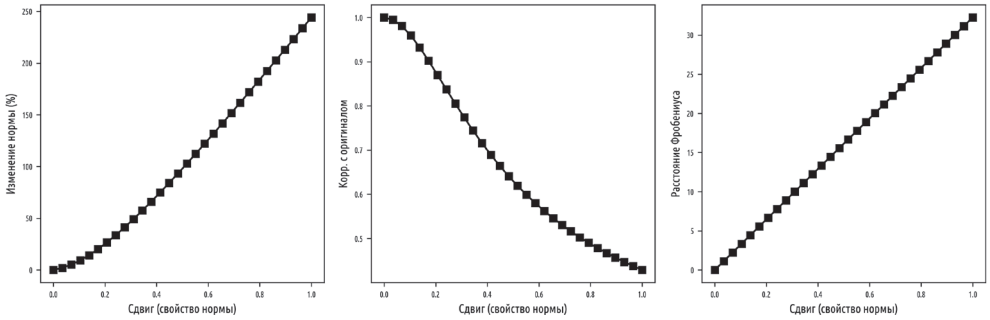


Рис. 6.8 ❖ Решение к упражнению 6.4

Упражнение 6.6

Продемонстрируйте правило сложения ранга матриц ($r(\mathbf{A} + \mathbf{B}) \leq r(\mathbf{A}) + r(\mathbf{B})$), создав три пары одноранговых матриц, просуммированных с

1) нуль-ранговой,
2) одноранговой и
3) двуранговой.

Затем повторите это упражнение, используя умножение матриц вместо их сложения.

Упражнение 6.7

Поместите исходный код из упражнения 6.5 в функцию Python, которая на входе принимает параметры M и r и на выходе выдает случайную матрицу $M \times M$ ранга r. В двойном цикле `for` создайте пары матриц 20×20 с отдельными рангами, варьирующимися от 2 до 15. Складывайте и умножайте эти матрицы и сохраняйте ранги этих результирующих матриц. Указанные ранги должны быть организованы в матрицу и визуализированы как функция рангов отдельных матриц (рис. 6.9).

Упражнение 6.8

Интересно, что все матрицы $\mathbf{A}$, $\mathbf{A}^{\mathrm{T}}$, $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ и $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ имеют одинаковый ранг. Напишите исходный код, чтобы это продемонстрировать, используя случайные матрицы различных размеров, очертаний (квадратные, высокие, широкие) и рангов.

Упражнение 6.9

Цель этого упражнения – ответить на вопрос: $\mathbf{v} \in C(\mathbf{A})$? Создайте матрицу $\mathbf{A} \in \mathbb{R}^{4\times3}$ ранга 3 и вектор $\mathbf{v} \in \mathbb{R}^4$ с использованием чисел, взятых из нормального распределения в случайном порядке. Следуйте описанному ранее алгоритму определения принадлежности/непринадлежности вектора столбцовому пространству матрицы. Повторите исходный код несколько раз,

чтобы попробовать найти неуклонную закономерность. Затем используйте матрицу $\mathbf{A} \in \mathbb{R}^{4\times4}$ ранга 4. Готов поспорить на один миллион биткойнов[1], что вы *всегда* будете находить, что $\mathbf{v} \in C(\mathbf{A})$, когда $\mathbf{A}$ – это случайная матрица 4×4 (при условии отсутствия ошибок программирования). Что именно вселяет в меня уверенность в вашем ответе[2]?

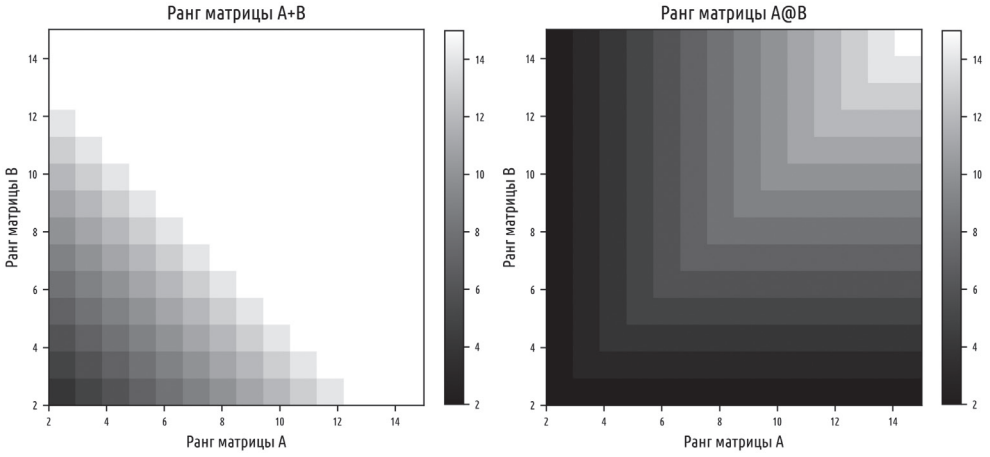


Рис. 6.9 ❖ Результаты упражнения 6.7

В целях дополнительной сложности поместите этот исходный код в функцию, которая возвращает `True` либо `False` в зависимости от результата проверки и которая вызывает исключение (то есть полезное сообщение об ошибке), если размер вектора не соответствует расширению матрицы.

Упражнение 6.10

Напомню, что определитель рангово-пониженной матрицы – теоретически – равен нулю. В данном упражнении вы проверите эту теорию на практике. Выполните следующие ниже шаги.

1. Создайте квадратную случайную матрицу.
2. Понизьте ранг матрицы. Ранее вы делали это путем умножения прямоугольных матриц; здесь задайте один столбец кратным другому столбцу.
3. Вычислите определитель и сохраните его абсолютное значение.

Выполните эти три шага в двойном цикле `for`: один цикл для матриц размером от 3×3 до 30×30 и второй цикл, который повторяет три шага сто раз (повторение эксперимента широко применяется при симулировании шумовых данных). Наконец, выведите определитель, усредненный по ста повторам, на линейный график как функцию от числа элементов в матрице. Теория линейной алгебры предсказывает, что эта линия (то есть определители всех

[1] Nota bene. У меня не так много биткойнов, ¯_(ツ)_/¯

[2] Потому что столбцовое пространство матрицы полного ранга охватывает все $\mathbb{R}^M$, и, следовательно, все векторы в $\mathbb{R}^M$ обязательно находятся в пространстве столбцов.

рангово-пониженных матриц) равна нулю независимо от размера матрицы. На рис. 6.10 показано иное, отражая вычислительные трудности, связанные с точным вычислением определителя. Я подверг данные логарифмическому преобразованию с целью повышения наглядности; вы должны проинспектировать график, используя логарифмическую и линейную шкалы.

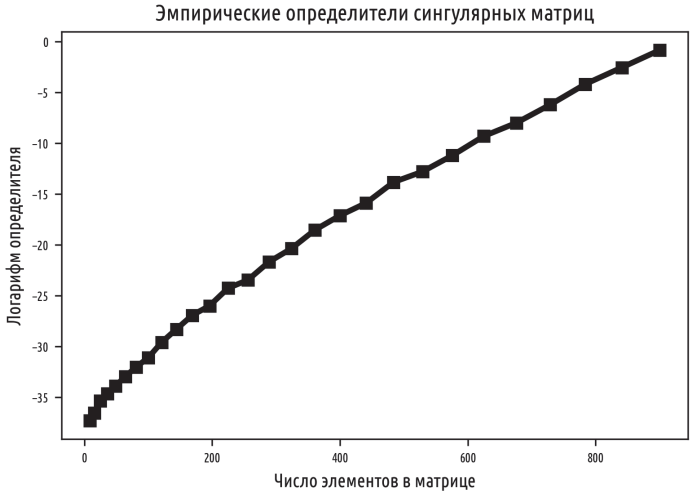


Рис. 6.10 ❖ Результаты упражнения 6.10

Глава 7

Применения матриц

Надеюсь, что теперь, после насыщенных теорией двух последних глав, вы чувствуете себя так, словно только что закончили интенсивную тренировку в тренажерном зале: измотанные, но полные энергии. Данная глава должна быть похожа на велосипедную прогулку по холмам в сельской местности: временами она будет требовать усилий, но при этом предлагать свежую и вдохновляющую перспективу.

Описанные в данной главе приложения в общих чертах основаны на приложениях, описанных в главе 4. Я сделал это, чтобы иметь несколько общих нитей изложения, которые связывают главы о векторах и о матрицах. И потому, что хочу, чтобы вы увидели, что, несмотря на все более усложняющиеся концепции и приложения, по мере продвижения по линейной алгебре фундамент по-прежнему строится на тех же простых принципах, таких как линейно-взвешенные комбинации и точечное произведение.

Матрицы ковариаций многопеременных данных

В главе 4 вы узнали, как вычислять коэффициент корреляции Пирсона в виде векторного точечного произведения двух переменных, деленного на произведение норм векторов. Эта формула была рассчитана на две переменные (например, рост и вес); а что, если у вас несколько переменных (например, рост, вес, возраст, еженедельные упражнения…)?

Вы могли бы представить, что пишете двойной цикл `for` для всех переменных и применяете формулу двухпеременной корреляции ко всем парам переменных. Но это будет громоздко и неэлегантно, а следовательно, противоречит духу линейной алгебры. Цель этого раздела – показать, как вычислять матрицы ковариаций и корреляций из наборов многопеременных[1] данных.

---

[1] Англ. *multivariate*; син. многопараметрический; относится к нескольким независимым переменным в статистической модели или модели МО. – *Прим. перев.*

Давайте начнем с *ковариации*. Ковариация – это просто числитель уравнения корреляции, другими словами, точечное произведение двух переменных, центрированных по среднему значению. Ковариация интерпретируется так же, как корреляция (положительная, когда переменные движутся вместе; отрицательная, когда переменные движутся порознь; нулевая, когда переменные не имеют линейной зависимости), за исключением того, что в ковариации сохраняется шкала данных и, следовательно, не ограничена ±1.

Ковариация также имеет коэффициент нормализации $n - 1$, где n – это число точек данных. Указанная нормализация не дает ковариации увеличиваться по мере суммирования большего числа значений данных (аналогично делению на N, чтобы преобразовать сумму в среднее значение). Вот уравнение ковариации:

$$c_{a,b} = (n-1)^{-1}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y}).$$

Как и в главе 4, если через $\tilde{\mathbf{x}}$ обозначить переменную, центрированную по среднему значению переменной $\mathbf{x}$, то ковариация будет равна $\tilde{\mathbf{x}}^{\mathrm{T}}\tilde{\mathbf{y}}/(n-1)$.

Ключевым моментом в реализации указанной формулы для нескольких переменных является то, что матричное умножение представляет собой организованный набор точечных произведений между строками левой матрицы и столбцами правой матрицы.

Итак, вот что мы делаем: создаем матрицу, в которой каждый столбец соответствует каждой переменной (переменная – это признак данных). Обозначим эту матрицу через $\mathbf{X}$. Теперь умножение $\mathbf{XX}$ не имеет смысла (и, вероятно, даже недопустимо, потому что матрицы данных, как правило, являются высокими, следовательно, $M > N$). Но если бы мы транспонировали первую матрицу, то *строки* матрицы $\mathbf{X}^{\mathrm{T}}$ соответствовали бы *столбцам* матрицы $\mathbf{X}$. Следовательно, в произведении матриц $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ кодируются все попарные ковариации (при условии что столбцы центрированы по среднему значению и затем делятся на $n - 1$). Другими словами, (i, j)-й элемент в матрице ковариаций является точечным произведением между признаками данных i и j.

Матричное уравнение для матрицы ковариаций выглядит элегантно и компактно:

$$\mathbf{C} = \mathbf{X}^{\mathrm{T}}\mathbf{X}\frac{1}{n-1}.$$

Матрица $\mathbf{C}$ симметрична. Это вытекает из доказательства в главе 5, что любая матрица, умноженная на ее транспонированную версию, является квадратно-симметричной, но это также имеет смысл статистически: ковариация и корреляция симметричны, и, стало быть, например, корреляция между высотой и весом такая же, как и корреляция между весом и ростом.

Каковы диагональные элементы $\mathbf{C}$? Они содержат ковариации каждой переменной с самой собой, которые в статистике называются *дисперсией*[1],

[1] Англ. *variance*. – *Прим. перев.*

и количественно определяют разбросанность вокруг среднего значения (дисперсия – это возведенное в квадрат стандартное отклонение).

Зачем транспонировать левую матрицу?

В транспонировании левой матрицы нет ничего особенного. Если ваша матрица данных организована как признаки по наблюдениям, тогда ее матрица ковариаций равна $\mathbf{X}\mathbf{X}^T$. Если вы когда-либо будете сомневаться в том, какую матрицу следует транспонировать: левую либо правую, – подумайте о том, как применить правила умножения матриц, чтобы получить матрицу в формате «признаки по признакам». Матрицы ковариаций всегда организованы как «признаки по признакам».

Пример в онлайновом исходном коде создает матрицу ковариаций из общедоступного набора данных статистики преступности. Набор данных содержит более ста признаков социальной, экономической, образовательной и жилищной информации в различных сообществах по всей территории США[1]. Цель набора данных – использовать указанные признаки для предсказывания уровня преступности, но здесь мы будем использовать его для инспектирования матриц ковариации и корреляции.

После импорта и небольшой обработки данных (описанной в онлайновом коде) у нас будет матрица данных под названием `dataMat`. Следующий ниже исходный код показывает, как вычислять матрицу ковариаций:

```
datamean = np.mean(dataMat,axis=0) # вектор средних значений признаков
dataMatM = dataMat - datamean      # среднецентрировать,
                                   # используя транслирование
covMat   = dataMatM.T @ dataMatM   # матрица данных, умноженная на
                                   # ее транспонированную версию
covMat  /= (dataMatM.shape[0]-1)   # разделить на N-1
```

На рис. 7.1 показано изображение матрицы ковариаций. Прежде всего она выглядит просто замечательно, не правда ли? На своей «повседневной работе» я работаю с наборами многопеременных данных в качестве профессора нейробиологии, и разглядывание матриц ковариаций никогда не переставало вызывать улыбку на моем лице.

В этой матрице светлые цвета указывают на переменные, которые ковариируют положительно (например, процент разведенных мужчин по сравнению с числом людей, живущих в бедности), темные цвета указывают на переменные, которые ковариируют отрицательно (например, процент разведенных мужчин по сравнению со средним доходом), а серые цвета указывают на переменные, которые друг с другом не связаны.

Как вы узнали в главе 4, вычисление корреляции на основе ковариации просто предусматривает шкалирование на нормы векторов. Его можно пере-

---

[1] *Редмонд М. А. и Бавея А.* Программный инструмент, управляемый данными, для обеспечения совместного обмена информацией между полицейскими департаментами (M. A. Redmond and A. Baveja, "A Data-Driven Software Tool for Enabling Cooperative Information Sharing Among Police Departments," European Journal of Operational Research 141 (2002): 660–678).

ложить в матричное уравнение, которое позволит вычислять матрицу корреляций данных без циклов `for`. Упражнение 7.1 и упражнение 7.2 познакомят с процедурой. Как я писал в главе 4, прежде чем переходить к следующему разделу, призываю вас проработать эти упражнения.

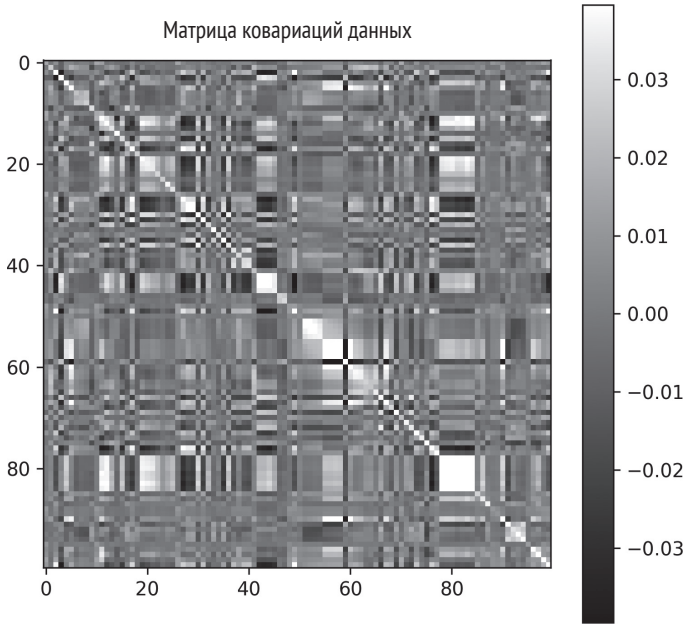


Рис. 7.1 ❖ Матрица ковариаций данных

Заключительное примечание: в NumPy есть функции вычисления матриц ковариаций и корреляций (соответственно, `np.cov()` и `np.corrcoef()`). На практике использовать эти функции удобнее, чем писать исходный код самостоятельно. Но – как всегда в этой книге – я хотел бы, чтобы вы поняли математику и механизмы, которые реализованы в этих функциях. Следовательно, в этих упражнениях вы должны реализовать ковариацию как прямое переложение формул вместо вызова функций NumPy.

Геометрические преобразования посредством умножения матриц на векторы

В главе 5 упоминалось, что одной из целей умножения матрицы на вектор является применение геометрического преобразования к множеству координат. В данном разделе вы это увидите на двумерных статичных изображениях и в анимации. Попутно вы узнаете о матрицах чистого поворота и о создании анимации данных на Python.

«Матрица чистого поворота» поворачивает вектор, сохраняя при этом его длину. Подумайте о стрелках аналоговых часов: с течением времени стрелки вращаются, но не меняются по длине. Двумерная матрица поворота выражается как

$$\mathbf{T} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}.$$

Матрица чистого поворота является примером *ортогональной матрицы*. Я опишу ортогональные матрицы подробнее в главе 9, но хотел бы указать, что столбцы матрицы **T** ортогональны (их точечное произведение равно $\cos(\theta)\sin(\theta) - \sin(\theta)\cos(\theta)$) и являются единичными векторами (вспомним тригонометрическое тождество, что $\cos^2(\theta) + \sin^2(\theta) = 1$).

Для того чтобы применить эту трансформационную матрицу, надо установить θ равной некоторому углу поворота по часовой стрелке, а затем умножить матрицу **T** на матрицу геометрических точек $2\times T$, где каждый столбец в этой матрице содержит координаты (X,Y) по каждой из T точек данных. Например, $\theta = 0$ не изменит местоположения точек (потому что $\theta = 0$ означает $\mathbf{T} = \mathbf{I}$); $\theta = \pi/2$ повернет точки на 90° вокруг начала координат.

В качестве простого примера возьмем множество точек, выровненных по вертикальной линии, и эффект умножения этих координат на **T**. На рис. 7.2 я задал $\theta = \pi/5$.

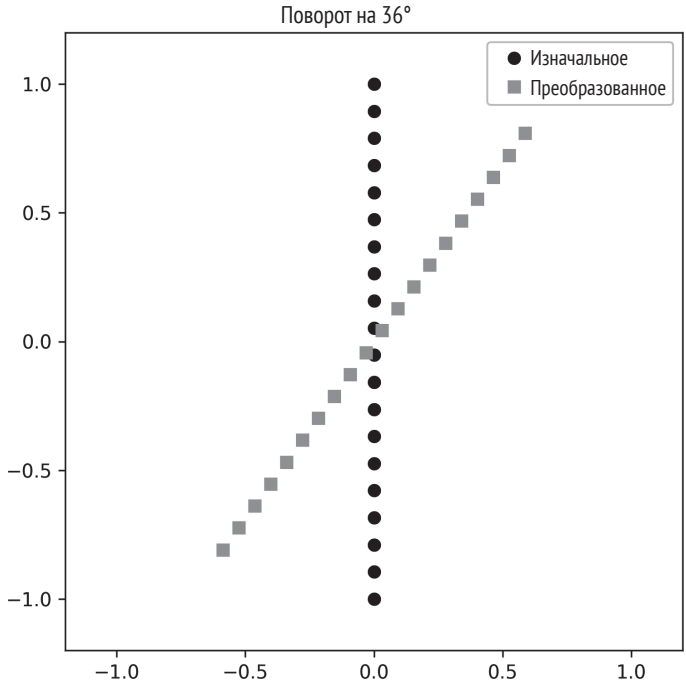


Рис. 7.2 ❖ Поворот точек вокруг начала координат с помощью матрицы чистого поворота

Прежде чем продолжить работу с этим разделом, пожалуйста, ознакомьтесь с онлайновым исходным кодом, которым генерируется этот рисунок. Убедитесь, что понимаете, как написанная выше математика переводится в исходный код, и найдите минутку, чтобы поэкспериментировать с разными углами поворота. Вы также можете попробовать выяснить, как сделать поворот против часовой стрелки вместо поворота по часовой стрелке; ответ находится в сноске[1].

Давайте сделаем обследование поворотов более захватывающим, применив «нечистые» повороты (то есть растягивание и поворот, а не только поворот) и анимировав преобразования. В частности, мы будем плавно корректировать трансформационную матрицу в каждом кадре фильма.

Анимации на Python создаются несколькими способами; применяемый здесь метод предусматривает определение функции Python, которая создает содержимое рисунка для каждого кадра фильма, затем вызывает процедуру библиотеки `matplotlib`, чтобы выполнять эту функцию на каждой итерации фильма.

Я называю этот фильм «Шаткий круг». Окружности определяются множеством точек cos(θ) и sin(θ) для вектора углов θ, которые варьируются от 0 до 2π. Трансформационная матрица задана в следующем виде:

$$\mathbf{T} = \begin{bmatrix} 1 & 1-\varphi \\ 0 & 1 \end{bmatrix}.$$

Почему я выбрал именно эти значения и как интерпретировать трансформационную матрицу? В общем случае диагональные элементы шкалируют координаты по осям *x* и *y*, тогда как внедиагональные элементы растягивают обе оси. Точные значения в приведенной выше матрице были выбраны путем подбора чисел до тех пор, пока я не нашел что-то, что, по моему мнению, выглядело замечательно. Позже, в упражнениях, у вас будет возможность обследовать эффекты изменения трансформационной матрицы.

По ходу фильма значение φ будет плавно переходить от 1 к 0 и обратно к 1, подчиняясь формуле $\varphi = x^2, -1 \leq x \leq 1$. Обратите внимание, что $\mathbf{T} = \mathbf{I}$ при $\varphi = 1$.

Исходный код анимации данных на Python можно подразделить на три части. Первая часть состоит в настройке рисунка:

```
theta  = np.linspace(0,2*np.pi,100)
points = np.vstack((np.sin(theta),np.cos(theta)))

fig,ax = plt.subplots(1,figsize=(12,6))
plth,  = ax.plot(np.cos(x),np.sin(x),'ko')
```

Результатом работы метода `ax.plot` является переменная `plth`, представляющая собой *дескриптор*, или указатель на объект графика. Этот дескриптор позволяет обновлять место расположения точек, не перерисовывая рисунок с нуля в каждом кадре.

Вторая часть состоит в определении функции, которая обновляет оси в каждом кадре:

---

[1] Поменяйте местами знаки минус в функциях синуса.

```
def aframe(ph):
    # создать и применить трансформационную матрицу
    T = np.array([ [1,1-ph],[0,1] ])
    P = T@points

    # обновить местоположение точек
    plth.set_xdata(P[0,:])
    plth.set_ydata(P[1,:])

    return plth
```

Наконец, определяем параметр преобразования φ и вызываем функцию `matplotlib`, которая создает анимацию:

```
phi = np.linspace(-1,1-1/40,40)**2
animation.FuncAnimation(fig, aframe, phi,
                        interval=100,
                        repeat=True)
```

На рис. 7.3 показан один кадр фильма, и вы можете просмотреть все видео, выполнив исходный код. По общему признанию, этот фильм вряд ли получит какие-либо награды, но он показывает, как умножение матриц применяется в анимации. Графика в CGI-фильмах и видеоиграх немного сложнее, потому что в них используются математические объекты, именуемые кватернионами, то есть векторы в $\mathbb{R}^4$, допускающие повороты и трансляции в 3D. Но принцип – умножение матрицы геометрических координат на трансформационную матрицу – в точности тот же.

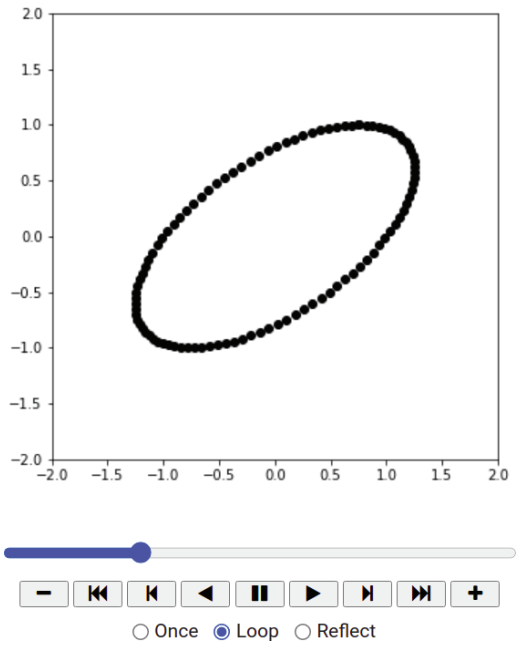


Рис. 7.3 ❖ Кадр из фильма «Шаткий круг»

Прежде чем приступать к выполнению упражнений данного раздела, рекомендую потратить немного времени на то, чтобы поиграть с исходным кодом этого раздела. В частности, измените трансформационную матрицу, установив одному из диагональных элементов значение .5 или 2, измените нижний левый внедиагональный элемент вместо верхнего правого внедиагонального элемента (либо в дополнение к нему), параметризуйте один из диагональных элементов вместо внедиагонального элемента и т. д. И вот вопрос: сможете ли вы выяснить, как заставить круг качаться влево, а не вправо? Ответ содержится в сноске[1].

Обнаружение признаков изображения

В данном разделе я познакомлю вас с фильтрацией изображений, то есть механизмом обнаружения признаков изображения. Фильтрация изображений на самом деле является расширением фильтра временного ряда, так что ознакомление с главой 4 пойдет вам здесь на пользу. Напомню, что для фильтрации или обнаружения признаков в сигнале временного ряда разрабатывается ядро, а затем создается временной ряд точечных произведений между ядром и накладывающимися друг на друга сегментами сигнала.

Фильтрация изображений работает таким же образом, только в 2D, а не в 1D. Мы конструируем двумерное ядро, а затем создаем новое изображение, содержащее «точечные произведения» между ядром и накладывающимися окнами изображения.

Я написал «точечные произведения» в кавычках, как бы извиняясь, потому что здесь операция формально не совпадает с точечным произведением векторов. Вычисление одно и то же – поэлементное умножение и сумма, однако операция выполняется между двумя матрицами, поэтому реализация представляет собой адамарово умножение и суммирование по всем элементам матрицы. График A на рис. 7.4 иллюстрирует указанную процедуру. Существуют дополнительные детали свертки, – например, дополнение изображения таким образом, чтобы результат был того же размера, – о которых вы узнаете из книги по обработке изображений. Здесь же я хотел бы, чтобы вы сосредоточились на аспектах линейной алгебры, в частности на идее о том, что точечное произведение количественно определяет взаимосвязь между двумя векторами (или матрицами) и его можно применять для обнаружения признаков и фильтрации.

Прежде чем перейти к анализу, кратко объясню, как создается двумерное гауссово ядро. Двумерный гауссиан задается следующим ниже уравнением:

$$G = \exp(-(X^2 + Y^2)/\sigma).$$

Несколько замечаний об этом уравнении: exp обозначает натуральную экспоненту (константа e = 2,71828...), а exp(x) используется вместо e^x, когда

[1] Установите значение нижнего правого элемента равным –1.

экспоненциальный член – длинный. X и Y – это двумерные решетки с координатами x, y, на которых вычисляется функция. Наконец, σ – это параметр функции, который часто называют «коэффициентом масштаба», или «шириной»: меньшие его значения делают гауссиан уже, тогда как бóльшие значения делают ее шире. В настоящий момент я зафиксирую данный параметр на определенных значениях, и вы сможете обследовать последствия изменения указанного параметра в упражнении 7.6.

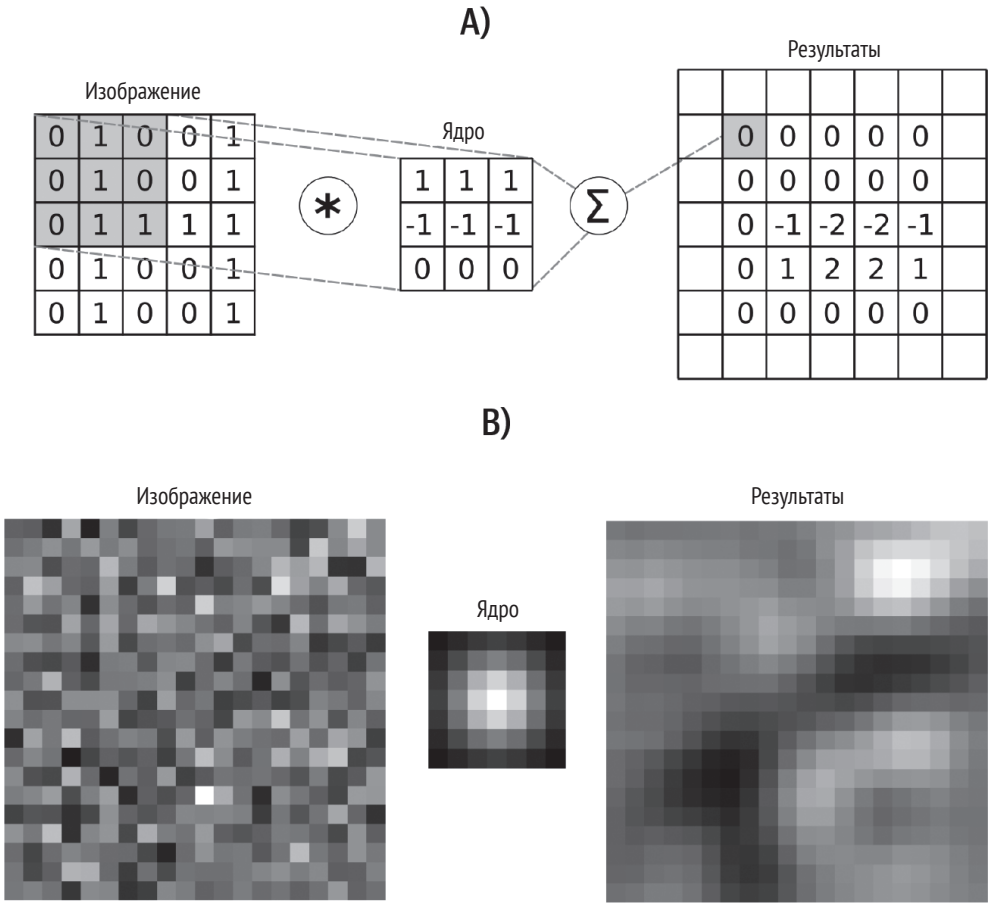


Рис. 7.4 ❖ Механизм свертки изображения

Вот как эта формула переводится в исходный код:

```
Y, X   = np.meshgrid(np.linspace(-3,3,21),
                     np.linspace(-3,3,21))
kernel = np.exp( -(X**2+Y**2) / 20 )
kernel = kernel / np.sum(kernel) # нормализовать
```

Решетки X и Y изменяются от –3 до +3 с шагом 21. Параметр ширины жестко задан равным 20. Третья строка нормализует значения в ядре таким

образом, чтобы сумма по всему ядру была равна 1. Это позволяет поддерживать изначальную шкалу данных. При надлежащей нормализации каждый шаг свертки – и, следовательно, каждый пиксел в отфильтрованном изображении – становится средневзвешенным значением окружающих пикселов, при этом веса определяются гауссианом.

Вернемся к текущей задаче: мы сгладим матрицу случайных чисел аналогично тому, как мы сглаживали временной ряд случайных чисел в главе 4. Матрица случайных чисел, гауссово ядро и результат свертки показаны на рис. 7.4.

Следующий ниже исходный код Python показывает реализацию свертки изображения. И вновь вспомните свертку временного ряда в главе 4, чтобы понять, что идея та же самая, но с дополнительным геометрическим измерением, требующим дополнительного цикла `for`:

```
for rowi in range(halfKr,imgN-halfKr):     # прокрутить строки
    for coli in range(halfKr,imgN-halfKr): # прокрутить столбцы

        # вырезать кусок изображения
        pieceOfImg = imagePad[ rowi-halfKr:rowi+halfKr+1:1,
                               coli-halfKr:coli+halfKr+1:1 ]

        # точечное произведение:
        # адамарово умножение и суммирование
        dotprod = np.sum( pieceOfImg*kernel )

        # сохранить результат для этого пиксела
        convoutput[rowi,coli] = dotprod
```

Реализация свертки в виде двойного цикла `for` на самом деле неэффективна с вычислительной точки зрения. Оказывается, что свертку можно реализовать быстрее и с меньшим объемом исходного кода в частотном диапазоне. Это происходит благодаря теореме о свертке, которая гласит, что свертка во временном (или пространственном) диапазоне равна умножению в частотном диапазоне. Полное изложение теоремы о свертке выходит за рамки этой книги; я упоминаю о ней здесь для того, чтобы обосновать рекомендацию использовать функцию SciPy `convolve2d` вместо двойного цикла `for`, в особенности в случае больших изображений.

Давайте попробуем сгладить реальный фотоснимок. Мы будем использовать фотографию музея Стеделийк в Амстердаме, которую я с любовью называю «ванна из космоса». Это изображение является трехмерной матрицей, поскольку в нем есть строки, столбцы и глубина – глубина содержит значения интенсивности пикселов из красного, зеленого и синего цветовых каналов. Указанное изображение хранится в виде матрицы размером $\mathbb{R}^{1675\times3000\times3}$. Формально она называется *тензором*, потому что является кубом, а не развернутой таблицей чисел.

Пока что мы сведем ее к двумерной матрице путем преобразования в оттенки серого. Это упрощает вычисления, хотя в этом нет необходимости. В упражнении 7.5 вы узнаете, как сглаживать трехмерное изображение. На рис. 7.5 показано изображение в оттенках серого до и после применения гауссова ядра сглаживания.

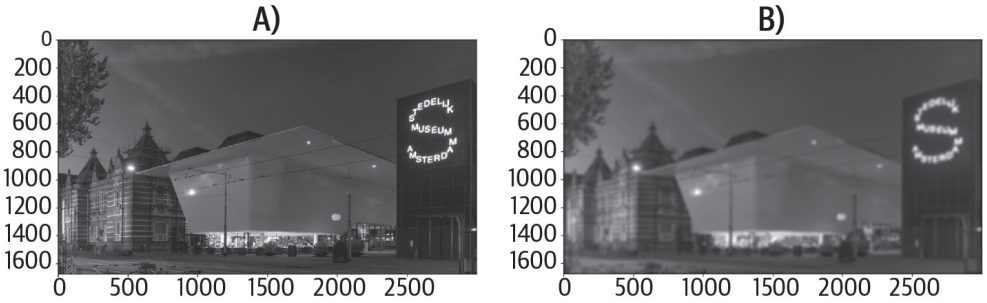


Рис. 7.5 ❖ Фотография музея-ванны до и после сносного сглаживания

В обоих этих примерах использовалось гауссово ядро. Сколько существует других ядер? Бесконечное число! В упражнении 7.7 вы протестируете два других ядра, которые применяются для выявления вертикальных и горизонтальных линий. Эти детекторы признаков широко применяются в обработке изображений (и используются нейронами мозга для обнаружения краев в узорах света, попадающих на сетчатку).

Ядра свертки изображений являются главнейшей темой в компьютерном зрении. Собственно говоря, невероятная результативность сверточных нейронных сетей (архитектуры глубокого обучения, оптимизированной под компьютерное зрение) полностью обусловлена способностью сети искусно создавать оптимальные фильтерные ядра посредством автоматического обучения.

Резюме

Буду говорить просто: вся суть в том, – повторю это еще раз – что невероятно важные и изощренные методы в науке о данных и машинном обучении построены на простых линейно-алгебраических принципах.

Упражнения по программированию

Упражнения по матрицам ковариаций и корреляций

Упражнение 7.1

В данном упражнении вы преобразуете матрицу ковариаций в матрицу корреляций. Процедура предусматривает деление каждого элемента матрицы (то есть ковариации между каждой парой переменных) на произведение дисперсий этих двух переменных.

Указанная процедура реализуется путем пред- и постпозиционного умножения матрицы ковариаций на диагональную матрицу, содержащую инвертированные стандартные отклонения каждой переменной (стандартное отклонение – это квадратный корень из дисперсии). Стандартные отклонения инвертируются, потому что нам нужно *делить* на отклонения, хотя мы будем *умножать* матрицы. Причиной пред- и постпозиционного умножения на стандартные отклонения является особое свойство пред- и постпозиционного умножения на диагональную матрицу, которое было объяснено в упражнении 5.11. Уравнение 7.1 показывает формулу.

Упражнение 7.1. Корреляция из ковариации

$$\mathbf{R} = \mathbf{SCS}.$$

C – это матрица ковариаций, а **S** – диагональная матрица взаимообратных стандартных отклонений по каждой переменной (то есть -я диагональ равна $1/\sigma_i$, где σ_i – это стандартное отклонение переменной i).

В данном упражнении ваша цель – вычислить матрицу корреляций из матрицы ковариацй, переложив уравнение 7.1 в исходный код Python. И тогда вы сможете воспроизвести рис. 7.6.

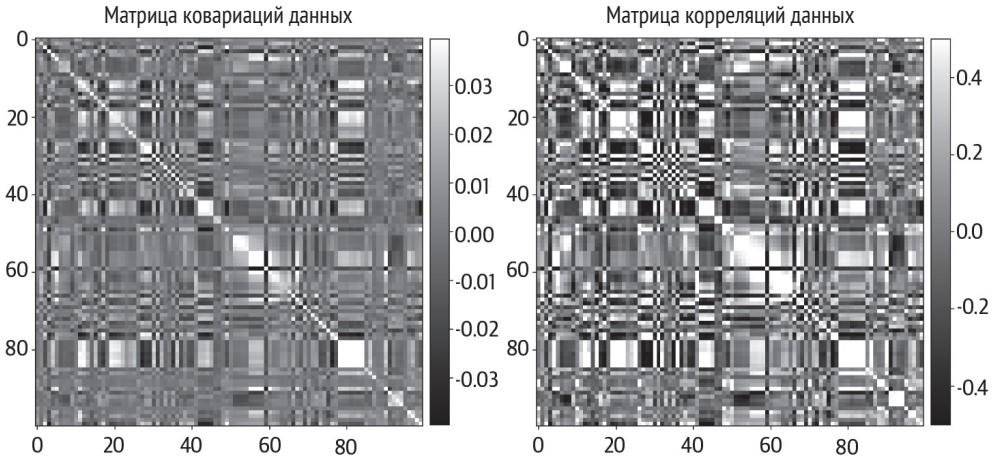


Рис. 7.6 ❖ Решение упражнения 7.1

Упражнение 7.2

В библиотеке NumPy есть функция `np.corrcoef()`, которая возвращает матрицу корреляций с учетом входной матрицы данных. Примените эту функцию, чтобы воспроизвести матрицу корреляций, созданную вами в предыдущем упражнении. Покажите обе матрицы и их разницу на рисунке, подобном рис. 7.7, чтобы подтвердить их одинаковость.

Далее проинспектируйте исходный код функции `np.corrcoef()`, выполнив `??np.corrcoef()`. В NumPy используется несколько иная реализация транслируемого деления на стандартные отклонения вместо пред- и постпозицион-

ного умножения на диагональную матрицу, состоящую из инвертированных стандартных отклонений, но вы должны суметь понять, как их реализация в исходном коде соответствует математике и исходному коду Python, который вы написали в предыдущем упражнении.

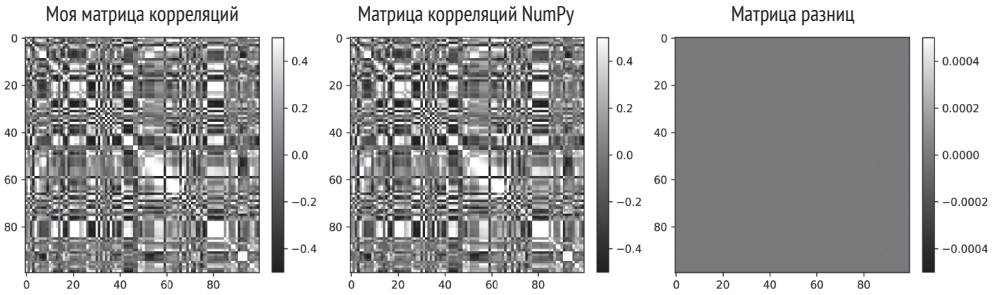


Рис. 7.7 ❖ Решение упражнения 7.2.
Обратите внимание на разницу в шкалировании цвета

Упражнения по геометрическим преобразованиям

Упражнение 7.3

Цель этого упражнения – показать точки в круге до и после применения преобразования, аналогично тому, как я показал линию до и после поворота на рис. 7.2. Примените следующую ниже матрицу преобразования, а затем создайте график, который выглядит как на рис. 7.8:

$$\mathbf{T} = \begin{bmatrix} 1 & .5 \\ 0 & .5 \end{bmatrix}.$$

Упражнение 7.4

Теперь перейдем к еще одному фильму. Я называю его «Сворачивающаяся ДНК». На рис. 7.9 показан один кадр фильма. Процедура та же, что и для «Шаткого круга», – настроить рисунок, создать функцию Python, которая применяет трансформационную матрицу к матрице координат, сообщить библиотеке `matplotlib`, что нужно создать анимацию с использованием этой функции. Используйте следующую ниже трансформационную матрицу:

$$\mathbf{T} = \begin{bmatrix} (1-\varphi/3) & 0 \\ 0 & \varphi \end{bmatrix},$$

$-1 \leq \varphi \leq 1.$

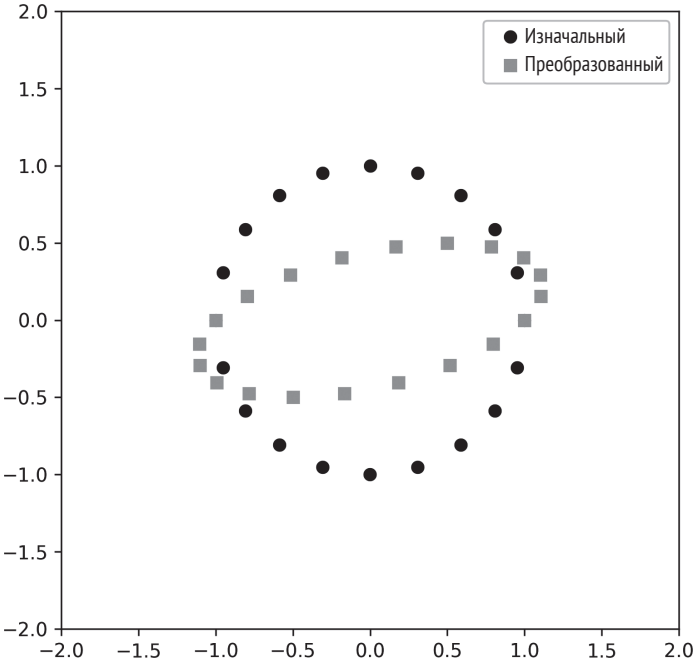


Рис. 7.8 ❖ Решение к упражнению 7.3

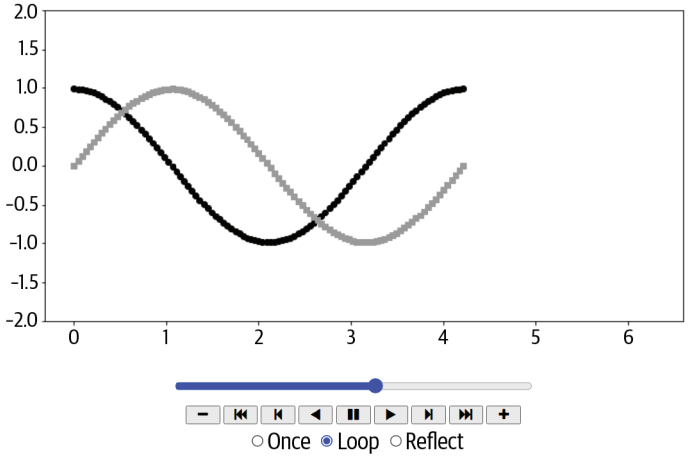


Рис. 7.9 ❖ Решение к упражнению 7.4

Упражнения по обнаружению признаков изображения

Упражнение 7.5

Сгладьте трехмерное изображение музея-ванны (если вам нужна подсказка, то обратитесь к сноске[1]).

Результат работы функции `convolve2d` имеет тип данных `float64` (вы можете убедиться в этом сами, введя `variableName.dtype`). Однако `plt.imshow` выдаст предупреждение об отсечении числовых значений, и изображение не будет отображаться надлежащим образом. Следовательно, вам нужно будет преобразовать результат свертки в `uint8`.

Упражнение 7.6

Для каждого цветового канала вовсе не нужно использовать одно и то же ядро. Измените параметр ширины гауссианы по каждому каналу в соответствии со значениями, показанными на рис. 7.10. Эффект на изображении незаметен, но разные размытия разных значений цвета придают ему немного трехмерный вид, как будто вы смотрите на красно-синий анаглиф без очков.

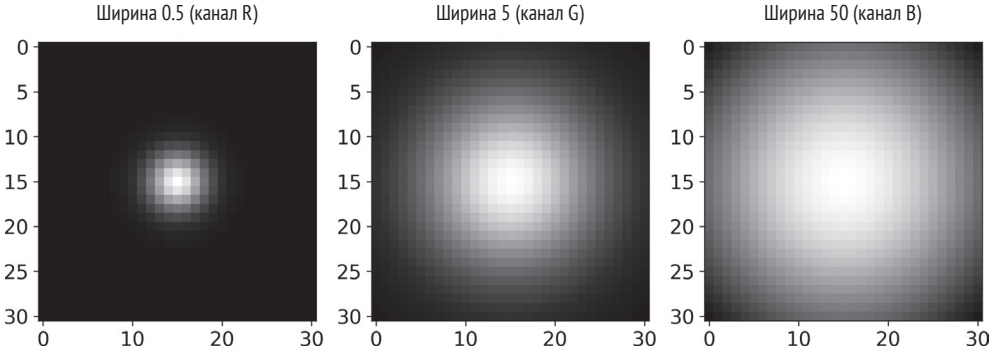


Рис. 7.10 ❖ Ядра для каждого цветового канала, используемые в упражнении 7.6

Упражнение 7.7

В техническом плане сглаживание изображения – это извлечение признаков, поскольку оно предусматривает извлечение сглаженных признаков сигнала при одновременном ослаблении резких признаков. Здесь мы изменим фильтерные ядра, чтобы решить другие задачи обнаружения признаков изображения: выявление горизонтальных и вертикальных линий.

Два ядра показаны на рис. 7.11, как и их влияние на изображение. Два ядра можно создать вручную, основываясь на их внешнем виде; они имеют размер 3×3 и содержат только числа –1, 0 и +1. Примените свертку с этими

[1] Подсказка: сглаживайте каждый цветовой канал по отдельности.

ядрами к двумерному фотоснимку в оттенках серого, чтобы создать карты признаков, показанные на рис. 7.11.

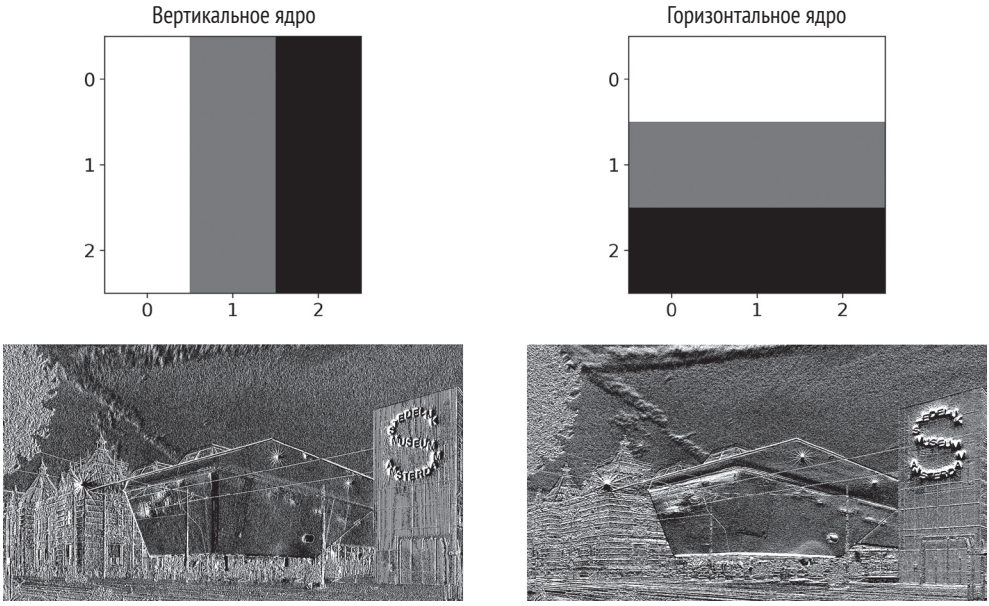


Рис. 7.11 ❖ Результаты упражнения 7.7

Глава 8

Обратные матрицы

Мы движемся к решению матричных уравнений. Матричные уравнения похожи на обычные уравнения (например, найти решение для x в $4x = 8$), но… в них есть матрицы. К этому моменту книги вы хорошо понимаете, что когда в дело вступают матрицы, все становится сложнее. Тем не менее мы должны принять эту сложность как само собой разумеющееся, потому что решение матричных уравнений – огромная часть науки о данных.

Обратная матрица играет центральную роль в решении матричных уравнений в практических приложениях, включая подгонку статистических моделей к данным (подумайте об общих линейных моделях и регрессии). К концу этой главы вы поймете, что такое обратная матрица, когда ее можно и нельзя вычислить, как ее вычислять и как ее интерпретировать.

Обратная матрица

Матрица, обратная матрице **A,** – это еще одна матрица $\mathbf{A}^{-1}$ (произносится как «обратная матрица, или инверсная матрица, матрицы **A**»), которая умножает матрицу **A**, производя единичную матрицу. Другими словами, $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$. Именно так матрица «отменяется». Еще одна концептуализация заключается в намерении преобразовать матрицу линейно в единичную матрицу; обратная матрица содержит это линейное преобразование, а матричное умножение является механизмом применения этого преобразования к матрице.

Но зачем вообще инвертировать матрицы? Нам приходится «отменять» матрицу, чтобы решать задачи, которые можно выразить в форме $\mathbf{Ax} = \mathbf{b}$, где **A** и **b** – это известные величины, и мы хотим найти решение для **x**. Решение имеет следующую ниже общую форму:

$$\begin{aligned}
\mathbf{Ax} &= \mathbf{b};\\
\mathbf{A}^{-1}\mathbf{Ax} &= \mathbf{A}^{-1}\mathbf{b};\\
\mathbf{Ix} &= \mathbf{A}^{-1}\mathbf{b};\\
\mathbf{x} &= \mathbf{A}^{-1}\mathbf{b}.
\end{aligned}$$

Она выглядит очень простой, но, как вы скоро узнаете, вычислять обратную матрицу обманчиво трудно.

Типы обратных матриц и условия обратимости

«Инвертировать матрицу» звучит так, как будто это должно работать всегда. Кому не хотелось бы инвертировать матрицу, если это удобно? К сожалению, жизнь не всегда так проста: не все матрицы можно инвертировать.

Есть три разных вида обратных матриц с разными условиями обратимости. Они представлены ниже; их подробности находятся в следующих далее разделах:

Полная обратная матрица

Она означает, что $\mathbf{A}^{-1}\mathbf{A} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$. Наличие у матрицы полной обратной матрицы обусловливается двумя свойствами: (1) она должна быть квадратной и (2) полноранговой. Каждая квадратная полноранговая матрица имеет обратную матрицу, и каждая матрица, имеющая полную обратную матрицу, является квадратной и полноранговой. Между прочим, здесь я использую термин «полная обратная матрица», чтобы отличить ее от следующих ниже двух возможностей; полная обратная матрица обычно называется просто обратной матрицей.

Односторонняя обратная матрица

Односторонняя обратная матрица может преобразовывать прямоугольную матрицу в единичную матрицу, но она работает только для одного порядка умножения. В частности, высокая матрица **T** может иметь левообратную матрицу, то есть $\mathbf{LT} = \mathbf{I}$, но $\mathbf{TL} \neq \mathbf{I}$. А широкая матрица **W** может иметь правообратную матрицу, то есть $\mathbf{WR} = \mathbf{I}$, но $\mathbf{RW} \neq \mathbf{I}$.

Неквадратная матрица имеет одностороннюю обратную матрицу только в том случае, если она имеет максимально возможный ранг. То есть высокая матрица имеет левообратную матрицу, если она имеет ранг N (полный столбцовый ранг), тогда как широкая матрица имеет правообратную матрицу, если она имеет ранг M (полный строчный ранг).

Псевдообратная матрица

Каждая матрица имеет псевдообратную матрицу, независимо от ее формы и ранга. Если матрица является квадратной полноранговой, то псевдообратная матрица равна полной обратной матрице. Точно так же если матрица является неквадратной и имеет максимально возможный ранг, то псевдообратная марица равна левообратной матрице (для высокой матрицы) или правообратной матрице (для широкой матрицы). Но рангово-пониженная матрица по-прежнему имеет псевдообратную матрицу, и в этом случае псевдообратная матрица преобразовывает сингулярную матрицу в еще одну матрицу, близкую, но не равную единичной матрице.

Матрицы, не имеющие ни полной, ни односторонней обратной матрицы, называются сингулярными, или необратимыми. Это то же самое, что пометить матрицу как *рангово-пониженную* или *рангово-дефицитную*.

Вычисление обратной матрицы

Обратная матрица звучит великолепно! Как ее вычислять? Давайте начнем, подумав о том, как вычислить обратное значение скаляра: надо просто инвертировать его число (взять его взаимообратную величину). Например, число, обратное числу 3, равно 1/3, что то же самое, что 3^{-1}. Тогда $3\times3^{-1} = 1$.

Основываясь на этом рассуждении, можно догадаться, что взятие обратной матрицы работает точно так же: инвертировать каждый элемент матрицы. Давай попробуем:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \begin{bmatrix} 1/a & 1/b \\ 1/c & 1/d \end{bmatrix}.$$

К сожалению, это не произведет желаемого результата, что легко продемонстрировать, умножив изначальную матрицу на матрицу отдельно инвертированных элементов:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}\begin{bmatrix} 1/a & 1/b \\ 1/c & 1/d \end{bmatrix} = \begin{bmatrix} 1+b/c & a/b+b/d \\ c/a+d/c & 1+c/b \end{bmatrix}.$$

Это допустимое умножение, но оно не дает единичной матрицы, и, стало быть, матрица с отдельно инвертированными элементами не является обратной матрицей.

Существует алгоритм вычисления матрицы для любой обратимой матрицы. Он долгий и утомительный (вот почему у нас есть компьютеры, которые делают всю числодробительную работу за нас!), но имеется несколько сокращенных версий для специальных матриц.

Обратная матрица матрицы 2×2

Для того чтобы инвертировать матрицу 2×2, надо поменять местами диагональные элементы, умножить внедиагональные элементы на –1 и разделить на определитель. Этот алгоритм создаст матрицу, которая преобразовывает изначальную матрицу в единичную.

Проследите за следующей ниже последовательностью уравнений:

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix};$$

$$\mathbf{A}^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix};$$

$$\begin{aligned} \mathbf{A}\mathbf{A}^{-1} &= \begin{bmatrix} a & b \\ c & d \end{bmatrix} \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \\ &= \frac{1}{ad-bc}\begin{bmatrix} ad-bc & 0 \\ 0 & ad-bc \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \end{aligned}$$

Давайте проработаем числовой пример:

$$\begin{bmatrix} 1 & 4 \\ 2 & 7 \end{bmatrix} \begin{bmatrix} 7 & -4 \\ -2 & 1 \end{bmatrix} \frac{1}{7-8} = \begin{bmatrix} (7-8) & (-4+4) \\ (14-14) & (-8+7) \end{bmatrix} \frac{1}{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Сработало замечательно.

Вычислить обратую матрицу на Python очень просто:

```
A = np.array([ [1,4],[2,7] ])
Ainv = np.linalg.inv(A)
A@Ainv
```

Вы можете подтвердить, что `A@Ainv` дает единичную матрицу, как и `Ainv@A`. Конечно же, `A*Ainv` не дает единичной матрицы, потому что операция `*` означает адамарово (поэлементное) умножение.

Матрица, обратная ей матрица и их произведение представлены на рис. 8.1.



Рис. 8.1 ❖ Обратная матрица матрицы 2×2

Давайте попробуем еще один пример:

$$\begin{bmatrix} 1 & 4 \\ 2 & 8 \end{bmatrix} \begin{bmatrix} 8 & -4 \\ -2 & 1 \end{bmatrix} \frac{1}{0} = \begin{bmatrix} (8-8) & (-4+4) \\ (16-16) & (-8+8) \end{bmatrix} \frac{1}{0} = ???.$$

В этом примере есть несколько проблем. Умножение матриц вместо $\lambda\mathbf{I}$ дает $\mathbf{0}$. Но тут есть более крупная проблема – определитель равен нулю! Математики веками предупреждали о том, что на ноль делить нельзя. Так давайте же не будем начинать это делать сейчас.

Чем отличается второй пример? Это рангово-пониженная матрица (ранг = 1). В нем показан числовой пример того, что рангово-пониженные матрицы необратимы.

Что делает Python в этом случае? Давайте выясним:

```
A = np.array([ [1,4],[2,8] ])
Ainv = np.linalg.inv(A)
A@Ainv
```

Python даже не будет пытаться вычислить результат, как это сделал я. Вместо этого он выдаст ошибку со следующим ниже сообщением[1]:

```
LinAlgError: Singular matrix
```

Рангово-пониженные матрицы не имеют обратной матрицы, и такие программы, как Python, даже не будут пытаться ее вычислить. Однако эта матрица имеет псевдообратную матрицу. Я вернусь к ней через несколько разделов.

Обратная матрица диагональной матрицы

Существует также сокращенная версия вычисления обратной матрицы квадратной диагональной матрицы. К этому сокращению ведет понимание, что произведение двух диагональных матриц – это просто умножение диагональных элементов на скаляр (обнаруженное в упражнении 5.12). Рассмотрим приведенный ниже пример; прежде чем продолжить чтение текста, попробуйте найти сокращенное вычисление обратной матрицы для диагональной матрицы:

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 4 \end{bmatrix} \begin{bmatrix} b & 0 & 0 \\ 0 & c & 0 \\ 0 & 0 & d \end{bmatrix} = \begin{bmatrix} 2b & 0 & 0 \\ 0 & 3c & 0 \\ 0 & 0 & 4d \end{bmatrix}.$$

Вы поняли, как вычислять обратную матрицу диагональной матрицы? Хитрость в том, что надо просто инвертировать каждый диагональный элемент, игнорируя внедиагональные нули. Это ясно из предыдущего примера, если задать $b = 1/2$, $c = 1/3$ и $d = 1/4$.

Что произойдет, если у вас диагональная матрица с нулем по диагонали? Инвертировать этот элемент не получится, потому что у вас будет 1/0. Таким образом, диагональная матрица хотя бы с одним нулем по диагонали не имеет обратной матрицы. (Также вспомните из главы 6, что диагональная матрица является полноранговой, только если все диагональные элементы отличны от нуля.)

---

[1] Линейно-алгебраическая ошибка: сингулярная матрица. – *Прим. перев.*

Обратная матрица диагональной важна тем, что она напрямую приводит к формуле вычисления псевдообратной матрицы. Подробнее об этом позже.

Инвертирование любой квадратной полноранговой матрицы

Честно говоря, я сомневался, стоит ли включать этот раздел в главу. Полный алгоритм инвертирования обратимой матрицы – длинный и утомительный, и вам никогда не понадобится его использовать в приложениях (вместо этого вы будете использовать функцию `np.linalg.inv` либо другие функции, вызывающие `inv`).

С другой стороны, реализация алгоритма на Python – это для вас отличная возможность попрактиковаться в переложении в исходный код Python алгоритма, описанного в уравнениях и на естественном языке. Поэтому здесь я объясню принцип работы алгоритма, не показывая никакого исходного кода. Я призываю вас реализовать алгоритм программно на языке Python, по ходу чения этого раздела, и потом вы сможете сравнить свое решение с моим в упражнении 8.2 онлайнового исходного кода.

Алгоритм вычисления обратной матрицы предусматривает четыре промежуточные матрицы, именуемые матрицей миноров, матрицей-решеткой, матрицей кофакторов и матрицей адъюгатов.

Матрица миноров

Эта матрица содержит определители подматриц. Каждый элемент $m_{i,j}$ матрицы миноров является определителем подматрицы, созданной путем исключения i-й строки и j-го столбца. На рис. 8.2 показан общий вид процедуры для матрицы 3×3.

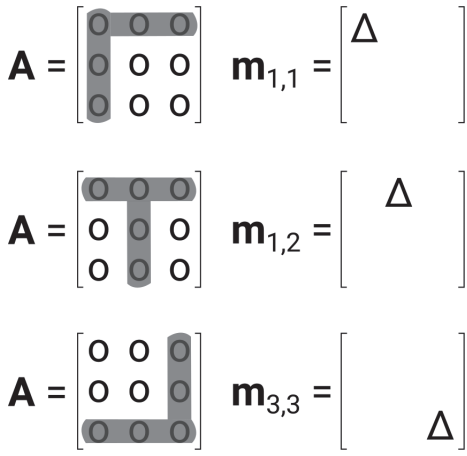


Рис. 8.2 ❖ Вычисление матрицы миноров (области, окрашенные серым цветом, удаляются, чтобы создать каждую подматрицу)

Матрица-решетка

Матрица-решетка – это шахматная доска с чередующимися значениями +1 и –1. Она рассчитывается по следующей ниже формуле:

$$g_{i,j} = -1^{i+j}.$$

При реализации этой формулы на Python будьте осмотрительны с индексацией и возведением в степень. Вы должны тщательно проверить матрицу, чтобы убедиться, что это шахматная доска с чередующимися знаками, с +1 в верхнем левом элементе.

Матрица кофакторов

Матрица кофакторов – это адамарово умножение матрицы миноров на матрицу-решетку.

Матрица адъюгатов[1]

Это транспонированная версия матрицы кофакторов, скалярно умноженной на взаимообратное значение определителя изначальной матрицы (матрицы, матричную инверсию которой вы вычисляете, а не матрицы кофакторов).

Матрица адъюгатов является инверсией изначальной матрицы.

На рис. 8.3 показаны четыре промежуточные матрицы, а также обратная матрица, возвращенная функцией `np.linalg.inv`, и единичная матрица, полученная в результате умножения изначальной матрицы на обратную ей матрицу, вычисленную в соответствии с ранее описанной процедурой. Изначальная матрица была матрицей случайных чисел.

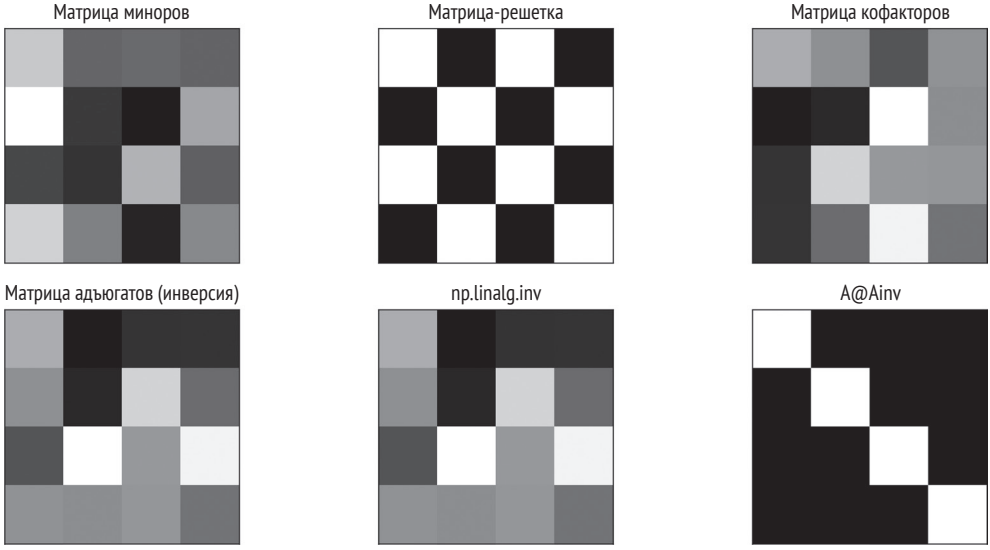


Рис. 8.3 ❖ Визуализация матриц, продуцирующих инверсию для матрицы случайных чисел

[1] Англ. *adjugate*; син. примыканий, присоединений. – *Прим. перев.*

Односторонние обратные матрицы

Высокая матрица не имеет полной обратной матрицы. То есть для матрицы $\mathbf{T}$ размера $M > N$ не существует высокой матрицы $\mathbf{T}^{-1}$ такой, что $\mathbf{T}\mathbf{T}^{-1} = \mathbf{T}^{-1}\mathbf{T} = \mathbf{I}$.

Но существует матрица $\mathbf{L}$ такая, что $\mathbf{L}\mathbf{T}^{-1}$. Сейчас наша цель – найти эту матрицу. Начнем с того, что сделаем матрицу $\mathbf{T}$ квадратной. Как превратить неквадратную матрицу в квадратную? Разумеется, вы знаете ответ – ее надо умножить на ее транспонированную версию.

Возникает следующий вопрос: что вычислять – $\mathbf{T}^{\mathrm{T}}\mathbf{T}$ или $\mathbf{T}\mathbf{T}^{\mathrm{T}}$? Обе являются квадратными... но $\mathbf{T}^{\mathrm{T}}\mathbf{T}$ – полноранговая, если $\mathbf{T}$ имеет полный столбцовый ранг. И почему это важно? Как вы уже догадались, все квадратные полноранговые матрицы имеют обратную матрицу. Прежде чем представить математический вывод левообратной матрицы, давайте продемонстрируем на Python, что высокая матрица, умноженная на ее транспонированную версию, имеет полную обратную матрицу:

```
T = np.random.randint(-10, 11, size=(40,4))
TtT = T.T @ T
TtT_inv = np.linalg.inv(TtT)
TtT_inv @ TtT
```

В приведенном выше исходном коде можно подтвердить, что последняя строка создает единичную матрицу (в пределах машинно зависимой ошибки прецизионности).

Давайте я переложу этот исходный код Python в математическое уравнение:

$$(\mathbf{T}^{\mathrm{T}}\mathbf{T})^{-1}(\mathbf{T}^{\mathrm{T}}\mathbf{T}) = \mathbf{I}.$$

Из исходного кода и формулы видно, что поскольку $\mathbf{T}^{\mathrm{T}}\mathbf{T}$ – это не та же матрица, что и $\mathbf{T}$, $(\mathbf{T}^{\mathrm{T}}\mathbf{T})^{-1}$ *не* является обратной матрицей матрицы $\mathbf{T}$.

Но – и вот ключевой момент – мы ищем матрицу, которая умножает $\mathbf{T}$ с левой стороны, чтобы получить единичную матрицу; нас на самом деле не волнует, какие другие матрицы нужно умножить, чтобы получить эту матрицу. Тогда давайте разобьем и перегруппируем умножения матриц:

$$\mathbf{L} = (\mathbf{T}^{\mathrm{T}}\mathbf{T})^{-1}\mathbf{T}^{\mathrm{T}} = \mathbf{I};$$

$$\mathbf{L}\mathbf{T} = \mathbf{I}.$$

Эта матрица $\mathbf{L}$ является левообратной матрицей матрицы $\mathbf{T}$.

Теперь можно завершить исходный код Python вычисления левообратной матрицы и подтвердить, что он соответствует нашей спецификации. Умножьте изначальную высокую матрицу с левой стороны, чтобы получить единичную матрицу:

```
L = TtT_inv @ T.T # левообратная матрица
L@T # производит единичную матрицу
```

Вы также можете подтвердить на Python, что $\mathbf{TL}$ (то есть умножение на *лево*обратную матрицу справа) *не* дает единичную матрицу. Вот почему левообратная матрица является односторонней обратной матрицей.

На рис. 8.4 показаны высокая матрица, ее левообратная матрица и два способа умножения левообратной матрицы на матрицу. Обратите внимание, что левообратная матрица не является квадратной и что постпозиционное умножение на левообратную матрицу дает результат, который определенно не является единичной матрицей.

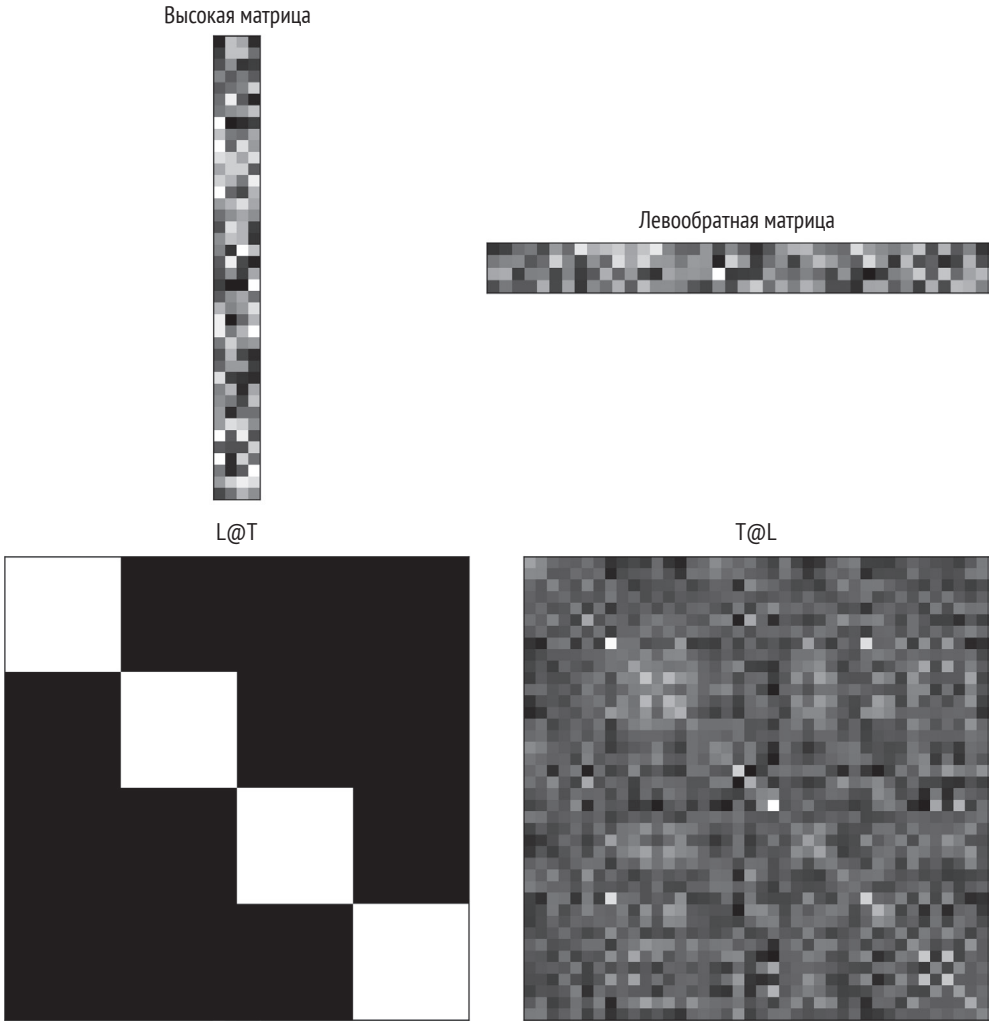


Рис. 8.4 ❖ Визуализация левообратной матрицы

Левообратная матрица имеет чрезвычайную важность. На самом деле, после того как вы узнаете о подгонке статистических моделей к данным и методе наименьших квадратов, вы повсюду будете встречать левообратную матрицу. Не будет преувеличением сказать, что левообратная матрица – один из самых важных вкладов линейной алгебры в современную человеческую цивилизацию.

Заключительное замечание о левообратной матрице, которое присутствовало в этом изложении неявно: левообратная матрица определена только для высоких матриц с полным столцовым рангом. Матрица размера $M > N$ ранга $r < N$ не имеет левообратной матрицы. Почему? Ответ – в сноске[1].

Теперь вы научились вычислять левообратную матрицу. А что насчет правообратной? Я отказываюсь учить вас ее вычислять! И не потому, что скрываю от вас тайное знание, и уж точно не потому, что вы мне не нравитесь. Наоборот, я хочу, чтобы вы бросили вызов самому себе, математически выведя правообратную матрицу и продемонстрировав ее в исходном коде с использованием Python. В упражнении 8.4 есть несколько подсказок, если они вам понадобятся, либо вы можете попробовать разобраться с этим вопросом сейчас, перед тем как переходить к следующему разделу.

Уникальность обратной матрицы

Обратная матрица уникальна, а это означает, что если матрица имеет обратную матрицу, то она имеет ровно одну обратную матрицу. Не может быть двух матриц $\mathbf{B}$ и $\mathbf{C}$ таких, что $\mathbf{AB} = \mathbf{I}$ и $\mathbf{AC} = \mathbf{I}$ и $\mathbf{B} \neq \mathbf{C}$.

Есть несколько доказательств этого утверждения. То, которое я покажу, основано на технике, именуемой доказательством путем отрицания. Оно означает, что мы пытаемся, но не можем доказать ложное утверждение, тем самым доказывая правильное утверждение. В данном случае мы начинаем с трех допущений:

1) матрица $\mathbf{A}$ обратима;
2) матрицы $\mathbf{B}$ и $\mathbf{C}$ – обратные матрицы матрице $\mathbf{A}$;
3) матрицы $\mathbf{B}$ и $\mathbf{C}$ различны, то есть $\mathbf{B} \neq \mathbf{C}$.

Следуйте каждому выражению слева направо и обратите внимание, что каждое последующее выражение основано на добавлении или удалении единичной матрицы, выраженной как матрица, умноженная на обратную ей матрицу:

$$\mathbf{C} = \mathbf{CI} = \mathbf{CAB} = \mathbf{IB} = \mathbf{B}.$$

Все утверждения эквивалентны, и, значит, первое и последнее выражения эквивалентны, и, стало быть, наше допущение о том, что $\mathbf{B} \neq \mathbf{C}$, ложно. Вывод: любые две матрицы, которые претендуют на то, чтобы быть обратными матрицами одной и той же матрицы, эквивалентны. Другими словами, обратимая матрица имеет ровно одну обратную матрицу.

---

[1] Потому что $\mathbf{T}^{\mathrm{T}}\mathbf{T}$ является рангово-пониженной и, следовательно, инвертировать ее невозможно.

Псевдообратная матрица Мура–Пенроуза

Как я писал ранее, преобразовать рангово-пониженную матрицу в единичную матрицу путем умножения матриц просто невозможно. Это означает, что рангово-пониженные матрицы не имеют ни полной, ни односторонней обратной матрицы. Но вот сингулярные матрицы имеют псевдообратные матрицы. Псевдообратные матрицы – это трансформационные матрицы, приближающие матрицу к единичной матрице.

Термин *псевдобратные матрицы* во множественном числе – это не опечатка: в отличие от уникальности полной обратной матрицы, псевдообратная матрица не уникальна. Рангово-пониженная матрица имеет бесконечное число псевдообратных матриц. Но некоторые из них лучше, чем другие, и на самом деле обсуждать стоит только одну псевдообратную матрицу, потому что она, скорее всего, будет единственной, которую вы когда-либо будете использовать.

Она называется *псевдообратной матрицей Мура–Пенроуза*, иногда сокращенно именуемой псевдообратной матрицей MP. Но поскольку она, безусловно, является наиболее часто используемой псевдообратной матрицей, можно всегда исходить из того, что термин псевдообратная матрица относится к псевдообратной матрице Мура–Пенроуза.

Следующая ниже матрица является псевдообратной матрицей сингулярной матрицы, которую вы встречали ранее в этой главе. Первая строка показывает псевдообратную матрицу матрицы, а вторая строка – произведение матрицы и псевдообратной ей матрицы:

$$\begin{bmatrix}1 & 4\\2 & 8\end{bmatrix}^{\dagger} = \frac{1}{85}\begin{bmatrix}1 & 2\\4 & 8\end{bmatrix};$$

$$\begin{bmatrix}1 & 4\\2 & 8\end{bmatrix}\frac{1}{85}\begin{bmatrix}1 & 2\\4 & 8\end{bmatrix} = \begin{bmatrix}.2 & .4\\.4 & .8\end{bmatrix}.$$

(Шкалирующий коэффициент 85 был извлечен, чтобы облегчить визуальный осмотр матрицы.)

Псевдообратная матрица обозначается надстрочным крестиком, знаком плюс либо звездочкой: $\mathbf{A}^{\dagger}$, $\mathbf{A}^{+}$ или $\mathbf{A}^{*}$.

В Python получение псевдообратной матрицы реализовано в функции `np.linalg.pinv`. Следующий ниже исходый код вычисляет псевдообратную матрицу сингулярной матрицы, для которой функция `np.linalg.inv` выдала сообщение об ошибке:

```
A = np.array([ [1,4],[2,8] ])
Apinv = np.linalg.pinv(A)
A@Apinv
```

Каков алгоритм вычисления псевдообратной матрицы? Он непостижимо труден либо интуитивно понятен; все зависит от вашего понимания сингулярного разложения. Я кратко объясню вычисление псевдообратной мат-

рицы, но если вы его не поймете, то, пожалуйста, не беспокойтесь: обещаю, что к концу главы 13 данный алгоритм станет интуитивно понятным. Для того чтобы вычислить псевдообратную матрицу, надо взять SVD матрицы, инвертировать ненулевые сингулярные числа без изменения сингулярных векторов и реконструировать матрицу путем умножения $\mathbf{U}\mathbf{\Sigma}^{+}\mathbf{V}^{\mathrm{T}}$.

Численная стабильность обратной матрицы

При вычислении обратной матрицы задействуется много FLOP'ов[1] (операций с плавающей точкой), включая большое число определителей. В главе 6 вы узнали, что определитель матрицы может быть численно нестабильным, поэтому вычисление *многочисленных* определителей приводит к численным неточностям, которые могут накапливаться и вызывать значительные проблемы при работе с большими матрицами.

По этой причине в низкоуровневых библиотеках, в которых реализованы численные вычисления (таких как LAPACK), обычно, когда это возможно, стараются избегать явного инвертирования матриц либо разлагают матрицы на произведение других матриц, которые являются более численно стабильными (например, путем QR-разложения, о котором вы узнаете в главе 9).

Матрицы, числовые значения которых находятся примерно в одном диапазоне, тяготеют к большей стабильности (хотя это и не гарантируется), поэтому с матрицами случайных чисел легко работать. Но матрицы с большим диапазоном числовых значений имеют высокий риск численной нестабильности. «Диапазон числовых значений» более формально определяется как кондиционное число матрицы, которое представляет собой отношение наибольшего сингулярного числа к наименьшему. Подробнее о кондиционном числе вы узнаете в главе 14; пока же достаточно сказать, что кондиционное число является мерой разброса числовых значений в матрице[2].

Примером численно нестабильной матрицы является матрица Гильберта. Каждый элемент в матрице Гильберта определяется простой формулой, показанной в уравнении 8.1.

Уравнение 8.1. Формула создания матрицы Гильберта. i и j – это индексы строк и столбцов

$$h_{i,j} = \frac{1}{i + j - 1}.$$

---

[1] Англ. *floating-point operations*. – *Прим. перев.*

[2] Англ. *condition number*; син. число обусловленности матрицы, кондиция матрицы. В частности, кондиционное число матрицы определяет степень чувствительности ответа к возмущениям во входных данных и к ошибкам округления, допущенным в процессе решения. – *Прим. перев.*

Вот пример матрицы Гильберта 3×3:

$$\begin{bmatrix} 1 & 1/2 & 1/3 \\ 1/2 & 1/3 & 1/4 \\ 1/3 & 1/4 & 1/5 \end{bmatrix}.$$

По мере того как матрица становится все больше, диапазон числовых значений увеличивается. Как следствие рассчитываемая компьютером матрица Гильберта быстро становится рангово-дефицитной. Даже полноранговые матрицы Гильберта имеют обратные матрицы в совершенно другом числовом диапазоне. Это проиллюстрировано на рис. 8.5, где показаны матрица Гильберта 5×5, обратная ей матрица и их произведение.

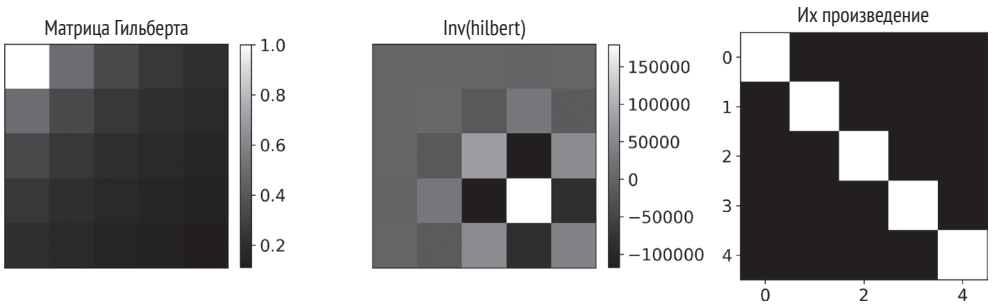


Рис. 8.5 ❖ Матрица Гильберта, обратная ей матрица и их произведение

Матрица произведения, безусловно, выглядит как единичная матрица, но в упражнении 8.9 вы обнаружите, что ее внешний вид бывает обманчивым, а ошибки округления резко возрастают с увеличением размера матрицы.

Геометрическая интерпретация обратной матрицы

В главах 6 и 7 вы научились концептуализировать умножение матрицы на вектор как геометрическое преобразование вектора или множества точек.

Следуя в этом направлении, обратную матрицу можно трактовать как откат назад геометрического преобразования, вызванного матричным умножением. На рис. 8.6 показан пример, вытекающий из рис. 7.8; я просто умножил преобразованные геометрические координаты на обратную матрицу трансформационной матрицы.

Этот геометрический эффект перестанет быть удивительным после обследования математики. В следующих ниже уравнениях $\mathbf{P}$ – это матрица $2\times N$ изначальных геометрических координат, $\mathbf{T}$ – трансформационная матрица,

Q – матрица преобразованных координат, а **U** – матрица обратно преобразованных координат:

$$\mathbf{Q} = \mathbf{TP};$$

$$\mathbf{U} = \mathbf{T}^{-1}\mathbf{Q};$$

$$\mathbf{U} = \mathbf{T}^{-1}\mathbf{TP}.$$

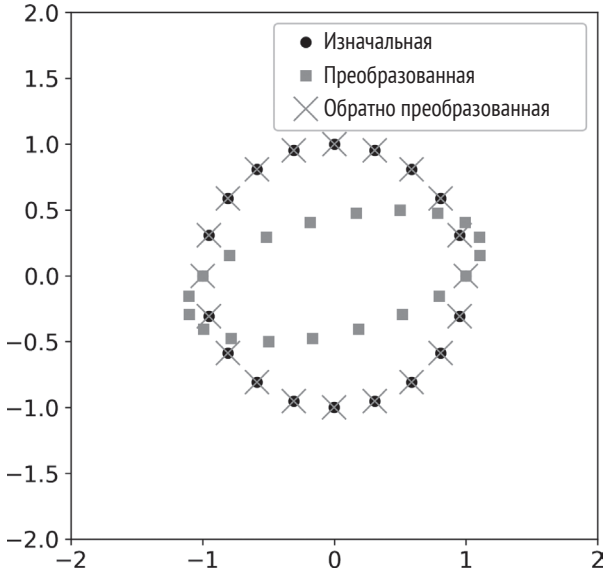


Рис. 8.6 ❖ Обратная матрица откатывает назад геометрическое преобразование. Исходный код создания этого рисунка является частью упражнения 8.8

Хотя этот результат и не удивителен, надеюсь, что он поможет сформировать некоторое геометрическое понимание предназначения обратной матрицы на интуитивном уровне: откат назад вызванного матрицей результата преобразования. Эта интерпретация пригодится, когда вы узнаете о диагонализации матрицы посредством собственного разложения.

Указанная геометрическая интерпретация также дает некоторое интуитивное понимание причины, по которой рангово-пониженная матрица не имеет обратной матрицы: геометрический эффект преобразования с помощью сингулярной матрицы заключается в том, что по меньшей мере одно измерение сглаживается. Когда измерение сглажено, это сглаживание нельзя отменить, точно так же, как невозможно увидеть свою спину, глядя в зеркало[1].

[1] Где-то здесь должна быть остроумная аналогия с Плосколандией, которую я не совсем в состоянии красноречиво высказать. Лучше уж прочтите книгу «Плосколандия» (Flatland).

Резюме

Мне очень понравилось писать эту главу, и я надеюсь, что вам понравилось учиться по ней. Вот краткое изложение ключевых выводов, которые следует вынести из этой главы.

- Обратная матрица – это матрица, которая преобразовывает максимально-ранговую матрицу в единичную матрицу путем умножения матриц. Обратная матрица имеет много предназначений, включая переставление матриц в уравнении (например, отыскание решения для $\mathbf{x}$ в $\mathbf{Ax} = \mathbf{b}$).
- Полноранговая квадратная матрица имеет полную обратную матрицу, высокая матрица полного столбцового ранга имеет левообратную матрицу, а широкая матрица полного строчного ранга имеет правообратную матрицу. Линейно преобразовать рангово-пониженные матрицы в единичную матрицу невозможно, но у них есть псевдообратная матрица, которая преобразовывает матрицу в еще одну матрицу, более близкую к единичной матрице.
- Обратная матрица является уникальной – если матрицу можно линейно преобразовать в единичную, то это можно сделать только одним способом.
- Вычисление обратных матриц некоторых видов, включая матрицы 2×2 и диагональные матрицы, предусматривает использование нескольких сокращенных способов. Указанные способы представляют собой упрощения полной формулы вычисления обратной матрицы.
- Из-за риска ошибок численной прецизионности в алгоритмах производственного уровня предпочитается избегать явного инвертирования матриц, либо матрица в них разлагается на другие матрицы, которые можно инвертировать с большей численной стабильностью.

Упражнения по программированию

Упражнение 8.1

Обратная матрица обратной матрицы будет изначальной матрицей; другими словами, $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$. Это аналогично следующему: $1/1/a = a$. Проиллюстрируйте это на Python.

Упражнение 8.2

Реализуйте полный алгоритм, описанный в разделе «Инвертирование любой квадратной полноранговой матрицы» на стр. 149, и воспроизведите рис. 8.3. Разумеется, из-за случайных чисел ваши матрицы будут выглядеть иначе, чем на рис. 8.3, хотя матрица-решетка и единичная матрица останутся теми же.

Упражнение 8.3

Реализуйте алгоритм генерирования полной обратной матрицы вручную для матрицы 2×2, используя матричные элементы *a*, *b*, *c* и *d*. Обычно в этой книге я не даю задания, решаемые вручную, но данное упражнение покажет, откуда берется сокращенный вариант. Вспомните, что определителем скаляра является его абсолютное значение.

Упражнение 8.4

Выведите правообратную матрицу для широких матриц математически, следуя логике, которая позволила обнаружить левообратную матрицу. Затем воспроизведите рис. 8.4 для широкой матрицы. (Подсказка: начните с исходного кода для левообратной матрицы и при необходимости откорректируйте его.)

Упражнение 8.5

Проиллюстрируйте на Python, что псевдообратная матрица (посредством функции `np.linalg.pinv`) равна полной обратной матрице (посредством функции `np.linalg.inv`) обратимой матрицы. Затем проиллюстрируйте, что псевдообратная матрица равна левообратной матрице высокой матрицы с полным столбцовым рангом и что она равна правообратной матрице широкой матрицы с полным строчным рангом.

Упражнение 8.6

Правило LIVE EVIL применимо к обратной матрице умноженных матриц. Проверьте эту применимость в исходном коде, создав две квадратные полноранговые матрицы **A** и **B**, а затем примените евклидово расстояние, чтобы сравнить

1) $(\mathbf{AB})^{-1}$,
2) $\mathbf{A}^{-1}\mathbf{B}^{-1}$ и
3) $\mathbf{B}^{-1}\mathbf{A}^{-1}$.

Перед тем как приступать к программированию, сделайте предсказание о том, какие результаты будут эквивалентными. Распечатайте свои результаты, используя форматирование, подобное приведенному ниже (я опустил свои результаты, чтобы не влиять на ваше решение!):

```
Расстояние между (AB)^-1 и (A^-1)(B^-1) равно ___
Расстояние между (AB)^-1 и (B^-1)(A^-1) равно ___
```

В качестве дополнительной задачи можно подтвердить, что правило LIVE EVIL применимо к более длинной цепочке матриц, например к четырем матрицам вместо двух.

$(\mathbf{T}^{\mathrm{T}}\mathbf{T})^{-1}(\mathbf{T}^{\mathrm{T}}\mathbf{T}) = \mathbf{I}$.

Упражнение 8.7

Применимо ли правило LIVE EVIL к односторонней обратной матрице? То есть будет ли верно, что $(\mathbf{T}^{\mathrm{T}}\mathbf{T})^{-1} = \mathbf{T}^{\mathrm{T}}\mathbf{T}^{-1}$? Как и в предыдущем упражнении, сделайте предсказание, а затем проверьте его на Python.

Упражнение 8.8

Напишите исходный код, чтобы воспроизвести рис. 8.6. Начните с копирования исходного кода из упражнения 7.3. После воспроизвения рисунка сделайте трансформационную матрицу необратимой, задав нижний левый элемент равным 1. Что еще нужно изменить в исходном коде, чтобы избежать ошибок?

Упражнение 8.9

Это и следующее упражнения *помогут* обследовать обратную матрицу и ее риск численной нестабильности, используя матрицу Гильберта. Начните с создания матрицы Гильберта. **Напишите** функцию Python, которая на входе принимает целое число и на выходе создает матрицу Гильберта, следуя уравнению 8.1. Затем воспроизведите рис. 8.5.

Я рекомендую писать вашу функцию Python, используя двойной цикл `for` по строкам и столбцам (матричные индексы *i* и *j*), следуя математической формуле. Убедившись в точности функции, можете бросить вызов самому себе и переписать функцию без циклов `for` (подсказка: используйте внешнее произведение). Точность своей функции можно подтвердить, сравнив ее с функцией Гильберта, которая находится в библиотеке `scipy.linalg`.

Упражнение 8.10

Используя матричную функцию Гильберта, создайте матрицу Гильберта, затем вычислите обратную ей матрицу, применив функцию `np.linalg.inv`, и вычислите произведение двух матриц. Это произведение должно равняться единичной матрице, и, стало быть, евклидово расстояние между этим произведением и истинной единичной матрицей, произведенной функцией `np.eye`, должно быть равно 0 (в пределах ошибки вычислительного округления). Вычислите это евклидово расстояние.

Поместите этот исходный код в цикл `for` по диапазону размеров матриц от 3×3 до 12×12. По каждому размеру матрицы сохраняйте евклидово расстояние и кондиционное число матрицы Гильберта. Как я уже писал ранее, кондиционное число является мерой разброса числовых значений в матрице и извлекается функцией `np.linalg.cond`.

Затем повторите описанный выше исходный код, но теперь вместо матрицы Гильберта используя матрицу гауссовых случайных чисел.

Наконец, нанесите все результаты на график, как показано на рис. 8.7. Я нанес расстояние и кондиционное число в логарифмической шкале, чтобы облегчить визуальную интерпретацию.

Пожалуйста, прочувствуйте вдохновение и продолжайте обследовать линейную алгебру, используя это упражнение! Попробуйте построить матрицу Гильберта, умноженную на обратную ей матрицу (подумайте о корректировке цветовой шкалы), используя матрицы большего размера либо другие специальные матрицы, извлекая другие свойства матрицы, такие как ранг или норма, и т. д. Вы отправились в путешествие по чудесной стране линейной алгебры, полное приключений, и Python – это ковер-самолет, который несет вас над ее чудесным ландшафтом.

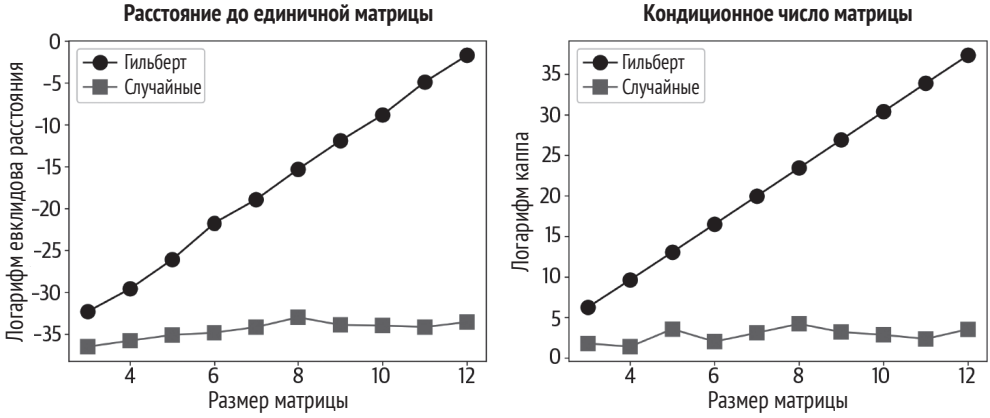


Рис. 8.7 ❖ Результаты упражнения 8.10

Глава 9

Ортогональные матрицы и QR-разложение

В этой книге вы познакомитесь с пятью главнейшими разложениями: ортогональным разложением векторов, QR-разложением, LU-разложением, собственным разложением и сингулярным разложением. Это не единственные разложения в линейной алгебре, но они наиболее важны в науке о данных и машинном обучении.

В этой главе вы познакомитесь с QR. И попутно узнаете о новом специальном типе матрицы (ортогональной). QR-разложение – это рабочая лошадка, которая приводит в движение приложения, включая получение обратной матрицы, подгонку моделей методом наименьших квадратов и получение собственного разложения. Таким образом, понимание и знакомство с QR-разложением помогут вам улучшить свои линейно-алгебраические навыки.

Ортогональные матрицы

Начну с того, что познакомлю вас с ортогональными матрицами. *Ортогональная матрица* – это специальная матрица, которая важна для нескольких разложений, включая QR, собственное разложение и сингулярное разложение. Буква **Q** часто используется для обозначения ортогональных матриц. Ортогональные матрицы обладают двумя свойствами:

Ортогональные столбцы
: Все столбцы попарно ортогональны.

Единично-нормные столбцы
: Норма (геометрическая длина) каждого столбца равна 1.

Эти два свойства транслируются в математическое выражение (вспомните, что $\langle \mathbf{a}, \mathbf{b} \rangle$ – это альтернативное обозначение точечного произведения):

$$\langle \mathbf{q}_i, \mathbf{q}_j \rangle = \begin{cases} 0, & \text{если } i \neq j \\ 1, & \text{если } i = j \end{cases}.$$

Что оно означает? Оно означает, что точечное произведение столбца с самим собой равно 1, а точечное произведение столбца с любым другим столбцом равно 0. Мы имеем большое число точечных произведений только с двумя возможными исходами. Мы можем организовать все точечные произведения среди всех пар столбцов, предпозиционно умножив матрицу на ее транспонированную версию. Вспомните, что умножение матриц определяется как точечное произведение между всеми строками левой матрицы со всеми столбцами правой матрицы; следовательно, строки $\mathbf{Q}^{\mathrm{T}}$ являются столбцами $\mathbf{Q}$.

Матричное уравнение, выражающее два ключевых свойства ортогональной матрицы, просто изумительно:

$$\mathbf{Q}^{\mathrm{T}}\mathbf{Q} = \mathbf{I}.$$

Выражение $\mathbf{Q}^{\mathrm{T}}\mathbf{Q} = \mathbf{I}$ является реально удивительным и действительно очень важным.

Почему оно имеет такую большую важность? Потому что $\mathbf{Q}^{\mathrm{T}}$ – это матрица, которая умножает матрицу $\mathbf{Q}$, производя единичную матрицу. Это то же самое определение, что и для обратной матрицы. Таким образом, обратной матрицей ортогональной матрицы является ее транспонированная версия. Это чертовски круто, потому что получать обратную матрицу утомительно трудно и она подвержена численным неточностям, тогда как транспонированную матрицу можно получить быстро и точно.

На самом ли деле такие матрицы существуют в дикой природе или это всего лишь плод воображения исследователя данных? Да, они действительно существуют. По сути дела, единичная матрица является примером ортогональной матрицы. Вот еще два примера:

$$\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad \frac{1}{3}\begin{bmatrix} 1 & 2 & 2 \\ 2 & 1 & -2 \\ -2 & 2 & -1 \end{bmatrix}.$$

Пожалуйста, найдите время, чтобы убедиться, что каждый столбец имеет единичную длину и ортогонален другим столбцам. Затем можно подтвердить на Python:

```
Q1 = np.array([ [1,-1], [1,1] ]) / np.sqrt(2)
Q2 = np.array([ [1,2,2], [2,1,-2], [-2,2,-1] ]) / 3

print( Q1.T @ Q1 )
print( Q2.T @ Q2 )
```

Оба результата являются единичной матрицей (в пределах ошибок округления порядка 10^{-15}). Что произойдет, если вычислить $\mathbf{QQ}^T$? Будет ли результат все той же единичной матрицей? Попробуйте – и узнаете[1]!

Еще одним примером ортогональной матрицы являются матрицы чистого поворота, о которых вы узнали в главе 7. Можете вернуться к исходному коду в той главе и убедиться, что трансформационная матрица, умноженная на ее транспонированную версию, является единичной матрицей, независимо от угла поворота (при условии что во всех матричных элементах используется одинаковый угол поворота). Матрицы перестановок тоже ортогональны. Матрицы перестановок используются для обмена строками матрицы; вы узнаете о них в следующей главе при изложении LU-разложения.

Как создавать такие величественные чудеса математики? Ортогональная матрица вычисляется из неортогональной матрицы посредством QR-разложения, которое, в сущности, представляет собой изощренную версию процедуры Грама–Шмидта. А как работает процедура Грама–Шмидта? По сути, это ортогональное разложение векторов, о котором вы узнали в главе 2.

Процедура Грама–Шмидта

Процедура Грама–Шмидта – это способ преобразования неортогональной матрицы в ортогональную. Указанная процедура имеет большое образовательное значение, но, к сожалению, очень мало прикладного значения. Причина, как вы уже несколько раз читали, – в численных нестабильностях, возникающих в результате многочисленных делений и умножений на крошечные числа. К счастью, есть более изощренные и численно стабильные методы QR-разложения, такие как отражения Хаусхолдера. Детали этого алгоритма выходят за рамки данной книги, но они обрабатываются вызываемыми из Python низкоуровневыми библиотеками численных вычислений.

Тем не менее я опишу процедуру Грама–Шмидта (иногда сокращенно GS или G-S), поскольку она показывает применение ортогонального разложения векторов, потому что вы собираетесь запрограммировать ее алгоритм на Python, основываясь на следующих ниже математике и описании, а также потому, что GS – это правильный способ концептуализировать принцип и причину работы QR-разложения, даже если низкоуровневая реализация немного отличается.

Матрица $\mathbf{V}$, содержащая столбцы с $\mathbf{v}_1$ по $\mathbf{v}_n$, преобразовывается в ортогональную матрицу $\mathbf{Q}$ со столбцами $\mathbf{q}_k$ согласно следующему ниже алгоритму.

Для всех векторов-столбцов в $\mathbf{V}$, начиная с первого (самого левого) и систематически двигаясь к последнему (крайнему правому):

1) ортогонализовать $\mathbf{v}_k$ ко всем предыдущим столбцам матрицы $\mathbf{Q}$, используя ортогональное векторное разложение. То есть вычислить ком-

[1] Это рассматривается подробнее в упражнении 9.1.

поненту $\mathbf{v}_k$, которая перпендикулярна $\mathbf{q}_{k-1}$, $\mathbf{q}_{k-2}$ и так далее вплоть до $\mathbf{q}_1$. Ортогонализованный вектор называется $\mathbf{v}_k^*$[1];

2) нормировать $\mathbf{v}_k^*$, приведя его к единичной длине. Теперь это $\mathbf{q}_k$, k-й столбец матрицы $\mathbf{Q}$.

Звучит просто, не правда ли? Реализация этого алгоритма в исходном коде может быть замысловатой из-за повторяющихся ортогонализаций. Но при некоторой настойчивости можно разобраться (упражнение 9.2).

QR-разложение

Процедура GS преобразовывает матрицу в ортогональную матрицу $\mathbf{Q}$. (Как я уже написал в предыдущем разделе, матрица $\mathbf{Q}$ на самом деле получается посредством серии отражений от векторной плоскости, именуемой преобразованием Хаусхолдера, но это связано с численными проблемами; GS – отличный способ концептуализировать принцип формирования матриц $\mathbf{Q}$.)

Что в звуке твоем?

«QR» в QR-разложении произносится как «кью ар»[2]. На мой взгляд, это настоящая упущенная возможность; линейную алгебру было бы интереснее изучать, если бы мы произносили ее как «QweRty-разложение». Но, к лучшему или к худшему, современные условности формируются историческим прецедентом.

Матрица $\mathbf{Q}$ очевидным образом отличается от изначальной матрицы (при условии что изначальная матрица не была ортогональной). И значит, мы потеряли информацию об этой матрице. К счастью, эту «потерянную» информацию можно легко восстановить и сохранить в другой матрице $\mathbf{R}$, которая умножает $\mathbf{Q}$[3]. Это приводит к вопросу о том, как создавать матрицу $\mathbf{R}$. На самом деле создание матрицы $\mathbf{R}$ является простым и вытекает из определения QR:

$$\mathbf{A} = \mathbf{QR};$$

$$\mathbf{Q}^T\mathbf{Q} = \mathbf{Q}^T\mathbf{QR};$$

$$\mathbf{Q}^T\mathbf{A} = \mathbf{R}.$$

Здесь вы видите всю красоту ортогональных матриц: мы можем решать матричные уравнения, не беспокоясь о вычислении обратной матрицы.

В следующем ниже исходном коде Python показано, как вычислять QR-разложение квадратной матрицы, а на рис. 9.1 показаны три матрицы:

---

[1] Первый вектор-столбец не ортогонализуется, поскольку предшествующих векторов нет; поэтому вы начинаете со следующего шага нормализации.

[2] Англ. *queue are*. – *Прим. перев.*

[3] Возможность восстановления $\mathbf{R}$ с помощью матричного умножения существует, поскольку GS – это серия линейных преобразований.

```
A = np.random.randn(6,6)
Q,R = np.linalg.qr(A)
```

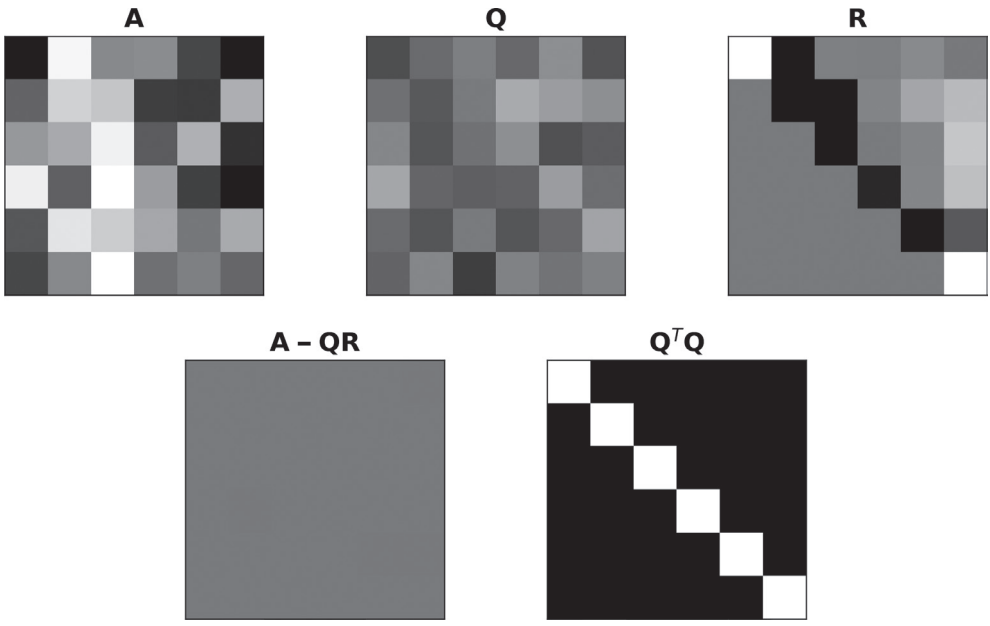


Рис. 9.1 ❖ QR-разложение матрицы случайных чисел

Несколько важных признаков QR-разложения видны на рис. 9.1, в том числе то, что **A** = **QR** (их разностью является матрица нулей), и то, что матрица **Q**, умноженная на ее транспонированую версию, дает единичную матрицу.

Посмотрите на матрицу **R**: это верхнетреугольная матрица (все элементы ниже диагонали равны нулю). Вряд ли это произошло случайно, учитывая, что мы начали со случайной матрицы. На самом деле матрица **R** *всегда* будет верхнетреугольной. Для того чтобы понять причину, нужно подумать об алгоритме GS и организации точечных произведений в умножении матриц. В следующем далее разделе я объясню причину, по которой матрица **R** является верхнетреугольной; но до этого хотел бы, чтобы вы придумали ответ.

Размеры матриц Q и R

Размеры матриц **Q** и **R** зависят от размера подлежащей разложению матрицы **A** и от того, является ли QR-разложение «экономным» (также именуемым «сокращенным», или «компактным») либо «полным». На рис. 9.2 представлен обзор всех возможных размеров.

Экономное QR-разложение по сравнению с полным применимо только к высоким матрицам. Вопрос в следующем: для высокой матрицы ($M > N$) мы создаем матрицу **Q** с N столбцами или же с M столбцами? Первый вариант

называется *экономным*, или *сокращенным*[1], и дает высокую матрицу **Q**, а последний вариант называется *полным*[2] и дает квадратную матрицу **Q**.

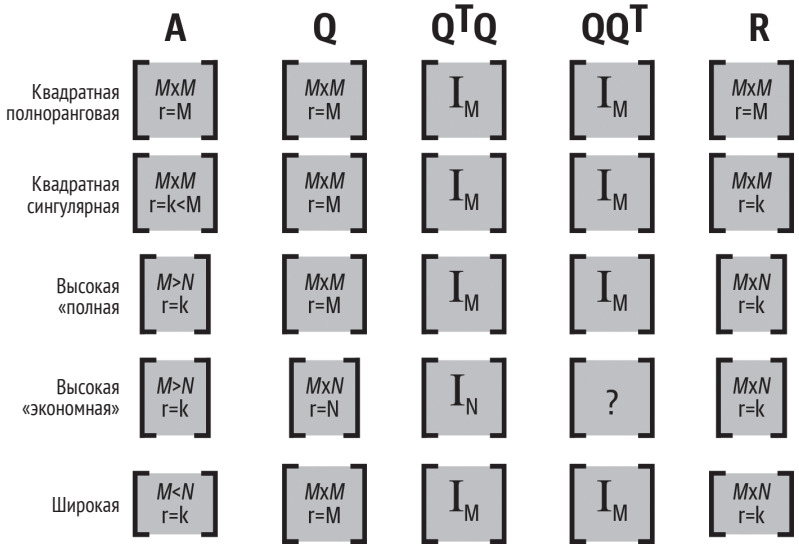


Рис. 9.2 ❖ Размеры матриц Q и R в зависимости от размера матрицы A. Символ «?» указывает на то, что матричные элементы зависят от значений в матрице A, т. е. это не единичная матрица

Возможно, покажется удивительным, что матрица **Q** может быть квадратной, когда матрица **A** – высокая (другими словами, **Q** может иметь больше столбцов, чем **A**): откуда берутся дополнительные столбцы? На самом деле ортогональные векторы можно создавать «из воздуха». Рассмотрим следующий ниже пример на Python:

```
A = np.array([ [1,-1] ]).T
Q,R = np.linalg.qr(A,'complete')
Q*np.sqrt(2) # шкалируется с помощью sqrt(2),
             # чтобы получить целые числа

>> array([[-1.,  1.],
          [ 1.,  1.]])
```

Обратите внимание на опциональный второй аргумент `'complete'`, который производит полное QR-разложение. Задание этого аргумента равным `'reduced'`, который используется по умолчанию, дает экономное QR-разложение, в котором матрица **Q** имеет тот же размер, что и матрица **A**.

Поскольку из матрицы с N столбцами можно создать более $M > N$ ортогональных векторов, ранг матрицы **Q** всегда является максимально возможным

[1] Англ. *economy*, *reduced*. – *Прим. перев.*

[2] Англ. *full*, *complete*. – *Прим. перев.*

рангом, который равен M для всех квадратных матриц $\mathbf{Q}$ и N для экономной матрицы $\mathbf{Q}$. Ранг матрицы $\mathbf{R}$ совпадает с рангом матрицы $\mathbf{A}$.

Разница в ранге между матрицами $\mathbf{Q}$ и $\mathbf{A}$ в результате ортогонализации означает, что $\mathbf{Q}$ охватывает все $\mathbb{R}^M$, даже если столбцовое пространство матрицы $\mathbf{A}$ является лишь низкоразмерным подпространством $\mathbb{R}^M$. Этот факт является ключевым в том, почему сингулярное разложение выступает таким удобным для выявления свойств матрицы, включая ее ранг и нуль-пространство. Еще одна причина с нетерпением ждать информации об SVD в главе 14!

Примечание об уникальности: QR-разложение не является уникальным для всех размеров и рангов матриц. Это означает, что можно получить $\mathbf{A} = \mathbf{Q}_1\mathbf{R}_1$ и $\mathbf{A} = \mathbf{Q}_2\mathbf{R}_2$, где $\mathbf{Q}_1 \neq \mathbf{Q}_2$. Однако все результаты QR-разложения обладают одинаковыми свойствами, описанными в этом разделе. QR-разложение можно сделать уникальным при наличии дополнительных ограничений (например, положительных значений на диагоналях матрицы $\mathbf{R}$), хотя в большинстве случаев это необязательно и не реализовано ни в Python, ни в MATLAB. Вы увидите эту неуникальность в упражнении 9.2 при сравнении GS с QR.

Почему матрица R является верхнетреугольной

Надеюсь, вы серьезно задумались над данным вопросом. В QR-разложении это сложный момент, поэтому если вы не смогли разобраться в нем самостоятельно, то, пожалуйста, прочитайте следующие несколько абзацев, а затем оторвите взгляд от книги/экрана и заново выведите аргумент математически.

Начну с того, что напомню три факта:

- матрица $\mathbf{R}$ получается из формулы $\mathbf{Q}^{\mathrm{T}}\mathbf{A} = \mathbf{R}$;
- нижний треугольник матрицы произведения содержит точечные произведения между *последующими* строками левой матрицы и *предшествующими* столбцами правой матрицы;
- строки матрицы $\mathbf{Q}^{\mathrm{T}}$ являются столбцами матрицы $\mathbf{Q}$.

Соединяя все вместе: поскольку ортогонализация проходит по столбцам слева направо, последующие столбцы в матрице $\mathbf{Q}$ ортогонализируются к *предшествующим* столбцам матрицы $\mathbf{A}$. Следовательно, нижний треугольник матрицы $\mathbf{R}$ получается из пар ортогонализированных векторов. Напротив, *предшествующие* столбцы в $\mathbf{Q}$ не ортогонализируются к *последующим* столбцам матрицы $\mathbf{A}$, поэтому их точечные произведения ожидаемо не будут равны нулю.

Заключительный комментарий: если бы столбцы i и j матрицы $\mathbf{A}$ уже были ортогональны, то соответствующий (i, j)-й элемент в $\mathbf{R}$ был бы равен нулю. На самом деле если вычислить QR-разложение ортогональной матрицы, то $\mathbf{R}$ будет диагональной матрицей, в которой диагональные элементы являются нормами каждого столбца в матрице $\mathbf{A}$. Это означает, что если $\mathbf{A} = \mathbf{Q}$, то $\mathbf{R} = \mathbf{I}$, что очевидно из уравнения, вычисленного для $\mathbf{R}$. Вы обследуете этот вопрос в упражнении 9.3.

QR и обратные матрицы

QR-разложение обеспечивает численно более стабильный способ вычисления обратной матрицы.

Давайте начнем с написания формулы QR-разложения и инвертирования обеих частей уравнения (обратите внимание на применение правила LIVE EVIL):

$$\mathbf{A} = \mathbf{QR};$$
$$\mathbf{A}^{-1} = (\mathbf{QR})^{-1};$$
$$\mathbf{A}^{-1} = \mathbf{R}^{-1}\mathbf{Q}^{-1};$$
$$\mathbf{A}^{-1} = \mathbf{R}^{-1}\mathbf{Q}^{T}.$$

Таким образом, обратную матрицу матрицы **A** можно получить как инверсию матрицы **R**, умноженную на транспонированную версию матрицы **Q**. Матрица **Q** численно устойчива из-за алгоритма отражения Хаусхолдера, а матрица **R** численно устойчива, потому что она является просто результатом матричного умножения.

Теперь нам все еще нужно инвертировать **R** в явной форме, но инвертирование треугольных матриц численно является очень стабильным, если выполняется с помощью процедуры, именуемой обратной подстановкой. Вы узнаете о ней подробнее в следующей главе, но ключевой момент заключается в следующем: важным применением QR-разложения является предоставление численно более стабильного способа инвертирования матриц по сравнению с алгоритмом, который был представлен в предыдущей главе.

С другой стороны, имейте в виду, что матрицы, которые теоретически являются обратимыми, но близкими к сингулярным, по-прежнему очень трудно инвертировать; QR-разложение может быть численно *более* стабильным, чем представленный в предыдущей главе традиционный алгоритм, но это не гарантирует высококачественную обратную матрицу. Если окунуть гнилое яблоко в мед, то оно по-прежнему останется гнилым.

Резюме

QR-разложение – великолепная вещь. Оно определенно входит в мой список пяти лучших матричных разложений в линейной алгебре. Вот ключевые выводы, которые следует вынести из этой главы.

- Ортогональная матрица имеет столбцы, которые попарно ортогональны и имеют норму, равную 1. Ортогональные матрицы являются ключом к нескольким матричным разложениям, включая QR-разложение, собственное разложение и сингулярное разложение. Ортогональные матрицы также важны в геометрии и компьютерной графике (например, матрицы чистого поворота).

- Неортогональная матрица преобразовывается в ортогональную матрицу посредством процедуры Грама–Шмидта, которая предусматривает применение ортогонального разложения векторов, чтобы изолировать компоненту каждого столбца, который ортогонален всем предыдущим столбцам («предыдущий» означает слева направо).
- QR-разложение является результатом процедуры Грама–Шмидта (технически оно реализовано более стабильным алгоритмом, но GS по-прежнему является правильным ключом к его пониманию).

Упражнения по программированию

Упражнение 9.1

Квадратная матрица **Q** имеет следующие ниже тождества:

$$\mathbf{Q}^T\mathbf{Q} = \mathbf{Q}\mathbf{Q}^T = \mathbf{Q}^{-1}\mathbf{Q} = \mathbf{Q}\mathbf{Q}^{-1} = \mathbf{I}.$$

Продемонстрируйте это в исходном коде, вычислив **Q** из матрицы случайных чисел, потом вычислив $\mathbf{Q}^T$ и $\mathbf{Q}^{-1}$. Затем покажите, что все четыре выражения дают единичную матрицу.

Упражнение 9.2

Реализуйте процедуру Грама–Шмидта, как описано ранее[1]. Используйте матрицу случайных чисел 4×4. Сравните свой ответ с матрицей **Q** из функции `np.linalg.qr`.

Важно: в преобразованиях типа отражения Хаусхолдера присутствует фундаментальная неопределенность знака. Это означает, что векторы могут «переворачиваться» (умножаться на –1) в зависимости от незначительных различий в алгоритме и реализации. Эта особенность существует во многих матричных разложениях, включая собственные векторы. В главе 13 имеется более глубокое и подробное изложение причины, по которой это происходит, и что это означает. А пока результат таков: надо *вычесть* свою матрицу **Q** из Python'овской матрицы **Q** и *сложить* свою матрицу **Q** и Python'овскую матрицу **Q**. Ненулевые столбцы в одной будут нулями в другой.

Упражнение 9.3

В этом упражнении вы узнаете, что происходит при применении QR-разложения к матрице, которая почти, но не совсем ортогональна. Во-первых, создайте ортогональную матрицу под названием **U** из QR-разложения матрицы случайных чисел 6×6. Вычислите QR-разложение матрицы **U** и подтвердите, что **R** = **I** (и убедитесь, что вы понимаете причину!).

Во-вторых, видоизмените нормы каждого столбца матрицы **U**. Установите нормы столбцов 1–6 равными 10–15 (то есть первый столбец матрицы **U** должен иметь норму 10, второй столбец должен иметь норму 11 и т. д.). Про-

[1] Не торопитесь с этим упражнением; оно довольно сложное.

гоните эту модулированную матрицу **U** через QR- разложение и подтвердите, что ее **R** является диагональной матрицей с диагональными элементами, равными 10–15. Какой будет $\mathbf{Q}^T\mathbf{Q}$ для этой матрицы?

В-третьих, нарушьте ортогональность матрицы **U**, установив элемент $u_{1.4} = 0$. Что произойдет с **R** и почему?

Упражнение 9.4

Цель этого упражнения – сравнить численные ошибки, полученные при использовании «традиционного» инверсного метода, с которым вы познакомились в предыдущей главе, с инверсным методом на основе QR. Мы будем использовать матрицы случайных чисел, помня о том, что они, как правило, численно стабильны и, следовательно, имеют точные обратные матрицы.

Вот что нужно сделать: скопируйте исходный код из упражнения 8.2 в функцию Python, которая на входе принимает матрицу и на выходе предоставляет обратную ей матрицу. (Вы также можете включить проверку того, что входная матрица является квадратной и является полноранговой.) Я назвал эту функцию `oldSchoolInv`. Затем создайте матрицу случайных чисел 5×5. Вычислите обратную ей матрицу, используя традиционный метод и представленный в этой главе метод на основе QR-разложения (для вычисления $\mathbf{R}^{-1}$ можете использовать свой «традиционный метод»). Рассчитайте ошибку вычисления обратной матрицы как евклидово расстояние от матрицы, умноженной на обратную ей матрицу, до истинной единичной матрицы из функции `np.eye`. Постройте гистограмму результатов, показав два метода на оси *x* и ошибку (евклидово расстояние до **I**) на оси *y*, как представлено на рис. 9.3.

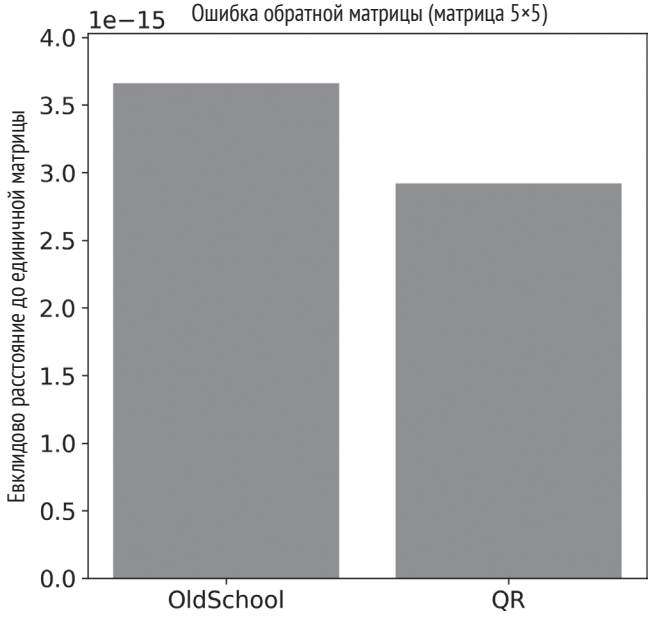


Рис. 9.3 ❖ Результаты упражнения 9.4

Выполняйте исходный код несколько раз и инспектируйте гистограмму. Вы обнаружите, что иногда традиционный метод работает лучше, а иногда лучше работает метод на основе QR (чем меньше числа, тем лучше; теоретически столбцы должны иметь нулевую высоту). Попробуйте еще раз, используя матрицу 30×30. Будут ли результаты более последовательными? На самом деле от прогона к прогону будет большой разброс. Это означает, что необходимо провести эксперимент, в котором сравнение повторяется много раз. Это следующее упражнение.

Упражнение 9.5

Поместите исходный код из предыдущего упражнения в цикл `for` с сотней итераций, в которых вы повторяете эксперимент, всякий раз используя другую матрицу случайных чисел. Сохраняйте ошибку (евклидово расстояние) по каждой итерации и постройте график, подобный рис. 9.4, который показывает среднее значение по всем прогонам эксперимента (серый столбик) и все отдельные ошибки (черные точки). Проведите эксперимент для матриц 5×5 и 30×30.

Вы также можете попробовать использовать функцию `np.linalg.inv`, чтобы вместо традиционного метода инвертировать **R** и увидеть, будет ли это иметь эффект.

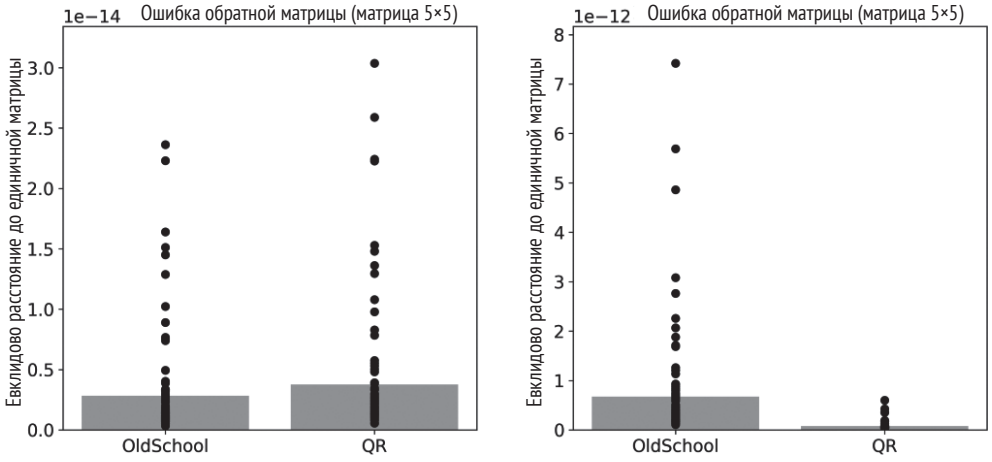


Рис. 9.4 ❖ Результаты упражнения 9.5. Обратите внимание на разницу в шкалировании оси *y* между левой и правой панелями

Упражнение 9.6

Интересное свойство квадратных ортогональных матриц состоит в том, что все их сингулярные числа (и их собственные числа) равны 1. Это означает, что они имеют индуцированную 2-норму, равную 1 (индуцированная норма является наибольшим сингулярным числом), и они имеют фробениусову норму *M*. Последний результат объясняется тем, что фробениусова норма

равна квадратному корню из суммы квадратов сингулярных чисел. В этом упражнении вы подтвердите эти свойства.

Создайте ортогональную матрицу $M \times M$ как QR-разложение случайной матрицы. Вычислите ее индуцированную 2-норму, используя функцию `np.linalg.norm`, и вычислите ее фробениусову норму, используя уравнение, которое вы узнали в главе 6, деленное на квадратный корень из M. Подтвердите, что обе величины равны 1 (в разумных пределах допуска ошибки округления). Проверьте, используя несколько разных значений M.

Далее обследуйте значение индуцированной нормы, используя умножение матрицы на вектор. Создайте случайный -элементный вектор-столбец **v**. Затем вычислите нормы векторов **v** и **Qv**. Эти нормы должны быть равны друг другу (хотя не следует ожидать, что они будут равны 1).

Наконец, возьмите лист бумаги и разработайте доказательство этой эмпирической демонстрации. Указанное доказательство напечатано в следующем абзаце, так что не смотрите вниз! Но вы можете обратиться к сноске, если вам нужна подсказка[1].

Искренне надеюсь, что вы это читаете, чтобы проверить свои рассуждения, а не потому, что жульничаете! Так или иначе, доказательство состоит в том, что векторную норму $||\mathbf{v}||$ можно вычислить как $\mathbf{v}^T\mathbf{v}$; поэтому векторная норма $||\mathbf{Qv}||$ вычисляется как $(\mathbf{Qv})^T\mathbf{Qv} = \mathbf{v}^T\mathbf{Q}^T\mathbf{Qv}$. $\mathbf{Q}^T\mathbf{Q}$ отменяется, давая единичную матрицу, при этом оставляя точечное произведение вектора с самим собой. Вывод состоит в том, что ортогональные матрицы могут поворачивать вектор, но не шкалировать его.

Упражнение 9.7

В этом упражнении будет освещена одна особенность матрицы **R**, которая имеет отношение к пониманию того, как использовать QR для реализации метода наименьших квадратов (глава 12): когда матрица **A** является высокой и имеет полный столбцовый ранг, первые N строк матрицы **R** являются верхнетреугольными, тогда как строки $N + 1$ вплоть до M – нулевыми. Подтвердите это на Python, используя случайную матрицу 10×4. Проследите, чтобы использовалось полное QR-разложение, а не экономное (компактное) разложение.

Конечно же, матрица **R** необратима, потому что она не квадратная. Но

1) подматрица, состоящая из первых N строк, является квадратной и полноранговой (когда матрица **A** имеет полный столбцовый ранг), следовательно, имеет полную обратную матрицу, и
2) высокая матрица **R** имеет псевдообратную матрицу.

Вычислите обе обратные матрицы и подтвердите, что полная обратная матрица первых N строк матрицы **R** равна первым N столбцам псевдообратной матрицы высокой матрицы **R**.

---

[1] Подсказка: распишите формулу точечного произведения для векторной нормы.

Глава 10

Приведение строк и LU-разложение

Теперь переходим к LU-разложению. LU, как и QR, является одним из вычислительных стержней, лежащих в основе алгоритмов обработки данных, включая подгонку модели методом наименьших квадратов и обратную матрицу. Таким образом, эта глава имеет ключевое значение для вашего образования в области линейной алгебры.

Особенность LU-разложения состоит в том, что его невозможно просто выучить за один присест. Вместо этого сначала придется познакомиться с системами уравнений, приведением строк и гауссовым устранением. И в ходе изучения этих тем вы также узнаете о ступенчатых матрицах и матрицах перестановок. О да, дорогой читатель, это будет захватывающая и насыщенная событиями глава.

Системы уравнений

В целях понимания LU-разложения и его применений нужно понять приведение строк и устранение по Гауссу. А в целях понимания этих тем нужно понять, как манипулировать уравнениями, преобразовывать их в матричное уравнение и решать это матричное уравнение, используя приведение строк.

Начнем с «системы» из одного уравнения:

$$2x = 8.$$

Уверен, что, научившись в школе, вы способны выполнять различные математические манипуляции с уравнением – при условии что вы делаете одно и то же с обеими частями уравнения. Это означает, что следующее уравнение не совпадает с предыдущим, но они связаны между собой простыми манипуляциями. Что еще важнее, любое решение одного уравнения является решением другого:

$$5(2x - 3) = 5(8 - 3).$$

Теперь перейдем к системе из двух уравнений:

$x = 4 - y$;

$y = x/2 + 2$.

В этой системе уравнений найти уникальные значения x и y только из одного из этих уравнений невозможно. Вместо этого, чтобы получить решение, нужно рассматривать оба уравнения одновременно. Если вы попытаетесь решить эту систему сейчас, то, вероятно, воспользуетесь стратегией подстановки y в первом уравнении правой частью второго уравнения. После отыскания решения для x в первом уравнении вы подставите это значение во второе уравнение, чтобы найти y. Эта стратегия похожа (хотя и не так эффективна) на обратную подстановку, определение которой я дам позже.

Важной особенностью системы уравнений является то, что отдельные уравнения можно складывать друг с другом либо вычитать. В следующих ниже уравнениях я два раза добавил второе уравнение к первому и вычел первое изначальное уравнение из второго (скобки добавлены для ясности):

$x + (2y) = 4 - y + (x + 4)$;

$y - (x) = x/2 + 2 - (4 - y)$.

Даю вам возможность поработать над арифметикой, но итогом является то, что x выпадает из первого уравнения, а y выпадает из второго уравнения. Это значительно упрощает вычисление решения ($x = 4/3$, $y = 8/3$). И вот важный момент: умножение уравнения на скаляр и добавление их к другим уравнениям облегчили отыскание решения системы. Опять же, модулированная и изначальная системы – это не одни и те же уравнения, но их решения одинаковы, поскольку две системы связаны серией линейных операций.

Это базовые знания, необходимые для того, чтобы научиться решать системы уравнений с помощью линейной алгебры. Но прежде чем научиться этому подходу, вам нужно научиться представлять систему уравнений матрицами и векторами.

Конвертирование уравнений в матрицы

Конвертирование систем уравнений в матрично-векторное уравнение применяется для решения систем уравнений и используется для настройки формулы общей линейной модели в статистике. К счастью, переложение уравнений в матрицы осуществляется концептуально просто и выполняется в два шага.

Во-первых, надо организовать уравнения так, чтобы константы находились в правой части уравнений. Константы – это числа, которые не привязаны к переменным (иногда именуемые *пересечениями*, или *сдвигами*). Переменные и умножающие их коэффициенты находятся в левой части уравнения в том же порядке (например, все уравнения должны иметь сначала

член x, затем член y и т. д.). Следующие ниже уравнения образуют систему уравнений, с которой мы работали ранее, но в правильной организации:

$$x + y = 4;$$

$$-x/2 + y = 2.$$

Во-вторых, надо выделить коэффициенты (числа, на которые умножаются переменные; отсутствующие в уравнении переменные имеют коэффициент, равный нулю) в матрицу с одной строкой в расчете на одно уравнение. Переменные помещаются в вектор-столбец, который умножает матрицу коэффициентов справа. А константы помещаются в вектор-столбец в правой части уравнения. Наша примерная система имеет матричное уравнение, которое выглядит следующим образом:

$$\begin{bmatrix} 1 & 1 \\ -1/2 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}.$$

И вуаля! Вы конвертировали систему уравнений в одно матричное уравнение. К этому уравнению можно обращаться как к $\mathbf{Ax} = \mathbf{b}$, где $\mathbf{A}$ – это матрица коэффициентов, $\mathbf{x}$ – вектор неизвестных переменных, для которых нужно найти решение (в данном случае $\mathbf{x}$ – это вектор, содержащий $[x\ y]$), а $\mathbf{b}$ – вектор констант.

Пожалуйста, найдите минутку, чтобы убедиться, что вы понимаете, как матричное уравнение отображается в систему уравнений. В частности, проработайте умножение матрицы на вектор, чтобы продемонстрировать, что оно эквивалентно изначальной системе уравнений.

Работа с матричными уравнениями

Матричными уравнениями можно манипулировать так же, как и обычными уравнениями, включая сложение, умножение, транспонирование и т. д., при условии что манипуляции допустимы (например, в случае сложения размеры матриц совпадают) и все манипуляции влияют на обе части уравнения. Например, допустима следующая ниже прогрессия уравнений:

$$\mathbf{Ax} = \mathbf{b};$$

$$\mathbf{v} + \mathbf{Ax} = \mathbf{v} + \mathbf{b};$$

$$(\mathbf{v} + \mathbf{Ax})^{\mathrm{T}} = (\mathbf{v} + \mathbf{b})^{\mathrm{T}}.$$

Главное различие между работой с матричными уравнениями и скалярными уравнениями заключается в том, что, поскольку умножение матриц зависит от стороны, матрицы необходимо умножать одинаково в обеих частях уравнения.

Например, допустима следующая ниже прогрессия уравнений:

$$\mathbf{AX} = \mathbf{B};$$

$$\mathbf{CAX} = \mathbf{CB}.$$

Обратите внимание, что **C** предпозиционно умножает обе части уравнения. Напротив, следующая ниже прогрессия недопустима:

$$\mathbf{AX} = \mathbf{B};$$

$$\mathbf{AXC} = \mathbf{CB}.$$

Проблема здесь в том, что **C** *постпозиционно* умножает в левой части, но *предпозиционно* умножает в правой части. Несомненно, есть несколько исключительных случаев, когда это уравнение будет допустимым (например, если **C** является единичной матрицей или матрицей нулей), но в общем случае эта прогрессия недопустима.

Давайте обратимся к примеру на Python. Мы найдем неизвестную матрицу **X** в уравнении **AX** = **B**. Следующий ниже исходный код генерирует матрицы **A** и **B** из случайных чисел. Вы уже знаете, что **X** можно найти, используя $\mathbf{A}^{-1}$. Вопрос в том, имеет ли значение порядок умножения[1].

```
A = np.random.randn(4,4)
B = np.random.randn(4,4)
# найти X
X1 = np.linalg.inv(A) @ B
X2 = B @ np.linalg.inv(A)
# остаток (должен быть матрицей нулей)
res1 = A@X1 - B
res2 = A@X2 - B
```

Если бы умножение матриц было коммутативным (означая, что порядок не имеет значения), то `res1` и `res2` должны были быть равны матрице нулей. Давайте посмотрим:

```
res1:

[[-0.  0.  0.  0.]
 [-0. -0.  0.  0.]
 [ 0.  0.  0.  0.]
 [ 0.  0. -0. -0.]]

res2:

[[-0.47851507  6.24882633  4.39977191  1.80312482]
 [ 2.47389146  2.56857366  1.58116135 -0.52646367]
 [-2.12244448 -0.20807188  0.2824044  -0.91822892]
 [-3.61085707 -3.80132548 -3.47900644 -2.372463  ]]
```

Теперь вы знаете, как выражать систему уравнений одним матричным уравнением. Я собираюсь вернуться к этому вопросу через несколько разделов; сперва мне нужно научить вас приведению строк и ступенчатой форме матрицы.

---

[1] Конечно же, вы знаете, что порядок имеет значение, но эмпирические демонстрации помогают развивать интуицию. И я хочу, чтобы вы привыкли использовать Python как инструмент для эмпирического подтверждения математических принципов.

Приведение строк

В традиционной линейной алгебре теме приведения строк уделяется много внимания, потому что это проверенный временем способ ручного решения систем уравнений. Я серьезно сомневаюсь, что в своей карьере исследователя данных вы будете решать какие-либо системы уравнений вручную. Но о приведении строк полезно знать, и оно ведет напрямую к LU-разложению, которое применяется на практике в прикладной линейной алгебре. Итак, начнем.

Приведение строк[1] означает итеративное применение двух операций – скалярного умножения и сложения – к строкам матрицы. Приведение строк основано на том же принципе, что и добавление уравнений к другим уравнениям в системе.

Запомните вот это утверждение: *цель приведения строк – преобразовать плотную матрицу в верхнетреугольную матрицу*.

Начнем с простого примера. В следующей ниже плотной матрице мы добавляем первую строку ко второй строке, тем самым выбивая –2. И таким образом мы преобразовали плотную матрицу в верхнетреугольную матрицу:

$$\begin{bmatrix} 2 & 3 \\ -2 & 2 \end{bmatrix} \xrightarrow{R_1+R_2} \begin{bmatrix} 2 & 3 \\ 0 & 5 \end{bmatrix}.$$

Полученная в результате приведения строк верхнетреугольная матрица называется *ступенчатой формой* матрицы.

Формально матрица имеет ступенчатую форму, если

1) самое левое ненулевое число в строке (именуемое *опорным элементом*[2]) находится справа от опорного элемента вышележащих строк и
2) любые строки из одних нулей находятся ниже строк, содержащих ненулевые числа.

Подобно манипулированию уравнениями в системе, матрица *после* приведения строк отличается от матрицы *до* приведения строк. Но две матрицы связаны линейным преобразованием. А поскольку линейные преобразования можно представлять матрицами, то для выражения приведения строк можно использовать матричное умножение:

$$\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}\begin{bmatrix} 2 & 3 \\ -2 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 0 & 5 \end{bmatrix}.$$

Я буду называть эту матрицу $\mathbf{L}^{-1}$ по причинам, которые станут ясны, когда я введу понятие LU-разложения[3]. Таким образом, в выражении ($\mathbf{L}^{-1}\mathbf{A} = \mathbf{U}$) матрица $\mathbf{L}^{-1}$ – это линейное преобразование, отслеживающее манипуляции,

---

[1] Англ. *Row Reduction*; син. сокращение строк. – *Прим. перев.*

[2] Англ. *Pivot*. – *Прим. перев.*

[3] Внимание, спойлер: LU-разложение предусматривает представление матрицы как произведения нижнетреугольной и верхнетреугольной матриц.

которые мы реализовали посредством приведения строк. Пока что не нужно сосредоточиваться на матрице $\mathbf{L}^{-1}$ – на самом деле при гауссовом отсеивании ее часто игнорируют. Но ключевой момент (слегка расширенный по сравнению с более ранним утверждением) заключается в следующем: *приведение строк предусматривает преобразовывание матрицы в верхнетреугольную матрицу посредством манипуляций со строками, реализуемых как препозиционное умножение на трансформационную матрицу*.

Вот еще один пример матрицы 3×3. Для преобразования этой матрицы в ступенчатую форму требуется два шага:

$$\begin{bmatrix} 1 & 2 & 2 \\ -1 & 3 & 0 \\ 2 & 4 & -3 \end{bmatrix} \xrightarrow{-2R_1+R_3} \begin{bmatrix} 1 & 2 & 2 \\ -1 & 3 & 0 \\ 0 & 0 & -7 \end{bmatrix} \xrightarrow{R_1+R_2} \begin{bmatrix} 1 & 2 & 2 \\ 0 & 5 & 2 \\ 0 & 0 & -7 \end{bmatrix}.$$

Процедура приведения строк утомительна (см. врезку «Всегда ли процедура приведения строк так проста?»). Наверняка должна существовать функция Python, которая делает это за нас! Все дело в том, что она есть и ее нет. Нет функции Python, которая возвращает ступенчатую форму, подобную той, что я создал в двух предыдущих примерах. Причина в том, что ступенчатая форма матрицы не уникальна. Например, в приведенной выше матрице 3×3 вы могли умножить вторую строку на 2, чтобы получить вектор-строку [0 10 4]. Это создает совершенно допустимую, но другую ступенчатую форму той же изначальной матрицы. И действительно, с этой матрицей связано бесконечное число ступенчатых матриц.

С учетом сказанного, двухступенчатые формы матрицы предпочтительнее бесконечно возможных ступенчатых форм. Эти две формы уникальны с учетом некоторых ограничений и называются строчно приведенной ступенчатой формой и матрицей **U** из LU-разложения. Я представлю обе позже; сначала пора научиться использовать приведение строк для решения систем уравнений.

Всегда ли процедура приведения строк так проста?

Овладение навыком приведения строк матрицы к ее ступенчатой форме требует усердной практики и усвоения нескольких приемов с крутыми названиями, такими как взаимообмен строк и взаимообмен строк и столбцов[1]. Приведение строк раскрывает несколько интересных свойств строчного и столбцового пространств матрицы. И хотя успешное приведение строк матрицы 5×6 с помощью ручного взаимообмена строк приносит чувство выполненного долга и удовлетворения, я полагаю, что ваше время лучше потратить на линейно-алгебраические концепции, которые имеют более прямое применение в науке о данных. Мы движемся к пониманию LU-разложения, и этого краткого введения в приведение строк будет для указанной цели вполне достаточно.

[1] Англ. *partial pivoting*; данный прием используется во избежание ошибок округления, которые могут возникать при делении каждого значения строки на опорный элемент. – *Прим. перев.*

Метод устранения по Гауссу

К этому моменту книги вы знаете, как решать матричные уравнения, используя обратную матрицу. Что, если бы я вам сказал, что вы можете решить матричное уравнение, не инвертируя никаких матриц[1]?

Этот метод называется устранением по Гауссу, или *гауссовым устранением*[2]. Несмотря на свое название, этот алгоритм на самом деле был разработан китайскими математиками почти за две тысячи лет до Гаусса, а затем открыт заново Ньютоном за сотни лет до Гаусса. Но Гаусс внес в этот метод важный вклад, в том числе технические приемы, реализованные в современных компьютерах.

Метод гауссова устранения работает просто: надо расширить матрицу коэффициентов вектором констант, привести строки к ступенчатой форме, а затем применить обратную подстановку, чтобы найти решение для каждой переменной по очереди.

Начнем с системы из двух уравнений, которую мы решили ранее:

$x = 4 - y;$

$y = x/2 + 2.$

Первым шагом является конвертирование этой системы уравнений в матричное уравнение. Данный шаг уже нами пройден; это уравнение напечатано здесь как напоминание:

$$\begin{bmatrix} 1 & 1 \\ -1/2 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}.$$

Далее расширим матрицу коэффициентов вектором констант:

$$\left[\begin{array}{cc|c} 1 & 1 & 4 \\ -1/2 & 1 & 2 \end{array}\right].$$

Затем выполним приведение строк в этой расширенной матрице. Обратите внимание, что при приведении строк вектор-столбец констант изменится:

$$\left[\begin{array}{cc|c} 1 & 1 & 4 \\ -1/2 & 1 & 2 \end{array}\right] \xrightarrow{1/2R_1+R_2} \left[\begin{array}{cc|c} 1 & 1 & 4 \\ 0 & 3/2 & 4 \end{array}\right].$$

Получив матрицу в ступенчатой форме, переводим расширенную матрицу обратно в систему уравнений. Она выглядит так:

$x + y = 4;$

$3/2y = 4.$

---

[1] Пожалуйста, вообразите в уме мем *Матрицы*, в которой Морфеус предлагает красную и синюю таблетки, соответствующие новым знаниям в противоположность приверженности тому, что вы уже знаете.

[2] Англ. *Gaussian Elimination*; син. исключение по Гауссу. – *Прим. перев.*

Метод гауссова устранения посредством приведения строк удалил член *x* во втором уравнении, а это означает, что отыскание *y* предусматривает всего лишь несколько арифметических действий. Решив *y* = 8/3, подставьте это значение в *y* в первом уравнении и найдите *x*. Эта процедура называется *обратной подстановкой*.

В предыдущем разделе я писал, что в Python нет функции для вычисления ступенчатой формы матрицы, поскольку она не уникальна. А затем написал, что существует одна уникальная ступенчатая матрица, именуемая *строчно приведенной ступенчатой формой* (нередко сокращенно RREF[1]), которую Python вычислит. Продолжайте читать, чтобы узнать подробности…

Метод устранения по Гауссу–Жордану

Давайте продолжим приведение строк нашей образцовой матрицы с целью превратить все опорные элементы – крайние левые ненулевые числа в каждой строке – в единицы. При наличии ступенчатой матрицы каждая строка просто делится на ее опорный элемент. В этом примере первая строка уже имеет 1 в крайнем левом положении, поэтому нам просто нужно скорректировать вторую строку. В результате получаем следующую ниже матрицу:

$$\begin{bmatrix} 1 & 1 & 4 \\ 0 & 1 & 8/3 \end{bmatrix}.$$

А теперь подходим к хитрости: мы продолжаем приведение строк *вверх*, чтобы отсеить все элементы над каждым опорным элементом. Другими словами, нам нужна ступенчатая матрица, в которой каждый опорный элемент равен 1, и это единственное ненулевое число в ее столбце.

$$\begin{bmatrix} 1 & 1 & 4 \\ 0 & 1 & 8/3 \end{bmatrix} \xrightarrow{-R_2+R_1} \begin{bmatrix} 1 & 0 & 4/3 \\ 0 & 1 & 8/3 \end{bmatrix}.$$

Это и есть строчно приведенная ступенчатая форма (RREF) нашей изначальной матрицы. Вы увидите единичную матрицу слева – RREF-форма всегда будет создавать единичную матрицу как подматрицу в левом верхнем углу изначальной матрицы. Это результат установки всех опорных элементов равными 1 и использования восходящего сведения строк, чтобы устранять все элементы над каждым опорным элементом.

Теперь мы продолжим гауссово устранение путем переложения матрицы обратно в систему уравнений:

$x = 4/3;$

$y = 8/3.$

[1] От англ. *Reduced Row Echelon Form*. – *Прим. перев.*

Нам больше не нужна ни обратная подстановка, ни даже базовая арифметика: модифицированный метод гауссова устранения, который называется методом устранения по Гауссу–Жордану, разъединил переплетенные переменные в системе уравнений и обнажил решения по каждой переменной.

Метод устранения по Гауссу–Жордану применялся людьми для решения систем уравнений вручную более чем за столетие до того, как появились компьютеры, чтобы помогать нам в числодробительной работе. На самом деле в компьютерах реализован все тот же метод, лишь с малыми модификациями в целях обеспечения численной стабильности.

RREF-форма уникальна, и, стало быть, матрица имеет только одну ассоциированную RREF-форму. В NumPy нет функции для вычисления RREF-формы матрицы, но в библиотеке sympy она есть (sympy – это библиотека для символьных математических вычислений на Python и мощный двигатель для математики «классной доски»):

```
import sympy as sym

# матрица, конвертированная в sympy
M = np.array([ [1,1,4],[-1/2,1,2] ])
symMat = sym.Matrix(M)

# RREF-форма
symMat.rref()[0]

>>
[[1, 0, 1.33333333333333],
 [0, 1, 2.66666666666667]]
```

Обратная матрица посредством метода устранения по Гауссу–Жордану

Ключевой момент метода устранения по Гауссу–Жордану заключается в том, что приведение строк производит последовательность манипуляций со строками, которая решает систему уравнений. Эти манипуляции со строками являются линейными преобразованиями.

Любопытно, что описание метода устранения по Гауссу–Жордану согласуется с описанием обратной матрицы, то есть линейного преобразования, которое решает систему уравнений. Но подождите, какую «систему уравнений» решает обратная матрица? Свежий взгляд на обратную матрицу даст несколько новых идей. Рассмотрим вот эту систему уравнений:

$$ax_1 + by_1 = 1;$$

$$cx_1 + dy_1 = 0.$$

Переведя в матричное уравнение, получим:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Посмотрите на вектор констант – это первый столбец единичной матрицы 2×2! Это означает, что применение RREF-формы к полноранговой квадратной матрице, расширенной первым столбцом единичной матрицы, покажет линейное преобразование, которое переводит матрицу в первый столбец единичной матрицы. А это, в свою очередь, означает, что вектор $[x_1\ y_1]^T$ является первым столбцом обратной матрицы.

Затем мы повторяем процедуру, но находя решение для второго столбца обратной матрицы:

$$ax_2 + by_2 = 0;$$

$$cx_2 + dy_2 = 1.$$

RREF-форма в этой системе дает вектор $[x_2\ y_2]^T$, который является вторым столбцом обратной матрицы.

Я отделил столбцы единичной матрицы, чтобы связать их с перспективой решения систем уравнений. Но мы можем расширить всю единичную матрицу целиком и найти обратную матрицу с помощью одной RREF-формы.

Вот вид с высоты птичьего полета на получение обратной матрицы посредством метода устранения по Гауссу–Жордану (квадратные скобки обозначают расширенные матрицы, при этом вертикальная линия отделяет две составляющие матрицы):

$$rref([\mathbf{A}\,|\,\mathbf{I}]) \Rightarrow [\mathbf{I}\,|\,\mathbf{A}^{-1}].$$

Это интересно, поскольку обеспечивает механизм вычисления обратной матрицы без вычисления определителей. С другой стороны, приведение строк предусматривает большое число делений, увеличивая риск ошибок численной прецизионности. Например, представьте, что даны два числа, которые по существу равны нулю плюс ошибка округления. Если мы в конечном итоге придем к тому, что поделим эти числа во время получения RREF, то мы получим дробь $10^{-15}/10^{-16}$, которая на самом деле равна 0/0, но ответ будет 10.

Вывод здесь аналогичен тому, который я изложил в предыдущей главе об использовании QR-разложения для вычисления обратной матрицы: применение метода устранения по Гауссу–Жордану для вычисления обратной матрицы, вероятно, будет более численно стабильным, чем полный алгоритм вычисления обратной матрицы, но инвертировать матрицу, близкую к сингулярной или имеющую высокое кондиционное число, будет трудно, независимо от используемого алгоритма.

LU-разложение

Буквы «LU» в LU-разложении означают «нижний верхний»[1], как в нижнем треугольном, верхнем треугольнике. Идея состоит в том, чтобы разложить матрицу на произведение двух треугольных матриц:

[1] Англ. *lower upper*. – *Прим. перев.*

$\mathbf{A} = \mathbf{LU}$.

Вот числовой пример:

$$\begin{bmatrix} 2 & 2 & 4 \\ 1 & 0 & 3 \\ 2 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 1/2 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 2 & 4 \\ 0 & -1 & 1 \\ 0 & 0 & -3 \end{bmatrix}.$$

А вот соответствующий исходный код Python (обратите внимание, что функция для LU-разложения находится в библиотеке SciPy):

```
import scipy.linalg # LU в библиотеке scipy
A = np.array([ [2,2,4], [1,0,3], [2,1,2] ])
_, L, U = scipy.linalg.lu(A)

# распечатать их
print('L: '), print(L)
print('U: '), print(U)

L:
[[1. 0.  0. ]
 [0.5 1.  0. ]
 [1. 1.  1. ]]

U:
[[ 2. 2.  4.]
 [ 0. -1.  1.]
 [ 0. 0. -3.]]
```

Откуда взялись эти две матрицы? На самом деле вы уже знаете ответ: приведение строк может быть выражено как ($\mathbf{L}^{-1}\mathbf{A} = \mathbf{U}$), где $\mathbf{L}^{-1}$ содержит набор манипуляций со строками, который преобразовывает плотную матрицу $\mathbf{A}$ в верхнетреугольную (ступенчатую) матрицу $\mathbf{U}$.

Поскольку ступенчатая форма не уникальна, LU-разложение не является обязательно уникальным. То есть существует бесконечная пара нижне- и верхнетреугольных матриц, которые можно перемножать, чтобы получать матрицу $\mathbf{A}$. Однако добавление ограничения, состоящего в том, что диагонали матрицы $\mathbf{L}$ равны 1, обеспечивает, чтобы LU-разложение было уникальным для квадратной полноранговой матрицы $\mathbf{A}$ (это видно в предыдущем примере). Уникальность LU-разложений рангово-пониженных матриц и неквадратных матриц потребует более продолжительного изложения, которое я здесь не буду рассматривать; однако алгоритм LU-разложения библиотеки SciPy является детерминированным, а это означает, что повторяющиеся LU-разложения заданной матрицы будут идентичными.

Взаимообмен строками посредством матриц перестановок

Некоторые матрицы очень легко преобразовываются в верхнетреугольную форму. Рассмотрим следующую ниже матрицу:

$$\begin{bmatrix} 3 & 2 & 1 \\ 0 & 0 & 5 \\ 0 & 7 & 2 \end{bmatrix}.$$

Она не находится в ступенчатой форме, но была бы в ней, если бы мы поменяли местами вторую и третью строки. Взаимообмен строками – это один из приемов приведения строк, который реализуется посредством матрицы перестановок:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}\begin{bmatrix} 3 & 2 & 1 \\ 0 & 0 & 5 \\ 0 & 7 & 2 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 1 \\ 0 & 7 & 2 \\ 0 & 0 & 5 \end{bmatrix}.$$

Матрицы перестановок часто обозначаются как **P**. Таким образом, полное LU-разложение на самом деле принимает следующий вид:

$$\mathbf{PA} = \mathbf{LU};$$

$$\mathbf{A} = \mathbf{P}^{\mathrm{T}}\mathbf{LU}.$$

Примечательно, что матрицы перестановок ортогональны, поэтому $\mathbf{P}^{-1} = \mathbf{P}^{\mathrm{T}}$. Вкратце: причина состоит в том, что все элементы матрицы перестановок равны либо 0, либо 1, а поскольку строки обмениваются местами только один раз, каждый столбец имеет ровно один ненулевой элемент (и действительно, все матрицы перестановок являются единичными матрицами со взаимообменом строк). Следовательно, точечное произведение любых двух столбцов равно 0, а точечное произведение столбца на самого себя равно 1, и, стало быть, $\mathbf{P}^{\mathrm{T}}\mathbf{P} = \mathbf{I}$.

Важно: приведенные выше формулы дают математическое описание LU-разложения. Библиотека Scipy на самом деле возвращает $\mathbf{A} = \mathbf{PLU}$, что также можно было бы записать как $\mathbf{P}^{\mathrm{T}}\mathbf{A} = \mathbf{LU}$. Упражнение 10.4 дает возможность обследовать этот дезориентирующий момент.

На рис. 10.1 показан пример LU-разложения, примененного к случайной матрице.

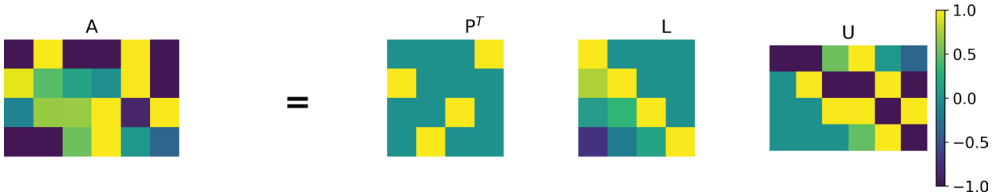


Рис. 10.1 ❖ Визуализация LU-разложения

LU-разложение используется в нескольких приложениях, включая вычисление определителя и обратной матрицы. В следующей главе вы увидите, как LU-разложение применяется в вычислении методом наименьших квадратов.

Резюме

Я открыл эту главу, пообещав увлекательное образовательное приключение. Надеюсь, вы испытали несколько всплесков адреналина, изучая новые взгляды на алгебраические уравнения, разложение матриц и обратную матрицу. Вот ключевые выводы, которые следует вынести из этой главы.

- Системы уравнений можно преобразовывать в матричное уравнение. Помимо обеспечения компактного представления, это обеспечивает возможность находить самые изощренные линейно-алгебраические решения систем уравнений.
- При работе с матричными уравнениями следует помнить, что манипуляции должны применяться к обеим частям уравнения и что умножение матриц некоммутативно.
- Приведение строк – это процедура, в которой строки матрицы $\mathbf{A}$ скалярно умножаются и складываются до тех пор, пока матрица не будет линейно преобразована в верхнетреугольную матрицу $\mathbf{U}$. Набор линейных преобразований можно хранить в еще одной матрице $\mathbf{L}^{-1}$, которая умножает матрицу $\mathbf{A}$ слева, чтобы произвести выражение $\mathbf{L}^{-1}\mathbf{A} = \mathbf{U}$.
- Приведение строк веками использовалось для ручного решения систем уравнений, включая обратную матрицу. Мы по-прежнему используем процедуру приведения строк, хотя компьютеры берут на себя всю арифметику.
- Приведение строк также используется для реализации LU-разложения. LU-разложение является уникальным в условиях некоторых ограничений, которые встроены в функцию `lu()` библиотеки SciPy.

Упражнения по программированию

Упражнение 10.1

LU-разложение бывает вычислительно емким, хотя оно и эффективнее других разложений, таких как QR. Интересно, что LU-разложение часто ис-

пользуется в качестве эталона для сравнения времени вычислений между операционными системами, аппаратными процессорами, компьютерными языками (например, C, Python и MATLAB) или реализуемыми алгоритмами. Из любопытства я проверил время, которое потребовалось Python и MATLAB на выполнение LU-разложения на тысяче матриц размера 100×100. На моем ноутбуке MATLAB заняло около 300 мс, а Python – около 410 мс. Исходный код Python в Google Colab занял около 1000 мс. Проверьте, сколько времени он займет на вашем компьютере.

Упражнение 10.2

Примените метод матричного умножения, чтобы создать матрицу 6×8 ранга 3. Возьмите его LU-разложение и покажите три матрицы с их рангами в заголовке, как на рис. 10.2. Обратите внимание на ранги трех матриц и на все единицы на диагонали матрицы **L**. Можете свободно обследовать ранги матриц с другими размерами и рангами.

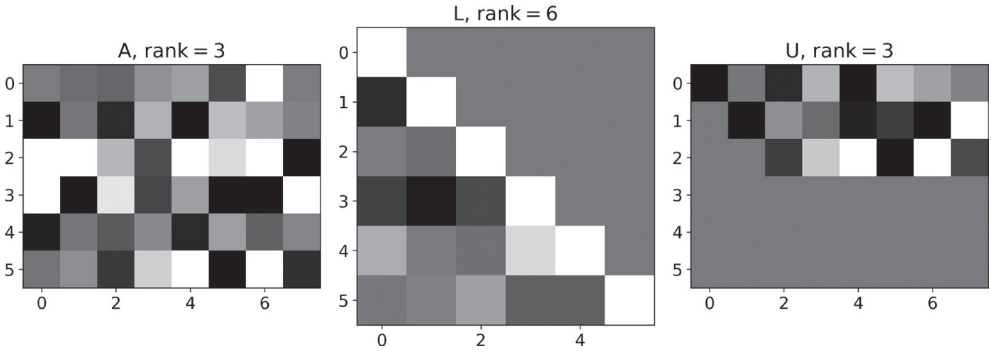


Рис. 10.2 ❖ Результаты упражнения 10.2

Упражнение 10.3

Одним из примерений LU-разложения является вычисление определителя[1]. Вот два свойства определителя: определитель треугольной матрицы равен произведению диагоналей, а определитель матрицы произведения равен произведению определителей (то есть $det(\mathbf{AB}) = det(\mathbf{A})det(\mathbf{B})$). Объединив эти два факта вместе, вы сможете вычислить определитель матрицы как произведение диагоналей матрицы **L**, умноженное на произведение диагоналей матрицы **U**. С другой стороны, поскольку все диагонали матрицы **L** равны 1 (при реализации на Python в целях обеспечения уникальности разложения), определитель матрицы **A** – это просто произведение диагоналей матрицы **U**. Попробуйте это на Python – и, перед тем как читать следующий далее абзац, сравните с результатом вызова функции `np.linalg.det(A)` – несколько раз с разными случайными матрицами.

[1] Это один из бесчисленных аспектов линейной алгебры, которые вы могли бы изучить в традиционном учебнике по линейной алгебре; они интересны сами по себе, но имеют меньшее отношение к науке о данных.

Получили ли вы тот же результат, что и Python? Допускаю, что вы обнаружили совпадение детерминантов по величине, но знаки, по-видимому, будут различаться случайным образом. Почему это произошло? Это произошло потому, что в инструкциях я опустил матрицу перестановок. Определитель матрицы перестановок равен +1 при четном числе взаимообменов строк и −1 при нечетном числе взаимообменов строк. Теперь вернитесь к своему исходному коду и вставьте определитель **P** в свои вычисления.

Упражнение 10.4

Следуя формуле из раздела «LU-разложение» на стр. 183, обратная матрица выражается как

$$\mathbf{A} = \mathbf{P}^{\mathrm{T}}\mathbf{LU};$$

$$\mathbf{A}^{-1} = (\mathbf{P}^{\mathrm{T}}\mathbf{LU})^{-1};$$

$$\mathbf{A}^{-1} = \mathbf{U}^{-1}\mathbf{L}^{-1}\mathbf{P}.$$

Реализуйте третье уравнение напрямую, используя данные на выходе из функции `scipy.linalg.lu` на матрице случайных чисел 4×4. Будет ли $\mathbf{AA}^{-1}$ единичной матрицей? Иногда да, а иногда нет, в зависимости от **P**. Это расхождение возникает по причине описанных мной данных на выходе из функции `scipy.linalg.lu`. Скорректируйте исходный код так, чтобы он соответствовал соглашению, принятому в библиотеке SciPy, а не математическому соглашению.

Вот вывод, который следует вынести из этого упражнения: отсутствие сообщений об ошибках не обязательно означает, что ваш исходный код является правильным. Пожалуйста, проверяйте исправность своего математического исходного кода как можно тщательнее.

Упражнение 10.5

Для матрицы **A** = **PLU** (с использованием упорядочения матрицы перестановок на Python) $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ может вычисляться как $\mathbf{U}^{\mathrm{T}}\mathbf{L}^{\mathrm{T}}\mathbf{LU}$ – *без матриц перестановок*. Почему можно отбросить матрицу перестановок? Ответьте на вопрос, а затем, используя случайные матрицы, подтвердите на Python, что $\mathbf{A}^{\mathrm{T}}\mathbf{A} = \mathbf{U}^{\mathrm{T}}\mathbf{L}^{\mathrm{T}}\mathbf{LU}$, даже при $\mathbf{P} \neq \mathbf{I}$.

Глава 11

Общие линейные модели и наименьшие квадраты

Вселенная – действительно большое и реально замысловатое место. Все животные на Земле обладают естественным любопытством, побуждающим их обследовать и пытаться понять окружающую среду, но мы, люди, наделены интеллектом, позволяющим разрабатывать научные и статистические инструменты, чтобы выводить наше любопытство на новый уровень. Вот почему у нас есть самолеты, аппараты МРТ, марсоходы, вакцины и, конечно же, книги, подобные этой.

Как мы познаем вселенную? Разрабатывая математически обоснованные теории и собирая данные, чтобы тестировать и совершенствовать эти теории. И это подводит нас к статистическим моделям. Статистическая модель – это упрощенное математическое представление какого-то отдельного аспекта мира. Некоторые статистические модели характерны своей простотой (например, предсказание роста фондового рынка в течение десятилетий); другие гораздо более изощренны, например проект Blue Brain, в котором симулируется деятельность мозга с такой точностью, что одна секунда симулируемой активности требует 40 минут вычислительного времени.

Ключевое отличие *статистических* моделей (в отличие от других математических моделей) заключается в том, что они содержат свободные параметры, которые подгоняются к данным. Например, я знаю, что фондовый рынок со временем вырастет, но не знаю, насколько. Поэтому я допускаю, что изменение цены на фондовом рынке с течением времени (то есть наклон[1]) является свободным параметром, числовое значение которого определяется данными.

Разработка статистической модели сопряжена с различными трудностями и требует творческого подхода, опыта и знаний. Но отыскание свободных параметров, основываясь на подгонке модели к данным, является элементарным вопросом линейной алгебры – на самом деле вы уже знаете всю математику, необходимую для этой главы; это просто вопрос соединения частей воедино и усвоения статистической терминологии.

---

[1] Син. угол наклона, угловой коэффициент. – *Прим. перев.*

Общие линейные модели

Статистическая модель представляет собой систему уравнений, связывающих предсказатели (именуемые *независимыми переменными*) с наблюдениями (именуемыми *зависимой переменной*). В модели фондового рынка независимой переменной является *время*, а зависимой переменной – *цена на фондовом рынке* (например, количественно определяемая как индекс S&P 500).

В этой книге я сосредоточусь на общих линейных моделях, сокращенно обозначаемых как GLM[1]. Например, регрессия является одним из типов общей линейной модели.

Терминология

Статистики используют несколько иную терминологию, чем линейные алгебраисты. В табл. 11.1 показаны ключевые буквы и описания векторов и матриц, используемых в общей линейной модели.

Таблица 11.1. Таблица членов общей линейной модели

Линейная алгебра	Статистика	Описание
$Ax = b$	$X\beta = y$	Общая линейная модель (GLM)
A	X	Расчетная матрица (столбцы = независимые переменные, предсказатели, регрессоры)
x	β	Коэффициенты регрессии или бета-параметры
b	y	Зависимая переменная, исход, мера результата, данные

Настройка общей линейной модели

Настройка общей линейной модели предусматривает:

1) определение уравнения, которое связывает предсказательные переменные с зависимой переменной;
2) отображение наблюдаемых данных в уравнения;
3) преобразование серии уравнений в матричное уравнение;
4) решение этого уравнения.

Я буду использовать простой пример, чтобы конкретизировать указанную процедуру. У меня есть модель, которая предсказывает рост взрослого человека на основе веса и роста родителей. Уравнение выглядит следующим образом:

$$y = \beta_0 + \beta_1 w + \beta_2 h + \epsilon,$$

[1] Англ. *General Linear Model*. – *Прим. перев.*

где y – это рост человека, w – его вес, h – рост его родителей (средние показатели матери и отца). ϵ – ошибка (также именуемая остатком), потому что невозможно разумно ожидать, что вес и рост родителей полностью определяют рост человека; наша модель не учитывает громадное число факторов, и дисперсия, не связанная с весом и ростом родителей, будет поглощена остатком.

Моя гипотеза состоит в том, что вес и рост родителей имеют важность для роста человека, но я не знаю степень важности каждой переменной. Теперь введем члены β: это коэффициенты, или веса, которые говорят мне о том, как комбинировать вес и рост родителей, чтобы предсказывать рост человека. Другими словами, это линейно-взвешенная комбинация, где β – это веса.

Член β_0 называется *пересечением*[1] (иногда его называют *константой*). Член пересечения – это вектор, состоящий из одних единиц. Без указанного члена линия наилучшей подгонки будет вынужденно проходить через начало координат. Я объясню причину и покажу демонстрацию ближе к концу главы.

Теперь у нас есть уравнение, модель Вселенной (ну, хотя бы одна крошечная ее часть). Далее необходимо отобразить наблюдаемые данные в уравнения. Для простоты я выдумаю немного данных и сведу их в табл. 11.2 (как вы, наверное, догадались, y и h измеряются в сантиметрах, а w – в килограммах).

Таблица 11.2. Выдуманные данные для статистической модели роста

y	w	h
175	70	177
181	86	190
159	63	180
165	62	172

Отображение наблюдаемых данных в нашу статистическую модель предусматривает повторение уравнения четыре раза (что соответствует четырем наблюдениям в наборе данных), всякий раз заменяя переменные y, w и h измеренными данными:

$$175 = \beta_0 + 70\beta_1 + 177\beta_2 + \epsilon;$$

$$181 = \beta_0 + 86\beta_1 + 190\beta_2 + \epsilon;$$

$$159 = \beta_0 + 63\beta_1 + 180\beta_2 + \epsilon;$$

$$165 = \beta_0 + 62\beta_1 + 172\beta_2 + \epsilon.$$

Пока что я опускаю член ϵ; об остатках я расскажу позже. Теперь нам нужно переложить эту систему уравнений в матричное уравнение. Я знаю, что вы знаете, как это делается, поэтому распечатаю здесь уравнение только для того, чтобы вы могли подтвердить то, что уже знаете из главы 10:

---

[1] Син. точка пересечения, коэффициент сдвига. – *Прим. перев.*

$$\begin{bmatrix} 1 & 70 & 177 \\ 1 & 86 & 190 \\ 1 & 63 & 180 \\ 1 & 62 & 172 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix} = \begin{bmatrix} 175 \\ 181 \\ 159 \\ 165 \end{bmatrix}.$$

И конечно же, это уравнение можно выразить кратко как $\mathbf{X\beta} = \mathbf{y}$.

Решение общих линейных моделей

Уверен, что вы уже знаете главную идею этого раздела: для того чтобы найти вектор неизвестных коэффициентов $\boldsymbol{\beta}$, надо просто умножить обе части уравнения слева на левообратную матрицу $\mathbf{X}$, то есть расчетную матрицу. Решение выглядит так:

$$\mathbf{X\beta} = \mathbf{y};$$
$$(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{X\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y};$$
$$\boldsymbol{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}.$$

Пожалуйста, всмотритесь в заключительное уравнение и удерживайте свой взор на нем до тех пор, пока оно не отпечатается в вашем мозгу навсегда. Оно называется *решением методом наименьших квадратов* и является одним из наиболее важных математических уравнений в прикладной линейной алгебре. Вы будете встречать его в научных публикациях, учебниках, блогах, лекциях, документационных литералах по функциям Python, на рекламных щитах в Таджикистане[1] и во многих других местах. При этом вы будете видеть другие буквы или, возможно, некоторые дополнения, например следующие ниже:

$$\mathbf{b} = (\mathbf{H}^T\mathbf{WH} + \lambda\mathbf{L}^T\mathbf{L})^{-1}\mathbf{H}^T\mathbf{x}.$$

Смысл этого уравнения и интерпретация дополнительных матриц не важны (это различные способы регуляризации подгонки модели); важно то, что вы способны увидеть формулу наименьших квадратов, встроенную в это замысловато выглядящее уравнение (например, представьте, что $\mathbf{W} = \mathbf{I}$ и $\lambda = 0$).

Решение методом наименьших квадратов посредством левообратной матрицы можно переложить напрямую в исходный код Python (переменная `X` – это расчетная матрица, а переменная `y` – вектор данных):

```
X_leftinv = np.linalg.inv(X.T@X) @ X.T

# найти коэффициенты
beta = X_leftinv @ y
```

[1] Признаться, никогда не видел это уравнение на таджикском рекламном щите, но суть в том, чтобы быть открытым ко всему новому.

Позже в этой главе я покажу результаты этих моделей – и то, как их интерпретировать; сейчас же я бы хотел, чтобы вы сосредоточились на том, как математические формулы переводятся в исходный код Python.



Левообратная матрица в сосоставлении с решателем методом наименьших квадратов NumPy

Исходный код в этой главе является прямым переложением математики в исходный код Python. Вычисление левообратной матрицы в явной форме – не самый численно стабильный способ решения общей линейной модели (хотя и точен для простых задач этой главы), но я хочу, чтобы вы увидели, что кажущаяся абстрактной линейная алгебра работает реально. Существуют более численно стабильные способы решения общей линейной модели, включая QR-разложение (который вы увидите позже в этой главе) и более численно стабильные методы Python (с которыми вы познакомитесь в следующей главе).

Является ли решение точным?

Уравнение $\mathbf{X\beta} = \mathbf{y}$ точно разрешимо, когда $\mathbf{y}$ находится в столбцовом пространстве расчетной матрицы $\mathbf{X}$. Поэтому вопрос заключается в том, гарантированно ли вектор данных находится в столбцовом пространстве статистической модели. Ответ – нет, такой гарантии нет. На самом деле вектор данных $\mathbf{y}$ почти никогда не находится в столбцовом пространстве матрицы $\mathbf{X}$.

В целях понимания причины, по которой такой гарантии нет, давайте представим опрос студентов университета, в ходе которого исследователи пытаются предсказать средний балл GPA (средний балл успеваемости) на основе поведения, связанного с употреблением алкоголя. Опрос может содержать данные, полученные от двух тысяч студентов, но имеет только три вопроса (например, сколько алкоголя вы употребляете; как часто вы теряете сознание; какой у вас средний балл). Данные содержатся в таблице 2000×3. Столбцовое пространство расчетной матрицы является 2D-подпространством внутри этой объемлющей размерности 2000D, а вектор данных является 1D-подпространством внутри той же объемлющей размерности.

Если данные находятся в столбцовом пространстве расчетной матрицы, то это означает, что модель учитывает 100 % дисперсии данных. Но этого почти никогда не происходит: реальные данные содержат шум и изменчивость отбора экземпляров, а модели представляют собой упрощения, не учитывающие всей изменчивости (например, средний балл GPA определяется громадным числом факторов, которые наша модель игнорирует).

Решение этой головоломки заключается в видоизменении уравнения общей линейной модели так, чтобы учесть расхождение между модельно-предсказанными и наблюдаемыми данными. Его можно выразить несколькими эквивалентными способами (с точностью до знака):

$$\mathbf{X\beta} = \mathbf{y} + \boldsymbol{\epsilon};$$

$$\mathbf{X\beta} + \boldsymbol{\epsilon} = \mathbf{y};$$

$$\boldsymbol{\epsilon} = \mathbf{X\beta} - \mathbf{y}.$$

Первое уравнение интерпретируется так: ϵ – это остаток, или член ошибки, который добавляется в вектор данных, чтобы он помещался внутри столбцового пространства расчетной матрицы. Второе уравнение интерпретируется следующим образом: остаточный член – это корректировка расчетной матрицы таким образом, чтобы она идеально вписывалась в данные. Наконец, интерпретация третьего уравнения такова: остаток определяется как разница между модельно-предсказанными и наблюдаемыми данными.

Существует еще одна, очень проницательная интерпретация, которая подходит к общим линейным моделям и методу наименьших квадратов с геометрической точки зрения. Я вернусь к ней в следующем разделе.

Суть же этого раздела в том, что наблюдаемые данные почти никогда не находятся внутри подпространства, охватываемого регрессорами. По этой причине также нередко можно увидеть, как общая линейная модель выражается как $\mathbf{X} = \boldsymbol{\beta}\hat{\mathbf{y}}$, где $\hat{\mathbf{y}} = \mathbf{y} + \boldsymbol{\epsilon}$.

Следовательно, цель общей линейной модели состоит в том, чтобы найти линейную комбинацию регрессоров, максимально приближенную к наблюдаемым данным. Подробнее об этом позже; теперь же я хочу познакомить вас с геометрической перспективой наименьших квадратов.

Геометрическая перспектива наименьших квадратов

До сих пор я представлял решение общей линейной модели с алгебраической точки зрения решения матричного уравнения. Существует также геометрическая перспектива общей линейной модели, которая обеспечивает альтернативную перспективу и помогает раскрыть несколько важных особенностей решения задачи о наименьших квадратах.

Предположим, что столбцовое пространство расчетной матрицы $C(\mathbf{X})$ является подпространством $\mathbb{R}^M$. В типичной ситуации это очень низкоразмерное подпространство (то есть $N << M$), потому что статистические модели, как правило, имеют гораздо больше наблюдений (строк), чем предсказателей (столбцов). Зависимая переменная – это вектор $\mathbf{y} \in \mathbb{R}^M$. Возникают вопросы: принадлежит ли вектор $\mathbf{y}$ столбцовому пространству расчетной матрицы, и если нет, то какая координата внутри столбцового пространства расчетной матрицы максимально близка к вектору данных?

Ответ на первый вопрос – нет, как я уже говорил в предыдущем разделе. Второй вопрос имеет большую важность, потому что вы уже узнали ответ в главе 2. Рассмотрите рис. 11.1, думая о решении.

Итак, наша цель – найти набор коэффициентов $\boldsymbol{\beta}$ такой, что взвешенная комбинация столбцов в $\mathbf{X}$ минимизирует расстояние до вектора данных $\mathbf{y}$. Мы можем назвать этот проекционный вектор $\boldsymbol{\epsilon}$. Как найти вектор $\boldsymbol{\epsilon}$ и коэффициенты $\boldsymbol{\beta}$? Ортогональная проекция вектора используется точно так же, как вы узнали в главе 2. Это означает, что можно применить тот же подход, что и в главе 2, но с использованием матриц вместо векторов. Ключевым

моментом является то, что кратчайшее расстояние между **y** и **X** определяется проекционным вектором **y** – **Xβ**, который пересекает **X** под прямым углом:

$$\begin{aligned}\mathbf{X}^T\boldsymbol{\epsilon} &= \mathbf{0};\\ \mathbf{X}^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) &= \mathbf{0};\\ \mathbf{X}^T\mathbf{y} - \mathbf{X}^T\mathbf{X}\boldsymbol{\beta} &= \mathbf{0};\\ \mathbf{X}^T\mathbf{X}\boldsymbol{\beta} &= \mathbf{X}^T\mathbf{y};\\ \boldsymbol{\beta} &= (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}.\end{aligned}$$

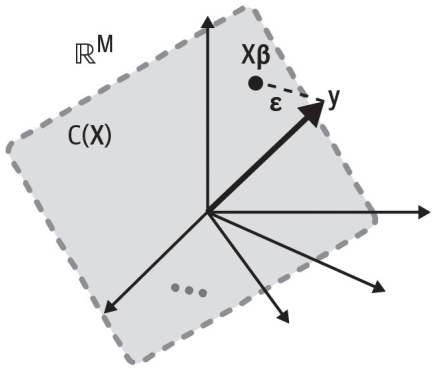


Рис. 11.1 ❖ Абстрактное геометрическое представление общей линейной модели: найдите точку в столбцовом пространстве расчетной матрицы, которая максимально близка к вектору данных

Приведенная выше прогрессия уравнений просто замечательна: мы начали с мысли об общей линейной модели как о геометрической проекции вектора данных на столбцовое пространство расчетной матрицы, применили принцип ортогональной проекции вектора, о котором вы узнали в начале книги, и вуаля! Мы повторно получили то же самое левообратное решение, что и при алгебраическом подходе.

В чем причина работы метода наименьших квадратов?

Почему этот метод называется «наименьшими квадратами»? Что это за так называемые квадраты и почему этот метод дает нам наименьший из них?

«Квадраты» здесь относятся к квадратам ошибок между предсказанными и наблюдаемыми данными. Для каждой -й предсказанной точки данных существует член ошибки, который определяется как $\epsilon_i = \mathbf{X}_i\boldsymbol{\beta} - \mathbf{y}_i$. Обратите внимание, что каждая точка данных предсказывается с использованием одного и того же набора коэффициентов (то есть одинаковых весов, служащих для комбинирования предсказателей в расчетной матрице). Все ошибки можно улавливать в одном векторе:

$$\boldsymbol{\epsilon} = \mathbf{X}\boldsymbol{\beta} - \mathbf{y}.$$

Если модель вписывается в данные хорошо, то ошибки должны быть малыми. Следовательно, можно сказать, что цель подгонки модели состоит в том, чтобы выбирать элементы в $\boldsymbol{\beta}$, которые минимизируют элементы в $\boldsymbol{\epsilon}$. Но только одна *минимизация* ошибок приведет к тому, что модель будет предсказывать значения в сторону отрицательной бесконечности. И следовательно, мы вместо этого минимизируем *квадраты* ошибок, которые соответствуют их геометрическому квадрату расстояния до наблюдаемых данных $\mathbf{y}$, независимо от того, является ли сама ошибка предсказания положительной либо отрицательной[1]. Это то же самое, что минимизировать квадрат нормы ошибок. Отсюда и название «наименьшие квадраты». Все это приводит к следующей ниже модификации:

$$||\boldsymbol{\epsilon}||^2 = ||\mathbf{X}\boldsymbol{\beta} - \mathbf{y}||^2.$$

Теперь на нее можно смотреть как на задачу оптимизации. В частности, мы хотим найти набор коэффициентов, который минимизирует квадраты ошибок. Эта минимизация выражается следующим образом:

$$\min_{\boldsymbol{\beta}} ||\mathbf{X}\boldsymbol{\beta} - \mathbf{y}||^2.$$

Решение этой оптимизации можно найти, установив производную целевой функции равной нулю и применив чуть-чуть дифференциального исчисления[2] и немного алгебры:

$$\begin{aligned} 0 = \frac{d}{d\boldsymbol{\beta}} ||\mathbf{X}\boldsymbol{\beta} - \mathbf{y}||^2 &= 2\mathbf{X}^{\mathrm{T}}(\mathbf{X}\boldsymbol{\beta} - \mathbf{y}); \\ 0 &= \mathbf{X}^{\mathrm{T}}\mathbf{X}\boldsymbol{\beta} - \mathbf{X}^{\mathrm{T}}\mathbf{y}; \\ \mathbf{X}^{\mathrm{T}}\mathbf{X}\boldsymbol{\beta} &= \mathbf{X}^{\mathrm{T}}\mathbf{y}; \\ \boldsymbol{\beta} &= (\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{y}. \end{aligned}$$

Удивительно, но мы начали с другой точки зрения – минимизировали квадрат расстояния между модельно-предсказанными и наблюдаемыми значениями – и снова переоткрыли то же самое решение задачи о наименьших квадратах, которое мы получили, просто используя линейно-алгебраическую интуицию.

На рис. 11.2 показано немного наблюдаемых данных (черными квадратами), их модельно-предсказанные значения (серыми точками) и расстояния между ними (серыми пунктирными линиями). Все модельно-предсказанные

---

[1] Если вам интересно, то вместо квадратов расстояний также можно минимизировать абсолютные расстояния. Эти две цели могут приводить к разным результатам; одним из преимуществ квадрата расстояния является удобная производная, которая приводит к решению методом наименьших квадратов.

[2] Если вас не устраивает матричное исчисление, то не беспокойтесь об уравнениях; дело в том, что мы взяли производную по $\boldsymbol{\beta}$, используя цепное правило.

значения лежат на одной линии; цель метода наименьших квадратов состоит в том, чтобы найти наклон и пересечение этой линии, которые минимизируют расстояния от предсказанных до наблюдаемых данных.

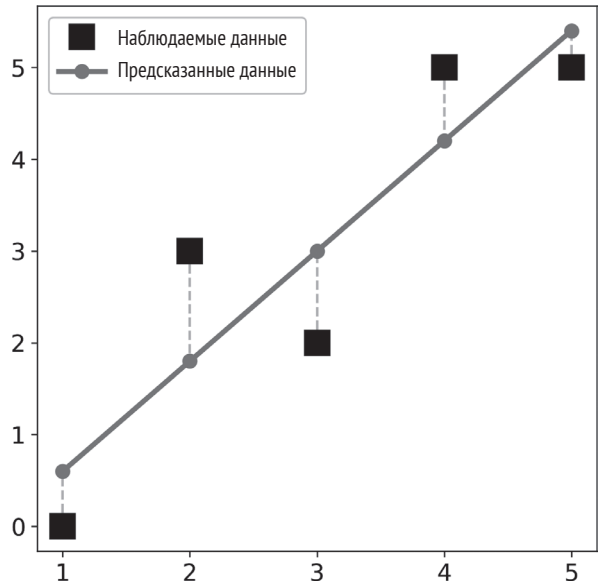


Рис. 11.2 ❖ Визуальное понимание наименьших квадратов на интуитивном уровне

Все дороги ведут к наименьшим квадратам

Вы познакомились с тремя способами получения решения задачи о наименьших квадратах. Примечательно, что все подходы приводят к одному и тому же заключению: надо умножить обе части уравнения общей линейной модели слева на левообратную матрицу расчетной матрицы **X**. Разные подходы имеют уникальные теоретические перспективы, которые обеспечивают понимание природы и оптимальности метода наименьших квадратов. Но вся красота в том, что независимо от того, как начинать свое приключение в подгонке линейной модели, все в конечном итоге сходится к одному и тому же выводу.

Общая линейная модель на простом примере

В следующей главе вы увидите несколько примеров с реальными данными; здесь же я хочу сосредоточиться на простом примере с поддельными данными. Поддельные данные получены в результате поддельного эксперимента, в ходе которого я опросил случайную группу из 20 моих поддельных студен-

тов и попросил их сообщить число пройденных ими моих онлайновых курсов и их общую удовлетворенность жизнью[1].

В табл. 11.3 показаны первые 4 (из 20) строки матрицы данных.

Таблица 11.3. Таблица данных

Число курсов	Жизненное счастье
4	25
12	54
3	21
14	80

Эти данные легче визуализировать в виде диаграммы рассеивания, которую вы видите на рис. 11.3.

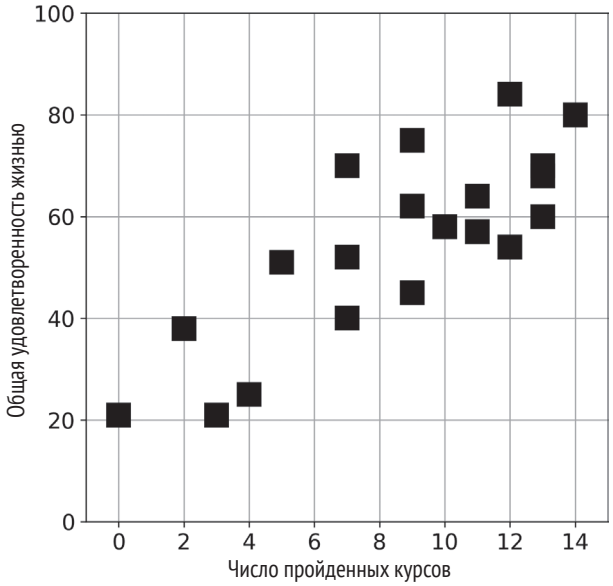


Рис. 11.3 ❖ Поддельные данные из поддельного опроса

Обратите внимание, что независимая переменная отложена по оси *x*, а зависимая переменная отложена по оси *y*. В статистике это общепринятое правило.

Нам нужно создать расчетную матрицу. Поскольку это простая модель только с одним предсказателем, расчетная матрица на самом деле представляет собой только один вектор-столбец. Матричное уравнение $\mathbf{X\beta} = \mathbf{y}$ выглядит так (опять же, только первые четыре значения данных):

[1] В случае если это еще не совсем ясно: в данном примере используются полностью выдуманные данные; любое сходство с реальным миром имеет случайный характер.

$$\begin{bmatrix}25\\54\\21\\80\end{bmatrix}[\beta] = \begin{bmatrix}4\\12\\3\\14\end{bmatrix}.$$

Следующий ниже исходный код Python показывает решение. Переменные `numcourses` и `happy` содержат данные; они обе являются списками и поэтому должны быть конвертированы в многомерные массивы NumPy:

```
X = np.array(numcourses,ndmin=2).T

# вычислить левообратную матрицу
X_leftinv = np.linalg.inv(X.T@X) @ X.T

# найти коэффициенты
beta = X_leftinv @ happiness
```

Формула наименьших квадратов говорит о том, что **β** = 5.95. Что означает это число? Это наклон в нашей формуле. Другими словами, по каждому дополнительному курсу, который кто-либо проходит, его самооценка жизненного счастья увеличивается на 5.95 балла. Давайте посмотрим на то, как этот результат выглядит на графике. На рис. 11.4 показаны данные (черными квадратами), предсказанные значения счастья (соединенными линией серыми точками) и остатки (пунктирной линией, соединяющей каждое наблюдаемое значение с предсказанным значением).

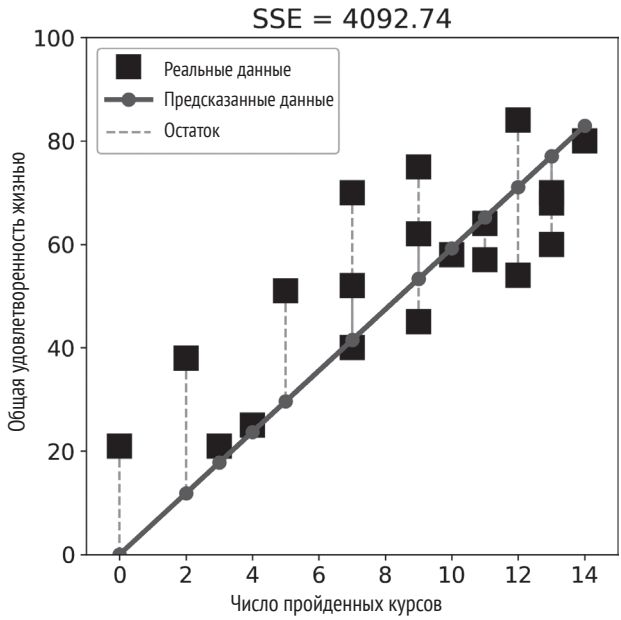


Рис. 11.4 ❖ Наблюдаемые и предсказанные данные (SSE = сумма квадратов ошибок)

Если, глядя на рис. 11.4, вы испытываете чувство беспокойства, то это хорошо – значит, вы мыслите критически и заметили, что модель не очень хорошо справляется с минимизацией ошибок. Можно легко представить, что левая часть линии наилучшей подгонки должна быть сдвинута вверх, чтобы получить более оптимальную подгонку. В чем тут проблема?

Проблема в том, что расчетная матрица не содержит пересечения. Уравнение линии наилучшей подгонки таково: $y = mx$, означая, что $y = 0$ при $x = 0$. В этой задаче указанное ограничение не имеет смысла – было бы очень печально, если бы кто-то, кто не проходит мои курсы, был бы полностью лишен удовлетворенности жизнью. Вместо этого мы хотим, чтобы наша линия имела форму $y = mx + b$, где b – это член пересечения, который дает линии наилучшей подгонки возможность пересекать ось y при любом значении. Статистически пересечение интерпретируется как ожидаемое числовое значение наблюдений, когда предсказатели заданы равными нулю.

Добавление члена пересечения в расчетную матрицу дает следующие ниже модифицированные уравнения (опять же, показаны только первые четыре строки):

$$\begin{bmatrix} 1 & 25 \\ 1 & 54 \\ 1 & 21 \\ 1 & 80 \end{bmatrix} \begin{bmatrix} \beta_1 \\ \beta_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 12 \\ 3 \\ 14 \end{bmatrix}.$$

Исходный код не меняется, за исключением создания расчетной матрицы:

```
X = np.hstack((np.ones((20,1)),
               np.array(numcourses,ndmin=2).T))
```

Теперь мы обнаруживаем, что $\boldsymbol{\beta}$ – это двухэлементный вектор [22.9, 3.7]. Ожидаемый уровень счастья для того, кто прошел ноль курсов, составляет 22.9, а каждый дополнительный курс увеличивает его счастье на 3.7 балла. Уверен, вы согласитесь, что рис. 11.5 выглядит намного лучше. И SSE составляет примерно половину того, что было, когда мы исключили пересечение.

Я позволю вам извлечь свои собственные выводы о поддельных результатах этого поддельного исследования, основанного на поддельных данных; вся суть была в том, чтобы увидеть численный пример того, как решать систему уравнений путем формирования надлежащей расчетной матрицы и отыскания решения для неизвестных регрессоров с помощью левообратной матрицы.

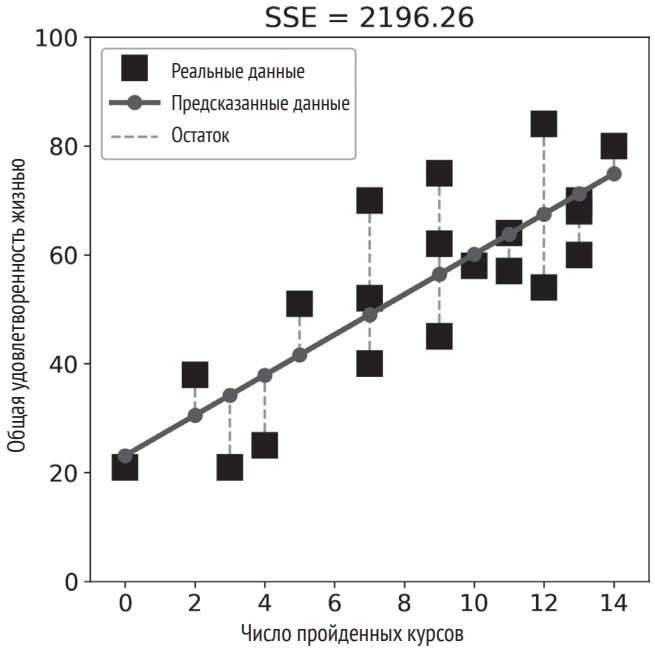


Рис. 11.5 ❖ Наблюдаемые и предсказанные данные, теперь с пересечением

Наименьшие квадраты посредством QR-разложения

Левообратный метод теоретически обоснован, но рискует численной нестабильностью. Отчасти это связано с тем, что для него требуется вычислять обратную матрицу, которая, как вы уже знаете, может быть численно нестабильной. Но оказывается, что сама матрица $\mathbf{X}^T\mathbf{X}$ может доставлять трудности. Умножение матрицы на ее транспонированную версию влияет на такие свойства, как норма и кондиционное число. В главе 14 вы узнаете о кондиционном числе подробнее, но я уже упоминал, что матрицы с высоким кондиционным числом бывают численно нестабильными и, следовательно, расчетная матрица с высоким кондиционным числом станет еще менее численно стабильной при возведении в квадрат.

QR-разложение обеспечивает более стабильный способ решения задачи о наименьших квадратах. Проследите за следующей ниже последовательностью уравнений:

$$\mathbf{X\beta} = \mathbf{y};$$
$$\mathbf{QX\beta} = \mathbf{y};$$
$$\mathbf{R\beta} = \mathbf{Q}^{\mathrm{T}}\mathbf{y};$$
$$\boldsymbol{\beta} = \mathbf{R}^{-1}\mathbf{Q}^{\mathrm{T}}\mathbf{y}.$$

Во-первых, эти уравнения немного упрощены по сравнению с реальными низкоуровневыми численными реализациями. Например, матрица **R** имеет то же очертание, что и **X**, то есть является высокой (и, следовательно, необратимой), хотя только первые N строк отличны от нуля (как было обнаружено в упражнении 9.7), а это означает, что строки с $N + 1$ по M не вносят вклад в решение (при умножении матриц строки нулей дают нулевые результаты). Эти строки можно удалить из **R** и из $\mathbf{Q}^{\mathrm{T}}\mathbf{y}$. Во-вторых, взаимообмены строк (реализованные посредством матриц перестановок) могут использоваться для повышения численной стабильности.

Но вот что самое приятное: отпадает всякая потребность в инвертировании матрицы **R** – она является верхнетреугольной, и поэтому решение отыскивается с помощью обратной подстановки. Это то же самое, что решать систему уравнений методом Гаусса–Жордана: расширить матрицу коэффициентов константами, выполнить приведение строк, чтобы получить RREF-форму, и извлечь решение из последнего столбца расширенной матрицы.

Вывод здесь таков: QR-разложение решает задачу о наименьших квадратах без возведения $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ в квадрат и без явного инвертирования матрицы. Главный риск численной нестабильности связан с вычислением матрицы **Q**, хотя при реализации посредством отражений Хаусхолдера она будет вполне численно стабильной.

Упражнение 11.3 проведет вас по этой реализации.

Резюме

Многие люди думают, что статистику трудно понять, потому что трудно понять лежащую в ее основе математику. Безусловно, существуют продвинутые статистические методы, предусматривающие продвинутую математику. Но многие широко используемые статистические методы базируются на линейно-алгебраических принципах, которые вы теперь понимаете. Это означает, что у вас больше нет оправданий, чтобы не освоить статистический анализ, который применяется в науке о данных!

Цель этой главы состояла в том, чтобы познакомить вас с терминологией и математикой, лежащими в основе общей линейной модели, геометрической интерпретацией и последствиями математики для минимизации разниц между модельно-предсказанными и наблюдаемыми данными. Я также показал применение регрессии на простом игрушечном примере. В следующей главе вы увидите метод наименьших квадратов, реализованный на реальных данных, и увидите расширения метода наименьших квадратов в регрессии, такие как полиномиальная регрессия и регуляризация.

Вот ключевые выводы, которые следует вынести из этой главы.

- Общая линейная модель (GLM) – это статистический каркас для понимания нашей богатой и прекрасной Вселенной. Она работает путем формирования системы уравнений, точно такой же, как системы уравнений, о которых вы узнали в предыдущей главе.
- Члены уравнений в линейной алгебре и статистике несколько отличаются; поняв терминологическую соотнесенность между ними, статистика станет проще, потому что вы уже знаете математику.
- Метод наименьших квадратов для решения уравнений посредством уравнения левообратной матрицы лежит в основе многих видов статистического анализа, и вы нередко будете видеть, как внутри кажущихся замысловатыми формул «спрятано» решение методом наименьших квадратов.
- Формула наименьших квадратов выводится посредством алгебры, геометрии или дифференциального исчисления. За счет этого обеспечивается несколько способов понимания и интерпретации метода наименьших квадратов.
- Умножение вектора наблюдаемых данных на левообратную матрицу в концептуальном плане является правильным образом мыслей о наименьших квадратах. На практике другие методы, такие как LU- и QR-разложение, являются более численно стабильными. К счастью, по этому поводу беспокоиться не приходится, потому что Python обращается к низкоуровневым библиотекам (в основном библиотеке LAPACK), в которых реализованы наиболее численно стабильные алгоритмы.

Упражнения по программированию

Упражнение 11.1

Я объяснил, что остатки ортогональны предсказанным данным (другими словами, $\epsilon^T\hat{\mathbf{y}} = 0$). Проиллюстрируйте данное утверждение на игрушечном наборе данных из этой главы. В частности, сделайте диаграмму рассеяния предсказанных данных относительно ошибок (как на рис. 11.6). Затем вычислите точечное произведение и коэффициент корреляции между остатками и модельно-предсказанными данными. Теоретически оба результата должны быть равны нулю, хотя будут некоторые ошибки округления. Какой из этих двух видов анализа (точечное произведение либо корреляция) меньше и почему?

Упражнение 11.2

Модельно-предсказанное счастье – это всего лишь один из способов линейного комбинирования столбцов расчетной матрицы. Но вектор остатков не только ортогонален этой одной линейно-взвешенной комбинации; как раз наоборот – вектор остатков ортогонален *всему подпространству*, охватываемому расчетной матрицей. Продемонстрируйте это на Python (подсказка: подумайте о левом нуль-пространстве и ранге).

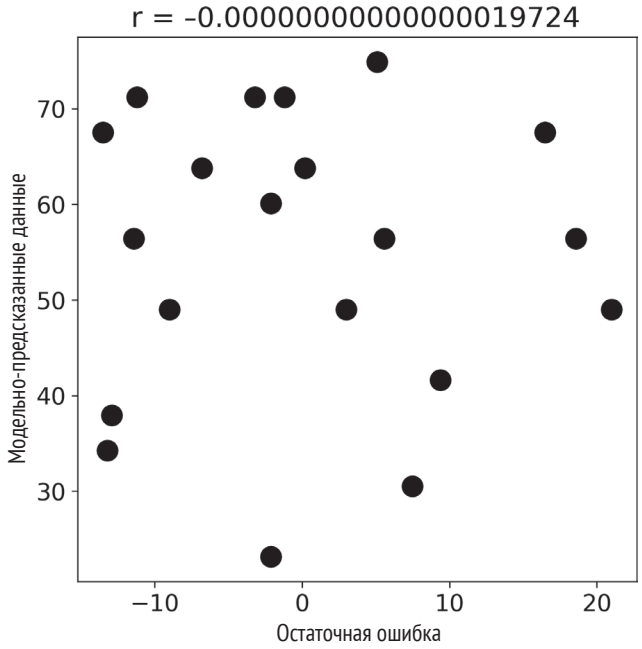


Рис. 11.6 ❖ Решение упражнения 11.1

Упражнение 11.3

Теперь вы собираетесь вычислить метод наименьших квадратов посредством QR-разложения, как я объяснял в разделе «Наименьшие квадраты посредством QR-разложения» на стр. 201. В частности, вычислите и сравните следующие ниже методы решения:

1) метод левообратной матрицы $(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$;
2) метод QR-разложения с обратной матрицей $\mathbf{R}^{-1}\mathbf{Q}^T\mathbf{y}$;
3) метод устранения по Гауссу–Жордану на матрице $\mathbf{R}$, расширенной вектором $\mathbf{Q}^T\mathbf{y}$.

Распечатайте бета-параметры из трех методов, как показано ниже. (Округление до трех цифр после запятой является дополнительным заданием по программированию.)

```
Беты из левообратной матрицы:
[23.13   3.698]

Беты из QR с inv(R):
[23.13   3.698]

Беты из QR с обратной подстановкой:
[[23.13   3.698]]
```

Наконец, распечатайте результирующие матрицы, полученные из QR-метода, как показано ниже:

```
Матрица R:
[[ -4.472 -38.237]
 [  0.     17.747]]

Матрица R|Q'y:
[[  -4.472  -38.237 -244.849]
 [   0.      17.747   65.631]]

Матрица RREF(R|Q'y):
[[ 1.    0.    23.13 ]
 [ 0.    1.     3.698]]
```

Упражнение 11.4

Выбросы – это значения данных, которые являются необычными или нерепрезентативными. Выбросы могут вызвать серьезные проблемы в статистических моделях и, следовательно, могут вызывать серьезную «головную боль» у исследователей данных. В этом упражнении мы создадим выбросы в данных о счастье, чтобы понаблюдать за их влиянием на результирующее решение задачи о наименьших квадратах.

В векторе данных измените первую наблюдаемую точку данных с 70 на 170 (имитация опечатки при вводе данных). Затем пересчитайте подгонку методом наименьших квадратов и выведите данные на график. Повторите эту симуляцию выброса, но измените окончательную точку данных с 70 на 170 (и верните первой точке данных ее исходное значение 70). Сравните с изначальными данными, создав визуализацию, как на рис. 11.7.

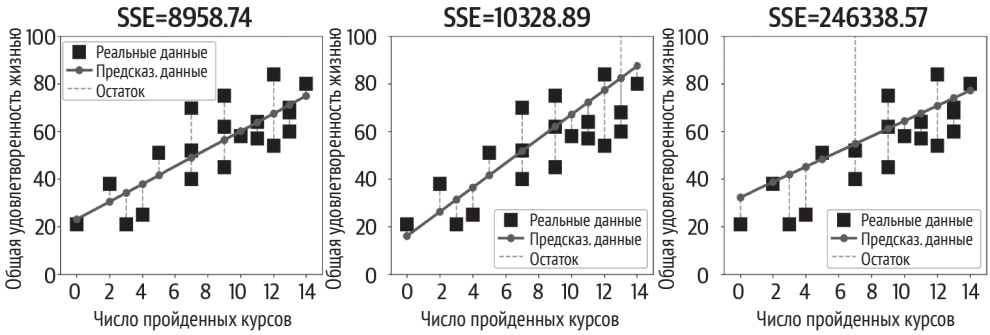


Рис. 11.7 ❖ Решение упражнения 11.4

Интересно, что выброс был идентичен в переменной исхода (в обоих случаях 70 превратились в 170), но влияние на подгонку модели к данным было совершенно разным из-за соответствующего значения по оси *x*. Это дифференцирующее влияние выбросов называется *рычагом*, и это то, о чем вы узнаете из более глубокого обсуждения статистики и подгонки моделей.

Упражнение 11.5

В этом упражнении вы вычислите обратную матрицу с помощью наименьших квадратов, следуя интерпретации, которую я представил в предыдущей

главе. Мы рассмотрим уравнение $\mathbf{XB} = \mathbf{Y}$, где $\mathbf{X}$ – это полноранговая квадратная матрица, которую необходимо инвертировать, $\mathbf{B}$ – матрица неизвестных коэффициентов (которая будет обратной матрицей), а $\mathbf{Y}$ – «наблюдаемые данные» (единичная матрица).

Вы вычислите $\mathbf{B}$ тремя способами. Во-первых, примените левообратный метод наименьших квадратов, чтобы вычислять матрицу по одному столбцу за раз. Это делается путем вычисления наименьших квадратов между матрицей $\mathbf{X}$ и каждым столбцом матрицы $\mathbf{Y}$ в цикле `for`. Во-вторых, примените левообратный метод, чтобы вычислить всю матрицу $\mathbf{B}$ в одной строке исходного кода. И наконец, вычислите $\mathbf{X}^{-1}$ с помощью функции `np.linalg.inv()`. Умножьте каждую из этих матриц $\mathbf{B}$ на $\mathbf{X}$ и покажите на рисунке, подобном рис. 11.8. Наконец, проверьте эквивалентность этих трех «разных» способов вычисления обратной матрицы (так и должно быть, потому что обратная матрица является уникальной).

Наблюдение: довольно странно (не говоря уже о том, чтобы зацикленно) использовать обратную матрицу матрицы $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ для вычисления обратной матрице $\mathbf{X}$ матрицы! Излишне говорить, это не вычислительный метод, который можно было бы реализовать на практике. Однако это упражнение усиливает интерпретацию обратной матрицы как преобразования, которое проецирует матрицу на единичную матрицу, и понимание того, что эта проекционная матрица может быть получена методом наименьших квадратов. Сравнение решения задачи о наименьших квадратах с функцией `np.linalg.inv` также иллюстрирует численные неточности, которые могут возникать при вычислении левообратной матрицы.



Рис. 11.8 ❖ Решение упражнения 11.5

Глава 12

Применения метода наименьших квадратов

В этой главе вы увидите несколько применений подгонки на основе наименьших квадратов к реальным данным. Попутно вы научитесь реализовывать метод наименьших квадратов, используя несколько разных и более численно стабильных функций Python, а также усвоите несколько новых понятий из статистики и машинного обучения, таких как мультиколлинеарность, полиномиальная регрессия и алгоритм поиска в параметрической решетке как альтернатива методу наименьших квадратов.

К концу этой главы у вас будет более глубокое понимание приемов применения метода наименьших квадратов в приложениях, включая важность численно стабильных алгоритмов для «трудных» ситуаций, связанных с рангово-пониженными расчетными матрицами. И вы увидите, что обеспечиваемое методом наименьших квадратов аналитическое решение превосходит эмпирический метод поиска значений параметров.

Предсказывание количеств велопрокатов на основе погоды

Я – большой поклонник велосипедов и большой поклонник пибимпапа (корейского блюда, приготовляемого из риса и овощей или мяса). Поэтому я был счастлив найти общедоступный набор данных о велопрокатах в Сеуле[1]. Этот набор данных содержит почти девять тысяч наблюдений о количестве вело-

[1] *Сатишкумар В. Э., Пак Чанву и Чо Йонгюн.* Применение методов глубокой переработки данных для предсказания спроса на велопрокат в столичном городе (V. E. Sathishkumar, Jangwoo Park, and Yongyun Cho, Using Data Mining Techniques for Bike Sharing Demand Prediction in Metropolitan City, Computer Communications, 153, (March 2020): 353–366); данные скачаны с https://archive.ics.uci.edu/ml/datasets/Seoul+Bike+Sharing+Demand.

сипедов, бравшихся напрокат внутри города, и переменных о погоде, включая температуру, влажность, количество осадков, скорость ветра и т. д.

Указанный набор данных предназначен для предсказания спроса на велопрокат в зависимости от погоды и времени года. Такая информация важна потому, что это поможет компаниям-арендодателям велосипедов и местным органам власти оптимизировать доступность более здоровых видов транспорта. Это отличный набор данных, с ним можно многое сделать, и я призываю вас потратить время на его обследование. В данной главе я сосредоточусь на разработке относительно простых регрессионных моделей предсказания количества велопрокатов на основе нескольких признаков.

Хотя эта книга посвящена линейной алгебре, а не статистике, тем не менее, прежде чем применять и интерпретировать какой-либо статистический анализ, важно тщательно инспектировать данные. Онлайновый исходный код содержит более подробную информацию об импорте и инспектировании данных с использованием библиотеки pandas. На рис. 12.1 показаны данные о велопрокатах (зависимая переменная) и количестве осадков (одна из независимых переменных).

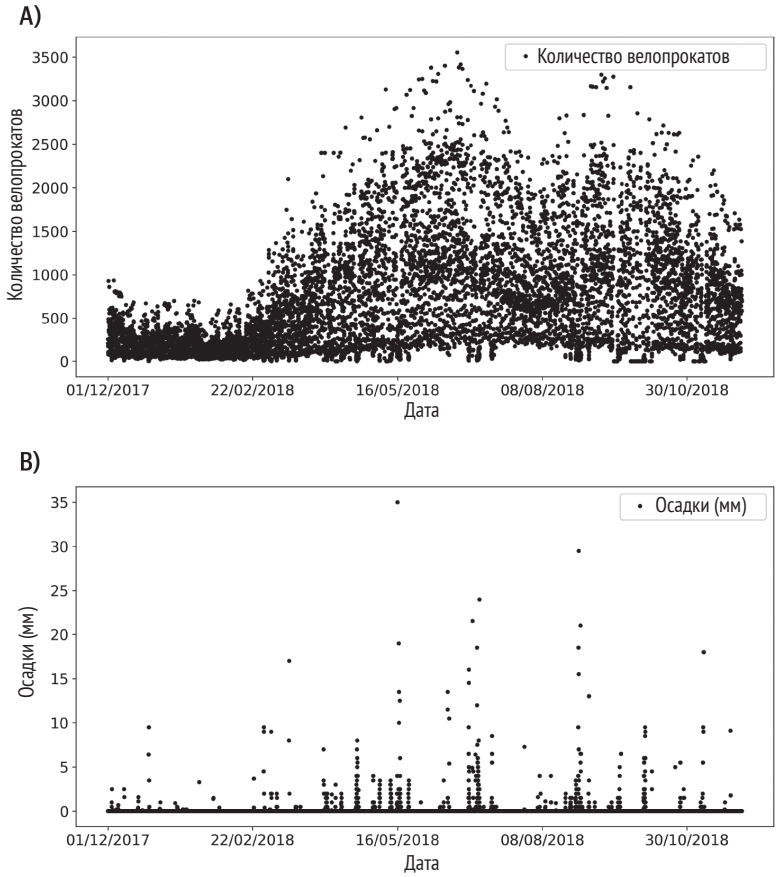


Рис. 12.1 ❖ Диаграммы рассеяния некоторых данных

Обратите внимание, что осадки являются разреженной переменной – в основном это нули с относительно малым числом ненулевых значений. Мы вернемся к этой теме в упражнениях.

На рис. 12.2 показана матрица корреляций четырех отобранных переменных. В целом перед началом статистического анализа всегда неплохо обследовать матрицы корреляций, потому что они будут показывать коррелирующие переменные (если они есть) и могут выявлять ошибки в данных (например, если две предположительно разные переменные полностью коррелированы). В этом случае мы видим, что количество велопрокатов положительно коррелирует со *временем* и *температурой* (люди берут больше велосипедов в более позднее время дня и в более теплую погоду) и отрицательно коррелирует с *количеством осадков*. (Обратите внимание, что я не показываю здесь статистическую значимость, поэтому данные интерпретации являются качественными.)

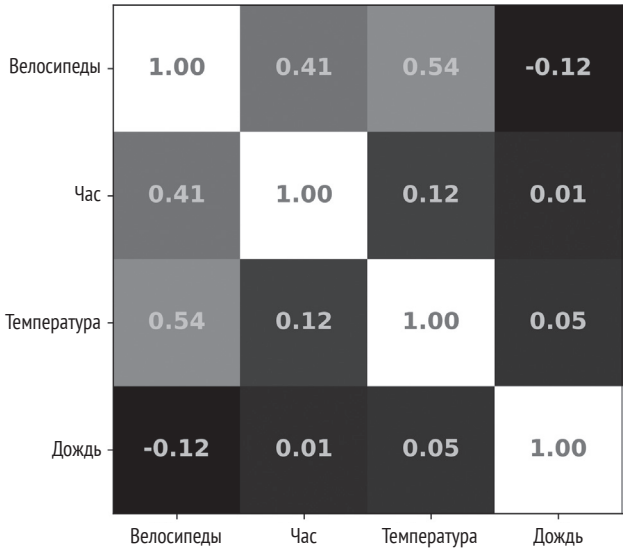


Рис. 12.2 ❖ Матрица корреляций четырех отобранных переменных

В первом анализе я хочу предсказать количество велопрокатов в зависимости от количества осадков и времени года. Времена года (зима, весна, лето, осень) – это текстовые метки в наборе данных, и нам нужно конвертировать их в числа, чтобы провести анализ. Мы могли бы перевести четыре сезона в числа от 1 до 4, но сезоны характеризуются цикличностью, а регрессии – линейностью. С этим можно справиться несколькими способами, в том числе применяя анализ ANOVA вместо регрессии, используя кодирование с одним активным состоянием[1] (которое применяется в моделях глубокого обучения)

[1] Англ. *one-hot-encoding*; данный термин происходит из терминологии цифровых интегральных микросхем, в которой он описывает конфигурацию микросхемы, допускающую, чтобы только один бит был положительным (активным). – *Прим. перев.*

или бинаризуя времена года. Я собираюсь использовать последний подход и обозначить осень и зиму «0», а весну и лето – «1». Это интерпретируется так: положительный бета-коэффициент указывает на большее количество велопрокатов весной/летом по сравнению с осенью/зимой.

(Примечание по касательной: с одной стороны, я мог бы все упростить, выбрав только непрерывные переменные. Но я хочу подчеркнуть, что наука о данных – это нечто большее, чем просто применение формулы к набору данных; есть много нетривиальных решений, которые влияют на пригодные для проведения виды анализа и, следовательно, виды результатов, которые можно получать.)

В левой части рис. 12.3 показана расчетная матрица, визуализированная в виде изображения. Это общепринятое представление расчетной матрицы, поэтому убедитесь, что вам удобно ее интерпретировать. Столбцы – это регрессоры, а строки – это наблюдения. Столбцы иногда нормализуются, чтобы облегчить визуальную интерпретацию, в случае если регрессоры имеют очень разные числовые шкалы, хотя здесь я этого не делал. Как видно по рисунку, количество осадков характеризуется разряженностью, и набор данных охватывает два осенне-зимних периода (черные области в средней колонке) и один весенне-летний период (белая область в середине). Пересечение очевидным образом имеет полностью белый цвет, потому что оно принимает одно и то же значение для каждого наблюдения.

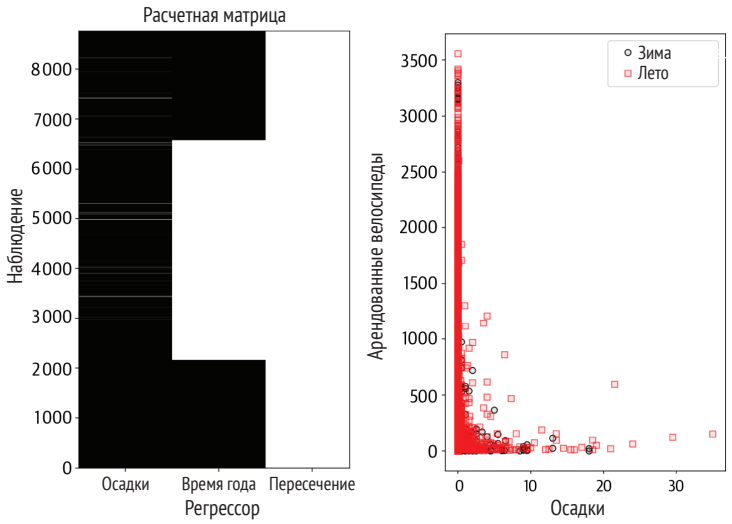


Рис. 12.3 ❖ Расчетная матрица и немного данных

В правой части рис. 12.3 показаны данные, нанесенные на график в виде количества осадков относительно велосипедов, бравшихся в прокат отдельно за два сезона. Очевидно, что данные не лежат на прямой, потому что на обеих осях есть много значений, равных или близких к нулю. Другими словами, визуальное инспектирование данных говорит о том, что взаимосвязи между переменными нелинейны, а это означает, что линейный подход к моделиро-

ванию может быть субоптимальным. Опять же, это подчеркивает важность визуального инспектирования данных и тщательного отбора подходящей статистической модели.

Тем не менее мы будем продвигаться вперед, используя линейную модель, подгоняемую к данным с применением метода наименьших квадратов. Следующий ниже исходный код показывает, как я создал расчетную матрицу (переменная `data` – это кадр данных библиотеки pandas):

```
# создать расчетную матрицу и добавить пересечение
desmat = data[['Осадки (mm)','Сезоны']].to_numpy()
desmat = np.append(desmat, np.ones((desmat.shape[0],1)), axis=1)

# извлечь зависимую переменную (DV)
y = data[['Велопрокаты']].to_numpy()

# выполнить подгонку модели к данным с
# использованием метода наименьших квадратов
beta = np.linalg.lstsq(desmat, y, rcond=None)
```

Значения бета для *осадков* и *сезона* составляют соответственно −80 и 369. Эти числа показывают, что во время дождя количество велопрокатов меньше, а весной/летом велопрокат больше, чем осенью/зимой.

На рис. 12.4 показаны предсказанные и наблюдаемые данные, отдельно по двум сезонам. Если бы модель вписывалась в данные идеально, то точки лежали бы на диагональной линии с наклоном, равным 1. Очевидно, это не так, а значит модель была не очень хорошо подогнана к данным. И действительно, R^2 составляет ничтожные 0.097 (другими словами, статистическая модель объясняет около 1 % дисперсии данных). Кроме того, хорошо видно, что модель предсказывает *отрицательное* количество велопрокатов, что не поддается интерпретации – количество велопрокатов является строго неотрицательным.

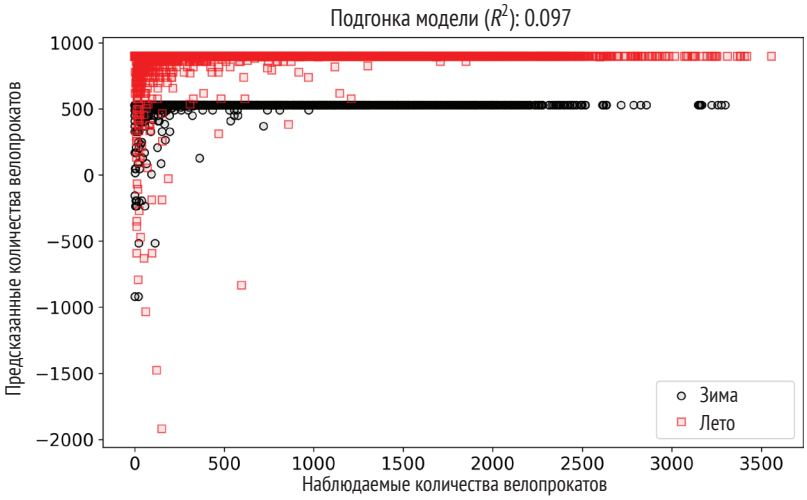


Рис. 12.4 ❖ Диаграмма рассеяния предсказанных данных относительно наблюдаемых данных

До сих пор в исходном коде мы не получали ни предупреждений, ни ошибок; мы не сделали ничего плохого ни с точки зрения математики, ни с точки зрения программирования. Однако примененная нами статистическая модель не является наиболее подходящей для этого исследовательского вопроса. В упражнениях 12.1 и 12.2 у вас будет возможность ее улучшить.

Регрессионная таблица с использованием библиотеки statsmodels

Не слишком углубляясь в статистику, хочу вам показать, как создавать регрессионную таблицу с помощью библиотеки statsmodels. Эта библиотека работает с кадрами данных pandas вместо массивов NumPy. В следующем ниже фрагменте исходного кода показано, как настраивать и вычислять регрессионную модель (OLS[1] означает *обычный метод наименьших квадратов*):

```
import statsmodels.api as sm

# извлечь данные (оставаясь с кадрами данных pandas)
desmat_df  = data[['Осадки (mm)', 'Сезоны']]
obsdata_df = data['Велопрокаты']

# создать модель и выполнить ее подгонку
# (необходимо добавить пересечение в явной форме)
desmat_df = sm.add_constant(desmat_df)
model = sm.OLS(obsdata_df,desmat_df).fit()
print( model.summary() )
```

Регрессионая таблица содержит много информации. Ничего страшного, если вы не все понимаете; ключевыми искомыми элементами в ней являются R^2 и коэффициенты регрессии (`coef`) для регрессоров:

```
==============================================================================
Dep. Variable:            Велопрокаты   R-squared:                       0.097
Model:                            OLS   Adj. R-squared:                  0.097
Method:                 Least Squares   F-statistic:                     486.8
Date:                Wed, 26 Jan 2022   Prob (F-statistic):
Time:                        08:40:31   Log-Likelihood:                -68654.
No. Observations:                8760   AIC:                         1.373e+05
Df Residuals:                    8757   BIC:                         1.373e+05
Df Model:                           2
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const        530.4946      9.313     56.963      0.000     512.239     548.750
Rainfall(mm) -80.5237      5.818    -13.841      0.000     -91.928     -69.120
Seasons      369.1267     13.127     28.121      0.000     343.395     394.858
```

[1] Англ. *Ordinary Least Squares*. – *Прим. перев.*

```
==============================================================================
Omnibus:                     1497.901   Durbin-Watson:                   0.240
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             2435.082
Skew:                           1.168   Prob(JB):                         0.00
Kurtosis:                       4.104   Cond. No.                         2.80
==============================================================================
```

Мультиколлинеарность

Если вы посещали курсы статистики, то, возможно, слышали о термине мультиколлинеарность. Его определение в Википедии гласит: «в модели множественной регрессии одна предсказательная переменная может с существенной степенью точности линейно предсказываться по другим переменным»[1].

Это означает, что в расчетной матрице существуют линейные зависимости. На языке линейной алгебры мультиколлинеарность – это просто причудливый термин, который обозначает *линейную зависимость*, что равносильно утверждению, что расчетная матрица имеет пониженный ранг либо что она сингулярна.

Рангово-пониженная расчетная матрица не имеет левообратной матрицы, а значит, задача о наименьших квадратах не решаема аналитически. Вы увидите последствия мультиколлинеарности в упражнении 12.3.

Решение общей линейной модели с мультиколлинеарностью

На самом деле существует возможность аналитического вывода решения для общих линейных моделей, которые имеют рангово-пониженную расчетную матрицу. Это делается с помощью модификации процедуры QR-разложения из предыдущей главы и с использованием псевдообратной матрицы Мура–Пенроуза. В случае рангово-пониженной расчетной матрицы единственного решения нет, но можно выбрать решение с минимальной ошибкой. Это называется *решением с минимальной нормой*, или просто решением min-norm, и часто используется, например, в биомедицинской визуализации. Несмотря на специальные применения, расчетная матрица с линейными зависимостями обычно указывает на проблему со статистической моделью и должна быть всячески обследована (распространенными источниками мультиколлинеарности являются ошибки ввода данных и ошибки программирования).

Регуляризация

Регуляризация – это зонтичный термин, который относится к различным способам модификации статистической модели с целью улучшения численной стабильности, преобразования сингулярных или плохо кондиционных матриц в полноранговые (и, следовательно, обратимые) или улучшения обобщаемости за счет уменьшения переподгонки. В зависимости от характера

[1] Википедия, поиск по «multicollinearity», https://en.wikipedia.org/wiki/Multicollinearity.

задачи и цели регуляризации применяется несколько форм регуляризации; некоторые конкретные методы, о которых вы, возможно, слышали, включают гребневую (она же L2), лассо (она же L1), тихоновскую и усадочную[1].

Разные методы регуляризации работают по-разному, но многие регуляризаторы «сдвигают» расчетную матрицу на некоторую величину. Из главы 5 вы помните, что сдвиг матрицы означает добавление некоторой константы к диагонали в виде $\mathbf{A} + \lambda\mathbf{I}$, а из главы 6 – что сдвиг матрицы может преобразовывать рангово-пониженную матрицу в полноранговую.

В этой главе мы выполним регуляризацию расчетной матрицы, сдвинув ее в соответствии с некоторой долей ее фробениусовой нормы. За счет этого видоизменится решение уравнения 12.1 наименьших квадратов.

Уравнение 12.1. Регуляризация

$$\boldsymbol{\beta} = (\mathbf{X}^{\mathrm{T}}\mathbf{X} + \gamma||\mathbf{X}||_F^2\mathbf{I})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{y}.$$

Ключевым параметром является γ (греческая буква гамма), который определяет степень регуляризации (обратите внимание, что $\gamma = 0$ соответствует отсутствию регуляризации). Выбор подходящего параметра γ нетривиален и нередко осуществляется с помощью статистических методов, таких как перекрестная валидация.

Наиболее очевидный эффект регуляризации заключается в том, что если расчетная матрица имеет пониженный ранг, то регуляризованная квадратная расчетная матрица имеет полный ранг. Регуляризация также уменьшает кондиционное число, которое измеряет «разброс» информации в матрице (это отношение наибольшего сингулярного числа к наименьшему; вы узнаете о нем в главе 14). Благодаря ей увеличивается численная стабильность матрицы. В статистическом плане последствием регуляризации является «сглаживание» решения за счет снижения чувствительности модели к отдельным точкам данных, которые могут быть выбросами либо нерепрезентативными, и, следовательно, вероятность наблюдать их в новых наборах данных весьма мала.

Почему я шкалирую на квадрат нормы Фробениуса? Учтите, что указанное значение γ, скажем $\gamma = 0.01$, может иметь огромное или незначительное влияние на расчетную матрицу в зависимости от диапазона числовых значений в матрице. Поэтому мы шкалируем в числовой диапазон матрицы, то есть мы интерпретируем параметр γ как *долю* регуляризации. Причина возведения в квадрат нормы Фробениуса заключается в том, что $||\mathbf{X}||_F^2 = ||\mathbf{X}^{\mathrm{T}}\mathbf{X}||_F$. Другими словами, квадрат нормы расчетной матрицы равен норме расчетной матрицы, умноженной на ее транспонированную версию.

На самом деле вместо нормы Фробениуса чаще используется среднее значение собственных чисел расчетной матрицы. Познакомившись с собственными числами в главе 13, вы сможете сравнить два метода регуляризации.

Реализации регуляризации в исходном коде посвящено упражнение 12.4.

---

[1] Англ. *shrinkage*. – *Прим. перев.*

Полиномиальная регрессия

Полиномиальная регрессия похожа на обычную регрессию, но независимыми переменными являются значения оси x, возводимые в более высокие степени. То есть каждый столбец i расчетной матрицы определяется как x^i, где x – это обычно время или пространство, но может быть и другими переменными, такими как дозировка лекарства или численность населения. Математическая модель выглядит так:

$$y = \beta_0 x^0 + \beta_1 x^1 + \cdots + \beta_n x^n.$$

Обратите внимание на $x_0 = 1$, которое дает пересечение модели. В противном случае это по-прежнему будет обычной регрессией – цель состоит в том, чтобы найти значения β, которые минимизируют квадраты разниц между предсказанными и наблюдаемыми данными.

Степень многочлена равна наибольшей степени i. Например, полиномиальная регрессия четвертой степени имеет члены до x^4 (если нет члена x^3, то это по-прежнему модель четвертой степени с $\beta_3 = 0$).

На рис. 12.5 показан пример отдельных регрессоров и расчетные матрицы многочлена третьей степени (имейте в виду, что многочлен n-й степени имеет $n + 1$ регрессоров, включая пересечение). Полиномиальные функции являются базисными векторами, служащими для моделирования наблюдаемых данных.

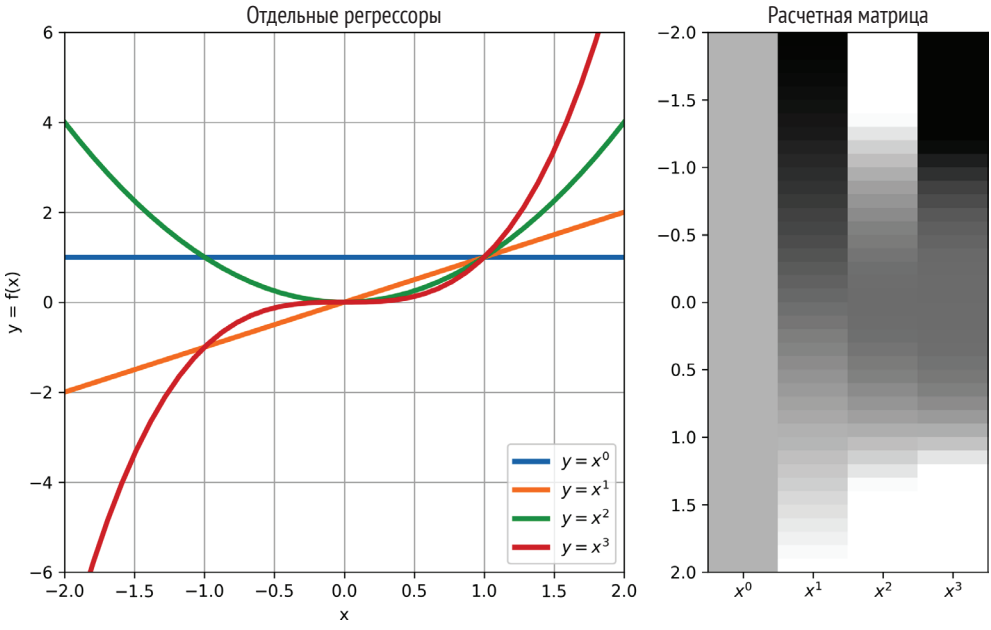


Рис. 12.5 ❖ Расчетная матрица полиномиальной регрессии

За исключением специальной расчетной матрицы, полиномиальная регрессия выполняется точно так же, как и любая другая регрессия: применить левообратную матрицу (либо более вычислительно стабильные альтернативы), чтобы получить набор коэффициентов, такой что взвешенная комбинация регрессоров (то есть предсказанные данные) совпадает с наблюдаемыми данными наилучшим образом.

Полиномиальные регрессии используются для подгонки кривых и аппроксимации нелинейных функций. Приложения включают моделирование временных рядов, динамику населения, функции доза–реакция в медицинских исследованиях и физические нагрузки на опорные балки. Полиномы также могут выражаться в 2D, которые используются для моделирования пространственной структуры, такой как распространение землетрясений и активность мозга.

Но довольно фоновой информации. Давайте поработаем на примере. Я выбрал набор данных, который основан на модели удвоения населения. Вопрос заключается в следующем: «Сколько времени потребуется для того, чтобы население человечества удвоилось (например, с пятисот миллионов до одного миллиарда)?» Если скорость прироста населения сама по себе увеличивается (поскольку у большего числа людей рождается больше детей, а у тех, кто вырастает, рождается еще больше детей), то время удвоения будет уменьшаться с каждым удвоением. С другой стороны, если рост населения замедляется (люди рожают меньше детей), то время удвоения будет увеличиваться по сравнению с последующими удвоениями.

Я нашел подходящий набор данных в интернете[1]. Это небольшой набор данных, поэтому все цифры доступны в онлайновом исходном коде и показаны на рис. 12.6. Указанный набор данных включает в себя как фактические измеренные данные, так и проекции до 2100 года. Эти проекции в будущее основаны на ряде допущений, и никто толком не знает, как сложится будущее (поэтому вам следует находить баланс между подготовкой к будущему и наслаждением моментом). Тем не менее данные на сей момент показывают, что за последние пятьсот лет (по меньшей мере) население человечества удваивалось с возрастающей частотой, и авторы набора данных предсказывают, что скорость удвоения немного увеличится в следующем столетии.

Для подгонки к данным я выбрал многочлен третьей степени, а затем создал модель и выполнил ее подгонку, используя следующий ниже исходный код (переменная `year` содержит координаты по оси *x*, а переменная `doubleTime` содержит зависимую переменную):

```
# расчетная матрица
X = np.zeros((N, 4))
for i in range(4):
    X[:, i] = np.array(year)**i

# выполнить подгонку модели и
# вычислить предсказанные данные
```

[1] *Розер Макс, Ричи Ханна и Ортис-Оспина Эстебан*. Прирост населения мира, OurWorldInData.org, 2013 г., https://ourworldindata.org/world-population-growth.

```
beta = np.linalg.lstsq(X, doubleTime, rcond=None)
yHat = X@beta[0]
```

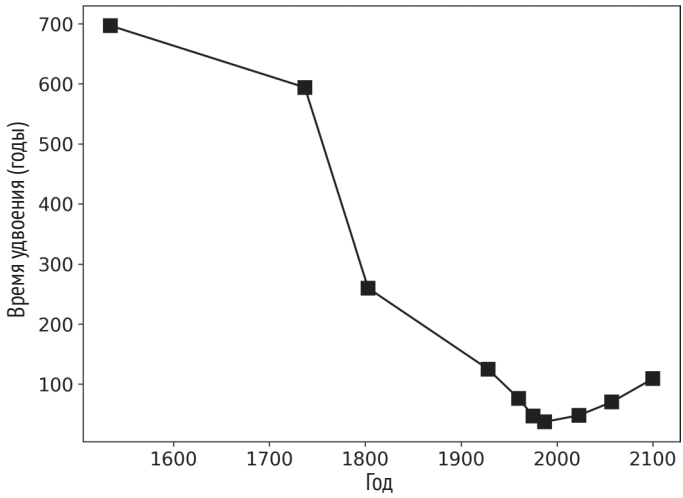


Рис. 12.6 ❖ График данных

На рис. 12.7 показаны предсказанные данные с использованием полиномиальной регрессии, созданной этим исходным кодом.

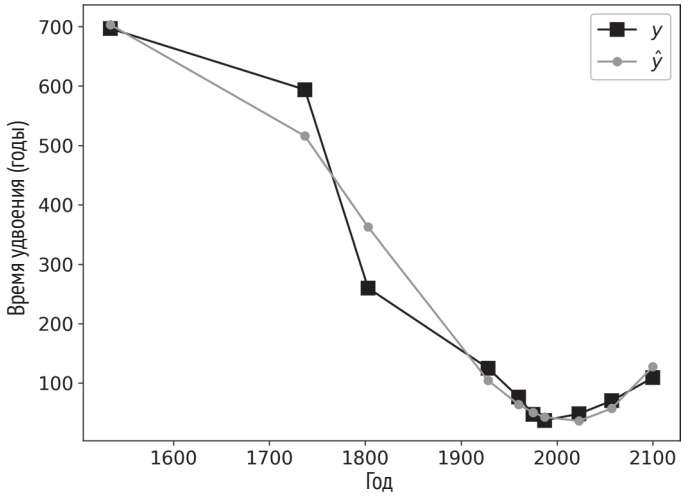


Рис. 12.7 ❖ График данных

Модель улавливает как нисходящий тренд, так и спроецированный виток вверх в данных. Без дальнейшего статистического анализа невозможно сказать, что это наилучшая модель или что указанная модель статистически значимо хорошо вписывается в данные. Но ясно одно, что полиномиальные

регрессии хорошо подходят для подгонки кривых. В упражнении 12.5 вы продолжите обследовать эту модель и данные, но рекомендую вам поэкспериментировать с исходным кодом, создающим рис. 12.7, пробуя разные параметры степени.

Полиномиальные регрессии находят широкое применение, и в библиотеке NumPy имеются специальные функции для создания и подгонки таких моделей:

```
beta = np.polyfit(year, doubleTime, 3) # 3-я степень
yHat = np.polyval(beta, year)
```

Поиск в параметрической решетке для отыскания модельных параметров

Метод наименьших квадратов посредством левообратной матрицы – отличный способ подгонки моделей к данным. Метод наименьших квадратов является точным, быстрым и детерминированным (это означает, что при каждом повторном прогоне исходного кода вы будете получать один и тот же результат). Но он работает только для подгонки линейных моделей, и не все модели можно подгонять с помощью линейных методов.

В этом разделе я познакомлю вас с еще одним методом оптимизации, используемым для выявления модельных параметров, именуемым *поиском в параметрической решетке*[1]. Поиск в параметрической решетке работает путем отбора значений параметра в параметрическом пространстве, вычисления подгонки модели к данным с каждым значением параметра, а затем отбора значения, дающего наилучшую подгонку модели.

В качестве простого примера возьмем функцию $y = x^2$. Нам нужно найти минимум этой функции. Разумеется, мы уже знаем, что минимум находится в $x = 0$; это поможет понять и оценить результаты поиска в решетке параметров.

При выполнении поиска в параметрической решетке мы начинаем с предопределенного набора тестируемых значений x. Давайте воспользуемся набором (−2, −1, 0, 1, 2). Это наша «решетка». Затем по каждому значению в решетке мы вычисляем функцию и получаем $y = (4, 1, 0, 1, 4)$. И находим, что минимум y происходит при $x = 0$. В этом случае решение на основе решетки совпадает с истинным решением.

Но поиск в параметрической решетке не гарантирует оптимального решения. Например, представьте, что наша решетка была бы (−2, −0.5, 1, 2.5); значения функции были бы $y = (4, 0.25, 1, 6.25)$, и мы бы заключили, что $x = -0.5$ является значением параметра, которое минимизирует функцию $y = x^2$. Этот

[1] Англ. *Grid Search*; идея поиска в параметрической решетке состоит в создании «решетки» параметров и просто опробывании всех возможных их комбинаций. – *Прим. перев.*

вывод будет «отчасти правильным», потому что в указанной решетке это лучшее решение. Ошибки при поиске в параметрической решетке также могут возникать из-за неправильно выбранного диапазона значений. Представьте, например, что наша решетка была бы (−1000, −990, −980, −970); мы пришли бы к выводу, что $y = x^2$ минимизируется при $x = -970$.

Дело в том, что и диапазон, и разрешающая способность (расстояние между точками решетки) имеют большую важность, потому что они определяют получаемое вами решение, которое может быть *наилучшим*, *довольно хорошим* либо *ужасным*. В приведенном выше игрушечном примере подходящий диапазон и разрешающая способность определяются легко. В многосложных, многопеременных, нелинейных моделях для поиска в параметрической решетке могут потребоваться дополнительная работа и обследование подходящих параметров.

Я выполнил поиск в параметрической решетке на данных о «счастливых студентах» из предыдущей главы (напомню: это были поддельные данные из поддельного опроса, показывающего, что люди, записавшиеся на большее количество моих курсов, были более удовлетворены жизнью). Модель этих данных имеет два параметра – пересечение и наклон, поэтому мы вычисляем эту функцию в каждой точке двумерной решетки возможных пар параметров. Результаты показаны на рис. 12.8.

Что означает этот график и как его интерпретировать? Две оси соответствуют значениям параметров, поэтому каждая координата на этом графике создает модель с соответствующими значениями параметров. Затем вычисляется подгонка каждой из этих моделей к данным, сохраняется и визуализируется в виде изображения.

Координата с наилучшей полгонкой к данным (наименьшая сумма квадратов ошибок) является оптимальным набором параметров. На рис. 12.8 также показано аналитическое решение с использованием метода наименьших квадратов. Они близки, но не накладываются друг на друга точно. В упражнении 12.6 у вас будет возможность реализовать этот поиск по параметрической решетке, а также обследовать вопрос важности разрешающей способности решетки для точности результатов.

Зачем вообще кому-то использовать поиск в параметрической решетке, когда метод наименьших квадратов работает лучше и быстрее? Все верно, не следует использовать метод поиска в параметрической решетке, когда жизнеспособным решением является метод наименьших квадратов. Вместе с тем поиск в параметрической решетке является полезным методом отыскания параметров в нелинейных моделях и нередко используется, например, для определения гиперпараметров в моделях глубокого обучения (*гиперпараметры* – это конструктивные особенности модельной архитектуры, которые отбираются исследователем, а не извлекаются из данных). В случае больших моделей поиск в параметрической решетке бывает времязатратным, но его параллелизация может делать его выполнение более приемлемым.

Вывод состоит в том, что поиск в параметрической решетке является нелинейным методом подгонки моделей к данным в ситуациях, когда линейные методы применить невозможно. По ходу своего путешествия по науке о данных и машинному обучению вы также узнаете о дополнительных не-

линейных методах, включая симплекс и знаменитый алгоритм градиентного спуска, который приводит в действие глубокое обучение.

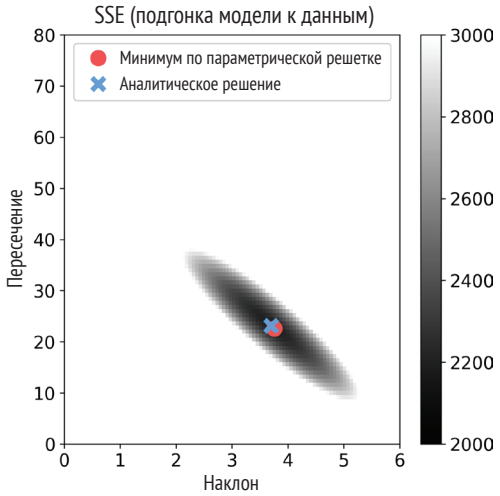


Рис. 12.8 ❖ Результаты поиска в параметрической решетке на наборе данных «Счастливый студент». Интенсивность показывает сумму квадратов ошибок, подогнанных к данным

Резюме

Надеюсь, вам понравилось читать о применении метода наименьших квадратов и сравнении с другими подходами к подгонке моделей. Следующие ниже упражнения являются наиболее важной частью этой главы, поэтому я не хочу отнимать у вас время подведением итогов. Вот важные моменты главы.

- Визуальное обследование данных имеет большую важность для отбора правильных статистических моделей и правильной интерпретации статистических результатов.
- Линейная алгебра используется для количественного оценивания наборов данных, включая матрицы корреляций.
- Методы визуализации матриц, о которых вы узнали в главе 5, полезны для инспектирования расчетных матриц.
- В разных областях математические понятия иногда получают разные названия. В этой главе понятие *мультиколлинеарности* является одним из таких примеров; оно означает линейные зависимости в расчетной матрице.

- Регуляризация предусматривает «сдвиг» расчетной матрицы на некоторую малую величину, в целях повышения численной стабильности и вероятности обобщения на новых данных.
- Глубокое понимание линейной алгебры поможет вам отбирать наиболее подходящий вид статистического анализа, интерпретировать результаты и предвидеть возможные проблемы.
- Полиномиальная регрессия аналогична «обычной» регрессии, но столбцы в расчетной матрице определяются как значения по оси *x*, возводимые в возрастающую степень. Полиномиальные регрессии используются для подгонки кривых.
- Поиск в параметрической решетке – это нелинейный метод подгонки моделей. Линейный метод наименьших квадратов является оптимальным подходом, когда модель является линейной.

Упражнения по программированию

Упражнения по аренде велосипедов

Упражнение 12.1

Возможно, часть проблемы с отрицательными велопрокатами на рис. 12.4 можно решить, исключив дни без дождя. Повторите анализ и график этого анализа, но выберите только те строки данных, в которых имеется нулевое количество осадков. Улучшатся ли результаты с точки зрения более высокого R^2 и положительного предсказанного количества велопрокатов?

Упражнение 12.2

Поскольку времена года – это категориальная переменная, дисперсионный анализ действительно был бы более подходящей статистической моделью, чем регрессия. Возможно, бинаризованным *временам года* не хватает чувствительности для предсказания количеств велопрокатов (например, осенью могут быть теплые солнечные дни, а весной – холодные дождливые дни), и поэтому температура может быть самым лучшим предсказателем.

Замените *времена года* в расчетной матрице *температурой* и выполните повторный прогон регрессии (можно использовать все дни, а не только дни без осадков из предыдущего упражнения) и воспроизведите рис. 12.9. Остается проблема предсказания отрицательных велопрокатов (это связано с линейностью модели), но R^2 получается выше, и предсказание выглядит качественно лучше.

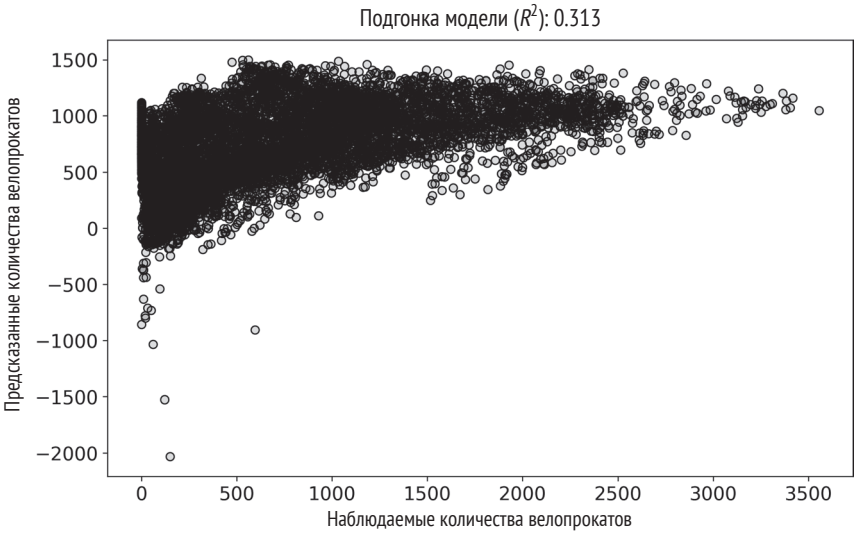


Рис. 12.9 ❖ Результаты упражнения 12.2

Упражнения по мультиколлинеарности

Упражнение 12.3

Это упражнение продолжает работу с моделью из упражнения 12.2[1]. Указанная модель содержит три регрессора, включая пересечение. Создайте новую расчетную матрицу, содержащую четвертый регрессор, определенный как некоторая линейно-взвешенная комбинация температуры и количества осадков. Дайте этой расчетной матрице другое имя переменной, потому что она понадобится вам в следующем упражнении. Подтвердите, что расчетная матрица имеет четыре столбца, но имеет ранг 3, и вычислите матрицу корреляций расчетной матрицы.

Обратите внимание, что в зависимости от перевесовки двух переменных корреляция ожидаемо не будет равна 1, даже при линейных зависимостях; здесь вы также не ожидаете воспроизвести точные корреляции:

```
Размер расчетной матрицы: (8760, 4)
Ранг расчетной матрицы: 3

Матрица корреляций расчетной матрицы:
[[1.      0.05028     nan 0.7057 ]
 [0.05028 1.          nan 0.74309]
 [    nan     nan     nan     nan]
 [0.7057  0.74309     nan 1.     ]]
```

[1] Если вы столкнулись с ошибками Python, то попробуйте выполнить повторный прогон предыдущего исходного кода, а затем повторно создать переменные расчетной матрицы.

Выполните подгонку модели, используя три разных программных подхода:

1) прямая реализация с левообратной матрицей, как вы узнали из предыдущей главы;
2) с использованием функции NumPy `lstsqr`;
3) с использованием статистических моделей.

Для всех трех методов вычислите R^2 и коэффициенты регрессии. Распечатайте результаты, как показано ниже. Отчетливо видна численная нестабильность функции `np.linalg.inv` на рангово-пониженной расчетной матрице.

```
ПОДГОНКА МОДЕЛИ К ДАННЫМ:
Левообратная матрица: 0.0615
np lstsqr           : 0.3126
statsmodels         : 0.3126

БЕТА-КОЭФФИЦИЕНТЫ:
Левообратная матрица: [[-1528.071  11.277  337.483  5.537 ]]
np lstsqr           : [[  -87.632   7.506  337.483  5.234 ]]
statsmodels         : [   -87.632   7.506  337.483  5.234 ]
```

Попутное замечание: никаких сообщений об ошибках, ни предупреждений не поступало; Python просто выдал результаты, хотя с расчетной матрицей явно что-то не так. Можно обсуждать достоинства такого поведения, но этот пример еще раз подчеркивает важность понимания линейной алгебры в науке о данных и что правильная наука о данных – это больше, чем просто знание математики.

Упражнения по регуляризации

Упражнение 12.4

Здесь вы займетесь обследованием эффектов регуляризации на рангово-пониженную расчетную матрицу, которую вы создали в предыдущем упражнении. Начните с реализации $(\mathbf{X}^T\mathbf{X} + \gamma||\mathbf{X}||_F^2\mathbf{I})^{-1}$, используя $\gamma = 0$ и $\gamma = 0.01$. Распечатайте размер и ранг двух матриц. Вот мои результаты (интересно отметить, что расчетная матрица ранга 3 является настолько численно нестабильной, что ее «обратная матрица» фактически имеет ранг 2):

```
размер inv(X'X + 0.0*I): (4, 4)
ранг inv(X'X + 0.0*I)  : 2

размер inv(X'X + 0.01*I): (4, 4)
ранг inv(X'X + 0.01*I)  : 4
```

Теперь об эксперименте. Здесь цель состоит в том, чтобы обследовать эффекты регуляризации на подгонку модели к данным. Напишите исходный код, который будет вычислять подгонку к данным как R^2, используя метод наименьших квадратов с регуляризацией на расчетных матрицах с мультиколлинеарностью и без нее. Поместите этот исходный код в цикл `for`, реали-

зующий диапазон значений γ от 0 до 0.2. Затем покажите результаты в виде рисунка, подобного рис. 12.10.

Между прочим, тривиальным для полноранговых расчетных матриц случаем является уменьшение подгонки модели с увеличением регуляризации – и действительно, регуляризация предназначена для того, чтобы делать модель менее чувствительной к данным. Важный вопрос заключается в том, улучшает ли регуляризация подгонку к тестовому набору данных или валидационному блоку, который был исключен при подгонке модели. Если регуляризация выгодна, то можно ожидать увеличение обобщаемости регуляризованной модели до некоторого γ, а затем снова ее уменьшение. Это тот уровень детализации, о котором вы узнаете из специальной книги по статистике или машинному обучению, хотя в главе 15 вы все же доберетесь до перекрестной валидации исходного кода.

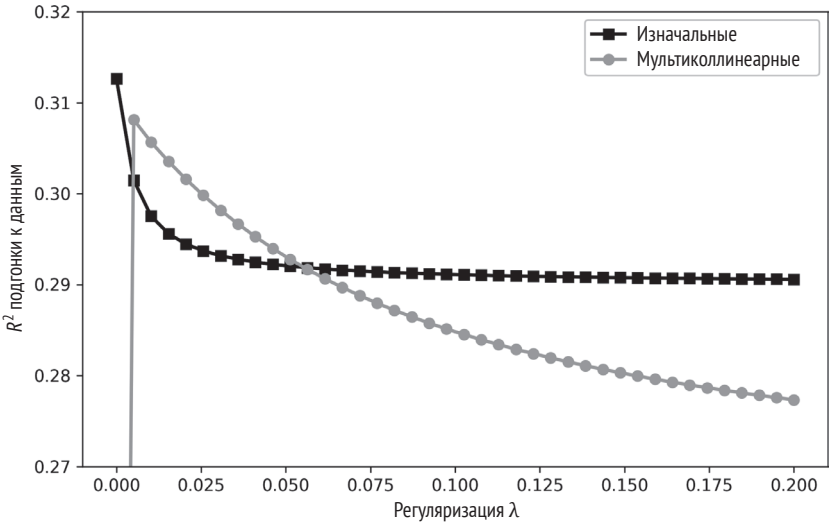


Рис. 12.10 ❖ Результаты упражнения 12.4

Упражнение по полиномиальной регрессии

Упражнение 12.5

Цель этого упражнения – выполнить подгонку полиномиальной регрессии, используя диапазон степеней от нуля до девяти. В цикле `for` перевычисляйте регрессию и предсказанные значения данных. Покажите результаты, как на рис. 12.11.

В этом упражнении высвечиваются проблемы недоподгонки и переподгонки. Модель со слишком малым числом параметров плохо справляется с предсказыванием данных. С другой стороны, модель с большим числом параметров слишком хорошо прилегает к данным и рискует оказаться чрезмерно чувствительной к шуму и быть не способной обобщать на новых данных. Стратегии отыскания баланса между недоподгонкой и переподгонкой

включают перекрестную валидацию и байесов информационный критерий; с этими темами вы познакомитесь в книге по машинному обучению или статистике.

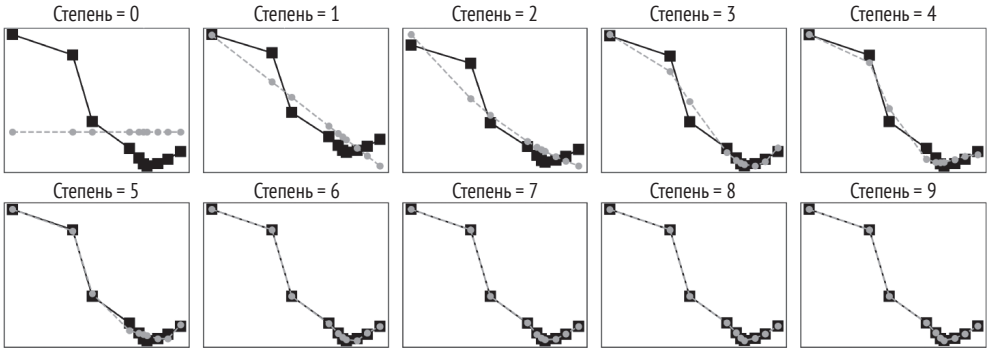


Рис. 12.11 ❖ Результаты упражнения 12.5

Упражнения по поиску в параметрической решетке

Упражнение 12.6

Здесь ваша цель будет простой: воспроизвести рис. 12.8, следуя инструкциям, представленным в тексте вокруг этого рисунка. Распечатайте коэффициенты регрессии для сравнения. Например, следующие ниже результаты были получены с использованием параметра разрешающей способности параметрической решетки, заданной равной 50:

```
Аналитический результат:
Пересечение: 23.13, наклон: 3.70

Эмпирический результат:
Пересечение: 22.86, наклон: 3.67
```

Когда у вас будет рабочий исходный код, попробуйте несколько разных параметров разрешающей способности. Я сделал рис. 12.8, используя разрешающую способность 100; вам также следует попробовать другие значения, например 20 или 500. Еще обратите внимание на время вычисления при более высоких значениях разрешающей способности – и это только двухпараметрическая модель! Исчерпывающий поиск в параметрической решетке с высокой разрешающей способностью для 10-параметрической модели потребует чрезвычайно больших вычислительных ресурсов.

Упражнение 12.7

Вы видели два разных метода оценивания подгонки модели к данным: сумма квадратов ошибок и R^2. В предыдущем упражнении для оценивания подгонки модели к данным вы использовали сумму квадратов ошибок;

в этом упражнении вы определите, является ли вариант с R^2 столь же жизнеспособным.

Часть исходного кода в этом упражнении будет простой: надо видоизменить исходный код из предыдущего упражнения, вычислив R^2 вместо SSE (проследите, чтобы видоизменения коснулись копии исходного кода, без перезаписи предыдущего упражнения).

Теперь самое сложное: вы обнаружите, что R^2 выглядит ужасно! Этот показатель дает совершенно неправильный ответ. Ваша задача – выяснить причину, по которой это так (онлайновое решение содержит изложение данного вопроса). Совет: сохраняйте предсказанные данные по каждой паре параметров, чтобы иметь возможность инспектировать предсказанные значения, а затем сравнивать их с наблюдаемыми данными.

Глава 13

Собственное разложение

Собственное разложение[1] – жемчужина линейной алгебры. Что такое жемчужина? Позвольте мне процитировать прямо из книги «20 000 лье под водой»:

> Для поэта жемчужина – слеза моря, для восточных народов – окаменевшая капля росы; для женщин – драгоценный овальной формы камень с перламутровым блеском, который они носят, как украшение, на руках, на шее, в ушах; для химика – соединение фосфорнокислых солей с углекислым кальцием; и, наконец, для натуралиста – просто болезненный нарост, представляющий собою шаровидные наплывы перламутра внутри мягкой ткани мантии у некоторых представителей двустворчатых моллюсков.
>
> *– Жюль Верн*

Дело в том, что один и тот же объект видится по-разному в зависимости от его использования. То же самое и с собственным разложением: собственное разложение имеет геометрическую интерпретацию (оси вращательной инвариантности), статистическую интерпретацию (направления максимальной ковариации), системно-динамическую интерпретацию (стабильные состояния системы), теоретико-графовую интерпретацию (влияние узла на его сеть), финансово-рыночную интерпретацию (выявление ковариирующих акций) и многие другие.

Собственное разложение (и SVD, которое, как вы узнаете в следующей главе, тесно связано с собственным разложением) – один из самых важных вкладов линейной алгебры в науку о данных. Цель этой главы – дать вам интуитивное понимание собственных чисел и собственных векторов – результатов собственного разложения матрицы. Попутно вы узнаете о диагонализации и других специальных свойствах симметричных матриц. В главе 14 – после того, как собственное разложение будет расширено на SVD, в главе 15 вы увидите несколько его приложений.

---

[1] Англ. *Eigendecomposition*; син. спектральное разложение. – *Прим. перев.*

Только для квадратов

Собственное разложение определено только для квадратных матриц. Матрицу размера *M*×*N* невозможно разложить, если только *M* не равно *N*. Неквадратные матрицы можно раскладывать с помощью SVD. Каждая квадратная матрица размера *M*×*M* имеет *M* собственных чисел (скаляров) и *M* соответствующих собственных векторов. Собственное разложение предназначено для выявления этих *M* векторно-скалярных пар.

Интерпретации собственных чисел и собственных векторов

В следующих разделах я опишу несколько способов интерпретации собственных чисел/векторов. Конечно же, математика в любом случае остается одинаковой, но наличие нескольких перспектив может облегчить интуитивное понимание, которое, в свою очередь, поможет разобраться в том, как и почему собственное разложение имеет такую важность в науке о данных.

Геометрия

На самом деле в главе 5 я уже познакомил вас с геометрической концепцией собственных векторов. На рис. 5.5 мы обнаружили, что существует особая комбинация матрицы и вектора, при которой матрица *растягивает* – но не *поворачивает* – этот вектор. Этот вектор является собственным вектором матрицы, а величина растяжения – собственным числом.

На рис. 13.1 показаны векторы до и после умножения на матрицу 2×2. Два вектора на левом графике ($\mathbf{v}_1$ и $\mathbf{v}_2$) являются собственными векторами, а два вектора на правом графике – нет. Собственные векторы указывают в одном направлении до и после постпозиционного умножения матрицы. В собственных числах кодируется степень растяжения; попытайтесь угадать собственные числа на основе визуального осмотра графика. Ответы – в сноске[1].

Вот геометрическая картина: собственный вектор означает, что умножение матрицы на вектор действует как умножение вектора на скаляр. Давайте посмотрим, сможем ли мы записать это в виде уравнения (мы можем, и это напечатано в уравнении 13.1).

Уравнение 13.1. Уравнение собственного числа

$$\mathbf{Av} = \lambda\mathbf{v}.$$

[1] Приблизительно 0.6 и 1.6.

Будьте осторожны с интерпретацией этого уравнения: оно не говорит, что матрица равна скаляру; в нем говорится, что влияние матрицы на вектор такое же, как влияние скаляра на тот же вектор.

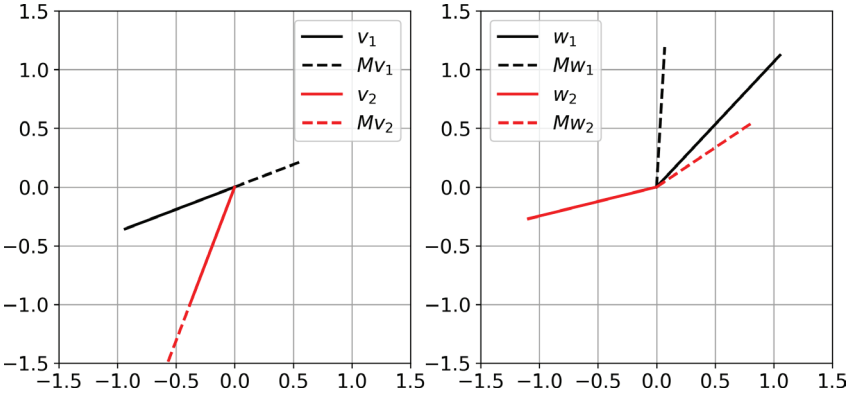


Рис. 13.1 ❖ Геометрия собственных векторов

Эта формула называется уравнением собственного числа, и это еще одна ключевая формула линейной алгебры, которую стоит запомнить. Вы будете сталкиваться с ней на протяжении всей этой главы, вы увидите ее небольшую вариацию в следующей главе, и вы будете сталкиваться с ней много раз, изучая многопеременную статистику, обработку сигналов, оптимизацию, теорию графов и бесчисленное число других приложений, в которых среди нескольких одновременно регистрируемых признаков выявляются закономерности.

Статистика (анализ главных компонент)

Одной из причин, по которой люди применяют статистику, является выявление и количественное оценивание взаимосвязей между переменными. Например, повышение глобальной температуры коррелирует с сокращением числа пиратов[1], но насколько сильна эта взаимосвязь? Разумеется, когда у вас есть только две переменные, будет достаточно простой корреляции (подобной той, о которой вы узнали в главе 4). Но в многопеременном наборе данных, включающем десятки или сотни переменных, двухпеременные корреляции не способны выявлять глобальные закономерности.

Давайте конкретизируем это на примере. Криптовалюты – это цифровые хранилища стоимости, закодированные в блочной цепи, так называемом блокчейне, который представляет собой систему отслеживания транзакций. Вы, наверное, слышали о биткойне и эфириуме; существуют десятки тысяч других криптомонет, которые имеют различное предназначение. Можно за-

[1] «Открытое письмо школьному совету Канзаса», Церковь летающего макаронного монстра, spaghettimonster.org/about/open-letter.

дасться вопросом, работает ли все криптопространство как единая система (имеется в виду, что стоимость всех монет движется вверх и вниз вместе) или же в этом пространстве есть независимые подкатегории (имеется в виду, что стоимость некоторых монет или групп монет меняется независимо от стоимости других монет).

Эту гипотезу можно проверить, выполнив анализ главных компонент на наборе данных, содержащем цены различных криптовалют с течением времени. Если бы весь крипторынок работал как единое целое, то график крутого склона[1] (график собственных чисел матрицы ковариаций в наборе данных) показал бы, что одна компонента отвечает за бóльшую часть дисперсии системы, а все остальные компоненты – за очень малую дисперсию (график A на рис. 13.2). Напротив, если бы рынок криптовалют имел, скажем, три главенствующие подкатегории с независимыми движениями цен, то мы бы ожидали увидеть три больших собственных числа (график B на рис. 13.2).

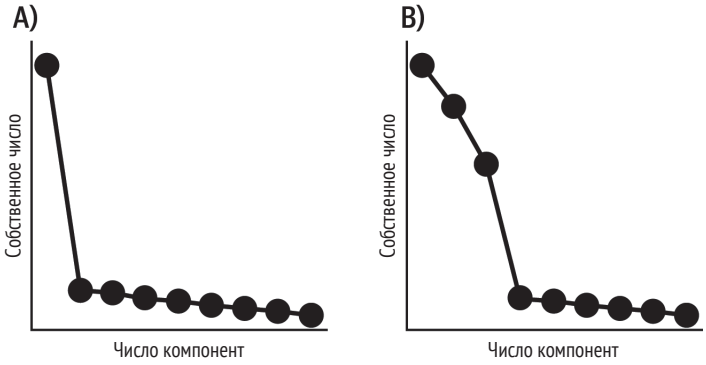


Рис. 13.2 ❖ Симулированные графики крутого склона на многопеременных наборах данных (данные просимулированы, чтобы проиллюстрировать возможные результаты)

Подавление шума

Большинство наборов данных содержат шум. Шум относится к вариациям в наборе данных, которые либо необъяснимы (например, случайные вариации), либо нежелательны (например, артефакты шума электрических линий в радиосигналах). Существует целый ряд способов ослабления или устранения шума, и оптимальная стратегия подавления шума зависит от характера и происхождения шума, а также от характеристик сигнала.

Одним из методов подавления случайного шума является идентификация собственных чисел и собственных векторов системы, а также «редукция» направлений в пространстве данных, связанных с малыми собственными числами. При этом принято исходить из допущения, что случайный шум вносит относительно малый вклад в общую дисперсию. «Редуцирование» размер-

[1] Англ. *scree plot*. – *Прим. перев.*

ности данных означает реконструкцию набора данных после приравнивания к нулю некоторых собственных чисел, которые ниже некоторого порога.

В главе 15 вы увидите пример использования собственного разложения для подавления шума.

Уменьшение размерности (сжатие данных)

Информационные коммуникационные технологии, такие как телефоны, интернет и телевидение, создают и передают огромный объем данных, таких как изображения и видео. Передача данных может занимать много времени и средств, и выгодно *сжимать* данные перед их передачей. Сжатие означает уменьшение размера данных (в байтах) при минимальном влиянии на качество данных. Например, файл изображения в формате TIFF может иметь размер 10 Мб, тогда как преобразованная в JPG версия может иметь размер 0.1 Мб, сохраняя при этом достаточно хорошее качество.

Один из способов размерно уменьшить набор данных состоит в том, чтобы взять его собственное разложение, отбросить собственные числа и собственные векторы, связанные с малыми направлениями в пространстве данных, а затем передать только относительно большие пары собственный вектор/число. На самом деле для сжатия данных чаще используется SVD (в главе 15 вы увидите пример), хотя принцип остается тем же.

Современные алгоритмы сжатия данных на самом деле работают быстрее и эффективнее, чем ранее описанный метод, но идея остается той же: разложить набор данных на набор базисных векторов, которые охватывают наиболее важные признаки данных, а затем восстановить высококачественную версию изначальных данных.

Отыскание собственных чисел

Для того чтобы выполнить собственное разложение квадратной матрицы, сначала нужно найти собственные числа, а затем использовать каждое собственное число для отыскания соответствующего собственного вектора. Собственные числа подобны ключам, которые вставляются в матрицу, чтобы получить доступ к мистическому собственному вектору.

Отыскание собственных чисел матрицы на Python выполняется очень легко:

```
matrix = np.array([
    [1,2],
    [3,4]
    ])

# получить собственные числа
evals = np.linalg.eig(matrix)[0]
```

Два собственных числа (округленные до сотых) равны –0.37 и 5.37.

Однако важный вопрос не в том, *какая функция возвращает собственные числа*, а в том, *как собственные числа матрицы идентифицируются*.

Для того чтобы найти собственные числа матрицы, мы начинаем с уравнения собственного числа, показанного в уравнении 13.1, и выполняем несколько простых арифметических действий, как показано в уравнении 13.2.

Уравнение 13.2. Реорганизованное уравнение собственного числа

$$\mathbf{Av} = \lambda\mathbf{v};$$

$$\mathbf{Av} - \lambda\mathbf{v} = \mathbf{0};$$

$$(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}.$$

Первое уравнение является точным повторением уравнения собственного числа. Во втором уравнении мы просто вычли правую часть, чтобы приравнять уравнение к вектору нулей.

Переход от второго к третьему уравнению требует некоторого пояснения. В левой части второго уравнения есть два векторных члена, оба из которых содержат **v**. И поэтому мы выносим этот вектор за скобки. Но в результате мы остаемся с операцией вычитания матрицы и скаляра ($\mathbf{A} - \lambda$), которая в линейной алгебре не определена[1]. Вместо этого мы *сдвигаем* матрицу на λ. Это подводит нас к третьему уравнению. (Попутное замечание: выражение $\lambda\mathbf{I}$ иногда называется *скалярной матрицей*.)

Что означает это третье уравнение? Оно означает, что *собственный вектор находится в нуль-пространстве матрицы, сдвинутой на ее собственное число*.

Если это поможет вам понять концепцию собственного вектора как нуль-пространственного вектора сдвинутой матрицы, то можете подумать о добавлении двух дополнительных уравнений:

$$\tilde{\mathbf{A}} = \mathbf{A} - \lambda\mathbf{I};$$

$$\tilde{\mathbf{A}}\mathbf{v} = \mathbf{0}.$$

Почему это утверждение столь проницательно? Вспомните, что в линейной алгебре тривиальные решения игнорируются, поэтому мы не считаем $\mathbf{v} = \mathbf{0}$ собственным вектором. А это означает, что сдвинутая на собственное число матрица является сингулярной, потому что только сингулярные матрицы имеют нетривиальное нуль-пространство.

А что еще мы знаем о сингулярных матрицах? Мы знаем, что их определитель равен нулю. Следовательно:

$$|\mathbf{A} - \lambda\mathbf{I}| = 0.$$

Хотите верьте, хотите нет, но это ключ к отысканию собственных чисел: надо сдвинуть матрицу на неизвестное собственное число λ, приравнять его

[1] Как я писал в главе 5, Python вернет результат, но это будет транслированное вычитание скаляра, которое не является линейно-алгебраической операцией.

определитель к нулю и найти λ. Давайте посмотрим, как это выглядит для матрицы 2×2:

$$\left|\begin{bmatrix} a & b \\ c & d \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right| = 0;$$

$$\begin{vmatrix} a-\lambda & b \\ c & d-\lambda \end{vmatrix} = 0;$$

$$(a-\lambda)(d-\lambda) - bc = 0;$$

$$\lambda^2 - (a+d)\lambda + (ad-bc) = 0.$$

Для того чтобы найти решение для двух значений λ, можно применить квадратичную формулу. Но сам ответ не важен; важно увидеть логическую последовательность математических понятий, сформулированных ранее в этой книге:

- умножение матрицы на вектор действует как умножение вектора на скаляр (уравнение собственного числа);
- мы приравниваем уравнение собственного числа к вектору нулей и выносим за скобки общие члены;
- этим показывается, что собственный вектор находится в нуль-пространстве матрицы, сдвинутой на собственное число. Мы не считаем вектор нулей собственным вектором, а значит, сдвинутая матрица является сингулярной;
- поэтому мы приравниваем определитель сдвинутой матрицы к нулю и находим неизвестное собственное число.

Приравненный к нулю определитель сдвинутой собственным числом матрицы называется *характеристическим многочленом* матрицы.

Обратите внимание, что в предыдущем примере мы начали с матрицы 2×2 и получили член λ^2, и, значит, это полиномиальное уравнение второй степени. Возможно, из школьного курса алгебры вы помните, что многочлен n-й степени имеет n решений, некоторые из которых могут быть комплекснозначными (это называется фундаментальной теоремой алгебры). И поэтому будет два удовлетворяющих уравнению числа λ.

Совпадающие двойки не случайны: характеристический многочлен матрицы M×M будет иметь член λ^M. По этой причине матрица M×M будет иметь M собственных чисел.



Утомительные практические задачи

На этом этапе в традиционном учебнике по линейной алгебре вам поручили бы найти вручную собственные числа десятков матриц 2×2 и 3×3. У меня по поводу такого рода упражнений смешанные чувства: с одной стороны, решение задач вручную действительно помогает усваивать механизм отыскания собственных чисел; но, с другой стороны, в этой книге я хочу сосредоточить внимание на концепциях, исходном коде и приложениях, не увязая в утомительной арифметике. Если вы чувствуете вдохновение решать задачи на собственные числа вручную, тогда дерзайте! Вы можете найти массу таких задач в традиционных учебниках или в интернете. Но я принял смелое

(и, возможно, спорное) решение избегать в этой книге задач, решаемых вручную, и вместо этого использовать упражнения, посвященные программированию и глубокому пониманию.

Отыскание собственных векторов

Как и в случае с собственными числами, отыскание собственных векторов на Python выполняется очень легко:

```
evals,evecs = np.linalg.eig(matrix)
print(evals), print(evecs)

[-0.37228132  5.37228132]

[[-0.82456484 -0.41597356]
 [ 0.56576746 -0.90937671]]
```

Собственные векторы находятся в столбцах матрицы `evecs` и в том же порядке, что и собственные числа (то есть собственный вектор в первом столбце матрицы `evecs` парно связан с первым собственным числом в векторе `evals`). Мне нравится использовать имена переменных `evals` и `evecs`, потому что они короткие и осмысленные. Вы также, возможно, заметите, что нередко используются имена переменных `L` и `V` или `D` и `V`. `L` – это Λ (заглавная буква λ), а `V` – это $\mathbf{V}$, матрица, в которой каждый столбец i является собственным вектором $\mathbf{v}_i$. `D` означает диагональ, потому что собственные числа часто хранятся в диагональной матрице по причинам, которые я объясню позже в этой главе.



Собственные векторы в столбцах, а не в строках!

Самое важное, что нужно помнить о собственных векторах при программировании, – это то, что они хранятся в *столбцах матрицы*, а не в строках! Такие размерно-индексационные ошибки легко возникают при работе с квадратными матрицами (потому что Python не будет реагировать сообщением об ошибке), но случайное использование строк вместо столбцов матрицы собственных векторов может иметь катастрофические последствия в приложениях. Если вы сомневаетесь, то вспомните изложение в главе 2 о том, что в линейной алгебре общепринято исходить из допущения, что векторы имеют ориентацию вдоль столбца.

Хорошо, но опять же, приведенный выше исходный код показывает, как получать собственные векторы матрицы из функции NumPy. Это можно узнать из документационного литерала docstring по функции `np.linalg.eig`. Важный вопрос: а откуда берутся собственные векторы и как их отыскивать?

На самом деле я уже писал, как отыскивать собственные векторы: надо найти вектор $\mathbf{v}$, который находится в нуль-пространстве матрицы, сдвинутой на λ. Другими словами:

$$\mathbf{v}_i \in N(\mathbf{A} - \lambda_i \mathbf{I}).$$

Давайте посмотрим на числовой пример. Ниже приведены матрица и ее собственные числа:

$$\begin{bmatrix}1 & 2\\ 2 & 1\end{bmatrix} \quad \Rightarrow \quad \lambda_1 = 3, \lambda_2 = -1.$$

Сосредоточимся на первом собственном числе. Для того чтобы выявить его собственный вектор, мы сдвигаем матрицу на 3 и находим вектор в его нуль-пространстве:

$$\begin{bmatrix}1-3 & 2\\ 2 & 1-3\end{bmatrix} = \begin{bmatrix}-2 & 2\\ 2 & -2\end{bmatrix} \quad \Rightarrow \quad \begin{bmatrix}-2 & 2\\ 2 & -2\end{bmatrix}\begin{bmatrix}1\\ 1\end{bmatrix} = \begin{bmatrix}0\\ 0\end{bmatrix}.$$

Это означает, что [1 1] является собственным вектором матрицы, ассоциированным с собственным числом 3.

Я нашел этот нуль-пространственный вектор, просто взглянув на матрицу. Как выявляют нуль-пространственные векторы (то есть собственные векторы матрицы) на практике?

Нуль-пространственные векторы можно найти методом Гаусса–Джордана, чтобы решить систему уравнений, где матрица коэффициентов – это λ-сдвинутая матрица, а вектор констант – вектор нулей. Это хороший способ концептуализировать решение. В реализации применяются более стабильные численные методы отыскания собственных чисел и собственных векторов, в том числе QR-разложение и процедура, именуемая степенным методом.

Неопределенность собственных векторов по знаку и шкале

Давайте я вернусь к числовому примеру из предыдущего раздела. Я написал, что [1 1] – это собственный вектор матрицы, потому что этот вектор является базисом нуль-пространства матрицы, сдвинутой на ее собственное число 3.

Посмотрите на сдвинутую матрицу и спросите себя, является ли [1 1] единственным возможным базисным вектором нуль-пространства? Даже близко ничего похожего! Можно также использовать [4 4] или [−5.4 −5.4], или... Я думаю, вы понимаете, к чему это идет: *любая* шкалированная версия вектора [1 1] является базисом этого нуль-пространства. Другими словами, если **v** – это собственный вектор матрицы, то таким же является и α**v** для любого действительно-значного α, кроме нуля.

В этой связи следует отметить, что важность собственных векторов обусловливается их *направлением*, а не их *модулем*.

Бесконечность возможных нуль-пространственных базисных векторов приводит к двум вопросам:

- *существует ли один «самый лучший» базисный вектор?* «Самого лучшего» базисного вектора как такового не существует, но удобно иметь

единично-нормализованные собственные векторы (евклидова норма 1). Это особенно полезно для симметричных матриц по причинам, которые будут объяснены позже в этой главе[1];

- *каков «правильный» знак собственного вектора?* Никакой. На самом деле при использовании разных версий NumPy, а также разных программ, таких как MATLAB, Julia или Mathematica, вы можете получать разные знаки собственных векторов из одной и той же матрицы. Неопределенность знака собственного вектора – это просто особенность жизни в нашей Вселенной. В таких применениях, как PCA, существуют принципиальные способы придания им знака, но это просто общепринятое соглашение, облегчающее интерпретацию.

Диагонализация квадратной матрицы

Уравнение собственного числа, с которым вы теперь знакомы, содержит одно собственное число и один собственный вектор. Это означает, что матрица $M \times M$ имеет M уравнений собственных чисел:

$$\mathbf{A}\mathbf{v}_1 = \lambda_1\mathbf{v}_1;$$
$$\vdots$$
$$\mathbf{A}\mathbf{v}_M = \lambda_M\mathbf{v}_M.$$

В этой серии уравнений нет ничего неправильного, но она довольно уродлива, а уродство нарушает один из принципов линейной алгебры: делать уравнения компактными и элегантными. Поэтому мы преобразовываем эту серию уравнений в одно матричное уравнение.

Ключевым моментом при написании уравнения матричного собственного числа является то, что каждый столбец матрицы собственных векторов шкалируется ровно одним собственным числом. Это реализуется с помощью постпозиционного умножения на диагональную матрицу (как вы узнали из главы 6).

Таким образом, вместо хранения собственных чисел в векторе мы храним собственные числа в диагонали матрицы. Следующее ниже уравнение показывает форму диагонализации для матрицы 3×3 (с использованием @ вместо числовых значений матрицы). В матрице собственных векторов первое подстрочное число соответствует собственному вектору, а второе подстрочное число – элементу собственного вектора. Например, v_{12} – это второй элемент первого собственного вектора:

---

[1] Для того чтобы снять неопределенность, следует отметить, что за счет этого матрица собственных векторов делается ортогональной матрицей.

$$\begin{bmatrix} @ & @ & @ \\ @ & @ & @ \\ @ & @ & @ \end{bmatrix}\begin{bmatrix} v_{11} & v_{21} & v_{31} \\ v_{12} & v_{22} & v_{32} \\ v_{13} & v_{23} & v_{33} \end{bmatrix} = \begin{bmatrix} v_{11} & v_{21} & v_{31} \\ v_{12} & v_{22} & v_{32} \\ v_{13} & v_{23} & v_{33} \end{bmatrix}\begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \lambda_3 \end{bmatrix}$$

$$= \begin{bmatrix} \lambda_1 v_{11} & \lambda_2 v_{21} & \lambda_3 v_{31} \\ \lambda_1 v_{12} & \lambda_2 v_{22} & \lambda_3 v_{32} \\ \lambda_1 v_{13} & \lambda_2 v_{23} & \lambda_3 v_{33} \end{bmatrix}.$$

Пожалуйста, найдите время, чтобы убедиться, что каждое собственное число шкалирует все элементы соответствующего собственного вектора, а не какие-либо другие собственные векторы.

В более общем случае матричное уравнение собственных чисел – также именуемое диагонализацией квадратной матрицы – таково:

$$\mathbf{AV} = \mathbf{V\Lambda}.$$

Функция `eig` библиотеки NumPy возвращает собственные векторы в матрице и собственные числа в векторе. Это означает, что для диагонализации матрицы в NumPy требуется немного дополнительного исходного кода:

```
evals,evecs = np.linalg.eig(matrix)
D = np.diag(evals)
```

Между прочим, в математике часто интересно и познавательно переставлять уравнения, находя решения для разных переменных. Рассмотрим следующий ниже список эквивалентных объявлений:

$$\mathbf{AV} = \mathbf{V\Lambda};$$

$$\mathbf{A} = \mathbf{V\Lambda V}^{-1};$$

$$\mathbf{\Lambda} = \mathbf{V}^{-1}\mathbf{AV}.$$

Второе уравнение показывает, что матрица **A** становится диагональной внутри пространства **V** (то есть **V** перемещает нас в «диагональное пространство», а затем $\mathbf{V}^{-1}$ возвращает нас назад из диагонального пространства). Это можно интерпретировать в контексте базисных векторов: матрица **A** является плотной в стандартном базисе, но затем мы применяем набор преобразований (**V**), чтобы повернуть матрицу в новый набор базисных векторов (собственных векторов), в которых информация разрежена и представлена диагональной матрицей. (В конце уравнения нам нужно вернуться назад в стандартное базисное пространство, отсюда и $\mathbf{V}^{-1}$.)

Особая удивительность симметричных матриц

Из предыдущих глав вы уже знаете, что симметричные матрицы обладают особыми свойствами, благодаря которым с ними очень удобно работать. Теперь вы готовы познакомиться с еще двумя специальными свойствами, относящимися к собственному разложению.

Ортогональные собственные векторы

Симметричные матрицы имеют ортогональные собственные векторы. Это означает, что все собственные векторы симметричной матрицы попарно ортогональны. Давайте я начну с примера, затем разберу последствия ортогональности собственных векторов и, наконец, покажу доказательство:

```
# всего лишь случайная симметричная матрица
A = np.random.randint(-3, 4, (3,3))
A = A.T@A

# ее собственное разложение
L,V = np.linalg.eig(A)

# все попарные точечные произведения
print( np.dot(V[:,0], V[:,1]) )
print( np.dot(V[:,0], V[:,2]) )
print( np.dot(V[:,1], V[:,2]) )
```

Все три точечных произведения равны нулю (в пределах вычислительных ошибок округления порядка 10^{-16}. (Обратите внимание, что я создал симметричные матрицы как случайную матрицу, умноженную на ее транспонированную версию.)

Свойство ортогональности собственного вектора означает, что точечное произведение между любой парой собственных векторов равно нулю, тогда как точечное произведение собственного вектора с самим собой не равно нулю (поскольку мы не считаем вектор нулей собственным вектором). Это можно записать как $\mathbf{V}^{\mathrm{T}}\mathbf{V} = \mathbf{D}$, где $\mathbf{D}$ – это диагональная матрица с диагоналями, содержащими нормы собственных векторов.

Но можно сделать еще лучше, чем просто диагональную матрицу: вспомните, что важность собственных векторов обусловлена их *направлением*, а не их *модулем*. И поэтому собственный вектор может иметь любой желаемый модуль (кроме, очевидно, нулевого модуля).

Давайте прошкалируем все собственные векторы так, чтобы они имели единичную длину. Вопрос к вам: если все собственные векторы ортогональны и имеют единичную длину, что произойдет, если умножить матрицу собственных векторов на ее транспонированную версию?

Вы, конечно же, знаете ответ:

$$\mathbf{V}^{\mathrm{T}}\mathbf{V} = \mathbf{I}.$$

Другими словами, матрица собственных векторов симметричной матрицы является ортогональной матрицей! Это имеет целый ряд последствий для науки о данных, в том числе то, что собственные векторы очень легко инвертировать (потому что их надо просто транспонировать). Существуют и другие последствия использования ортогональных собственных векторов для таких применений, как анализ главных компонент, о которых я расскажу в главе 15.

В главе 1 я писал, что в этой книге сравнительно мало доказательств. Но ортогональные собственные векторы симметричных матриц – это настолько важная концепция, что вам действительно нужно убедиться, что данное утверждение доказано.

Цель этого доказательства – показать, что точечное произведение любой пары собственных векторов равно нулю. Мы исходим из двух допущений:

1) матрица $\mathbf{A}$ – это симметричная матрица и
2) λ_1 и λ_2 – это отдельные собственные числа матрицы $\mathbf{A}$ (отдельные в смысле, что они не могут быть равны друг другу) с $\mathbf{v}_1$ и $\mathbf{v}_2$ в качестве соответствующих им собственных векторов.

Попробуйте проследить за каждым шагом равенства слева направо в уравнении 13.3.

Уравнение 13.3. Доказательство ортогональности собственных векторов для симметричных матриц

$$\lambda_1\mathbf{v}_1^T\mathbf{v}_2 = (\mathbf{A}\mathbf{v}_1)^T\mathbf{v}_2 = \mathbf{v}_1^T\mathbf{A}^T\mathbf{v}_2 = \mathbf{v}_1^T\lambda_2\mathbf{v}_2 = \lambda_2\mathbf{v}_1^T\mathbf{v}_2.$$

Члены в середине – это просто преобразования; обратите внимание на первый и последний члены. Они переписываются в уравнении 13.4, а затем вычитаются для обнуления.

Уравнение 13.4. Продолжение доказательства ортогональности собственных векторов...

$$\lambda_1\mathbf{v}_1^T\mathbf{v}_2 = \lambda_2\mathbf{v}_1^T\mathbf{v}_2;$$

$$\lambda_1\mathbf{v}_1^T\mathbf{v}_2 = \lambda_2\mathbf{v}_1^T\mathbf{v}_2 = 0.$$

Оба члена содержат точечное произведение $\mathbf{v}_1^T\mathbf{v}_2$, которое можно исключить. Это подводит нас к заключительной части доказательства, показанной в уравнении 13.5.

Уравнение 13.5. Доказательство ортогональности собственных векторов, часть 3

$$(\lambda_1 - \lambda_2)\mathbf{v}_1^T\mathbf{v}_2 = 0.$$

Приведенное выше последнее уравнение говорит о том, что два числа умножаются, чтобы получить 0, а это означает, что одно или оба этих числа должны быть равны нулю. Разность $\lambda_1 - \lambda_2$ не может быть равна нулю, поскольку мы исходили из допущения, что они различаются. Следовательно,

$\mathbf{v}_1^T\mathbf{v}_2$ должно быть равно нулю, а это означает, что два собственных вектора ортогональны. Вернитесь к уравнениям, чтобы убедиться, что это доказательство не работает для несимметричных матриц, когда $\mathbf{A}^T \neq \mathbf{A}$. Таким образом, собственные векторы несимметричной матрицы не ограничены быть ортогональными (они будут линейно независимыми для всех различающихся собственных чисел, но я опущу это обсуждение и доказательство).

Действительно-значные собственные числа

Второе особое свойство симметричных матриц состоит в том, что они имеют действительно-значные собственные числа (и, следовательно, действительно-значные собственные векторы).

Давайте я начну с демонстрации того, что матрицы – даже со всеми действительно-значными элементами – могут иметь комплексно-значные собственные числа:

```
A = np.array([[-3, -3, 0],
              [ 3, -2, 3],
              [ 0,  1, 2]])

# ее собственное разложение
L,V = np.linalg.eig(A)
L.reshape(-1,1) # напечатать в виде вектора-столбца

>> array([[-2.744739  +2.85172624j],
          [-2.744739  -2.85172624j],
          [ 2.489478  +0.j        ]])
```

(Будьте осторожны с интерпретацией этого массива NumPy; это не матрица 3×2; это вектор-столбец 3×1, содержащий комплексные числа. Обратите внимание на j и отсутствие запятой между числами.)

3×3-матрица **A** имеет два комплексно-значных собственных числа и одно действительно-значное собственное число. Сопряженные с комплексно-значными собственными числами собственные векторы сами будут комплексно-значными. В этой конкретной матрице нет ничего особенного; я буквально сгенерировал ее из случайных целых чисел от −3 до +3. Интересно, что комплексно-значные решения выводятся сопряженными парами. Это означает, что если существует $\lambda_i = a + ib$, то существует и $\lambda_k = a - ib$. Соответствующие им собственные векторы также являются комплексными сопряженными парами.

Не буду вдаваться в подробности комплексно-значных решений, кроме как покажу, что комплексные решения задачи собственного разложения элементарны[1].

Симметричные матрицы гарантированно имеют действительно-значные собственные числа, следовательно, также действительно-значные собствен-

[1] Под «элементарными» я подразумеваю математически ожидаемые; интерпретация сложных решений в собственном разложении далеко не элементарна.

ные векторы. Давайте я начну с того, что видоизменю приведенный выше пример, чтобы сделать матрицу симметричной:

```
A = np.array([[-3, -3, 0],
              [-3, -2, 1],
              [ 0,  1, 2]])

# ее собственное разложение
L,V = np.linalg.eig(A)
L.reshape(-1,1) # напечатать в виде вектора-столбца

>> array([[-5.59707146],
          [ 0.22606174],
          [ 2.37100972]])
```

Это только один конкретный пример; может, нам тут повезло? Рекомендую сделать паузу и обследовать этот вопрос самостоятельно в онлайновом исходном коде; вы можете создать случайную симметричную матрицу (путем создания случайной матрицы и собственного разложения $\mathbf{A}^T\mathbf{A}$) любого размера, чтобы подтвердить, что собственные числа являются действительно-значными.

Получение гарантированных действительно-значных собственных чисел из симметричных матриц – это настоящая удача, потому что с комплексными числами зачастую сложно работать. Многие используемые в науке о данных матрицы являются симметричными, поэтому если в своих приложениях в этой области вы видите комплексные собственные числа, то, возможно, проблема связана с исходным кодом или данными.



Эффективное применение симметрии

Если вы знаете, что работаете с симметричной матрицей, то вместо функции `np.linalg.eig` можете использовать функцию `np.linalg.eigh` (или вместо функции SciPy `eig` – функцию `eigh`). Символ `h` означает «эрмитову матрицу»[1], то есть комплексную версию симметричной матрицы. Функция `eigh` в целом работает быстрее и является более численно стабильной, чем функция `eig`, но оперирует только симметричными матрицами.

Собственное разложение сингулярных матриц

Я вставил этот раздел сюда, потому что обнаружил, что у студентов часто возникает идея, что сингулярные матрицы невозможно разложить процедурой собственного разложения или что собственные векторы сингулярной матрицы должны быть какими-то необычными.

Эта идея совершенно неверна. Применять собственное разложение к сингулярным матрицам – это совершенно нормально. Вот краткий пример:

```
# сингулярная матрица
A = np.array([[1,4,7],
```

[1] Англ. *Hermitian*. – *Прим. перев.*

```
            [2,5,8],
            [3,6,9]])

# ее собственное разложение
L,V = np.linalg.eig(A)
```

Распечатка ранга, собственных чисел и собственных векторов этой матрицы приведена ниже:

```
print( f'Ранг = {np.linalg.matrix_rank(A)}\n' )
print('Собственные числа: '), print(L.round(2)), print(' ')
print('Собственные векторы:'), print(V.round(2))

>> Ранг = 2

Собственные числа:
[16.12 -1.12 -0.  ]

Собственные векторы:
[[-0.46 -0.88  0.41]
 [-0.57 -0.24 -0.82]
 [-0.68  0.4   0.41]]
```

Эта матрица ранга 2 имеет одно нуль-значное собственное число с ненулевым собственным вектором. Вы можете использовать онлайновый исходный код, чтобы обследовать собственное разложение других рангово-пониженных случайных матриц.

Существует одно особое свойство собственного разложения сингулярных матриц, при котором по крайней мере одно собственное число гарантированно равно нулю. Это не означает, что количество ненулевых собственных чисел равно рангу матрицы – это верно для сингулярных чисел (скаляров из SVD), но не для собственных чисел. Но если матрица – сингулярная, то хотя бы одно собственное число равно нулю.

Верно и обратное: каждая полноранговая матрица имеет ноль нуль-значных собственных чисел.

Одно из объяснений причины, по которой это происходит, заключается в том, что сингулярная матрица уже имеет нетривиальное нуль-пространство, а это означает, что $\lambda = 0$ обеспечивает нетривиальное решение уравнения $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$. Это видно в приведенном выше примере матрицы: ассоциированный с $\lambda = 0$ собственный вектор – это нормализованный вектор [1 −2 1], то есть линейно-взвешенная комбинация столбцов (или строк), которая дает вектор нулей.

Основные выводы, которые следует вынести из этого раздела, таковы:

1) собственное разложение допустимо для рангово-пониженных матриц и
2) наличие по меньшей мере одного нуль-значного собственного числа указывает на рангово-пониженную матрицу.

Квадратичная форма, определенность и собственные числа

Следует признать, что термины *квадратичная форма* и *определенность* выглядят пугающе. Но не беспокойтесь – оба этих термина просты и открывают доступ к продвинутой линейной алгебре и таким ее применениям, как анализ главных компонент и симуляции методом Монте-Карло. И что еще лучше: интеграция исходного кода Python в ваше обучение даст вам огромное преимущество в изучении этих понятий по сравнению с традиционными учебниками по линейной алгебре.

Квадратичная форма матрицы

Рассмотрим следующее ниже выражение:

$$\mathbf{v}^{\mathrm{T}}\mathbf{A}\mathbf{w} = \alpha.$$

Другими словами, мы пред- и постпозиционно умножаем квадратную матрицу на один и тот же вектор **w** и получаем скаляр. (Обратите внимание, что это умножение допустимо только для квадратных матриц.)

Это называется *квадратичной формой* матрицы **A**.

Какую матрицу и какой вектор мы используем? Идея квадратичной формы заключается в использовании одной конкретной матрицы и набора всех возможных векторов (соответствующего размера). Важный вопрос касается знаков α всех возможных векторов. Давайте посмотрим на пример:

$$\begin{bmatrix} x & y \end{bmatrix}\begin{bmatrix} 2 & 4 \\ 0 & 3 \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix} = 2x^2 + (0+4)xy + 3y^2.$$

Для этой конкретной матрицы не существует возможной комбинации x и y, которая может давать отрицательный ответ, потому что квадраты членов ($2x^2$ и $3y^2$) всегда будут перевешивать перекрестный член ($4xy$), даже если x или y – отрицательные. Кроме того, α может быть неположительным только при $x = y = 0$.

Это нетривиальный результат квадратичной формы. Например, следующая ниже матрица может иметь положительное или отрицательное α в зависимости от значений x и y:

$$\begin{bmatrix} x & y \end{bmatrix}\begin{bmatrix} -9 & 4 \\ 3 & 9 \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix} = -9x^2 + (3+4)xy + 9y^2.$$

Вы можете подтвердить, что задание [x y] равным [−1 1] дает отрицательный результат квадратичной формы, а [−1 −1] дает положительный результат.

Как вообще можно узнать, какой скаляр (положительный, отрицательный либо нулевой) будет произведен квадратичной формой для всех возможных векторов? Ключ происходит из учитывания того, что полноранговая матрица собственных векторов охватывает все $\mathbb{R}^M$, и, следовательно, каждый вектор из $\mathbb{R}^M$ может быть выражен как некоторая взвешенная линейная комбинация собственных векторов[1]. Затем мы начинаем с уравнения собственного числа и умножаем его слева на собственный вектор, чтобы вернуться к квадратичной форме:

$$\mathbf{Av} = \lambda\mathbf{v};$$

$$\mathbf{v}^{\mathrm{T}}\mathbf{Av} = \lambda\mathbf{v}^{\mathrm{T}}\mathbf{v};$$

$$\mathbf{v}^{\mathrm{T}}\mathbf{Av} = \lambda||\mathbf{v}||^2.$$

Последнее уравнение является ключевым. Обратите внимание, что $||\mathbf{v}^{\mathrm{T}}\mathbf{v}||^2$ – строго положительный (модули векторов не могут быть отрицательными, и мы игнорируем вектор нулей), а это означает, что знак правой части уравнения полностью определяется собственным числом λ.

В этом уравнении используется только одно собственное число и его собственный вектор, но нам нужно знать о любом возможном векторе. Ключевой для понимания момент состоит в том, чтобы учесть, что если уравнение допустимо для каждой пары «собственный вектор – собственное число», то оно допустимо для любой комбинации пар «собственный вектор – собственное число». Например:

$$\mathbf{v}_1^{\mathrm{T}}\mathbf{Av}_1 = \lambda_1||\mathbf{v}_1||^2;$$

$$\mathbf{v}_2^{\mathrm{T}}\mathbf{Av}_2 = \lambda_2||\mathbf{v}_2||^2;$$

$$(\mathbf{v}_1 + \mathbf{v}_2)^{\mathrm{T}}\mathbf{A}(\mathbf{v}_1 + \mathbf{v}_2) = (\lambda_1 + \lambda_2)||\mathbf{v}_1 + \mathbf{v}_2||^2;$$

$$\mathbf{u}^{\mathrm{T}}\mathbf{Au} = \zeta||\mathbf{u}||^2.$$

Другими словами, мы можем задать любой вектор $\mathbf{u}$ как некоторую линейную комбинацию собственных векторов и некоторый скаляр ζ как ту же самую линейную комбинацию собственных чисел. Во всяком случае, это не меняет того принципа, что знак правой части – а значит, и знак квадратичной формы – определяется знаком собственных чисел.

Теперь давайте подумаем об этих уравнениях в условиях разных допущений о знаках собственных чисел λ.

Все собственные числа положительны

Правая часть уравнения – всегда положительная, а это означает, что $\mathbf{v}^{\mathrm{T}}\mathbf{Av}$ всегда положителен для любого вектора $\mathbf{v}$.

Собственные числа положительны либо равны нулю

$\mathbf{v}^{\mathrm{T}}\mathbf{Av}$ неотрицателен и будет равен нулю при $\lambda = 0$ (что происходит, когда матрица является сингулярной).

---

[1] Ради краткости я здесь опускаю некоторые тонкости относительно редких случаев, когда матрица собственных векторов не охватывает всемерное подпространство.

Собственные числа отрицательны либо равны нулю
Результат квадратичной формы будет нулевым или отрицательным.

Собственные числа – отрицательные
Результат квадратичной формы будет отрицательным для всех векторов.

Определенность

Определенность – это характеристика квадратной матрицы и определяется знаками собственных чисел матрицы, которые равносильны знакам результатов квадратичной формы. Определенность влияет на обратимость матрицы, а также на продвинутые методы анализа данных, такие как обобщенное собственное разложение (используемое в многопеременных линейных классификаторах и обработке сигналов).

Как показано в табл. 13.1, существует пять категорий определенности; знаки + и – указывают знаки собственных чисел.

Таблица 13.1. Категории определенности

Категория	Квадратичная форма	Собственные числа	Обратимая
Положительно определенная	Положительная	+	Да
Положительно полуопределенная	Неотрицательная	+ и 0	Нет
Неопределенная	Положительная и отрицательная	+ и –	Зависит
Отрицательно полуопределенная	Неположительная	– и 0	Нет
Отрицательно определенная	Отрицательная	–	Да

«Зависит» в таблице означает, что матрица может быть обратимой или сингулярной в зависимости от чисел матрицы, а не от категории определенности.

$\mathbf{A}^T\mathbf{A}$ является положительной (полу)определенной

Любая матрица, которая может быть выражена как произведение матрицы и ее транспонированной версии (то есть $\mathbf{S} = \mathbf{A}^T\mathbf{A}$), гарантированно будет положительно определенной либо положительно полуопределенной. Комбинация этих двух категорий часто записывается как «положительно (полу) определенная».

Все матрицы ковариаций данных являются положительно (полу)определенными, поскольку они определяются как произведение матрицы данных на ее транспонированую версию. Это означает, что все матрицы ковариаций имеют неотрицательные собственные числа. Все собственные числа будут положительными, когда матрица данных является полноранговой (имеет полный столбцовый ранг, если данные хранятся в виде наблюдений по при-

знакам), и будет иметься по меньшей мере одно нуль-значное собственное число, если матрица данных является рангово-пониженной.

Доказательство того, что **S** положительно (полу)определена, основано на выписывании его квадратичной формы и применении нескольких алгебраических манипуляций. (Обратите внимание, что для перехода от первого уравнения ко второму требуется просто переставить скобки; в линейной алгебре такое «доказательство с помощью скобок» является общепринятым.)

$$\begin{aligned}\mathbf{w}^T\mathbf{S}\mathbf{w} &= \mathbf{w}^T(\mathbf{A}^T\mathbf{A})\mathbf{w} \\ &= (\mathbf{w}^T\mathbf{A}^T)\mathbf{A}\mathbf{w} \\ &= (\mathbf{A}\mathbf{w})^T(\mathbf{A}\mathbf{w}) \\ &= ||\mathbf{A}\mathbf{w}||^2.\end{aligned}$$

Дело в том, что квадратичная форма $\mathbf{A}^T\mathbf{A}$ равна квадрату модуля матрицы, умноженной на вектор. Модули не могут быть отрицательными и могут быть равны нулю только тогда, когда вектор равен нулю. А если $\mathbf{A}\mathbf{w} = \mathbf{0}$ для нетривиального $\mathbf{w}$, то $\mathbf{A}$ является сингулярной.

Следует иметь в виду, что хотя все матрицы $\mathbf{A}^T\mathbf{A}$ симметричны, не все симметричные матрицы могут быть выражены как $\mathbf{A}^T\mathbf{A}$. Другими словами, матричная симметрия сама по себе не гарантирует положительной (полу) определенности, потому что не все симметричные матрицы могут быть выражены как произведение матрицы и ее транспонированной версии.

Кого вообще волнуют квадратичная форма и определенность?

Положительная определенность востребована в науке о данных, потому что некоторые линейно-алгебраические операции применимы только к таким «хорошо обеспеченным» матрицам, включая разложение Холецкого, используемое для создания коррелированных наборов данных в симуляциях Монте-Карло. Положительно определенные матрицы также важны для задач оптимизации (например, градиентного спуска), потому что можно гарантированно находить минимум. В своем бесконечном стремлении улучшить свое мастерство в науке о данных вы можете столкнуться с техническими документами, в которых используется аббревиатура SPD[1]: симметричная положительно определенная.

Обобщенное собственное разложение

Обратите внимание, что следующее ниже уравнение совпадает с фундаментальным уравнением собственного числа:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{I}\mathbf{v}.$$

[1] Англ. *Symmetric Positive Definite*. – *Прим. перев.*

Это очевидно, поскольку $\mathbf{Iv} = \mathbf{v}$. Обобщенное собственное разложение предусматривает замену единичной матрицы еще одной матрицей (не единичной либо матрицей нулей):

$$\mathbf{Av} = \lambda\mathbf{Bv}.$$

Обобщенное собственное разложение также называется *одновременной диагонализацией двух матриц*. Результирующая пара (λ, $\mathbf{v}$) не является собственным числом/вектором только матрицы **A** или только матрицы **B**. Вместо этого две матрицы имеют общие пары «собственное число/вектор».

В концептуальном плане обобщенное собственное разложение может трактоваться как «обычное» собственное разложение матрицы произведения:

$$\mathbf{C} = \mathbf{AB}^{-1};$$

$$\mathbf{Cv} = \lambda\mathbf{v}.$$

Это только концептуально; на практике же обобщенное собственное разложение не требует от матрицы **B** быть обратимой.

Здесь не тот случай, когда любые две матрицы могут быть диагонализированы одновременно. Но эта диагонализация возможна, если **B** положительно (полу)определена.

В библиотеке NumPy обобщенное собственное разложение не вычисляется, но в SciPy такая возможность есть. Если вы знаете, что две матрицы симметричны, то можете применить функцию `eigh`, которая является численно более стабильной:

```
# создать коррелированные матрицы
A = np.random.randn(4,4)
A = A@A.T
B = np.random.randn(4,4)
B = B@B.T + A/10

# обобщенное собственное разложение
from scipy.linalg import eigh
evals,evecs = eigh(A,B)
```

Помните о порядке входимых аргументов: второй аргумент является концептуально инвертированным.

В науке о данных обобщенное собственное разложение используется в анализе классификации. В частности, линейный дискриминантный анализ Фишера основан на обобщенном собственном разложении двух матриц ковариации данных. В главе 15 вы увидите соответствующий пример.

Несметное число тонкостей собственного разложения

О свойствах собственного разложения можно было бы сказать гораздо больше. Вот несколько примеров: сумма собственных чисел равна следу матрицы, а произведение собственных чисел равно определителю; не все квадратные матрицы можно диагонализовать; некоторые матрицы имеют повторяющиеся собственные числа, что влияет на их

собственные векторы; комплексные собственные числа действительно-значных матриц находятся внутри круга на комплексной плоскости. Математические познания в области собственных чисел имеют глубокие корни, но эта глава дает все базовые знания, необходимые для работы с собственным разложением в приложениях.

Резюме

Вот это была глава так глава! Напомню ее ключевые моменты.

- Собственное разложение идентифицирует M пар скаляр/вектор матрицы $M \times M$. Эти пары «собственное число / собственный вектор» отражают особые направления в матрице и имеют громадное число применений в науке о данных (обычно это анализ главных компонент), а также в геометрии, физике, вычислительной биологии и массе других технических дисциплин.
- Собственные числа отыскиваются путем принятия допущения о том, что матрица, сдвинутая на неизвестный скаляр λ, является сингулярной, посредством приравнивания ее определителя к нулю (именуемого характеристическим многочленом) и вычисления чисел λ.
- Собственные векторы отыскиваются путем отыскания базисного вектора для нуль-пространства λ-сдвинутой матрицы.
- Диагонализация матрицы означает представление матрицы в виде $\mathbf{V\Lambda V}^{-1}$, где $\mathbf{V}$ – это матрица с собственными векторами в столбцах, а $\mathbf{\Lambda}$ – диагональная матрица с собственными числами в диагональных элементах.
- Симметричные матрицы обладают рядом особых свойств в собственном разложении; наиболее важным для науки о данных является то, что все собственные векторы попарно ортогональны. Это означает, что матрица собственных векторов является ортогональной матрицей (когда собственные векторы единично-нормализованы), что, в свою очередь, означает, что обратная матрица матрицы собственных векторов является ее транспонированной версией.
- *Определенность* матрицы относится к знакам ее собственных чисел. В науке о данных наиболее релевантными категориями являются положительно (полу)определенные, что означает, что все собственные числа либо неотрицательны, либо положительны.
- Матрица, умноженная на ее транспонированную версию, всегда положительно (полу)определена, что означает, что все матрицы ковариаций имеют неотрицательные собственные числа.
- Учение о собственном разложении носит насыщенный и подробный характер, и в этой области исследований было открыто много захватывающих тонкостей, частных случаев и применений. Надеюсь, что обзор этой темы в данной главе обеспечит прочную основу для ваших потребностей как исследователя данных и, возможно, вдохновит вас узнать еще больше о фантастической красоте собственного разложения.

Упражнения по программированию

Упражнение 13.1

Интересно, что собственные векторы $\mathbf{A}^{-1}$ сопадают с собственными векторами $\mathbf{A}$, а собственные числа равны λ^{-1}. Докажите, что это так, выписав собственное разложение $\mathbf{A}$ и $\mathbf{A}^{-1}$. Затем проиллюстрируйте это, используя случайную полноранговую симметричную матрицу 5×5.

Упражнение 13.2

Воссоздайте левую панель рис. 13.1, но вместо столбцов используйте *строки* матрицы $\mathbf{V}$. Конечно же, вы знаете, что это ошибка программирования, но результаты будут поучительными: тест на геометрию не будет пройден, поскольку матрица, умноженная на ее собственный вектор, только растягивается.

Упражнение 13.3

Цель этого упражнения – продемонстрировать, что собственные числа неразрывно сопряжены со своими собственными векторами. Диагонализируйте симметричную матрицу случайных целых чисел[1], созданную с помощью аддитивного метода (см. упражнение 5.9), но переупорядочите собственные числа в случайном порядке (назовем эту матрицу $\tilde{\mathbf{\Lambda}}$) без переупорядочивания собственных векторов.

Сначала продемонстрируйте, что вы можете реконструировать изначальную матрицу как $\mathbf{V\Lambda V}^{-1}$. Точность реконструкции можно вычислить как расстояние Фробениуса между изначальной и реконструированной матрицами. Далее, попытайтесь реконструировать матрицу, используя $\tilde{\mathbf{\Lambda}}$. Насколько близка реконструированная матрица к изначальной? Что произойдет, если вместо случайного переупорядочивания взаимно поменять местами только два самых больших собственных числа? Как насчет двух наименьших собственных чисел?

Наконец, создайте гистограмму, показывающую расстояния Фробениуса до изначальной матрицы для разных вариантов взаимообмена (рис. 13.3). (Разумеется, из-за случайных матриц – и, следовательно, случайных собственных чисел – ваш график не будет выглядеть точно так же, как мой.)

Упражнение 13.4

Одно интересное свойство случайных матриц состоит в том, что их комплексно-значные собственные числа распределены по кругу с радиусом, пропорциональным размеру матрицы. В целях демонстрации этого свойства вычислите 123 случайные матрицы 42×42, извлеките их собственные числа, разделите на квадратный корень из размера матрицы (42) и нанесите собственные числа на комплексную плоскость, как показано на рис. 13.4.

---

[1] В упражнениях я очень часто использую симметричные матрицы; это связано с тем, что они имеют действительно-значные собственные числа, но это не меняет ни принципа, ни математики, а просто облегчает визуальный осмотр решений.

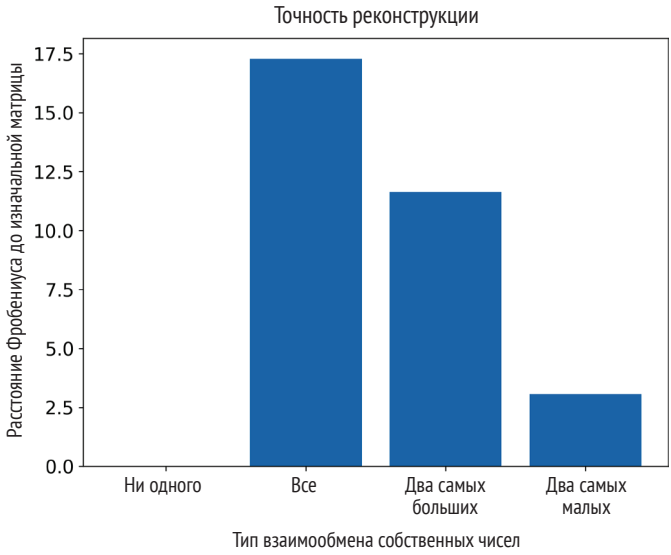


Рис. 13.3 ❖ Результаты упражнения 13.3

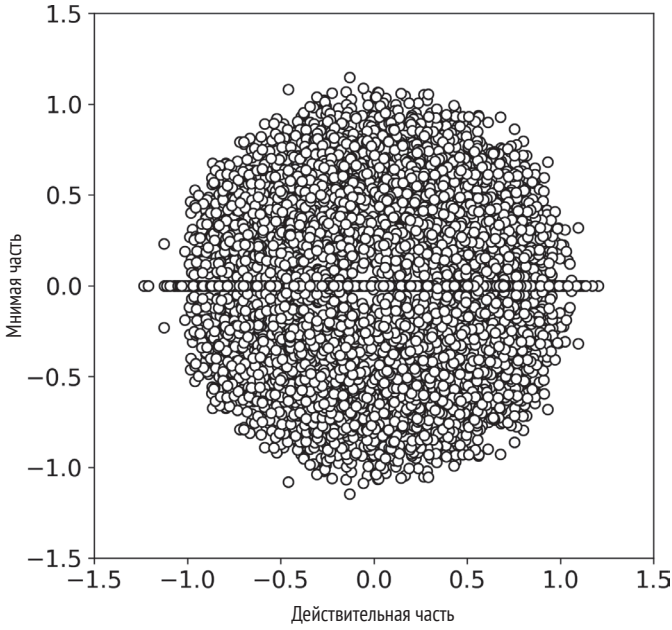


Рис. 13.4 ❖ Результаты упражнения 13.4

Упражнение 13.5

Это упражнение поможет вам получше разобраться в том, что собственный вектор является базисом нуль-пространства сдвинутой собственным числом матрицы, а также выявит риски числовых ошибок прецизионности. Примените процедуру собственного разложения к случайной симметричной мат-

рице 3×3. Затем по каждому собственному числу примените функцию `scipy.linalg.null_space()`, чтобы найти базисный вектор нуль-пространства каждой сдвинутой матрицы. Совпадают ли эти векторы с собственными векторами? Обратите внимание, что вам, возможно, понадобится принять во внимание нормы и знаковые неопределенности собственных векторов.

При выполнении исходного кода несколько раз на разных случайных матрицах вы, скорее всего, будете получать ошибки Python. Ошибка возникает из-за пустого нуль-пространства λ-сдвинутой матрицы. Указанная ошибка при обследовании возникает из-за того, что сдвинутая матрица имеет полный ранг. (Не верьте мне на слово; подтвердите это сами!) Этого не должно происходить, что еще раз подчеркивает, что

1) конечно-прецизионная математика на компьютерах не всегда соответствует математике на «классной доске» и
2) вам следует использовать конкретно ориентированные и более численно стабильные функции, вместо того чтобы пытаться делать прямое переложение формул в исходный код.

Упражнение 13.6

В данном упражнении я собираюсь научить вас третьему методу создания случайных симметричных матриц[1]. Начните с создания диагональной матрицы 4×4, по диагонали которой расположены положительные числа (например, это могут быть числа 1, 2, 3, 4). Затем создайте 4×4-матрицу **Q** из QR-разложения матрицы случайных чисел. Используйте эти матрицы в качестве собственных чисел и собственных векторов и умножьте их соответствующим образом, чтобы собрать матрицу. Убедитесь, что собранная матрица симметрична и что ее собственные числа равны собственным числам, которые указали вы.

Упражнение 13.7

Вернемся к упражнению 12.4. Повторите это упражнение, но вместо квадрата фробениусовой нормы расчетной матрицы используйте среднее значение собственных чисел (это называется усадочной регуляризацией). Как полученное число соотносится с числом из главы 12?

Упражнение 13.8

Это и следующее упражнение тесно связаны между собой. Мы создадим суррогатные данные с заданной матрицей корреляций (в этом упражнении), а затем удалим корреляцию (в следующем упражнении). Формула для создания данных с заданной структурой корреляции такова:

$$\mathbf{Y} = \mathbf{V}\mathbf{\Lambda}^{1/2}\mathbf{X},$$

где $\mathbf{V}$ и $\mathbf{\Lambda}$ – это собственные векторы и собственные числа матрицы корреляций, а $\mathbf{X}$ – $N \times T$-матрица некоррелированных случайных чисел (N каналов и T временных точек).

[1] Первые два были мультипликативным и аддитивным методами.

Примените эту формулу, чтобы создать 3×10 000-матрицу **Y** данных со следующей ниже структурой корреляции:

$$\mathbf{R} = \begin{bmatrix} 1 & .2 & .9 \\ .2 & 1 & .3 \\ .9 & .3 & 1 \end{bmatrix}.$$

Затем вычислите эмпирическую корреляционную матрицу матрицы **X**. Она не будет точно равна **R**, потому что мы отбираем образцы в случайном порядке из конечного набора данных. Но она должна быть достаточно близкой (например, в пределах 0.01).

Упражнение 13.9

Теперь давайте удалим эти навязанные корреляции с помощью *отбеливания*. Термин *отбеливание* используется в обработке сигналов и изображений и обозначает устранение корреляций. Многопеременный временной ряд можно отбелить, реализовав следующую ниже формулу:

$$\tilde{\mathbf{Y}} = \mathbf{Y}^{\mathrm{T}}\mathbf{V}\mathbf{\Lambda}^{-1/2}.$$

Примените эту формулу к матрице данных из предыдущего упражнения и убедитесь, что матрица корреляций является единичной матрицей (опять же, с некоторым допуском на случайный отбор данных).

Упражнение 13.10

В обобщенном собственном разложении собственные векторы не ортогональны, даже когда обе матрицы симметричны. Подтвердите на Python, что $\mathbf{V}^{-1} \neq \mathbf{V}^{\mathrm{T}}$. Это происходит потому, что хотя и **A**, и **B** симметричны, **C** = **AB** не симметрична[1].

Однако собственные векторы ортогональны относительно **B**, и, стало быть, $\mathbf{V}^{\mathrm{T}}\mathbf{B}\mathbf{V} = \mathbf{I}$. Подтвердите эти свойства, выполнив обобщенное собственное разложение двух симметричных матриц и получив рис. 13.5.

Упражнение 13.11

Давайте обследуем шкалирование собственных векторов. Начните с создания 4×4-матрицы случайных целых чисел, извлеченных в интервале между −14 и +14. Диагонализируйте матрицу и эмпирически подтвердите, что $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$. Подтвердите, что евклидова норма каждого собственного вектора равна 1. Обратите внимание, что квадрат комплексного числа вычисляется как это число, умноженное на его комплексно-сопряженное число (подсказка: используйте функцию `np.conj()`).

Затем умножьте матрицу собственных векторов на любой ненулевой скаляр. Я использовал π без особой веской причины, кроме того что его было интересно набирать на клавиатуре. Влияет ли этот скаляр на точность ре-

[1] Причина, по которой произведение двух симметричных матриц не является симметричным, совпадает с причиной, по которой матрица **R** из QR-разложения имеет нули на нижней диагонали.

конструированной матрицы и/или норм собственных векторов? Почему да или почему нет?

Наконец, повторите эту процедуру, но используя симметричную матрицу, и замените $\mathbf{V}^{-1}$на $\mathbf{V}^{\mathrm{T}}$. Изменят ли эти изменения умозаключение?

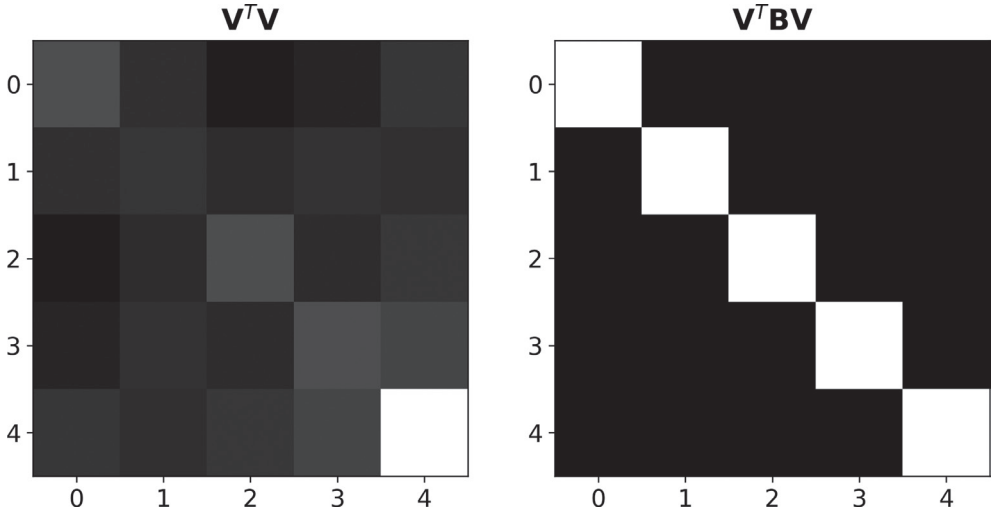


Рис. 13.5 ❖ Результаты упражнения 13.10

Глава 14

Сингулярное разложение

Предыдущая глава была по-настоящему насыщенной! Я изо всех сил старался сделать ее понятной и строгой, не слишком увязая в деталях, которые имеют меньшую релевантность для науки о данных.

К счастью, бóльшая часть того, что вы узнали о собственном разложении, применима и к сингулярному разложению (SVD)[1]. Это означает, что данная глава будет проще и короче.

Сингулярное разложение предназначено для разложения матрицы на произведение трех матриц, именуемых левыми сингулярными векторами ($\mathbf{U}$), сингулярными числами ($\mathbf{\Sigma}$) и правыми сингулярными векторами ($\mathbf{V}$):

$$\mathbf{A} = \mathbf{U\Sigma V}^{\mathrm{T}}.$$

Это разложение, должно быть, выглядит похоже на собственное разложение. На самом деле сингулярное разложение можно трактовать как обобщение собственного разложения на неквадратные матрицы – либо трактовать собственное разложение как частный случай собственного разложения для квадратных матриц[2].

Сингулярные числа сравнимы с собственными числами, а матрицы сингулярных векторов сравнимы с собственными векторами (эти два набора величин совпадают при некоторых обстоятельствах, которые я объясню позже).

Общая картина сингулярного разложения

Сначала я познакомлю вас с идеей и интерпретацией матриц, а позже в этой главе объясню, как вычислять сингулярное разложение.

---

[1] Англ. *Singular Value Decomposition*. – *Прим. перев.*

[2] Сингулярное разложение – это не то же самое, что собственное разложение для всех квадратных матриц; подробнее об этом позже.

На рис. 14.1 показан общий вид сингулярного разложения.

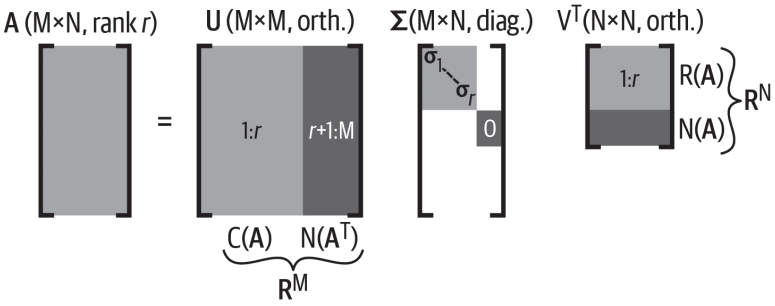


Рис. 14.1 ❖ Общая картина сингулярного разложения

На этой диаграмме видны многие важные признаки сингулярного разложения; эти признаки будут рассмотрены подробнее в данной главе, но если их сгруппировать, то получится вот такой список:

- и **U**, и **V** – это квадратные матрицы, даже если **A** не является квадратной;
- матрицы сингулярных векторов **U** и **V** ортогональны, а значит, $\mathbf{U}^T\mathbf{U} = \mathbf{I}$ и $\mathbf{V}^T\mathbf{V} = \mathbf{I}$. Напомним, что это означает, что каждый столбец ортогонален другому столбцу и любое подмножество столбцов ортогонально любому другому (непересекающемуся) подмножеству столбцов;
- первые r столбцов матрицы **U** обеспечивают ортогональные базисные векторы для столбцового пространства матрицы **A**, тогда как остальные столбцы предоставляют ортогональные базисные векторы для левого нуль-пространства (если только r не равно M, в коем случае матрица имеет полный столбцовый ранг, а левое нуль-пространство является пустым);
- первые r строк матрицы $\mathbf{V}^T$ (которые являются столбцами матрицы **V**) предоставляют ортогональные базисные векторы для строчного пространства, тогда как остальные строки предоставляют ортогональные базисные векторы для нуль-пространства;
- матрица сингулярных чисел – это диагональная матрица того же размера, что и **A**. Сингулярные числа всегда сортируются от наибольшего (вверху слева) к наименьшему (внизу справа);
- все сингулярные числа неотрицательны и действительно-значные. Они не могут быть ни комплексными, ни отрицательными, даже если матрица содержит комплексно-значные числа;
- количество ненулевых сингулярных чисел равно рангу матрицы.

Возможно, самое удивительное в сингулярном разложении то, что оно показывает все четыре подпространства матрицы: столбцовое пространство и левое пустое пространство охватываются первыми r и последними от $M - r$ до M столбцами матрицы **U**, тогда как строчное пространство и пустое пространство охватываются первыми r и последними $N - r$ до N строками матрицы $\mathbf{V}^T$. В случае прямоугольной матрицы если $r = M$, то левое

нуль-пространство является пустым, а если $r = N$, то нуль-пространство является пустым.

Сингулярные числа и ранг матрицы

Ранг матрицы определяется как количество ненулевых сингулярных чисел. Причина пристекает из предыдущего изложения, что столцовое пространство и строчное пространство матрицы определяются как левый и правый сингулярные векторы, которые шкалируются соответствующими им сингулярными числами, чтобы иметь некий «объем» в матричном пространстве, тогда как левое и правое нуль-пространства определяются как левый и правый сингулярные векторы, шкалированные в нули. Таким образом, размерность столбцового и строчного пространств определяется количеством ненулевых сингулярных чисел.

Собственно говоря, можно заглянуть в функцию NumPy `np.linalg.matrix_rank`, чтобы увидеть, как Python вычисляет ранг матрицы (я немного отредактировал исходный код, чтобы сосредоточиться на ключевых понятиях):

```
S = svd(M, compute_uv=False) # вернуть только сингулярные числа
tol = S.max() * max(M.shape[-2:]) * finfo(S.dtype).eps
return count_nonzero(S > tol)
```

Возвращаемое значение – это количество сингулярных чисел, которые превышают пороговое значение `tol`. Что такое `tol`? Это уровень допуска, учитывающий возможные ошибки округления. Он определяется как машинная прецизионность для этого типа данных (`eps`), шкалированная по наибольшему сингулярному числу и размеру матрицы.

Таким образом, мы еще раз видим разницу между «математикой на классной доске» и прецизионной математикой, реализованной на компьютерах: ранг матрицы на самом деле вычисляется не как количество ненулевых сингулярных чисел, а как количество сингулярных чисел, которые больше некоторого малого числа. Существует риск того, что малые, но по-настоящему ненулевые сингулярные числа будут проигнорированы, но это перевешивает риск неправильного завышения ранга матрицы, когда по-настоящему нуль-значные сингулярные числа оказываются ненулевыми из-за ошибок прецизионности.

Сингулярное разложение на Python

На языке Python сингулярное разложение вычисляется довольно просто:

```
U, s, Vt = np.linalg.svd(A)
```

Следует помнить о двух особенностях функции `svd` в библиотеке NumPy. Во-первых, сингулярные числа возвращаются в виде вектора, а не в виде мат-

рицы того же размера, что и **A**. Это означает, что вам нужен дополнительный исходный код получения матрицы **Σ**:

```
S = np.zeros(np.shape(A))
np.fill_diagonal(S, s)
```

Сначала вы, возможно, подумаете об использовании `np.diag(s)`, но она создает правильную матрицу сингулярных чисел только для квадратной матрицы **A**. Поэтому я сначала создаю матрицу нулей правильного размера, а затем заполняю диагональ сингулярными числами.

Вторая особенность заключается в том, что NumPy возвращает матрицу $\mathbf{V}^T$, а не **V**. Возможно, это будет дезориентировать читателей, знакомых с MATLAB, потому что функция MATLAB `svd` возвращает матрицу **V**. Подсказка находится в документационном литерале, описывающем матрицу `vh`, где буква `h` обозначает эрмитов, то есть имя симметричной комплексно-значной матрицы.

На рис. 14.2 показаны результаты на выходе из функции `svd` (в которых сингулярные числа конвертированы в матрицу).

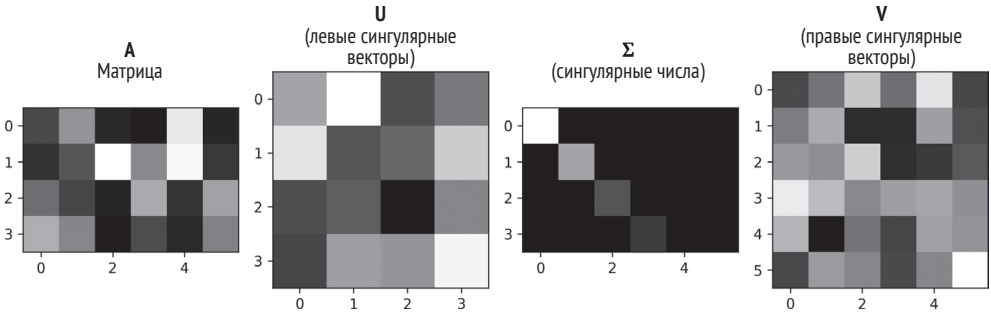


Рис. 14.2 ❖ Общая картина сингулярного разложения образцовой матрицы

Сингулярное разложение и одноранговые «слои» матрицы

Первое уравнение, которое я показал в предыдущей главе, было векторно-скалярной версией уравнения собственного числа ($\mathbf{Av} = \lambda\mathbf{v}$). Я открыл эту главу с матричного уравнения сингулярного разложения ($\mathbf{A} = \mathbf{U\Sigma V}^T$); как это уравнение выглядит для одного вектора? Его можно написать двумя разными способами, которые подчеркивают разные признаки сингулярного разложения:

$$\mathbf{Av} = \mathbf{u}\sigma;$$

$$\mathbf{u}^T\mathbf{A} = \sigma\mathbf{v}^T.$$

Эти уравнения чем-то похожи на уравнение собственного числа, за исключением того, что вместо одного вектора используются два. И следовательно, интерпретации немного более тонкие: в общем случае эти уравнения говорят о том, что влияние матрицы на один вектор такое же, как влияние скаляра на другой вектор.

Обратите внимание, что первое уравнение означает, что **u** находится в столбцовом пространстве матрицы **A**, а **v** обеспечивает веса для комбинирования столбцов. То же самое относится и ко второму уравнению, но **v** находится в строчном пространстве матрицы **A**, а **u** обеспечивает веса.

Но это не то, на чем я хотел бы сосредоточиться в данном разделе; я хочу рассмотреть, что происходит при умножении одного левого сингулярного вектора на один правый сингулярный вектор. Поскольку сингулярные векторы парно соединены с одним и тем же сингулярным числом, нам нужно умножить *i*-й левый сингулярный вектор на *i*-е сингулярное число на *i*-й правый сингулярный вектор.

Обратите внимание на ориентацию в этом умножении вектора на вектор: столбец слева, строка справа (рис. 14.3). Это означает, что результатом будет внешнее произведение того же размера, что и изначальная матрица. Кроме того, это внешнее произведение представляет собой одноранговую матрицу, норма которой определяется сингулярным числом (поскольку сингулярные векторы имеют единичную длину):

$$\mathbf{u}_1\sigma_1\mathbf{v}_1^{\mathrm{T}} = \mathbf{A}_1.$$

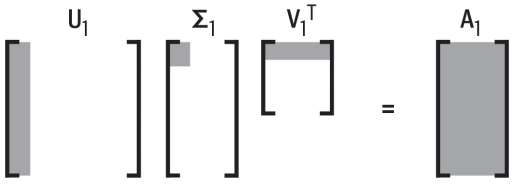


Рис. 14.3 ❖ Внешнее произведение сингулярных векторов создает матричный «слой»

Подстрочная 1 в уравнении указывает на использование первых сингулярных векторов и первого (наибольшего) сингулярного числа. Я обозначил результат как $\mathbf{A}_1$, потому что это не изначальная матрица **A**, а одноранговая матрица того же размера, что и **A**. И не просто любая одноранговая матрица – это самый важный «слой» матрицы. Он является самым важным, потому что имеет наибольшее сингулярное число (подробнее об этом – в следующем разделе).

С учетом этого изначальная матрица реконструируется путем суммирования всех «слоев» сингулярного разложения, ассоциированных с $\sigma > 0$[1]:

$$\mathbf{A} = \sum_{i=1}^{r} \mathbf{u}_i\sigma_i\mathbf{v}_i^{\mathrm{T}}.$$

[1] Суммировать нуль-значные сингулярные числа смысла нет, потому что это просто сложение матриц нулей.

Суть этого суммирования в том, что не обязательно использовать все r слоев; вместо этого можно реконструировать какую-то другую матрицу, назовем ее $\tilde{\mathbf{A}}$, содержащую первые $k < r$ слоев. Это называется *низкоранговой аппроксимацией* матрицы $\mathbf{A}$ – в данном случае -ранговой аппроксимацией.

Низкоранговая аппроксимация применяется, например, в очистке данных. Идея состоит в том, что информация, ассоциированная с малыми сингулярными числами, вносит малый вклад в суммарную дисперсию набора данных и, следовательно, может отражать шум, который можно удалить. Подробнее об этом – в следующей главе.

Сингулярное разложение из собственного разложения

Итак, к этому моменту главы вы уже знакомы с основами понимания и интерпретации матриц сингулярного разложения. Уверен, вам интересно, что это за волшебная формула, которая производит сингулярное разложение. Может быть, она настолько невероятно многосложна, что его мог понять только Гаусс? А может, ее придется так долго объяснять, что в одну главу не влезет?

Отнюдь нет!

На самом деле сингулярное разложение – по-настоящему элементарное (концептуально; другое дело – расчитывать сингулярное разложение вручную). Оно вычисляется из матрицы собственного разложения, умноженной всего лишь на ее транспонированную версию. Следующие ниже уравнения показывают, как получать сингулярные числа и левые сингулярные векторы:

$$\begin{aligned}\mathbf{A}\mathbf{A}^{\mathrm{T}} &= (\mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathrm{T}})(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathrm{T}})^{\mathrm{T}} \\ &= \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathrm{T}}\mathbf{V}\mathbf{\Sigma}^{\mathrm{T}}\mathbf{V}^{\mathrm{T}} \\ &= \mathbf{U}\mathbf{\Sigma}^{2}\mathbf{U}^{\mathrm{T}}.\end{aligned}$$

Другими словами, собственные векторы матрицы $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ – это левосингулярные векторы матрицы $\mathbf{A}$, а квадраты собственных чисел матрицы $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ – это сингулярные числа матрицы $\mathbf{A}$.

Этот ключевой для понимания момент раскрывает три признака сингулярного разложения:

1) сингулярные числа неотрицательны, потому что числа в квадрате не могут быть отрицательными;
2) сингулярные числа – действительно-значные, потому что симметричные матрицы имеют действительно-значные собственные числа;
3) сингулярные векторы ортогональны, потому что собственные векторы симметричной матрицы ортогональны.

Правые сингулярные числа получаются в результате предпозиционного умножения транспонированной матрицы:

$$\begin{aligned}\mathbf{A}^T\mathbf{A} &= (\mathbf{U\Sigma V}^T)^T(\mathbf{U\Sigma V}^T)\\ &= \mathbf{V\Sigma}^T\mathbf{U}^T\mathbf{U\Sigma V}^T\\ &= \mathbf{V\Sigma}^2\mathbf{V}^T.\end{aligned}$$

На самом деле уравнение сингулярного разложения можно реорганизовать под решение правосингулярных векторов, не вычисляя собственного разложения $\mathbf{A}^T\mathbf{A}$:

$$\mathbf{V}^T = \mathbf{\Sigma}^{-1}\mathbf{U}^T\mathbf{A}.$$

Разумеется, есть комплементарное уравнение получения **U**, если уже известна **V**.

Сингулярное разложение матрицы $\mathbf{A}^T\mathbf{A}$

Вкратце: если матрица может быть выражена как $\mathbf{S} = \mathbf{A}^T\mathbf{A}$, то ее левые и правые сингулярные векторы равны. Другими словами:

$$\mathbf{S} = \mathbf{U\Sigma V}^T = \mathbf{V\Sigma U}^T = \mathbf{U\Sigma U}^T = \mathbf{V\Sigma V}^T.$$

Доказательство этого утверждения происходит из расписывания сингулярного разложения **S** и $\mathbf{S}^T$, а затем рассмотрения последствия $\mathbf{S} = \mathbf{S}^T$. Я оставляю его вам на самостоятельное обследование! И также призываю вас подтвердить его на Python, используя случайные симметричные матрицы.

По сути, для симметричной матрицы сингулярное разложение – это то же самое, что собственное разложение. Этот факт имеет последствия для анализа главных компонент (PCA), поскольку PCA может выполняться с использованием собственного разложения матрицы ковариаций в данных, сингулярного разложения матрицы ковариаций либо сингулярного разложения матрицы данных.

Конвертация сингулярных чисел в дисперсию: объяснение

Сумма сингулярных чисел представляет собой общую величину «дисперсии» в матрице. Что это значит? Если представить, что информация в матрице содержится в пузыре, то сумма сингулярных чисел подобна объему этого пузыря.

Причина, по которой вся дисперсия содержится в сингулярных числах, заключается в том, что сингулярные векторы нормированы в единичный модуль, а это означает, что они не предоставляют информации о модуле (то есть $||\mathbf{Uw}|| = ||\mathbf{w}||$)[1]. Другими словами, сингулярные векторы указывают, а сингулярные числа говорят, насколько далеко.

[1] Доказательство этого утверждения находится в упражнении 14.3.

«Сырые» сингулярные числа находятся в числовой шкале матрицы. Это означает, что если умножить данные на скаляр, то сингулярные числа увеличатся. А это, в свою очередь, означает, что сингулярные числа трудно интерпретировать и практически невозможно сравнивать между разными наборами данных.

По этой причине нередко удобно конвертировать сингулярные числа в процент общей объясненной дисперсии. Формула проста; каждое сингулярное число i нормализуется следующим образом:

$$\tilde{\sigma}_i = \frac{100\sigma_i}{\sum \sigma}.$$

Эта нормализация распространена в анализе главных компонент, например чтобы определять число компонент, на которые приходится 99 % дисперсии. Данное число можно интерпретировать как показатель сложности системы.

Важно отметить, что указанная нормализация не влияет на относительные расстояния между сингулярными числами; она просто меняет числовую шкалу на более удобочитаемую.

Кондиционное число

В этой книге я несколько раз намекал, что кондиционное число матрицы используется для обозначения численной стабильности матрицы. Теперь, когда вы знаете о сингулярных числах, вы способны лучше понять, как вычислять и интерпретировать кондиционное число.

Кондиционное число матрицы определяется как отношение наибольшего сингулярного числа к наименьшему. Оно часто обозначается буквой к (греческой буквой каппа):

$$\kappa = \frac{\sigma_{\max}}{\sigma_{\min}}.$$

Кондиционное число очень часто используется в статистике и машинном обучении с целью оценивания стабильности матрицы при вычислении ее обратной матрицы и при использовании для решения систем уравнений (например, наименьших квадратов). Разумеется, необратимая матрица имеет кондиционное число NaN, потому что $\sigma/0 = ⁉$.

Но полноранговая матрица с большим кондиционным числом все же может быть численно нестабильной. Обратная матрица, хотя и являясь теоретически обратимой, на практике бывает ненадежной. Такие матрицы называются *плохо обусловленными*. Вы, возможно, встречали этот термин в предупреждающих сообщениях Python, иногда сопровождаемый такими фразами, как «точность результата не гарантируется».

В чем проблема с плохо обусловленной матрицей? По мере увеличения кондиционного числа матрица тяготеет к сингулярности. Следовательно, плохо обусловленная матрица является «почти сингулярной», а обратная

ей матрица становится недостоверной из-за повышенного риска числовых ошибок.

Влияние плохо обусловленной матрицы можно трактовать несколькими способами. Одним из них является снижение прецизионности решения из-за ошибок округления. Например, кондиционное число порядка 10^5 означает, что решение (например, обратная матрица или задача наименьших квадратов) теряет пять значащих цифр (это означало бы, например, переход от прецизионности 10^{-16} к 10^{-11}).

Вторая связанная с предыдущей интерпретация – это коэффициент усиления шума. Если у вас есть матрица с кондиционным числом порядка 10^4, то шум может повлиять на решение задачи наименьших квадратов на 10^4. Это может показаться много, но оно будет незначительным усилением, если ваши данные имеют прецизионность 10^{-16}.

В-третьих, кондиционное число указывает на чувствительность решения к возмущениям в матрице данных. Хорошо обусловленная матрица может пертурбироваться (добавлением большего шума) минимальным изменением решения. Напротив, добавление малого шума к плохо обусловленной матрице может приводить к совершенно другим решениям.

Каков порог плохой обусловленности матрицы? Никакого. Не существует магического числа, которое отличает хорошо обусловленную матрицу от плохо обусловленной. Различные алгоритмы будут применять разные пороги, которые зависят от числовых значений в матрице.

Ясно одно: следует серьезно относиться к предупреждающим сообщениям о плохо обусловленных матрицах. Обычно они указывают на то, что что-то не так, и результатам не следует доверять.

Что делать с плохо обусловленными матрицами?

К сожалению, это не тот вопрос, на который я могу дать конкретный ответ. Правильный поступок при наличии плохо обусловленной матрицы во многом зависит от матрицы и от задачи, которую вы пытаетесь решить. Для ясности: плохо обусловленные матрицы по своей сути не являются плохими, и вы никогда не должны легкомысленно отбрасывать матрицу только из-за ее кондиционного числа. Плохо обусловленная матрица несет в себе потенциальные проблемы только для определенных операций и, следовательно, актуальна только для определенных матриц, таких как матрицы статистических моделей или матрицы ковариаций.

Варианты исправления плохо обусловленной матрицы включают регуляризацию, уменьшение размерности и улучшение качества данных или выделение признаков.

Сингулярное разложение и псевдообратная матрица Мура – Пенроуза

Сингулярное разложение обратной матрицы выглядит довольно элегантно. Исходя из допущения, что матрица – квадратная и обратимая, мы получаем:

$$\begin{aligned}\mathbf{A}^{-1} &= (\mathbf{U\Sigma V}^{\mathrm{T}})^{-1} \\ &= \mathbf{V\Sigma}^{-1}\mathbf{U}^{-1} \\ &= \mathbf{V\Sigma}^{-1}\mathbf{U}^{\mathrm{T}}.\end{aligned}$$

Другими словами, нам нужно только инвертировать $\mathbf{\Sigma}$, потому что $\mathbf{U}^{-1} = \mathbf{U}^{\mathrm{T}}$. Кроме того, поскольку $\mathbf{\Sigma}$ является диагональной матрицей, обратная ей матрица получается путем простого инвертирования каждого диагонального элемента. С другой стороны, этот метод по-прежнему подвержен численной нестабильности, потому что крошечные сингулярные числа, которые могут отражать ошибки прецизионности (например, 10^{-15}), становятся галактически большими при инвертировании.

Теперь что касается алгоритма вычисления псевдообратной матрицы Мура–Пенроуза. Вы ждали этого во многих главах, и я ценю ваше терпение.

Псевдообратная матрица Мура–Пенроуза вычисляется почти точно так же, как полная обратная матрица, показанная в предыдущем примере; единственное видоизменение состоит в инвертировании *ненулевых* диагональных элементов в $\mathbf{\Sigma}$ вместо попыток инвертировать все диагональные элементы. (На практике «ненулевое значение» реализуется как превышение порога, чтобы учитывать ошибки прецизионности.)

Вот и все! Вот так вычисляется псевдообратная матрица. Вы видите, что это очень просто и интуитивно понятно, но для понимания потребовался значительный объем базовых знаний о линейной алгебре.

А еще лучше, поскольку сингулярное разложение работает с матрицами любого размера, псевдообратная матрица Мура–Пенроуза может применяться к неквадратным матрицам. На самом деле псевдообратная матрица Мура–Пенроуза высокой матрицы равна ее левообратной матрице, а псевдообратная матрица широкой матрицы равна ее правообратной. (Напомню, что псевдообратная матрица обозначается как $\mathbf{A}^{\dagger}$, $\mathbf{A}^{+}$ или $\mathbf{A}^{*}$.)

Вы получите больше опыта работы с псевдообратной матрицей, реализуя ее самостоятельно в упражнениях.

Резюме

Надеюсь, вы согласитесь, что после того, как вы приложили усилия, чтобы узнать о собственном разложении, теперь потребовалось лишь немного дополнительных усилий, чтобы разобраться в сингулярном разложении. Сингулярное разложение, пожалуй, является самым важным разложением в линейной алгебре, потому что оно раскрывает насыщенную и подробную информацию о матрице. Вот ключевые моменты, которые следует вынеси из этой главы.

- Сингулярное разложение разлагает матрицу (любого размера и ранга) в произведение трех матриц, именуемых левыми сингулярными векторами $\mathbf{U}$, сингулярными числами $\mathbf{\Sigma}$ и правыми сингулярными векторами $\mathbf{V}^{\mathrm{T}}$.

- Первые r (где r – это ранг матрицы) левых сингулярных векторов обеспечивают ортонормальный базис столбцового пространства матрицы, тогда как последующие сингулярные векторы обеспечивают ортонормальный базис левого нуль-пространства.
- Аналогичная история с правыми сингулярными векторами: первые r векторов обеспечивают ортонормальный базис строчного пространства, а последующие векторы обеспечивают ортонормальный базис нуль-пространства. Следует учесть, что правильные сингулярные векторы на самом деле являются *строками* матрицы $\mathbf{V}$, то есть *столбцами* матрицы $\mathbf{V}^{\mathrm{T}}$.
- Количество ненулевых сингулярных чисел равно рангу матрицы. На практике бывает трудно отличить очень малые ненулевые сингулярные числа от ошибок прецизионности нуль-значных сингулярных чисел. Такие программы, как Python, будут использовать порог допуска, чтобы проводить это различие.
- Внешнее произведение k-го левого сингулярного вектора и k-го правого сингулярного вектора, скаляр, умноженный на k-е сингулярное число, дает одноранговую матрицу, которую можно интерпретировать как «слой» матрицы. Реконструкция матрицы на основе слоев имеет множество применений, включая подавление шума и сжатие данных.
- В концептуальном плане сингулярное разложение можно получить из собственного разложения $\mathbf{A}\mathbf{A}^{\mathrm{T}}$.
- Суперважная псевдообратная матрица Мура–Пенроуза вычисляется как $\mathbf{V}\mathbf{\Sigma}^{+}\mathbf{U}^{\mathrm{T}}$, где $\mathbf{\Sigma}^{+}$ получается путем инвертирования ненулевых сингулярных чисел на диагонали.

Упражнения по программированию

Упражнение 14.1

Вы узнали, что для симметричной матрицы сингулярные числа и собственные числа совпадают. Как насчет сингулярных векторов и собственных векторов? Для ответа на этот вопрос примените Python с использованием случайной 5×5-матрицы $\mathbf{A}^{\mathrm{T}}\mathbf{A}$. Далее попробуйте еще раз, используя аддитивный метод создания симметричной матрицы ($\mathbf{A}^{\mathrm{T}} + \mathbf{A}$). Обратите внимание на знаки собственных чисел матрицы $\mathbf{A}^{\mathrm{T}} + \mathbf{A}$.

Упражнение 14.2

Python может дополнительно возвращать «экономное» сингулярное разложение, то есть матрицы сингулярных векторов усекаются до меньшего из M или N. Обратитесь к документационному литералу, чтобы выяснить, как это сделать. Подтвердите с высокими и широкими матрицами. Обратите внимание, что обычно требуется возвращать полные матрицы; экономное сингулярное разложение в основном используется для по-настоящему больших матриц и/или реально ограниченной вычислительной мощности.

Упражнение 14.3

Одним из важных признаков ортогональных матриц (таких как матрицы левых и правых сингулярных векторов) является то, что они поворачивают, но не масштабируют вектор. Это означает, что после умножения на ортогональную матрицу величина вектора сохраняется. Докажите, что $||\mathbf{Uw}|| = ||\mathbf{w}||$. Затем продемонстрируйте это эмпирически на Python, используя сингулярную матрицу векторов из сингулярного разложения случайной матрицы и случайного вектора.

Упражнение 14.4

Создайте случайную высокую матрицу с указанным кондиционным числом. Сделайте это, создав две случайные квадратные матрицы **U** и **V** и прямоугольную **Σ**. Подтвердите, что это эмпирическое кондиционное число матрицы $\mathbf{U\Sigma V}^{\mathrm{T}}$ совпадает с указанным вами числом. Визуализируйте свои результаты в виде рисунка, подобного рис. 14.4. (Я использовал кондиционное число 42.4[1].)

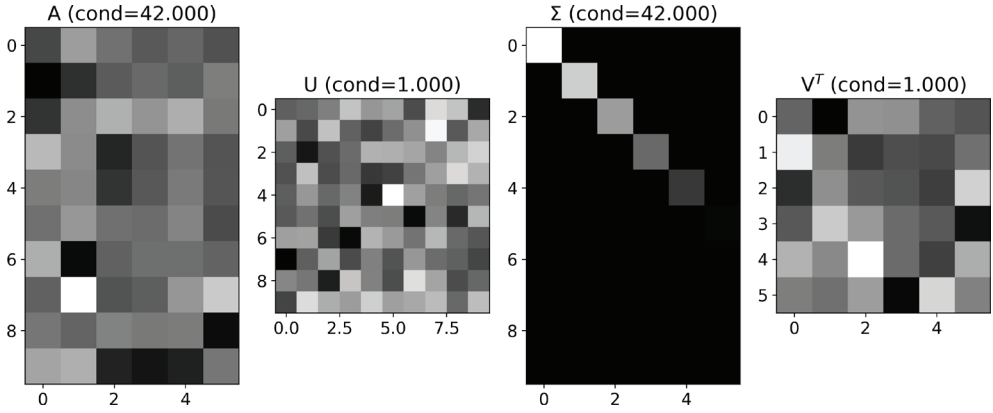


Рис. 14.4 ❖ Результаты упражнения 14.3

Упражнение 14.5

Здесь ваша цель проста: написать исходный код, воспроизводящий рис. 14.5. Что показывает этот рисунок? На панели A показана случайная матрица 30×40, которую я создал путем сглаживания случайных чисел (реализованного как 2D-свертка между 2D-гауссианом и случайным числом; если вы незнакомы с обработкой и фильтрацией изображений, то можете свободно скопировать исходный код и создать эту матрицу из моего исходного кода решения). На остальной части панели A представлены матрицы сингулярного разложения. Интересно отметить, что более ранние сингулярные векторы (ассоциированные с большими сингулярными числами) выглядят глаже, а более поздние – более неровными; это происходит из-за пространственной фильтрации.

[1] Да, и это еще одна отсылка к книге «Автостопом по Галактике».

На панели B продемонстрирован «график крутого склона», на котором показаны объясненные сингулярные числа, нормализованные к проценту объясненной дисперсии. Обратите внимание, что на первые несколько компонент приходится бóльшая часть дисперсии изображения, в то время как на более поздние компоненты приходится относительно малая дисперсия. Подтвердите, что сумма по всем нормализованным сингулярным числам равна 100. Панель C показывает первые четыре «слоя» – одноранговые матрицы, определенные как $\mathbf{u}_i\sigma_i\mathbf{v}_i^{\mathrm{T}}$, в верхней строке и совокупную сумму этих слоев в нижней строке. Вы видите, что каждый слой добавляет в матрицу больше информации; правое нижнее изображение (под названием «L 0:3») представляет собой одноранговую матрицу, но визуально очень похоже на изначальную 30-ранговую матрицу на панели A.

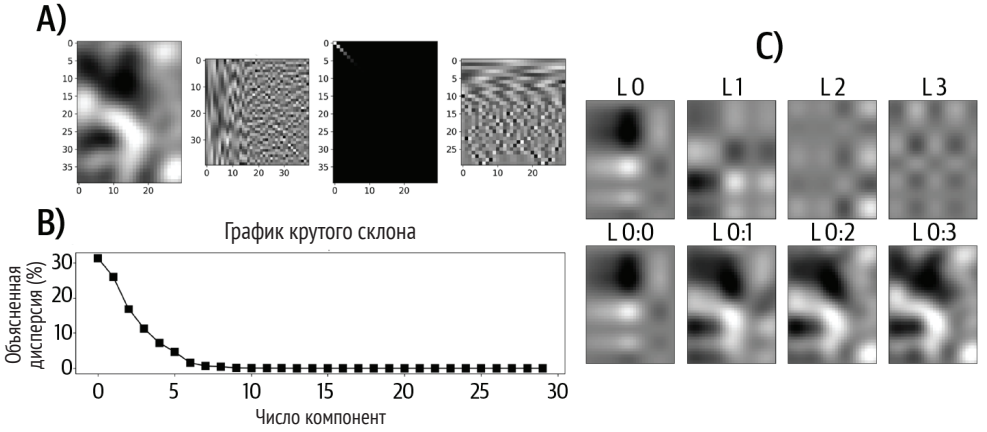


Рис. 14.5 ❖ Результаты упражнения 14.5

Упражнение 14.6

Реализуйте псевдообратную матрицу Мура–Пенроуза на основе описания в этой главе. Вам нужно определить допуск для игнорирования крошечных, но ненулевых сингулярных чисел. Пожалуйста, не ищите реализацию NumPy – и не возвращайтесь к более раннему исходному коду этой главы – вместо этого используйте свои знания линейной алгебры, чтобы придумать свой собственный уровень допуска.

Протестируйте свой исходный код на 3-ранговой матрице 5×5. Сравните свой результат с результатом на выходе из функции NumPy `pinv`. Наконец, проинспектируйте исходный код `np.linalg.pinv`, чтобы убедиться, что вы понимаете реализацию.

Упражнение 14.7

Продемонстрируйте, что псевдообратная матрица Мура–Пенроуза равна левообратной матрице для матрицы с полным столбцовым рангом путем вычисления явной левообратной матрицы высокой полной матрицы

$((\mathbf{A}^{\mathrm{T}}\mathbf{A})^{-1}\mathbf{A}^{\mathrm{T}})$ и псевдообратной матрицы матрицы **A**. Повторите эти действия для правообратной матрицы широкой матрицы с полным строчным рангом.

Упражнение 14.8

Рассмотрите уравнение собственного числа $\mathbf{Av} = \lambda\mathbf{v}$. Теперь, когда вы знаете о псевдообратной матрице, вы можете немного поэкспериментировать с этим уравнением. В частности, используйте матрицу 2×2, которая применялась в начале главы 13, чтобы вычислить $\mathbf{v}^{+}$ и подтвердить, что $\mathbf{vv}^{+} = 1$. Затем подтвердите следующие ниже тождества:

$\mathbf{v}^{+}\mathbf{Av} = \lambda\mathbf{v}^{+}\mathbf{v}$;

$\mathbf{Avv}^{+} = \lambda\mathbf{vv}^{+}$.

Глава 15

Применения собственного и сингулярного разложений

Собственное и сингулярное разложения – это жемчужины, которыми линейная алгебра наградила современную человеческую цивилизацию. Их важность в современной прикладной математике невозможно недооценить, и их применения бесчисленны и разбросаны по множеству дисциплин.

В этой главе я выделю три применения, с которыми вы, вероятно, столкнетесь в науке о данных и смежных областях. Моя главная цель – показать, что кажущиеся сложными методы науки о данных и машинного обучения на самом деле вполне разумны и легко понятны, как только вы изучите линейно-алгебраические темы этой книги.

Анализ главных компонент с использованием собственного и сингулярного разложений

Анализ главных компонент (PCA) предназначен для отыскания набора базисных векторов набора данных, которые указывают в направлении, максимизирующем ковариацию между переменными.

Представьте, что набор данных N-D существует в пространстве N-D, в котором каждая точка данных является координатой в этом пространстве. Это разумно, когда вы думаете о хранении данных в матрице с N наблюдениями (каждая строка является наблюдением) по M признаков (каждый столбец является признаком, также именуемым переменной, или результатом измерений); данные обитают в $\mathbb{R}^M$ и содержат N векторов или координат.

Пример в 2D показан на рис. 15.1. Левая панель показывает данные в их изначальном пространстве данных, в котором каждая переменная обеспечивает базисный вектор для данных. Ясно, что две переменные (оси *x* и *y*) связаны друг с другом, и очевидно, что есть направление в данных, которое отражает это отношение лучше, чем любой базисный вектор признаков.

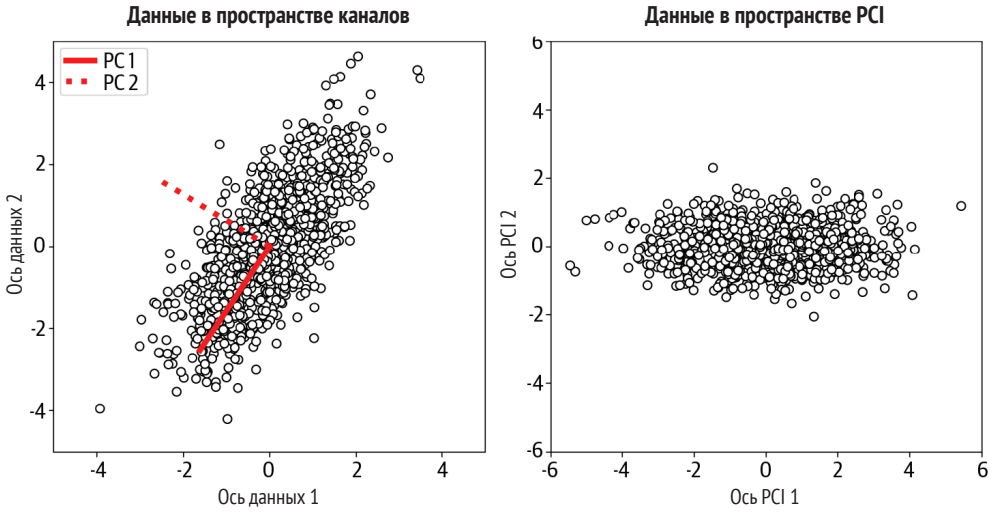


Рис. 15.1 ❖ Графический обзор PCA в 2D

Цель PCA – найти новый набор базисных векторов, такой чтобы линейные взаимосвязи между переменными были максимально выровнены с базисными векторами – это то, что показано на правой панели рис. 15.1. Важно отметить, что PCA имеет ограничение, заключающееся в том, что новые базисные векторы являются ортогональными поворотами изначальных базисных векторов. В упражнениях вы увидите последствия этого ограничения.

В следующем разделе я расскажу о математике и процедурах вычисления PCA; в упражнениях у вас будет возможность реализовать PCA, используя собственное и сингулярное разложения, и сравнить свои результаты с реализацией PCA на Python.

Математика анализа главных компонент

В анализе главных компонент (PCA) совмещена статистическая концепция дисперсии с линейно-алгебраической концепцией линейно-взвешенной комбинации. Дисперсия, как вы знаете, является мерой разброса набора данных вокруг его среднего значения. Метод PCA основан на том, что дисперсия – это хорошо, а направления в пространстве данных, которые имеют бóльшую дисперсию, более важны (иначе говоря, «дисперсия = релевантность»).

Но в PCA нас интересует не просто дисперсия *внутри* одной переменной; вместо этого мы хотим найти линейно-взвешенную комбинацию всех переменных, которая максимизирует дисперсию данной компоненты (*компонента* – это линейно-взвешенная комбинация переменных).

Запишем это математически. Матрица $\mathbf{X}$ – это наша матрица данных (высокая матрица с полным столбцовым рангом, имеющая формат «наблюдения по признакам»), а $\mathbf{w}$ – вектор весов. Наша цель в PCA – найти набор весов в $\mathbf{w}$ такой, что $\mathbf{Xw}$ имеет максимальную дисперсию. Дисперсия – это скаляр, поэтому мы можем записать ее как

$$\lambda = ||\mathbf{Xw}||^2.$$

Квадрат векторной нормы – это на самом деле то же самое, что и дисперсия, когда данные центрированы по среднему (т. е. каждая переменная данных имеет нулевое среднее значение)[1]; я опустил шкалирующий коэффициент $1/(N-1)$, потому что он не влияет на решение цели оптимизации.

Проблема с этим уравнением в том, что $\mathbf{w}$ можно задать ОГРОМНЫМИ числами; чем больше веса, тем больше дисперсия. Решение состоит в шкалировании нормы взвешенной комбинации переменных нормой весов:

$$\lambda = \frac{||\mathbf{Xw}||^2}{||\mathbf{w}||^2}.$$

Теперь у нас есть соотношение двух векторных норм. Эти нормы можно расширить до точечных произведений, чтобы получить ясное представление об уравнении:

$$\lambda = \frac{\mathbf{w}^T\mathbf{X}^T\mathbf{Xw}}{\mathbf{w}^T\mathbf{w}};$$

$$\mathbf{C} = \mathbf{X}^T\mathbf{X};$$

$$\lambda = \frac{\mathbf{w}^T\mathbf{Cw}}{\mathbf{w}^T\mathbf{w}}.$$

Теперь мы обнаружили, что решение PCA совпадает с решением для отыскания направленного вектора, который максимизирует *нормализованную* квадратичную форму (норма вектора – это член нормализации) матрицы ковариаций в данных.

Это все хорошо, но как на самом деле найти элементы вектора $\mathbf{w}$, которые максимизируют λ?

Здесь линейно-алгебраический подход заключается в том, чтобы рассматривать не просто одно векторное решение, а все множество решений. Таким образом, мы перепишем уравнение, используя вместо вектора $\mathbf{w}$ матрицу $\mathbf{W}$. В результате получим матрицу в знаменателе, что в линейной алгебре недопустимо; поэтому умножаем на взаимообратную:

$$\mathbf{\Lambda} = (\mathbf{W}^T\mathbf{W})^{-1}\mathbf{W}^T\mathbf{CW}.$$

[1] В онлайновом исходном коде это продемонстрировано.

Теперь применим немного алгебры и посмотрим, что произойдет:

$$\mathbf{\Lambda} = (\mathbf{W}^T\mathbf{W})^{-1}\mathbf{W}^T\mathbf{C}\mathbf{W};$$

$$\mathbf{\Lambda} = \mathbf{W}^{-1}\mathbf{W}^{-T}\mathbf{W}^T\mathbf{C}\mathbf{W};$$

$$\mathbf{\Lambda} = \mathbf{W}^{-1}\mathbf{C}\mathbf{W};$$

$$\mathbf{W}\mathbf{\Lambda} = \mathbf{C}\mathbf{W}.$$

Примечательно, что мы обнаружили, что решение PCA заключается в выполнении собственного разложения матрицы ковариаций данных. Собственные векторы – это веса переменных, а соответствующие им собственные числа – это дисперсии данных по каждому направлению (каждый столбец матрицы $\mathbf{W}$).

Поскольку матрицы ковариаций симметричны, их собственные векторы и, следовательно, главные компоненты ортогональны. Это имеет важные последствия для пригодности PCA для анализа данных, которые вы обнаружите в упражнениях.

Доказательство PCA

Здесь я изложу доказательство того, что собственное разложение решает задачу оптимизации PCA. Если вы незнакомы с дифференциальным исчислением и множителями Лагранжа, то можете свободно пропустить эту вкладку; я вставил ее сюда для полноты, а не потому, что вам нужно понять это доказательство, чтобы решать упражнения или применять PCA на практике.

Наша цель – максимизировать $\mathbf{w}^T\mathbf{C}\mathbf{w}$, при условии что $\mathbf{w}^T\mathbf{w} = 1$. Эту оптимизацию можно выразить с помощью множителя Лагранжа:

$$L(\mathbf{w}, \lambda) = \mathbf{w}^T\mathbf{C}\mathbf{w} - \lambda(\mathbf{w}^T\mathbf{w} - 1);$$

$$0 = \frac{d}{d\mathbf{w}}(\mathbf{w}^T\mathbf{C}\mathbf{w} - \lambda(\mathbf{w}^T\mathbf{w} - 1));$$

$$0 = \mathbf{C}\mathbf{w} - \lambda\mathbf{w};$$

$$\mathbf{C}\mathbf{w} = \lambda\mathbf{w}.$$

Вкратце: идея состоит в применении множителя Лагранжа, чтобы сбалансировать оптимизацию ограничением, взять производную по вектору весов, приравнять производную к нулю, продифференцировать по $\mathbf{w}$ и обнаружить, что $\mathbf{w}$ является собственным вектором матрицы ковариаций.

Шаги выполнения PCA

Оставив в стороне математику, ниже перечислены шаги реализации анализа главных компонент[1].

[1] В упражнении 15.3 вы также узнаете, как реализовывать PCA с помощью библиотеки Python scikit-learn.

1. Вычислить матрицу ковариаций в данных. Результирующая матрица коварий будет в формате «признаки по признакам». Перед вычислением ковариации каждый признак в данных должен быть центрирован по среднему значению.
2. Взять собственное разложение этой матрицы ковариаций.
3. Отсортировать собственные числа по убыванию модуля и соответствующим образом отсортировать собственные векторы. Собственные числа PCA иногда называются *показателями латентных факторов*[1].
4. Вычислить «показатели компонент» как взвешенную комбинацию всех признаков данных, где собственный вектор обеспечивает веса. Собственный вектор, ассоциированный с наибольшим собственным числом, является «наиболее важной» компонентой, то есть с наибольшей дисперсией.
5. Конвертировать собственные числа в процент объясненной дисперсии, чтобы облегчить интерпретацию.

PCA посредством сингулярного разложения

PCA можно выполнить эквивалентным образом посредством собственного разложения, как описано ранее, либо посредством сингулярного разложения. При этом существует два способа выполнения PCA с помощью сингулярного разложения:

- взять сингулярное разложение матрицы ковариаций. Процедура идентична ранее описанной, потому что для матриц ковариаций сингулярное и собственное разложения совпадают;
- взять сингулярное разложение матрицы данных напрямую. В данном случае правые сингулярные векторы (матрица $\mathbf{V}$) эквивалентны собственным векторам матрицы ковариаций (это будут левые сингулярные векторы, если матрица данных хранится как признаки по наблюдениям). Перед вычислением сингулярного разложения данные должны быть центрированы по среднему значению. Квадратный корень сингулярных чисел эквивалентен собственным числам матрицы ковариаций.

Какое разложение следует использовать для выполнения PCA? Возможно, вы подумали, что сингулярное разложение проще, потому что не требует матрицы ковариаций. Это верно для относительно малых и чистых наборов данных. Но большие или более сложные наборы данных потребуют отбора данных или будут потреблять слишком много памяти, чтобы получить сингулярное разложение всей матрицы данных. В этих случаях сначала вычисление матрицы ковариаций может повышать гибкость анализа. Но выбор собственного разложения вместо сингулярного разложения часто зависит от личных предпочтений.

---

[1] Англ. *latent factor scores*. – *Прим. перев.*

Линейный дискриминантный анализ

Линейный дискриминантный анализ (LDA)[1] – это метод многопеременной классификации, который часто используется в машинном обучении и статистике. Первоначально он был разработан Рональдом Фишером[2], которого часто считают «дедушкой» статистики за его многочисленные и важные вклады в математические основы статистики.

Цель LDA – найти направление в пространстве данных, максимально разделяющее категории данных. Пример набора данных конкретной задачи показан на графике A рис. 15.2. Визуально очевидно, что эти две категории можно разделить, но они не разделимы ни по одной из осей данных – это ясно из визуального осмотра маржинальных (частных) распределений.

Теперь введем линейный дискриминантный анализ. LDA найдет базисные векторы в пространстве данных, которые максимально разделяют две категории. График B на рис. 15.2 показывает те же данные, но в пространстве LDA. Теперь классификация будет элементарной: наблюдения с отрицательными значениями по оси 1 помечаются как категория «0», а любые наблюдения с положительными значениями по оси 1 помечаются как категория «1». Данные полностью не разделимы на оси 2, что указывает на то, что для точной категоризации в этом наборе данных одного измерения достаточно.

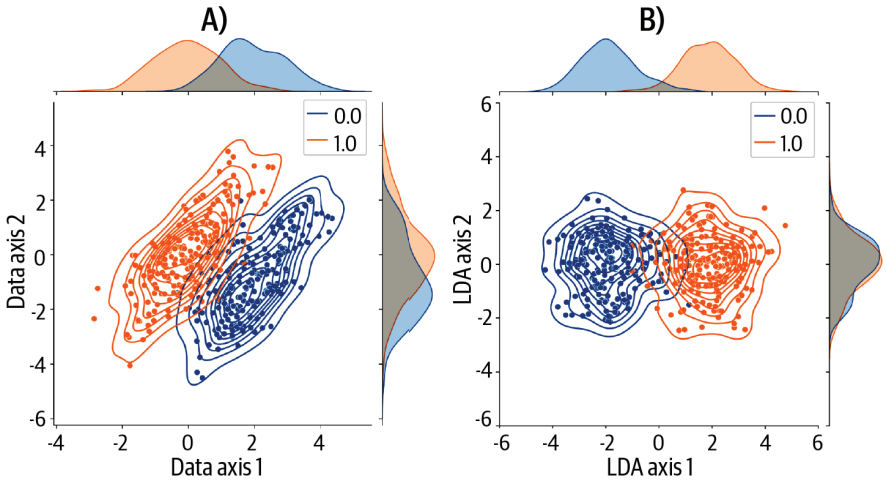


Рис. 15.2 ❖ Пример 2D-задачи для LDA

Звучит здорово, не правда ли? Но как работает такое чудо математики? На самом деле оно довольно просто устроено и основано на обобщенном собственном разложении, о котором вы узнали ближе к концу главы 13.

---

[1] Англ. *Linear Discriminant Analysis*. – *Прим. перев.*

[2] Более того, линейный дискриминантный анализ также называют дискриминантным анализом Фишера.

Давайте я начну с целевой функции: наша цель – найти набор весов, такой чтобы взвешенная комбинация переменных максимально разделяла категории. Эта целевая функция записывается аналогично целевой функции PCA:

$$\lambda = \frac{||\mathbf{X}_B\mathbf{w}||^2}{||\mathbf{X}_W\mathbf{w}||^2}.$$

На естественном языке она звучит так: мы хотим найти набор весов признаков $\mathbf{w}$, который максимизирует *отношение* дисперсии признака $\mathbf{X}_B$ к дисперсии признака $\mathbf{X}_W$ данных. Обратите внимание, что ко всем наблюдениям применяются одинаковые веса. (После изложения математики я напишу о признаках B и W подробнее.)

Линейно-алгебраическое решение вытекает из аналогичного аргумента, как описано в разделе PCA. Во-первых, расширяем $||\mathbf{X}_B\mathbf{w}||^2$ до $\mathbf{w}^{\mathrm{T}}\mathbf{X}_B^{\mathrm{T}}\mathbf{X}_B\mathbf{w}$ и представляем его как $\mathbf{w}^{\mathrm{T}}\mathbf{C}_B\mathbf{w}$; во-вторых, рассматриваем множество решений вместо одного решения; в-третьих, заменяем деление взаимообратным умножением; и, наконец, выполняем несколько алгебраических действий и смотрим, что произойдет:

$$\begin{aligned}
\mathbf{\Lambda} &= (\mathbf{W}^{\mathrm{T}}\mathbf{C}_W\mathbf{W})^{-1}\mathbf{W}^{\mathrm{T}}\mathbf{C}_B\mathbf{W};\\
\mathbf{\Lambda} &= \mathbf{W}^{-1}\mathbf{C}_W^{-1}\mathbf{W}^{-\mathrm{T}}\mathbf{W}^{\mathrm{T}}\mathbf{C}_B\mathbf{W};\\
\mathbf{\Lambda} &= \mathbf{W}^{-1}\mathbf{C}_W^{-1}\mathbf{C}_B\mathbf{W};\\
\mathbf{W}\mathbf{\Lambda} &= \mathbf{C}_W^{-1}\mathbf{C}_B\mathbf{W};\\
\mathbf{C}_W\mathbf{W}\mathbf{\Lambda} &= \mathbf{C}_B\mathbf{W}.
\end{aligned}$$

Другими словами, решение LDA получается из обобщенного собственного разложения двух матриц ковариаций. Собственные векторы – это веса, а обобщенные собственные числа – это коэффициенты дисперсии каждой компоненты[1].

Если не учитывать математику, то какие признаки данных используются для построения $\mathbf{X}_B$ и $\mathbf{X}_W$? Дело в том, что есть разные способы реализации этой формулы, в зависимости от природы задачи и конкретной цели анализа. Но в типичной модели LDA $\mathbf{X}_B$ происходит из межкатегориальной ковариации, а $\mathbf{X}_W$ – из внутрикатегориальной ковариации.

Внутрикатегориальная ковариация – это просто среднее значение ковариаций образцов данных внутри каждого класса. Межкатегориальная ковариация происходит из создания новой матрицы данных, содержащей средние значения признаков внутри каждого класса. Я проведу вас по данной процедуре в упражнениях. Если вы знакомы со статистикой, то поймете, что эта формула аналогична отношению межгрупповой суммы квадратов ошибок к внутригрупповой сумме в моделях ANOVA.

Два заключительных комментария: собственные векторы обобщенного собственного разложения не ограничены ортогональностью. Это вызвано

[1] Я не буду приводить нагруженное вычислениями доказательство, но это всего лишь второстепенный вариант доказательства, приведенного в разделе PCA.

тем, что $\mathbf{C}_W^{-1}\mathbf{C}_B$, как правило, не является симметричной матрицей, хотя две матрицы ковариаций сами по себе симметричны. Несимметричные матрицы не имеют ограничения по ортогональности собственного вектора. Вы увидите это в упражнениях.

Наконец, LDA всегда будет находить *линейное* решение (да, это видно и из самого названия LDA), даже если данные не являются линейно разделимыми. Нелинейное разделение потребует преобразования данных или применения нелинейного метода категоризации, такого как искусственные нейронные сети. LDA по-прежнему будет работать в смысле получения результата; вам как исследователю данных решать, подходит ли этот результат для поставленной задачи и можно ли его интерпретировать.

Низкоранговая аппроксимация посредством сингулярного разложения

Я объяснил концепцию низкоранговых аппроксимаций в предыдущей главе (например, в упражнении 14.5). Ее идея состоит в том, чтобы взять сингулярное разложение матрицы данных или изображения, а затем реконструировать эту матрицу данных, используя некоторое подмножество компонент сингулярного разложения.

Этого можно добиться, задав отобранные значения σ равными нулю или создав новые прямоугольные матрицы сингулярного разложения с подлежащими удалению векторами и удаленными сингулярными числами. Этот второй подход будет предпочтительнее, поскольку он уменьшает объем сохраняемых данных, как вы увидите в упражнениях. При таком подходе сингулярное разложение можно использовать для сжатия данных до меньшего размера.

Применяется ли сингулярное разложение в компьютерах для сжатия изображений?

Короткий ответ – нет, не используется.

Алгоритмы, лежащие в основе распространенных форматов сжатия изображений, таких как JPG, предусматривают поблочное сжатие, в которое встроены принципы человеческого восприятия (включая, например, то, как мы воспринимаем контраст и пространственные частоты), что позволяет им достигать более качественных результатов при меньшей вычислительной мощности по сравнению с одним сингулярным разложением всего изображения.

Тем не менее принцип остается тем же, что и при сжатии на основе сингулярного разложения: выявить небольшое число базисных векторов, которые сберегают важные признаки изображения, такие что низкоранговая реконструкция является точной аппроксимацией оригинала в полной разрешающей способности.

С учетом всего вышесказанного сингулярное разложение обычно используется для сжатия данных в других областях науки, включая биомедицинскую визуализацию.

Сингулярное разложение для шумоподавления

Шумоподавление посредством сингулярного разложения – это просто применение низкоранговой аппроксимации. Единственное отличие состоит в том, что компоненты сингулярного разложения отбираются для их исключения на основании шума, который они представляют, в отличие от внесения малого вклада в матрицу данных.

Подлежащие удалению компоненты могут быть слоями, связанными с наименьшими сингулярными числами, – это может иметь место в случае низкоамплитудного шума, ассоциированного с малыми дефектами оборудования. Но более крупные источники шума, оказывающие более сильное влияние на данные, могут иметь более высокие сингулярные числа. Указанные шумовые компоненты могут выявляться алгоритмом, основанным на их характеристиках, или путем визуального осмотра. В упражнениях вы увидите пример использования сингулярного разложения для выделения источника шума, добавленного в изображение.

Резюме

Вы дочитали книгу до конца (за исключением приведенных ниже упражнений)! Поздравляю! Найдите минутку, чтобы погордиться собой и своим стремлением учиться и инвестировать в свой мозг (в конце концов, это ваш самый ценный ресурс). Я вами горжусь, и если бы мы встретились лично, то дал бы вам пять, ударил бы кулаком о кулак, похлопал локтем или сделал бы что угодно, что в данный момент приемлемо с социальной/медицинской точки зрения.

Надеюсь, вы почувствовали, что эта глава помогла вам увидеть невероятную важность собственного и сингулярного разложений для применений в статистике и машинном обучении. Ниже – краткое изложение ключевых моментов, которые я обязан включить согласно контракту.

- Цель анализа главных компонент (PCA) – найти набор весов, такой чтобы линейно-взвешенная комбинация признаков данных имела максимальную дисперсию. Эта цель отражает лежащее в основе PCA допущение, а именно что «дисперсия равна релевантности».
- PCA-анализ реализуется как собственное разложение матрицы ковариаций данных. Собственные векторы – это коэффициенты перевесовки признаков, а собственные числа могут шкалироваться, чтобы кодировать процент дисперсии, учитываемой каждой компонентой (компонента – это линейно-взвешенная комбинация).
- PCA-анализ можно эквивалентным образом реализовать с использованием матрицы ковариаций либо матрицы данных сингулярного разложения.

- Линейный дискриминантный анализ (LDA) используется для линейной категоризации многопеременных данных. Его можно трактовать как расширение PCA-анализа: в отличие от PCA, который максимизирует дисперсию, LDA максимизирует отношение дисперсий между двумя признаками данных.
- LDA-анализ реализуется как обобщенное собственное разложение на двух матрицах ковариаций, формируемых из двух разных признаков данных. Двумя признаками данных часто являются межклассовая ковариация (для максимизации) и внутриклассовая ковариация (для минимизации).
- Низкоранговые аппроксимации предусматривают реконструкцию матрицы из подмножества сингулярных векторов/чисел и используются для сжатия данных и шумоподавления.
- В сжатии данных ассоциированные с наименьшими сингулярными числами компоненты удаляются; в шумоподавлении данных удаляются компоненты, улавливающие шум или артефакты (соответствующие им сингулярные числа могут быть малыми либо большими).

Упражнения

Анализ главных компонент (PCA)

Я люблю кофе по-турецки. Он делается из зерен очень мелкого помола и без фильтра. Весь ритуал его приготовления и питья выглядит прекрасно. И если вы пьете его из турки, то весьма возможно, что вам удастся погадать.

Но это упражнение касается не кофе по-турецки, а проведения PCA-анализа на наборе данных[1], который содержит временной ряд Стамбульской фондовой биржи, а также биржевые данные из нескольких других фондовых индексов в разных странах. Мы могли бы использовать этот набор данных, чтобы узнать, например, управляются ли международные фондовые биржи одним общим фактором или же в разных странах существуют независимые финансовые рынки.

Упражнение 15.1

Перед выполнением PCA-анализа импортируйте и обследуйте данные. Я сделал несколько графиков, показанных на рис. 15.3; вы можете воспроизвести эти графики и/или использовать другие методы обследования данных.

---

[1] Первоисточник данных: *Акбилгич Огуз.* Стамбульская фондовая биржа. Репозиторий машинного обучения UCI. 2013. Веб-сайт источников данных: https://archive-beta.ics.uci.edu/ml/datasets/istanbul+stock+exchange.

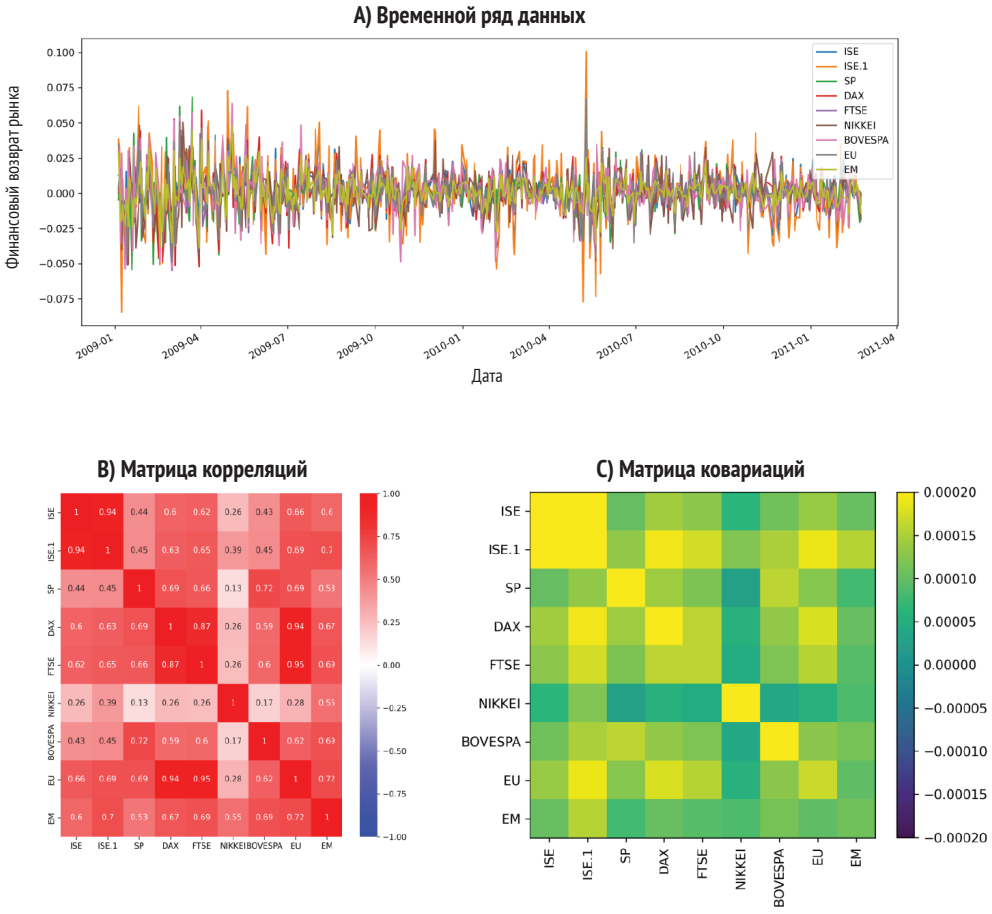


Рис. 15.3 ❖ Обследование набора данных международной фондовой биржи

Теперь приступим к PCA-анализу. Реализуйте PCA, используя пять описанных ранее в этой главе шагов. Визуализируйте результаты, как показано на рис. 15.4. Используйте исходный код, чтобы продемонстрировать несколько признаков PCA:

1. Дисперсия компонентного временного ряда (с использованием функции `np.var`) равна собственному числу, ассоциированному с этой компонентой. Результаты для первых двух компонент приведены ниже:

```
Дисперсия первых двух компонент:
[0.0013006 0.00028585]

Первые два собственных числа:
[0.0013006 0.00028585]
```

2. Корреляция между главными компонентами (то есть взвешенными комбинациями бирж) 1 и 2 равна нулю, т. е. ортогональна.

3. Визуализируйте веса собственных векторов первых двух компонент. Веса показывают величину вклада, вносимую в компонент каждой переменной.

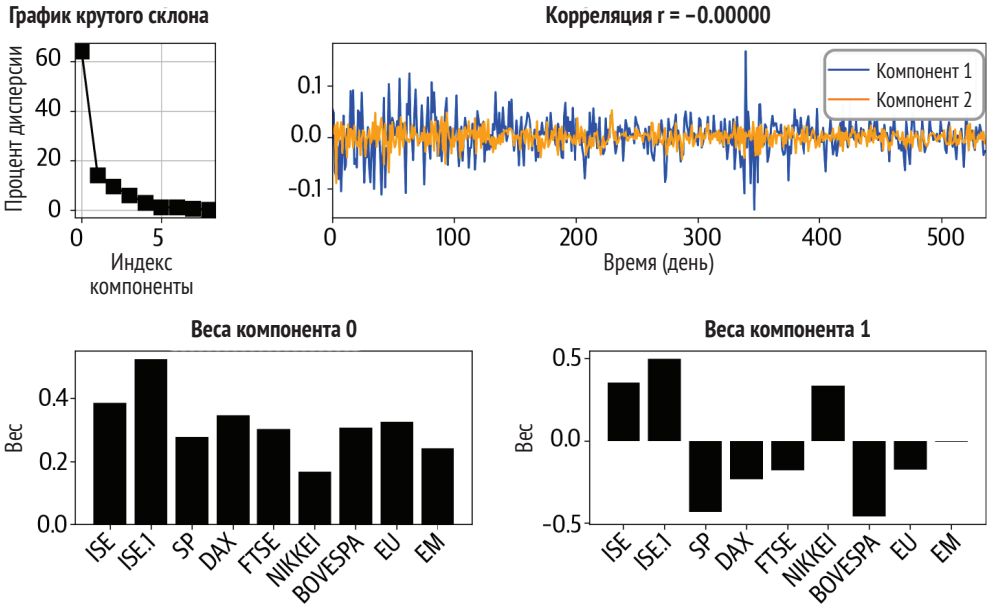


Рис. 15.4 ❖ Результаты PCA на наборе данных Стамбульской фондовой биржи

Обсуждение: график крутого склона убедительно свидетельствует о том, что международные фондовые биржи управляются общим фактором мировой экономики: на долю одной крупной компоненты приходится около 64 % дисперсии данных, тогда как на другие компоненты приходится менее 15 % дисперсии (в чисто случайном наборе данных мы ожидаем, что каждая компонента будет составлять 100/9 = 11 % дисперсии плюс/минус шум).

Строгое оценивание статистической значимости этих компонент выходит за рамки данной книги, но, основываясь на визуальном осмотре графика крутого склона, мы не имеем права интерпретировать компоненты после первой; похоже, что бóльшая часть дисперсии в этом наборе данных четко укладывается в одно измерение.

С точки зрения уменьшения размерности мы могли бы сократить весь набор данных до компоненты, ассоциированной с наибольшим собственным числом (она часто называется верхней компонентой), тем самым представив этот набор 9D-данных с помощью 1D-вектора. Разумеется, мы теряем информацию – 36 % информации в наборе данных удаляется, если сосредоточиться только на верхней компоненте, – но будем надеяться, что важные признаки сигнала находятся в верхней компоненте, тогда как менее важные признаки, включая случайный шум, игнорируются.

Упражнение 15.2

Воспроизведите результаты, используя:

1) сингулярное разложение матрицы ковариаций данных и
2) сингулярное разложение самой матрицы данных. Напомню, что собственные числа $\mathbf{X}^T\mathbf{X}$ являются квадратами сингулярных чисел $\mathbf{X}$; кроме того, для того чтобы найти эквивалентность, шкалирующий коэффициент на матрице ковариаций должен применяться к сингулярным числам.

Упражнение 15.3

Сравните свой «ручной» PCA-анализ с результатом работы PCA-программы на Python. Вам придется провести небольшое онлайновое исследование, чтобы понять, как выполнять указанную программу на Python (это один из самых важных навыков в программировании на Python!), но я дам вам подсказку: она находится в библиотеке sklearn.decomposition.



Библиотека sklearn либо ручная реализация PCA?

Следует ли вычислять PCA путем написания исходного кода вычисления и собственного разложения матрицы ковариаций или же лучше использовать реализацию в библиотеке sklearn? Всегда существует компромисс между использованием собственного исходного кода с целью максимальной адаптации под конкретно-прикладную задачу и использованием готового исходного кода с целью максимального упрощения. Одно из бесчисленного числа удивительных преимуществ понимания математики, лежащей в основе анализа данных, заключается в том, что вы имеете возможность адаптировать все виды анализа под свои потребности. В своем собственном исследовании я обнаружил, что моя личная реализация PCA дает мне больше свободы и гибкости.

Упражнение 15.4

Теперь выполните PCA-анализ на симулированных данных, что позволит выявить одно из потенциальных ограничений PCA. Цель – создать набор данных, состоящий из двух «потоков» данных, и вывести главные компоненты поверх, как показано на рис. 15.5.

Ниже приводится процедура создания данных.

1. Создать матрицу 1000×2 случайных чисел, взятых из нормального (гауссова) распределения, в котором шкала второго столбца уменьшена на 0.05.
2. Создать матрицу 2×2 чистого поворота (см. главу 7).
3. Наложить две копии данных по вертикали: один раз с данными, повернутыми на $\theta = -\pi/6$, и один раз с данными, повернутыми на $\theta = -\pi/3$. Результирующая матрица данных будет иметь размер 2000×2.

Примените сингулярное разложение, чтобы реализовать PCA. Для проведения визуального осмотра я прошкалировал сингулярные векторы с коэффициентом 2.

Обсуждение: PCA отлично подходит для уменьшения размерности высокоразмерного набора данных. За счет него облегчаются сжатие данных, очистка данных и сглаживаются проблемы численной стабильности (например, представьте, что 200-мерный набор данных с кондиционным числом

10^{10} сокращается до самых больших 100 измерений с кондиционным числом 10^5). Но сами измерения могут быть плохим вариантом для извлечения признаков из-за ограничения по ортогональности. И действительно, главные направления дисперсии на рис. 15.5 верны в математическом смысле, но я уверен, что у вас сложилось впечатление, что это не лучшие базисные векторы, для того чтобы улавливать признаки данных.

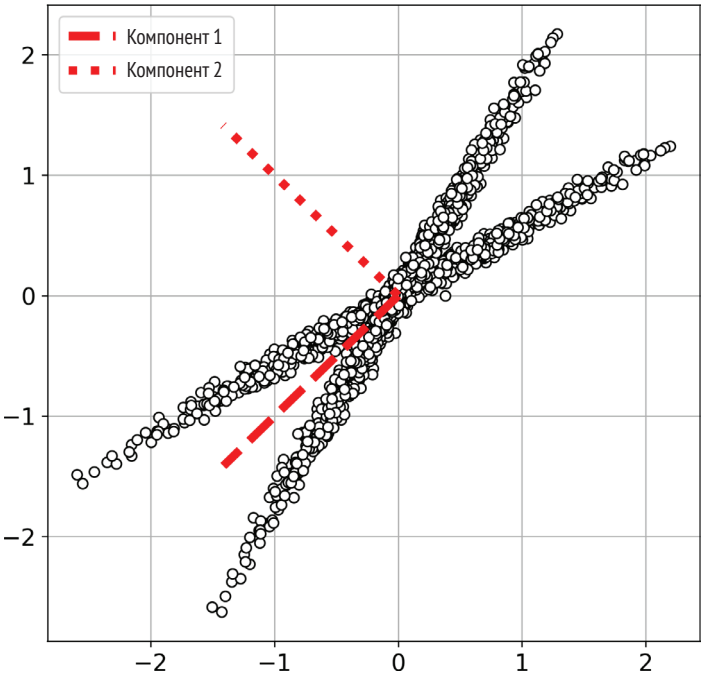


Рис. 15.5 ❖ Результаты упражнения 15.4

Линейный дискриминантный анализ (LDA)

Упражнение 15.5

Вы собираетесь выполнить LDA на симулированных 2D-данных. Симулированные данные удобны тем, что можно манипулировать размерами эффектов, величиной и характером шума, числом категорий и т. д.

Создаваемые вами данные показаны на рис. 15.2. Создайте два набора нормально распределенных случайных чисел, каждый размером 200×2, с добавлением второго измерения к первому (это вводит корреляцию между переменными). Затем добавьте в первый набор чисел *xy*-смещение, равное [2 −1]. Будет удобно создать матрицу 400×2, содержащую оба класса данных, а также 400-элементный вектор меток классов (я использовал 0 для первого класса и 1 для второго класса).

Примените функции `sns.jointplot` и `plot_joint`, чтобы воспроизвести график А рис. 15.2.

Упражнение 15.6

Теперь приступаем к LDA-анализу. Напишите исходный код с использованием NumPy и/или SciPy вместо применения встроенной библиотеки, такой как sklearn (мы вернемся к ней позже).

Матрица внутриклассовых ковариаций $\mathbf{C}_W$ создается путем вычисления ковариации каждого класса отдельно и последующего усреднения этих матриц ковариаций. Матрица межклассовых ковариаций $\mathbf{C}_B$ создается путем вычисления среднего значения каждого признака данных (в данном случае *xy*-координат) внутри каждого класса, конкатенации этих векторов усредненных признаков по всем классам (в разультате будет создана матрица 2×2 для двух признаков и двух классов), а затем вычисления матрицы ковариаций этой конкатенированной матрицы.

Вспомните из главы 13, что обобщенное собственное разложение реализуется с помощью функции SciPy `eigh`.

Проецируемые в пространство LDA данные вычисляются как $\tilde{\mathbf{X}}\mathbf{V}$, где $\tilde{\mathbf{X}}$ содержит конкатенированные данные из всех классов, центрированные по среднему значению по каждому признаку, а $\mathbf{V}$ – это матрица собственных векторов.

Вычислите классификационную точность, которая заключается просто в том, имеет ли каждый образец данных отрицательную («класс 0») либо положительную («класс 1») проекцию на первую компоненту LDA. График С рис. 15.6 показывает предсказанную метку класса по каждому образцу данных.

Наконец, покажите результаты, как видно на рис. 15.6.

Упражнение 15.7

В главе 13 я утверждал, что для обобщенного собственного разложения матрица собственных векторов $\mathbf{V}$ неортогональна, но ортогональна в пространстве матрицы – «знаменателе». Здесь ваша цель – продемонстрировать это эмпирически.

Вычислите и обследуйте результаты $\mathbf{V}^{\mathrm{T}}\mathbf{V}$ и $\mathbf{V}^{\mathrm{T}}\mathbf{C}_W\mathbf{V}$. Если игнорировать крошечные ошибки вычислительной прецизионности, то какой из них создает единичную матрицу?

Упражнение 15.8

Теперь воспроизведите результаты, используя библиотеку Python sklearn. Примените функцию `LinearDiscriminantAnalysis` из библиотечного модуля `sklearn.distribunt_analysis`. Создайте график, подобный рис. 15.7, и убедитесь, что совокупная точность предсказания совпадает с результатами вашего «ручного» LDA-анализа в предыдущем упражнении. Эта функция допускает применение нескольких разных решателей; примените решатель `eigen` для соотнесения с предыдущим упражнением, а также выполнения следующего упражнения.

Нанесите сверху предсказанные метки из вашего «ручного» LDA; вы должны обнаружить, что предсказанные метки обоих подходов одинаковы.

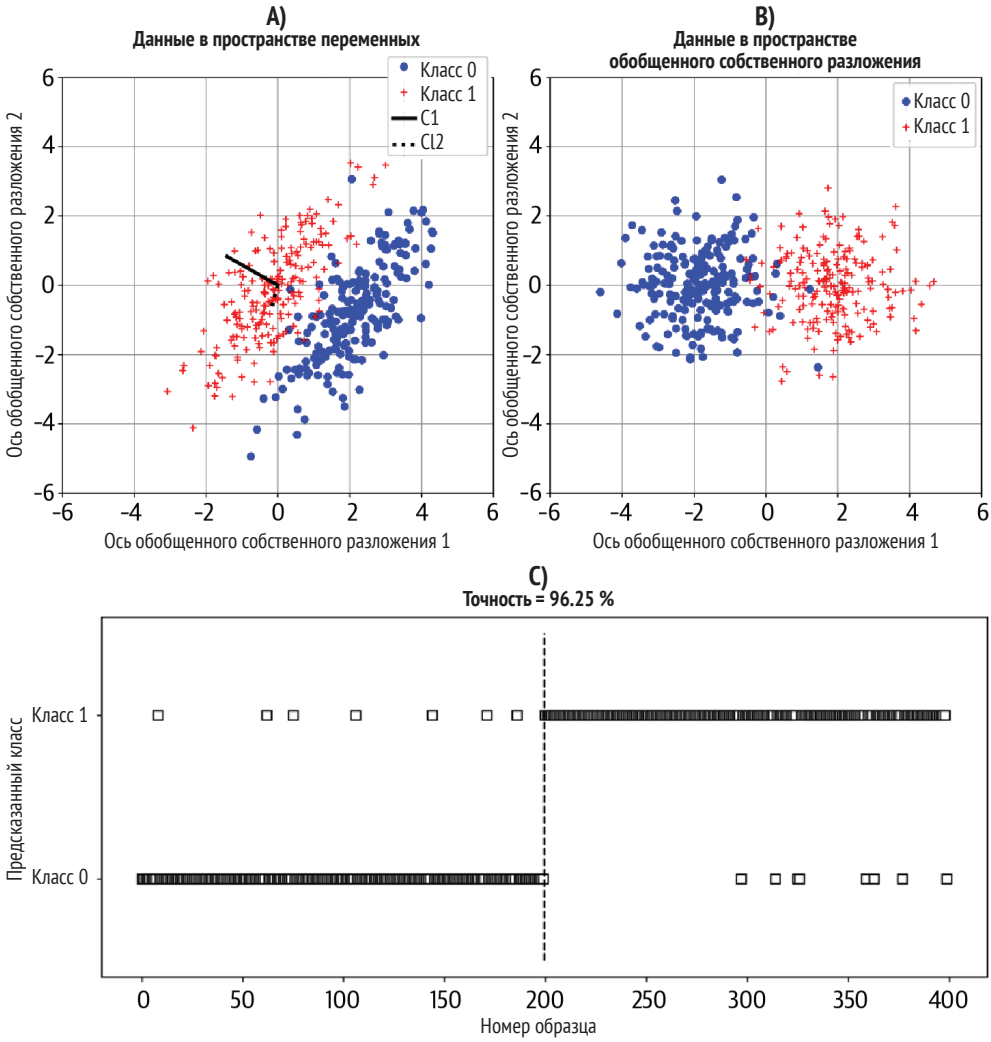


Рис. 15.6 ❖ Результаты упражнения 15.6

Упражнение 15.9

Давайте воспользуемся библиотекой sklearn, чтобы обследовать эффекты усадочной регуляризации. Как я писал в главах 12 и 13, вполне очевидно, что усадка снизит результативность на тренировочных данных; важный вопрос заключается в том, улучшает ли регуляризация точность предсказания на не встречавшихся ранее данных (иногда именуемых валидационным, или тестовым, набором). Поэтому вам нужно написать исходный код, чтобы реализовать тренировочный/тестовый подраздел. Я сделал это путем случайной перестановки индексов выборки между 0 и 399, тренировки на первых 350,

а затем тестирования на последних 50. Поскольку такого количества образцов будет для усреднения мало, я повторил этот случайный отбор 50 раз и взял среднюю точность как точность в расчете на величину усадки на рис. 15.8.

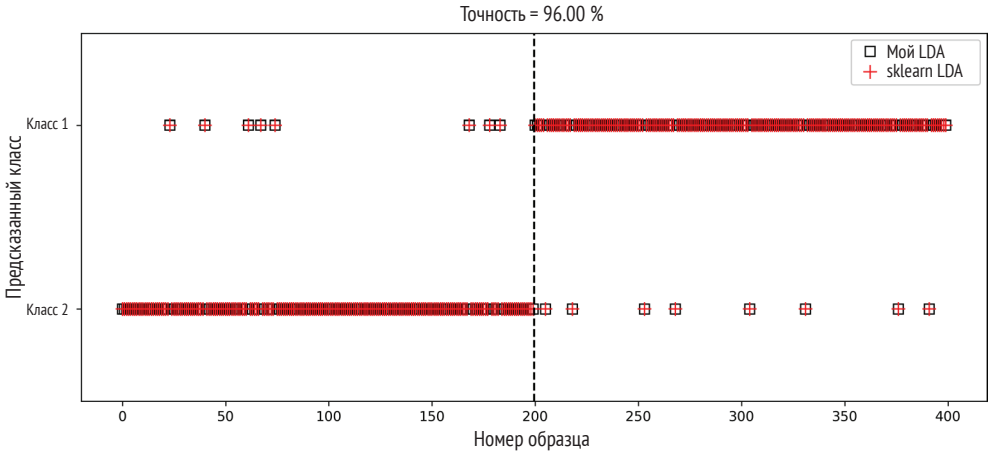


Рис. 15.7 ❖ Результаты упражнения 15.8

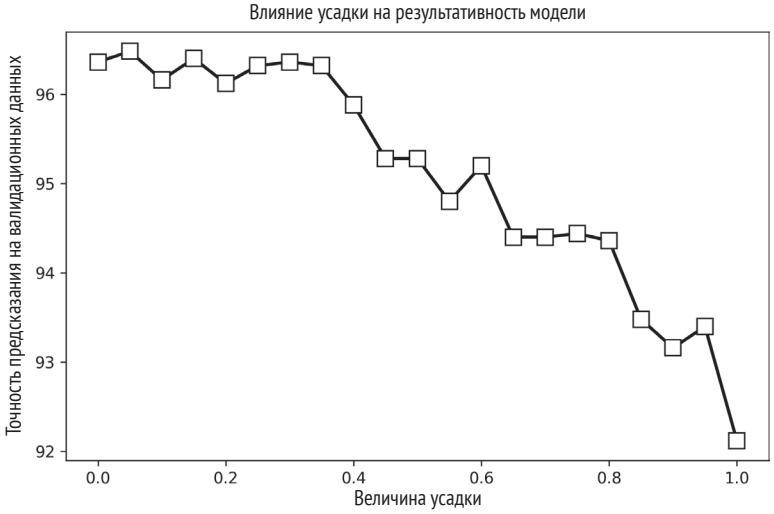


Рис. 15.8 ❖ Результаты упражнения 15.9

Обсуждение: усадка обычно сказывалась отрицательно на валидационной результативности. Хотя, похоже, результативность и улучшалась при некоторой усадке, многократное повторение исходного кода показало, что это были просто случайные колебания. Более глубокое погружение в регуляризацию больше подходит для отдельной книги по машинному обучению, но здесь я хотел подчеркнуть, что многие разработанные в машинном обучении «уловки» не обязательно выгодны во всех случаях.

Сингулярное разложение для низкоранговых аппроксимаций

Упражнение 15.10

Игорь Стравинский был одним из величайших композиторов всех времен (ИМХО) и, безусловно, одним из самых влиятельных в XX веке. Он также сделал много наводящих на размышления заявлений о природе искусства, средств массовой информации и критики, включая одну из моих любимых цитат: «Чем больше искусство ограничено, тем больше оно свободно». Есть знаменитый и пленительный рисунок Стравинского, написанный не кем иным, как великим Пабло Пикассо. Изображение этого рисунка доступно в Википедии, и мы будем работать с ним в следующих нескольких упражнениях. Как и в случае с другими изображениями, с которыми мы работали в этой книге, изначально это 3D-матрица (640×430×3), но для удобства мы конвертируем ее в оттенки серого (2D).

Цель этого упражнения – повторить упражнение 14.5, в котором вы воссоздали близкую аппроксимацию сглаженного по шуму изображения, основываясь на четырех «слоях» сингулярного разложения (пожалуйста, вернитесь к этому упражнению, чтобы освежить его в памяти). Создайте рисунок, подобный рис. 15.9, используя изображение Стравинского. Вот главный вопрос: дает ли реконструкция изображения по первым четырем компонентам такой же хороший результат, как в предыдущей главе?

Упражнение 15.11

Что ж, ответ на вопрос, поставленный в конце предыдущего упражнения, – решительное «Нет!». 4-ранговая аппроксимация выглядит просто ужасно! Она не похожа на изначальное изображение. Цель этого упражнения – реконструировать изображение, используя большее число слоев, чтобы низкоранговая аппроксимация стала достаточно точной, а затем вычислить величину полученного сжатия.

Начните с создания рис. 15.10, на котором показаны изначальное изображение, реконструированное изображение и карта ошибок, представляющая собой квадрат разницы между изначальным изображением и его аппроксимацией. Для этого рисунка я выбрал $k = 80$ компонент, но рекомендую вам обследовать другие значения (то есть другие ранговые аппроксимации).

Далее вычислите коэффициент сжатия, то есть процент числа байтов, используемых в низкоранговой аппроксимации, по сравнению с числом байтов, используемых в изначальном изображении. Мои результаты для $k = 80$ показаны ниже[1]. Имейте в виду, что при низкоранговой аппроксимации не требуется хранить ни полное изображение, ни полные матрицы сингулярного разложения!

```
Оригинал: 2.10 Мб
Реконструкция: 2.10 Мб
Векторы реконструкции: 0.65 Мб (при k=80 компонентах)
Сжатие: 31.13%
```

[1] Существует некоторая двусмысленность в отношении того, что считать одним мегабайтом: 1000^2 либо 1024^2 байт; я использовал последнее, но это не влияет на степень сжатия.

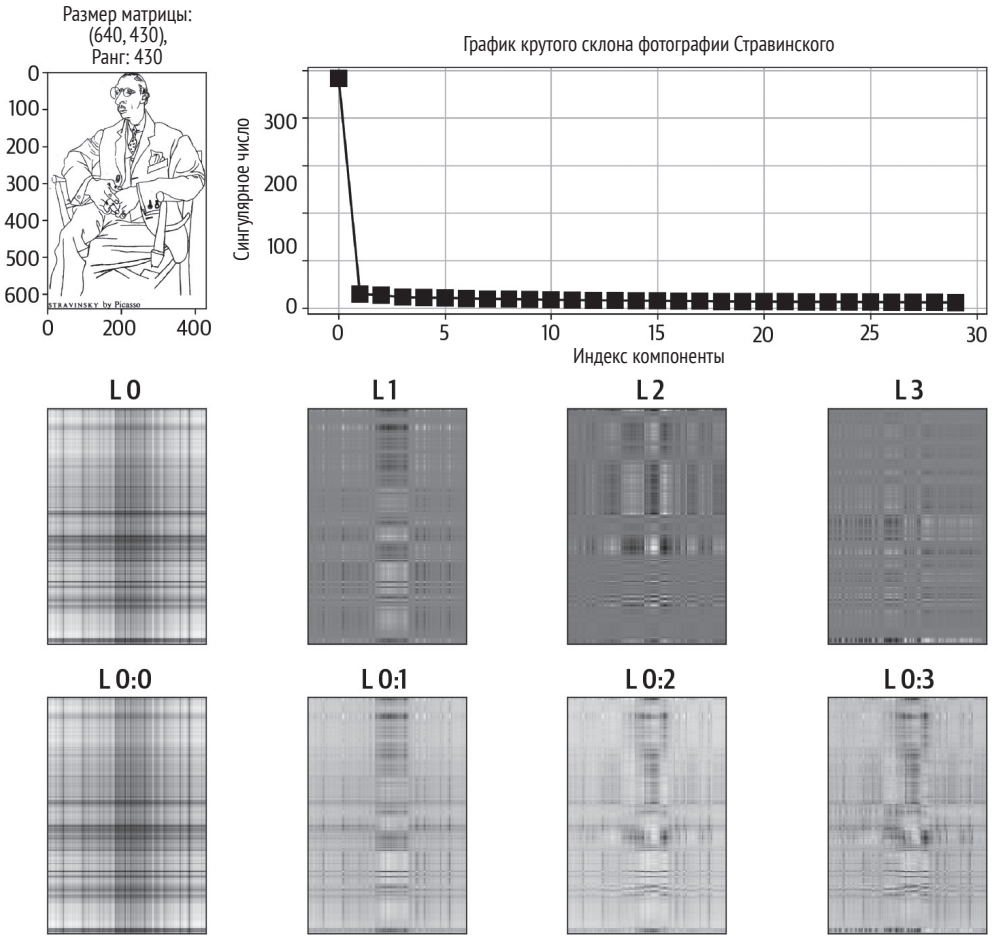


Рис. 15.9 ❖ Результаты упражнения 15.10

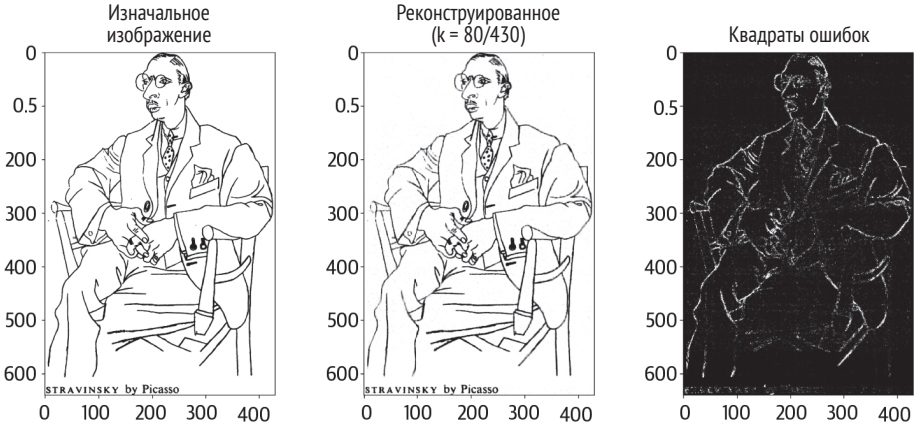


Рис. 15.10 ❖ Результаты упражнения 15.11

Упражнение 15.12

Почему я выбрал $k = 80$, а не, например, 70 или 103? Если честно, то этот выбор был сделан довольно произвольно. Цель этого упражнения – увидеть, можно ли использовать карту ошибок для определения подходящего параметра ранга.

В цикле `for` над рангами реконструкции от 1 до количества сингулярных чисел создайте низкоранговую аппроксимацию и вычислите расстояние Фробениуса между оригиналом и -ранговой аппроксимацией. Затем постройте график ошибки как функции от ранга, как показано на рис. 15.11. Ошибка, безусловно, уменьшается с увеличением ранга, однако нет четкого ранга, который выглядит самым лучшим. Иногда в алгоритмах оптимизации более информативной будет производная от функции ошибок; попробуйте!

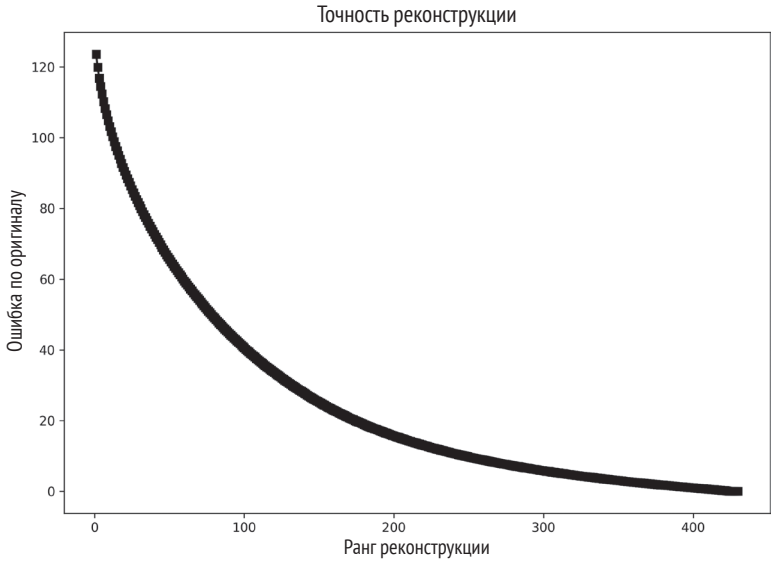


Рис. 15.11 ❖ Результаты упражнения 15.12

Заключительная мысль по поводу этого упражнения: ошибка реконструкции при $k = 430$ (т. е. полном сингулярном разложении) должна равняться 0. Так ли это? Очевидно, что нет; иначе я бы не поставил вопрос. Но вы должны убедиться в этом сами. Это еще одна демонстрация ошибок вычислительной прецизионности в прикладной линейной алгебре.

Сингулярное разложение для шумоподавления в изображениях

Упражнение 15.13

Давайте посмотрим, сможем ли мы расширить концепцию низкоранговой аппроксимации, чтобы убрать шум из фотографии Стравинского. Цель этого

упражнения – добавить шум и проинспектировать результаты сингулярного разложения, а затем следующее упражнение будет содержать «редуцирование» искажений.

Шумом здесь будет пространственная синусоидная волна. На рис. 15.12 показаны шум и искаженное изображение.

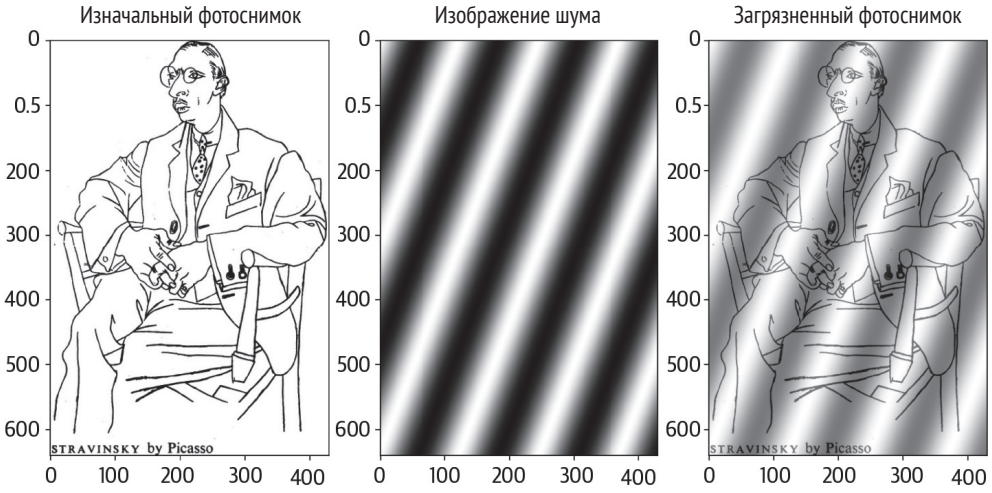


Рис. 15.12 ❖ Подготовка к упражнению 15.13

Теперь я опишу, как создавать двумерную синусоидную волну (также именуемую *синусоидной обрешеткой*). Это хорошая возможность попрактиковаться в навыках переложения математики в исходный код. Формула двумерной синусоидной обрешетки такова:

$$\mathbf{Z} = \sin\left(2\pi f(\mathbf{X}\cos(\theta) + \mathbf{Y}\sin(\theta))\right).$$

В приведенной выше формуле f – это частота синусоиды, θ – параметр поворота, а π – константа 3.14.... $\mathbf{X}$ и $\mathbf{Y}$ – это местоположения на решетке, в которых вычисляется функция; я задал их как целые числа от –100 до 100 с числом шагов, установленным в соответствии с размером фотографии Стравинского. Я установил $f = 0.02$ и $\theta = \pi/6$.

Прежде чем перейти к остальной части упражнения, рекомендую вам потратить некоторое время на исходный код синусоидной обрешетки путем обследования влияния изменений параметров на результирующее изображение. Тем не менее, пожалуйста, используйте параметры, которые я написал ранее, чтобы воспроизвести мои результаты, которые показаны ниже.

Затем испортите фотографию Стравинского, добавив в изображение шум. Вы должны сначала прошкалировать шум в диапазон от 0 до 1, затем сложить шум и изначальную фотографию, а потом перешкалировать. Шкалирование изображения между 0 и 1 достигается применением следующей ниже формулы:

$$\tilde{\mathbf{R}} = \frac{\mathbf{R} - \min(\mathbf{R})}{\max(\mathbf{R}) - \min(\mathbf{R})}.$$

Хорошо, теперь у вас есть искаженное шумом изображение. Воспроизведите рис. 15.13, который аналогичен рис. 15.9, но с использованием зашумленного изображения.

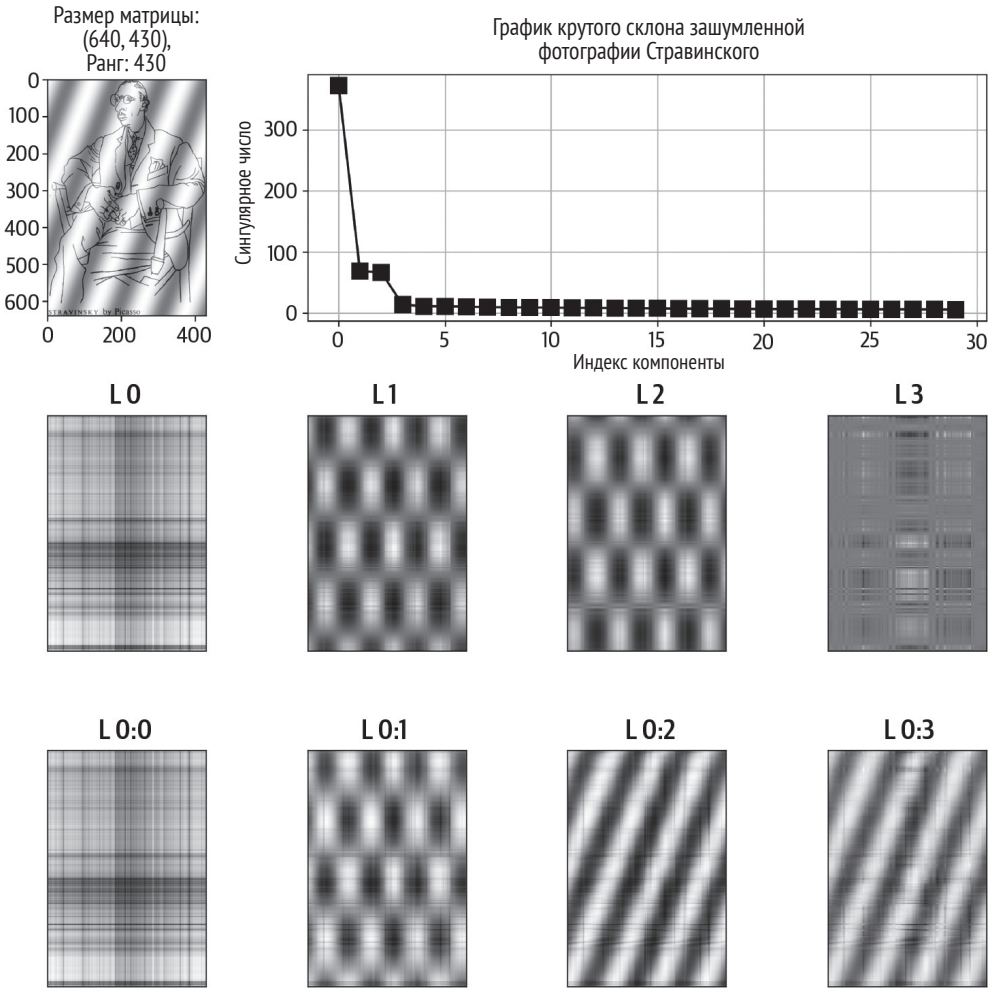


Рис. 15.13 ❖ Результаты упражнения 15.13

Обсуждение: интересно сравнить рис. 15.13 с рис. 15.9. Хотя мы создали шум на основе одного признака (синусоидная обрешетка), сингулярное разложение разделило обрешетку на две компоненты равной важности (примерно равные сингулярные числа)[1]. Эти две компоненты не являются

[1] Вероятно, это объясняется тем, что мы имеем двумерную сингулярную плоскость, а не пару сингулярных векторов; базисными векторами могли быть любые два линейно независимых вектора на этой плоскости, и Python выбрал ортогональную пару.

синусоидными обрешетками, а представляют собой вертикально ориентированные заплатки. Однако их сумма дает диагональные полосы обрешетки.

Упражнение 15.14

Теперь обратимся к шумоподавлению. Похоже, что шум содержится во второй и третьей компонентах, поэтому теперь ваша цель – реконструировать изображение, используя все компоненты, кроме этих двух. Создайте рисунок, как на рис. 15.14.

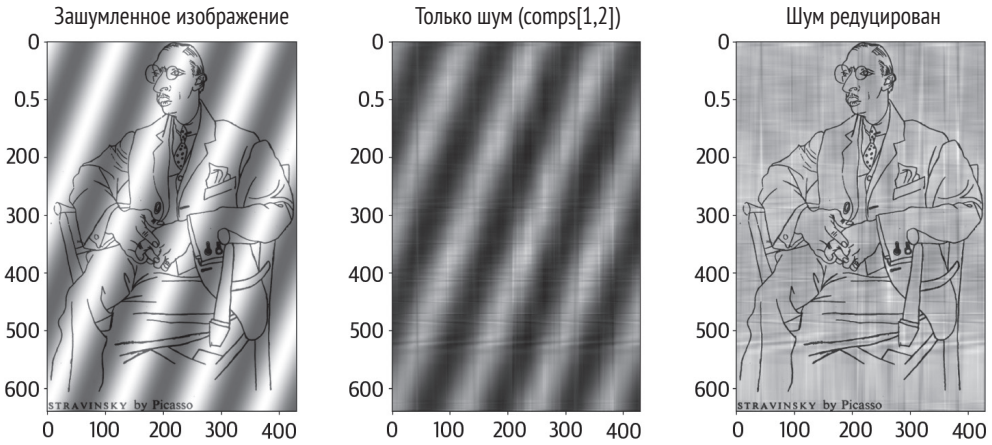


Рис. 15.14 ❖ Результаты упражнения 15.14

Обсуждение: шумоподавление выглядит достойно, но, конечно, не идеально. Одна из причин несовершенства заключается в том, что шум содержится в двух измерениях не полностью (обратите внимание, что средняя часть рис. 15.14 не полностью совпадает с изображением шума). Более того, редукция шума (изображение, составленное из компонент 1 и 2) имеет отрицательные значения и распределена вокруг нуля, хотя синусоидная обрешетка не имела отрицательных значений. (Это можно подтвердить, построив гистограмму изображения шума, которую я показываю в онлайновом исходном коде.) Остальная часть изображения должна иметь колебания значений с учетом этого, чтобы полная реконструкция имела только положительные значения.

Глава 16

Краткое руководство по языку Python

Как я объяснял в главе 1, эта глава представляет собой ускоренный курс по программированию на Python. Он предназначен для того, чтобы быстро освоиться и прослеживать исходный код в остальной части книги, но не для использования в качестве *полного* источника для овладения языком Python. Если вы ищете специальную книгу по Python, то рекомендую «Освоение языка Python» Марка Лутца (O'Reilly)[1].

При работе с данной главой откройте сеанс Python. (Я объясню, как это делать, позже.) Вы не освоите язык Python, просто прочитав эту главу; вам нужно читать, набирать исходный код Python на клавиатуре, изменять его, тестировать и т. д.

Кроме того, в данной главе вы должны набирать вручную весь исходный код, который вы видите напечатанным ниже. Исходный код всех остальных глав этой книги доступен в интернете, но я хотел бы, чтобы вы набирали исходный код этой главы вручную. Когда вы знакомы с программированием на Python, ввод большого объема исходного кода вручную представляет утомительную трату времени. Но когда вы только учитесь программировать, то вам просто необходимо программировать, то есть использовать пальцы, чтобы набирать все подряд, а не просто смотреть на исходный код, напечатанный на странице.

Почему Python и какие есть альтернативы?

Python разработан как язык программирования общего назначения. Его можно использовать для анализа текста, обработки веб-форм, создания алгоритмов и множества других применений. Python также широко используется в науке о данных и машинном обучении; в этих приложениях Python – в основном просто калькулятор. Но чрезвычайно мощный и универсальный каль-

[1] См. Learning Python, https://oreil.ly/learning-python. – *Прим. перев.*

кулятор, и мы используем Python, потому что мы (люди) недостаточно умны, чтобы выполнять все числовые вычисления в уме или с ручкой и бумагой.

Python в настоящее время (в 2022 году) является наиболее часто используемой программой числовой обработки в науке о данных (другие претенденты включают R, MATLAB, Julia, JavaScript, SQL и C). Останется ли Python доминирующим языком науки о данных? Понятия не имею, но сомневаюсь. История информатики полна «финальных языков», которые якобы будут существовать в веках. (Вы когда-нибудь программировали на FORTRAN, COBOL, IDL, Pascal и т. д.?) Но Python сейчас очень популярен, и сейчас вы изучаете прикладную линейную алгебру. В любом случае, хорошая новость заключается в том, что языки программирования обладают хорошей переносимостью знаний, а это означает, что владение Python поможет вам изучить другие языки. Иными словами, время, потраченное на изучение Python, будет потрачено не зря.

Интерактивные среды разработки

Python – это язык программирования, и вы можете выполнять Python в самых разных приложениях, именуемых *средами*. Разные среды создаются разными разработчиками с разными предпочтениями и потребностями. Некоторые распространенные интерактивные среды разработки (IDE)[1], с которыми вы можете столкнуться, включают Visual Studio, Spyder, PyCharm и Eclipse. Возможно, наиболее часто используемая IDE для изучения Python называется блокнотами Jupyter.

Исходный код к этой книге был написан с использованием среды Google Colab Jupyter (подробнее об этом – в следующем разделе). После того как вы познакомитесь с работой на Python на основе Jupyter, вы можете потратить некоторое время на опробование других IDE, чтобы увидеть, насколько они лучше соответствуют вашим потребностям и предпочтениям. Тем не менее здесь я рекомендую использовать Jupyter, потому что эта среда поможет вам прослеживать исходный код и воспроизводить рисунки.

Использование Python локально и онлайн

Поскольку Python является бесплатным и легковесным языком, его можно выполнять в самых разных операционных системах, как на вашем компьютере, так и на облачном сервере:

Работа с исходным кодом Python локально

Вы можете установить Python в любую главенствующую операционную систему (Windows, Mac, Linux). Если вы освоились с установкой программ

[1] Англ. *Interactive Development Environment*. – *Прим. перев.*

и пакетов, то можете устанавливать библиотеки по мере необходимости. В этой книге вам в основном понадобятся библиотеки NumPy, matplotlib, SciPy и sympy.

Но если вы читаете эту главу, то ваши навыки Python, вероятно, ограничены. В этом случае я рекомендую установить Python посредством программного пакета Anaconda. Этот пакет бесплатен и прост в установке, и Anaconda автоматически установит все библиотеки, которые вам понадобятся в этой книге.

Работа с исходным кодом Python онлайн

Изучая эту книгу, я рекомендую выполнять Python в интернете. Преимущества облачного Python заключаются в том, что не нужно ничего устанавливать локально, не нужно использовать собственные вычислительные ресурсы, и вы можете обращаться к своему исходному коду из любого браузера на любом компьютере и в любой операционной системе. Я предпочитаю среду Google Colaboratory, потому что она синхронизируется с моим Google Диском. За счет этого я могу хранить файлы исходного кода Python на моем Google Диске, а затем открывать их по адресу https://colab.research.google.com. Если вы избегаете сервисов Google, то можно использовать и другие облачные среды Python (хотя я не уверен, что это вообще возможно).

Google Colab можно использовать бесплатно. Вам понадобится учетная запись Google, чтобы получать к ней доступ, но это тоже бесплатно. И тогда вы сможете просто закачивать файлы исходного кода на свой Google Диск и открывать их в Colab.

Работа с файлами исходного кода в Google Colab

Теперь я объясню, как скачивать и получать доступ к файлам блокнотов Python этой книги. Как я уже писал ранее, к этой главе нет файлов исходного кода.

Есть два способа переноса исходного кода книги на Google Диск:

- перейти на https://github.com/mikexcohen/LinAlg4DataScience, кликнуть по зеленой кнопке с надписью Code (Исходный код), а затем кликнуть Download ZIP (Скачать ZIP) (рис. 16.1). В результате репозиторий исходного кода будет скачан, и вы сможете закачать эти файлы на свой Google Диск. Теперь на вашем Google Диске можно дважды кликнуть по файлу либо кликнуть правой кнопкой мыши и выбрать **Open with** (Открыть с помощью), а затем **«Google Colaboratory»**;
- перейти непосредственно на https://colab.research.google.com, выбрать вкладку GitHub и выполнить поиск по «mikexcohen» в поисковой строке. Вы найдете все мои общедоступные репозитории GitHub; вам нужен тот, который называется «LinAlg4DataScience». Оттуда вы можете кликнуть по одному из файлов, чтобы открыть блокнот.

Обратите внимание, что это копия записной книжки, предназначенная только для чтения; любые сделанные вами изменения сохранены не будут. Поэтому рекомендую скопировать этот файл на свой Google Диск.

A) github.com/mikexcohen/LinAlg4DataScience B) colab.research.google.com

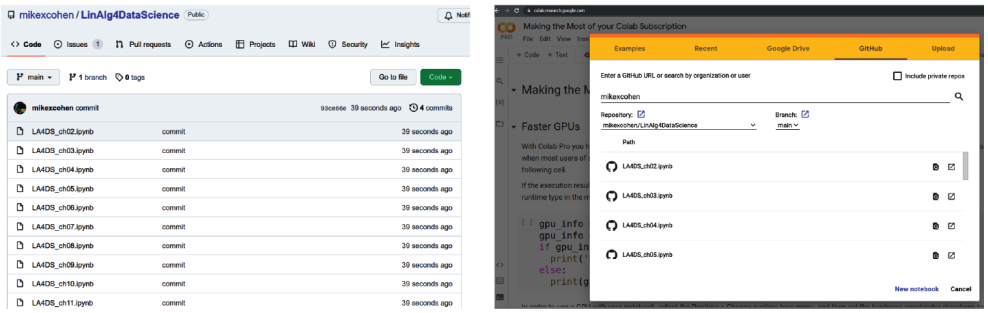


Рис. 16.1 ❖ Перенос исходного кода из GitHub (слева) в Colab (справа)

Теперь, когда вы знаете, как импортировать файлы исходного кода книги в Google Colab, самое время создать новый блокнот, чтобы начать работу с этой главой. Кликните по пункту меню **File** (Файл), а затем по **New Notebook** (Новый блокнот), чтобы создать новый блокнот. Он будет называться «Untitled1.ipynb» или что-то подобное. (Расширение *ipynb* означает «интерактивный блокнот Python», от англ. interactive Python notebook.) Я рекомендую изменить имя файла, кликнув на имени файла в верхнем левом углу экрана. По умолчанию новые файлы помещаются на ваш Google Диск в папку Colab Notebooks (Блокноты Colab).

Переменные

Язык Python можно использовать в качестве калькулятора. Давайте попробуем; наберите следующее в ячейку исходного кода:

```
4 + 5.6
```

После набора этого фрагмента исходного кода ничего не произойдет. Вам нужно сообщить Python, что нужно его исполнить. Это делается путем нажатия сочетания клавиш **Ctrl**-**Enter** (**Command**-**Enter** в Mac) на клавиатуре, когда эта ячейка *активна* (ячейка исходного кода является активной, если вы видите, что курсор мигает внутри ячейки). Существуют также опции меню для исполнения исходного кода в ячейке, но программирование становится проще и быстрее при использовании сочетаний клавиш.

Найдите минутку, чтобы обследовать арифметику. Для группировки можно использовать разные числа, круглые скобки и различные операции, такие как `-`, `/` и `*`. Также обратите внимание, что интервал не влияет на результат: `2*3` – это то же самое, что и `2 * 3`. (Интервал важен для других аспектов программирования Python, и мы вернемся к этому позже.)

Работа с отдельными числами в приложениях не масштабируется. Именно поэтому мы пользуемся переменными. Переменная – это имя, которое ссылается на данные, хранящиеся в памяти. Это аналогично использованию

слов в естественных языках для обозначения объектов в реальном мире. Например, меня зовут Майк, и я человек, состоящий из триллионов клеток, которые каким-то образом способны ходить, говорить, есть, мечтать, рассказывать плохие шутки и делать множество других вещей. Но это слишком сложно объяснить, поэтому для удобства люди называют меня «Майк Икс Коэн». Итак, переменная в Python – это просто удобная ссылка на хранимые данные, такие как число, изображение, база данных и т. д.

Мы создаем переменные в Python, присваивая им значение. Наберите следующее:

```
var1 = 10
var2 = 20.4
var3 = 'привет, меня зовут Майк'
```

Выполнение этой ячейки создаст переменные. Теперь вы можете начать их использовать! Например, в новой ячейке исполните следующий ниже фрагмент исходного кода:

```
var1 + var2
```

```
>> 30.4
```


Результаты

Знак >>, который вы видите в блоках исходного кода, является результатом исполнения ячейки исходного кода. Последующий текст – это то, что вы видите на экране, когда вычисляете исходный код в ячейке.

Теперь попробуйте следующее:

```
var1 + var3
```

А-а-а, вы только что получили свою первую ошибку Python! Добро пожаловать в клуб :) Не волнуйтесь, ошибки программирования очень распространены. На самом деле разница между хорошим программистом и плохим состоит в том, что хорошие программисты учатся на своих ошибках, тогда как плохие программисты думают, что хорошие программисты никогда не ошибаются.

Ошибки в Python бывает трудно понять. Ниже приведено сообщение об ошибке на моем экране:

```
TypeError                       Traceback (most recent call last)
<ipython-input-3-79613d4a2a16> in <module>()
      3 var3 = 'hello, my name is Mike'
      4
----> 5 var1 + var3
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Python указывает стрелкой ошибочную строку. Сообщение об ошибке, которое, как мы надеемся, поможет нам понять, что именно пошло не так и как это исправить, напечатано внизу. В этом случае сообщением об ошибке является TypeError (Ошибка типа). Что это значит и что такое «типы»?

Типы данных

Оказывается, у переменных есть типы, описывающие тип данных, которые эти переменные хранят. Наличие разных типов делает вычисления эффективнее, поскольку операции работают по-разному с разными типами данных.

В Python существует много типов данных. Здесь я представлю четыре из них, и по мере чтения книги вы узнаете больше о других типах данных:

Целочисленный
: Они называются `int` и представляют собой целые числа, такие как –3, 0, 10 и 1234.

Вещественный
: Они называются `float`, но это всего лишь причудливый термин для чисел с (плавающей) десятичной точкой, таких как –3.1, 0.12345 и 12.34. Имейте в виду, что числа с плавающей точкой и целые могут выглядеть одинаково для нас, людей, но функции Python обрабатывают их по-разному. Например, 3 – это целое число, а 3.0 – это число с плавающей точкой.

Строковый
: Они называются `str` и являются текстом. Здесь также помните о различии между `5` (строковым значением, соответствующим символу 5) и `5` (`int`, соответствующим числу 5).

Списковый
: Список – это коллекция элементов, каждый из которых может иметь разный тип данных.

Списки очень удобны и повсеместно используются в программировании на Python. Следующий ниже фрагмент исходного кода иллюстрирует три важные особенности списков: (1) они обозначаются квадратными скобками [ ], (2) запятые отделяют элементы списка и (3) отдельные элементы списка могут иметь разные типы данных:

```
list1 = [ 1,2,3 ]
list2 = [ 'hello', 123.2, [3, 'qwerty'] ]
```

Второй список показывает, что списки могут содержать другие списки. Иными словами, третий элемент списка `list2` сам по себе является списком.

Что делать, если вы хотите обратиться только ко второму элементу списка `list2`? Отдельные элементы списка извлекаются с помощью индексации, о которой я расскажу в следующем разделе.

Тип данных определяется с помощью функции `type`. Например, вычислите следующее в новой ячейке:

```
type(var1)
```

Эй, подождите, а что такое «функция»? Вы, возможно, с нетерпением хотите узнать о применении и создании функций, но сначала я хотел бы обратиться к индексации.



Как называть свои переменные?

Есть несколько жестких правил именования переменных. Имена переменных нельзя начинать с цифр (хотя они могут содержать цифры), а также нельзя включать пробелы или не буквенно-цифровые символы, такие как `!@#$%^&*()`. Символы подчеркивания `_` разрешены.

Существуют также рекомендации по именованию переменных. Самое важное правило – делать имена переменных осмысленными и интерпретируемыми. Например, имя `rawDataMatrix` гораздо лучше, чем `q`. В своем исходном коде можно создавать десятки переменных, и совсем не помешает возможность понимать данные, на которые переменная ссылается, по ее имени.

Индексация

Индексация означает доступ к определенному элементу в списке (и связанным типам данных, включая векторы и матрицы). Вот как извлекается второй элемент списка:

```
aList = [ 19,3,4 ]
aList[1]

>> 3
```

Обратите внимание, что индексация выполняется с использованием квадратных скобок после имени переменной, а затем числа, которое вы хотите индексировать.

Но постойте: я написал, что нам нужен второй элемент; почему исходный код обращается к элементу 1? Это не опечатка! В языке Python индексация начинается с 0, то есть индекс 0 – это первый элемент (в данном случае число 19), индекс 1 – второй элемент и т. д.

Если вы новичок в языках программирования, в которых применяется отсчет от 0, то, возможно, это покажется странным и запутанным. Полностью сочувствую. Хотел бы написать, что это станет второй натурой после некоторой практики, но правда в том, что индексация с отсчетом от 0 всегда будет источником путаницы и ошибок. Это просто то, о чем нужно всегда помнить.

Как обратиться к числу 4 в списке `aList`? Его можно индексировать напрямую как `aList[2]`. Но индексация Python имеет удобную функцию, благодаря которой можно индексировать элементы списка в обратном порядке. Для того чтобы обратиться к последнему элементу списка, наберите `aList[-1]`. `-1` можно трактовать как отсчет по кругу от конца списка. Аналогичным образом предпоследним элементом списка будет `aList[-2]` и т. д.

Функции

Функция – это фрагмент исходного кода, который можно выполнять без необходимости снова и снова набирать все отдельные его элементы. Некото-

рые функции – короткие и состоят всего из нескольких строк кода, тогда как другие состоят из сотен или тысяч строк кода.

Функции обозначаются в Python с помощью круглых скобок сразу после имени функции. Вот несколько общих функций:

```
type()  # возвращает тип данных
print() # печатает текстовую информацию в блокнот
sum()   # складывает числа
```

Комментарии

Комментарии – это куски кода, которые Python игнорирует. Комментарии помогают вам и другим интерпретировать и понимать исходный код. Комментарии в Python обозначаются символом #. Любой текст после # игнорируется. Комментарии могут находиться в отдельных строках либо справа от фрагмента исходного кода.

Функции могут принимать входные данные и могут предоставлять выходные данные. Общая анатомия функции Python выглядит так:

```
output1, output2 = functionname(input1, input2, input3)
```

Возвращаясь к предыдущим функциям:

```
dtype = type(var1)
print(var1+var2)
total = sum([1,3,5,4])

>> 30.4
```

`print()` – очень полезная функция. Python выводит результат только последней строки в ячейке и только тогда, когда эта строка не содержит присваивание значения переменной. Например, напишите следующий ниже фрагмент исходного кода:

```
var1+var2
total = var1+var2
print(var1+var2)
newvar = 10

>> 30.4
```

Приведенный выше исходный код состоит из четырех строк, поэтому можно было ожидать, что Python выдаст четыре результата. Но выдается только один результат, который соответствует функции `print()`. Первые две строки не выводят свой результат, потому что они не являются последней строкой, а последняя строка – потому что это присваивание значения переменной.

Методы в качестве функций

Метод – это функция, которая вызывается непосредственно на переменной. Различные типы данных имеют разные методы, а это означает, что метод, работающий на списках, не будет работать на строковых литералах.

Например, списковый тип данных имеет метод `append`, который добавляет дополнительный элемент в существующий список. Вот пример:

```
aSmallList = [ 'one','more' ]
print(aSmallList)

aSmallList.append( 'time' )
print(aSmallList)

>> ['one','more']
['one','more','time']
```

Обратите внимание на форматирование синтаксиса: методы похожи на функции тем, что они имеют круглые скобки, и (у некоторых методов) есть входные аргументы. Но методы присоединяются к имени переменной точкой и могут изменять переменную напрямую без выдачи явного результата.

Потратьте немного времени, чтобы изменить исходный код, используя другой тип данных, например строковое значение вместо списка. Повторное выполнение исходного кода приведет к следующему ниже сообщению об ошибке:

```
AttributeError: 'str' object has no attribute 'append'
```

Это сообщение об ошибке означает, что строковый тип данных распознает функцию `append` (*атрибут* – это свойство переменной, а метод – один из таких атрибутов).

Методы являются стержневой частью объектно-ориентированного программирования и классов. Этим аспектам Python можно было бы посвятить отдельную книгу по Python, но не беспокойтесь – для того чтобы изучать линейную алгебру с помощью этой книги, полное понимание объектно-ориентированного программирования вам не потребуется.

Написание своих собственных функций

В Python имеется много функций. Слишком много, чтобы сосчитать. Но никогда не найдется той идеальной функции, которая делает именно то, что вам нужно. И в итоге вам придется писать свои собственные функции.

Создавать собственные функции легко и удобно; для определения функции используется встроенное ключевое слово `def` (ключевое слово – это зарезервированное имя, которое нельзя переопределять как переменную или

функцию), затем указывается имя функции и любые возможные входные данные, и строка заканчивается двоеточием.

Любые строки кода после этого включаются в функцию, если они отделены двумя пробелами[1]. Python очень требователен к отступам в начале строки (но не к отступам в других частях строки). Любые результаты обозначаются ключевым словом `return`.

Начнем с простого примера:

```
def add2numbers(n1, n2):
  total = n1+n2
  print(total)
  return total
```

Эта функция принимает два входных аргумента и вычисляет, печатает и выводит их сумму. Теперь пришло время вызвать функцию:

```
s = add2numbers(4,5)
print(s)

>> 9
9
```

Почему число 9 появилось дважды? Оно было напечатано один раз, потому что функция `print()` была вызвана внутри функции, а затем оно было напечатано во второй раз, когда я вызвал `print(s)` после функции. Для того чтобы в этом убедиться, попробуйте поменять строку после вызова функции на `print(s+1)`. (Видоизменение исходного кода, чтобы увидеть результат на выходе, – отличный способ изучить Python; только не забывайте отменять свои изменения.)

Обратите внимание, что имя переменной, назначенное для вывода внутри функции (`total`), может отличаться от имени переменной, которое я использовал при вызове функции (`s`).

Написание конкретно-прикладных функций обеспечивает бóльшую гибкость, например установка дополнительных входных данных и используемых по умолчанию параметров, проверка входных данных на тип данных и согласованность и т. д. Но в этой книге базового понимания функций будет достаточно.

Когда писать функции?

Если у вас есть строки исходного кода, которые нужно выполнять десятки, сотни или, может быть, миллиарды раз, то написание функции – это, безусловно, правильный путь. Некоторым людям действительно нравится писать функции, и они будут писать специальные функции, даже если они вызываются только один раз.

Я предпочитаю создавать функцию только тогда, когда она будет вызываться несколько раз в разных контекстах или частях исходного кода. Но вы являетесь хозяином своего собственного исходного кода, и по мере накопления опыта программирования вы сможете выбирать ситуации, когда помещать исходный код в функции.

[1] В некоторых IDE допускается два или четыре пробела; в других принимается только четыре пробела. Я считаю, что два пробела выглядят чище.

Библиотеки

Python сконструирован таким образом, чтобы его можно было легко и быстро устанавливать и выполнять. Но недостатком является то, что базовый пакет Python поставляется с малым числом встроенных функций.

Поэтому разработчики создают коллекции функций, ориентированных на определенную тему, и такие коллекции называются библиотеками. После того как вы импортируете библиотеку в Python, вы можете обращаться ко всем функциям, типам переменных и методам, доступным в этой библиотеке.

Согласно результатам поиска в Google, существует более 130 000 библиотек Python. Не волнуйтесь, вам не нужно запоминать их все! В этой книге мы будем использовать лишь несколько библиотек, предназначенных для числовой обработки и визуализации данных. Самая важная библиотека для линейной алгебры называется NumPy; ее название состоит из комбинации слов numerical Python (численный Python).

Библиотеки Python не входят в базовую установку Python, а это значит, что вам нужно скачать их из интернета, а затем импортировать в Python. За счет этого обеспечивается их доступность для использования внутри Python. Их нужно скачивать только один раз, но импортировать их придется повторно в каждом сеансе Python[1].

NumPy

Для того чтобы импортировать библиотеку NumPy в Python, наберите:

```
import numpy as np
```

Обратите внимание на общую формулировку импорта библиотек: `import libraryname as abbreviation` (импортировать имя библиотеки как аббревиатуру). Аббревиатура является удобным сокращением. Для получения доступа к функциям в библиотеке пишется сокращенное имя библиотеки, точка и имя функции. Например:

```
average = np.mean([1,2,3])
sorted1 = np.sort([2,1,3])
theRank = np.linalg.matrix_rank([[1,2],[1,3]])
```

Третья строка исходного кода показывает, что библиотеки могут иметь вложенные в них подбиблиотеки или модули. В данном случае у NumPy есть много функций, а внутри NumPy есть библиотека под названием `linalg`, которая содержит свои функции, специально связанные с линейной алгеброй.

NumPy имеет собственный тип данных под названием *массив NumPy*. Массивы NumPy изначально кажутся похожими на списки в том смысле, что

[1] Если вы установили Python через Anaconda или используете среду Google Colab, то скачивать какие-либо библиотеки для этой книги не нужно, но вам нужно будет их импортировать.

в них обоих хранятся коллекции информации. Но в массивах NumPy хранятся только числа, и у них есть атрибуты, удобные для математического программирования. В следующем ниже фрагменте исходного кода показано, как создавать массив NumPy:

```
vector = np.array([ 9,8,1,2 ])
```

Индексация и нарезка в NumPy

Хотел бы вернуться к теме доступа к одному элементу внутри переменной. К одному элементу массива NumPy можно обращаться с помощью индексации точно так же, как индексируется список. В следующем ниже блоке кода я использую функцию `np.arange`, чтобы создать массив целых чисел от –4 до 4. Это не опечатка в коде – второй аргумент равен +5, но возвращаемые значения заканчиваются на 4. В Python часто используются *эксклительные* верхние границы; это означает, что Python считает до указанного вами конечного числа, но *не* включает его:

```
ary = np.arange(-4,5)
print(ary)
print(ary[5])

>> [-4 -3 -2 -1  0  1  2  3  4]
1
```

Это все хорошо, но что, если вы хотите получить доступ, например, к первым трем элементам? Или каждому второму элементу? Тут самое время перейти от *индексации* к *нарезке*.

Нарезка работает просто: надо указать начальный и конечный индексы с двоеточием между ними. Просто помните, что диапазоны Python имеют эксклительные верхние границы. Таким образом, для того чтобы получить первые три элемента массива, мы нарезаем до индекса 3 + 1 = 4, но тогда нам нужно учитывать индексацию на основе 0, то есть первые три элемента имеют индексы 0, 1 и 2, и мы нарезаем их, используя `0:3`:

```
ary[0:3]

>> array([-4, -3, -2])
```

Каждый второй элемент проиндексируется с помощью оператора пропуска:

```
ary[0:5:2]

>> array([-4, -2, 0])
```

Формулировка индексации с пропуском такова: `[начало:конец:пропуск]`. Весь массив можно просматривать в обратном порядке, указывая пропуск, равный –1, например вот так: `ary[::-1]`.

Знаю, это немного сбивает с толку, но обещаю: на практике все будет легче.

Визуализация

Многие концепции линейной алгебры и большинства других областей математики лучше всего понять, увидев их на экране компьютера.

Большинство визуализаций данных на Python обрабатываются библиотекой matplotlib. Некоторые аспекты графических отображений зависят от интерактивной среды разработки. Однако весь исходный код в этой книге работает как есть в любой среде Jupyter (посредством Google Colab, другого облачного сервера или локально). Если вы используете другую интерактивную среду разработки, то вам может потребоваться внести несколько незначительных изменений.

Вводить `matplotlib.pyplot` становится очень утомительно, поэтому название этой библиотеки часто сокращают до `plt`. Вы увидите это в следующем ниже блоке кода.

Начнем с рисования точек и линий. Посмотрите, сможете ли вы понять, как следующий ниже фрагмент исходного кода отображается на рис. 16.2:

```
import matplotlib.pyplot as plt
import numpy as np

plt.plot(1,2,'ko')          # 1) начерить черный кружок
plt.plot([0,2],[0,4],'r--') # 2) начерить линию
plt.xlim([-4,4])            # 3) задать пределы оси x
plt.ylim([-4,4])            # 4) задать пределы оси y
plt.title('Имя графика')    # 5) указать имя графика
```

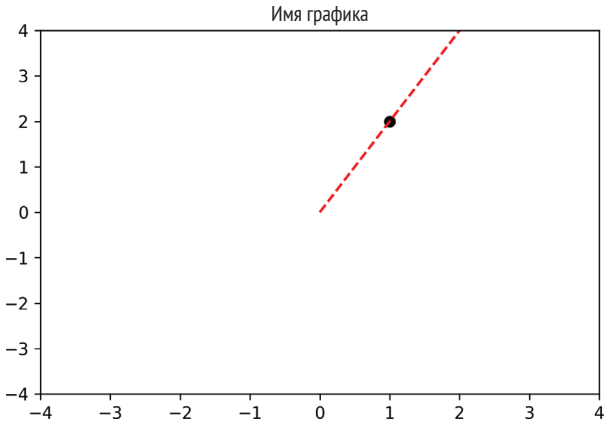


Рис. 16.2 ❖ Визуализация данных, часть 1

Удалось расшифровать исходный код? Строка кода № 1 говорит, что нужно нарисовать черный кружок (`ko` – `k` означает черный, а `o` – кружок) в местопо-

ложении xy `1,2`. Строка кода № 2 предоставляет списки чисел вместо отдельных чисел. В ней определяется линия, которая начинается в координате *xy* (0, 0) и заканчивается в координате (2, 4). `r--` обозначает красную пунктирную линию. Строки кода № 3 и № 4 устанавливают ограничения по осям *x* и *y*, и, конечно же, строка № 5 создает заголовок.

Прежде чем двигаться дальше, уделите немного времени обследованию этого исходного кода. Нарисуйте дополнительные точки и линии, попробуйте разные маркеры (подсказка: попробуйте буквы `o`, `c` и `p`) и разные цвета (попробуйте `r`, `k`, `b`, `y`, `g` и `m`).

В следующем ниже блоке кода представлены графики и изображения. Подграфик – это способ разделения графической области (именуемой figure, то есть рисунком) на решетку отдельных осей, по которым можно рисовать разные визуализации. Как и в случае с предыдущим блоком кода, прежде чем читать мое описание, проверьте свое понимание процедуры создания рис. 16.3 приведенным ниже фрагментом исходного кода:

```
_,axs = plt.subplots(1,2,figsize=(8,5)) # 1) создать подграфики
axs[0].plot(np.random.randn(10,5))      # 2) линейный график слева
axs[1].imshow(np.random.randn(10,5))    # 3) изображение справа
```

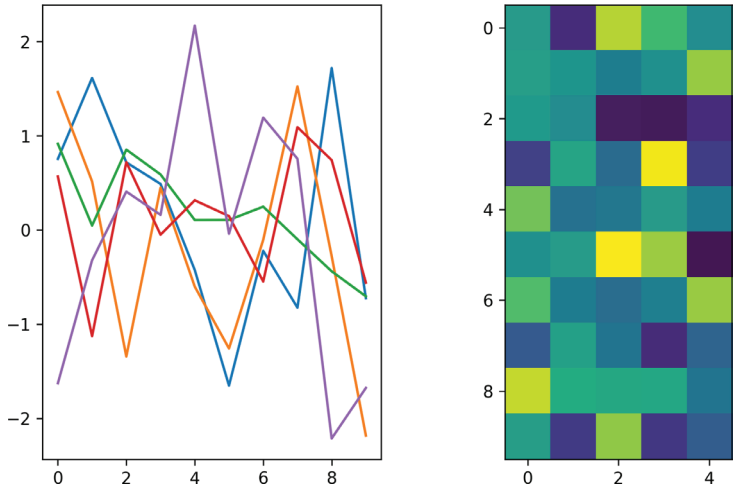


Рис. 16.3 ❖ Визуализация данных, часть 2

Строка кода № 1 создает подграфики. Первые два аргумента функции `plt.subplots` определяют геометрию решетки – в данном случае это матрица подграфиков 1×2, то есть одна строка и два столбца, что означает два графика бок о бок. Первый аргумент задает общий размер рисунка, причем два элемента в этом кортеже соответствуют ширине, а затем высоте в дюймах. (Размеры всегда указываются как ширина, высота. Мнемоника для запоминания порядка – WH как в «White House».) Функция `plt.subplots` предоставляет два результата. Первый – это дескриптор всего рисунка, который нам не нужен, поэтому вместо имени переменной я использовал подчеркивание. Второй

результат – это массив NumPy, содержащий дескрипторы каждой оси. Дескриптор – это особый тип переменной, которая указывает на объект на рисунке.

Теперь о строке кода № 2. Это должно быть знакомо по предыдущему блоку кода; два новых понятия – построение графика на определенной оси вместо всего рисунка (с использованием `plt.`). И передача на вход матрицы вместо отдельных чисел. Python создает отдельную строку для каждого столбца матрицы, поэтому на рис. 16.3 вы видите пять линий.

Наконец, строка кода № 3 показывает, как создавать изображение. Как вы узнали из главы 5, матрицы нередко визуализируются как изображения. Цвет каждого маленького блока в изображении соотносится с числовым значением в матрице.

Что ж, о создании графики на Python можно было бы сказать гораздо больше. Но я надеюсь, что этого введения будет достаточно, чтобы начать работу.

Переложение формул в исходный код

Переложение математических уравнений в исходный код Python иногда не представляет никаких трудностей, а иногда сопряжено со сложностями. Но это важный навык, и вы улучшите его с практикой. Начнем с простого примера в уравнении 16.1.

Уравнение 16.1. Уравнение

$y = x^2$.

Возможно, вы подумаете, что следующий ниже фрагмент исходного кода будет работать:

```
y = x**2
```

Но вы получите сообщение об ошибке (`NameError: name x is not defined`). Проблема в том, что мы пытаемся использовать переменную `x` до ее определения. Так как же определять `x`? На самом деле, когда вы смотрите на математическое уравнение, вы определяете `x`, даже не задумываясь об этом: `x` начинается в отрицательной бесконечности и уходит в положительную бесконечность. Но вы не очерчиваете функцию так далеко – для построения этой функции вы, вероятно, выбрали бы ограниченный диапазон, возможно от –4 до +4. И этот диапазон – именно то, что необходимо указать на Python:

```
x = np.arange(-4,5)
y = x**2
```

На рис. 16.4 показан график функции, созданный с использованием `plt.plot(x,y,'s-')`.

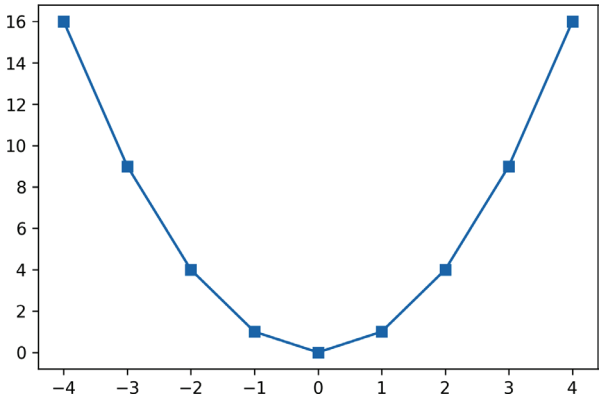


Рис. 16.4 ❖ Визуализация данных, часть 3

Выглядит неплохо, но я думаю, что он получился слишком прерывистым; хотелось бы, чтобы линия выглядела глаже. Этого можно добиться, увеличив разрешающую способность, то есть увеличив число точек в диапазоне от −4 до +4. Я буду использовать функцию `np.linspace()`, которая принимает три аргумента: начальное значение, конечное значение и число промежуточных точек:

```
x = np.linspace(-4,4,42)
y = x**2
plt.plot(x,y,'s-')
```

Теперь у нас 42 точки, расположенные линейно (равномерно) между −4 и +4. В результате график делается более гладким (рис. 16.5). Обратите внимание, что функция `np.linspace` выводит вектор, оканчивающийся на +4. Эта функция имеет инклюзивные границы. Немного дезориентирует, что приходится запоминать, какие функции имеют инклюзивные, а какие – эксклюзивные границы. Не волнуйтесь, вы с этим справитесь.

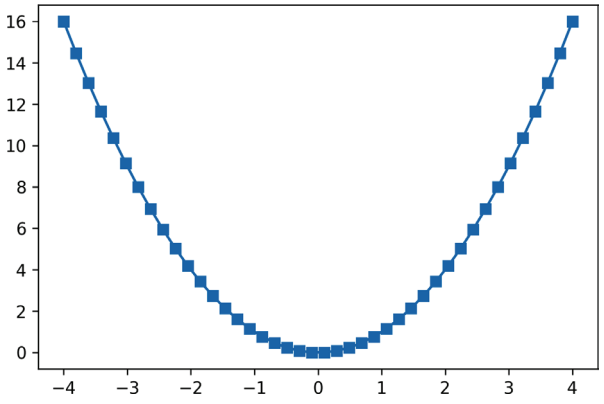


Рис. 16.5 ❖ Визуализация данных, часть 4

Давайте попробуем еще один перевод функции в исходный код. Я также воспользуюсь этой возможностью, чтобы познакомить вас с концепцией, именуемой *мягким программированием*, которая означает создание переменных для параметров, которые вы, возможно, захотите изменить позже.

Пожалуйста, перед тем как смотреть на мой следующий ниже фрагмент исходного кода, переведите следующую математическую функцию в исходный код и создайте график:

$$f(x) = \frac{\alpha}{1 + e^{-\beta x}};$$

$$\alpha = 1.4;$$

$$\beta = 2.$$

Эта функция называется *сигмоидой* и часто используется в прикладной математике, например как нелинейная активационная функция в моделях глубокого обучения. α и β – это параметры уравнения. Здесь я установил для них определенные значения. Но после того, как ваш исходный код заработает, вы сможете обследовать влияние изменения этих параметров на результирующий график. На самом деле использование исходного кода для понимания математики – это, ИМХО[1], самый лучший способ изучения математики.

Эту функцию можно запрограммировать двумя способами. Один из них заключается в помещении числовых значений α и β непосредственно в функцию. Это пример жесткого программирования, поскольку значения параметров реализуются в функции напрямую.

Альтернативой является установка переменных Python равными этим двум параметрам, а затем использование этих параметров при создании математической функции. Это и есть мягкое программирование, и оно упрощает чтение, видоизменение и отладку исходного кода:

```
x = np.linspace(-4,4,42)
alpha = 1.4
beta  = 2

num = alpha               # числитель
den = 1 + np.exp(-beta*x) # знаменатель
fx  = num / den
plt.plot(x,fx,'s-')
```

Обратите внимание, что я разделил создание функции на три строки кода, которые определяют числитель и знаменатель, а затем их отношение. Это делает исходный код чище, и его легче читать. Всегда стремитесь к тому, чтобы исходный код легко читался, потому что это (1) снижает риск ошибок и (2) облегчает отладку.

---

[1] Мне сказали, что ИМХО – это тысячелетний жаргонизм от англ. *in my humble opinion* (по моему скромному мнению).

На рис. 16.6 показана результирующая сигмоида. Потратьте несколько минут, чтобы поэкспериментировать с исходным кодом: измените пределы и разрешающую способность переменной `x`, измените значения параметров `alpha` и `beta`, возможно, даже измените саму функцию. *Математика прекрасна, Python – это ваш холст, а исходный код – ваша кисть!*

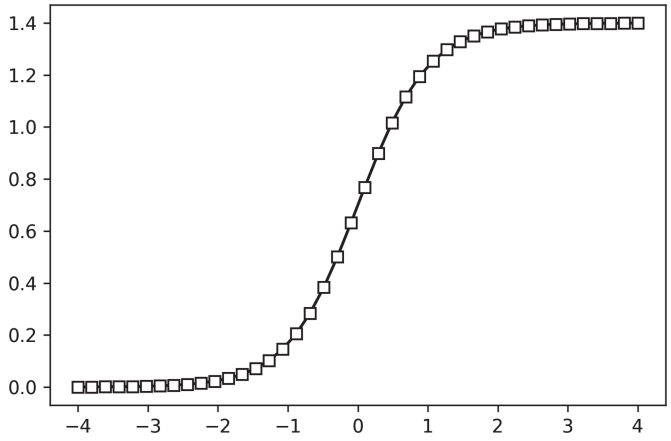


Рис. 16.6 ❖ Визуализация данных, часть 5

Форматирование печати и F-строки

Вы уже знаете, как распечатывать переменные с помощью функции `print()`. Но эта фукция применяется только для вывода одной переменной без другого текста. F-строки дают вам больше контроля над выходным форматом. Например:

```
var1 = 10.54
print(f'Значение переменной равно {var1}, и я от этого счастлив.')

>> Значение переменной равно 10.54, и я от этого счастлив.
```

Обратите внимание на две ключевые особенности f-строки: начальную букву `f` перед первой кавычкой и фигурные скобки {}, заключающие имена переменных, которые заменяются значениями переменных.

Следующий ниже блок кода еще больше подчеркивает гибкость f-строк:

```
theList = ['Майк', 7]
print(f'{theList[0]} ест {theList[1]*100} гр. шоколада каждый день.')

>> Майк ест 700 гр. шоколада каждый день.
```

Из этого примера следует усвоить два ключевых момента: (1) не волнуйтесь, на самом деле я не ем столько шоколада (ну, не каждый день хотя бы),

и (2) вы можете использовать индексацию и исходный код внутри фигурных скобок, и Python распечатает результат вычисления.

И последняя особенность форматирования f-строк:

```
pi = 22/7
print(f'{pi}, {pi:.3f}')

>> 3.142857142857143, 3.143
```

Ключевым дополнением в приведенном выше фрагменте исходного кода является форматное выражение `:.3f`, которое управляет форматированием результата. Этот фрагмент кода сообщает Python, что надо напечатать три числа после запятой. Посмотрите, что происходит, если заменить 3 на другое целое число, и что происходит, если вставить целое число перед двоеточием.

Существует много других вариантов форматирования и иных способов гибкого вывода текста, но в этой книге вам требуется лишь базовое понимание принципа работы f-строк.

Поток управления

Сила и гибкость программирования исходят из того, что вы наделяете свой исходный код способностью адаптировать свое поведение в зависимости от состояния определенных переменных или вводимых пользователем данных. Динамичность исходного кода обеспечивается инструкциями *управления потоком*.

Компараторы

Компараторы – это специальные символы, которые позволяют сравнивать разные значения. Результатом компаратора является тип данных, именуемый *булевым* типом, который принимает одно из двух значений: `True` либо `False`. Вот несколько примеров:

```
print( 4<5 )  # 1
print( 4>5 )  # 2
print( 4==5 ) # 3
```

Результаты этих строк таковы: `True` в #1 и `False` в #2 и #3.

Третье выражение содержит двойной знак равенства. Он сильно отличается от одиночного знака равенства, который, как вы уже знаете, используется для присваивания значений переменной.

Еще два компаратора таковы: `<=` (меньше или равно) и `>=` (больше или равно).

Инструкции if

Инструкции `if` интуитивно понятны, потому что они применяются все время: *если (if) я устану, то я отдохну*.

Базовая инструкция `if` состоит из трех частей: ключевого слова `if`, условного выражения и *содержимого исходного кода*. Условное выражение – это фрагмент исходного кода, который вычисляется как истина либо ложь, за которым следует двоеточие (:). Если условие истинно, то исполняется весь исходный код, расположенный ниже и с отступом; если условие ложно, то ни одна строка исходного кода с отступом не исполняется, и Python продолжит исполнение исходного кода без отступа.

Вот пример:

```
var = 4
if var==4:
  print(f'{var} равно 4!')

print("Я -снаружи цикла +for+.")

>> 4 равно 4!
Я - снаружи цикла +for+.
```

А вот еще пример:

```
var = 4
if var==5:
  print(f'{var} равно 5!')

print("Я -снаружи цикла +for+.")

>> Я - снаружи цикла +for+.
```

Первое сообщение пропускается, так как 4 не равно 5; следовательно, условное выражение ложно, и поэтому Python игнорирует весь исходный код с отступом.

Инструкции elif и else

Эти два примера показывают базовую форму инструкции `if`. Инструкции `if` могут содержать дополнительные условные конструкции для повышения сложности потока информации. Прежде чем читать мое объяснение следующего ниже фрагмента исходного кода и прежде чем набирать его на Python на своем компьютере, попытайтесь понять исходный код и предсказать, какие сообщения будут напечатаны:

```
var = 4

if var==5:
  print('var is 5') # фрагмент 1
elif var>5:
```

```
  print('var > 5') # фрагмент 2
else:
  print('var < 5') # фрагмент 3

print('За пределами if-elif-else')
```

Когда Python встречает такую инструкцию, он вычисляет сверху вниз. Таким образом, Python начнет с первого условного выражения после `if`. Если это условие истинно, то Python исполнит фрагмент 1, а затем *пропустит все последующие условия*. То есть как только Python встречает истинное условие, то исполняется исходный код с отступом, и инструкция `if` заканчивается. Истинность последующих условных выражений не имеет значения; Python не будет их проверять, ни исполнять их исходный код с отступом.

Если первое условное условие ложно, то Python перейдет к следующему условному выражению, которым является `elif` (сокращенно от «else if», то есть «если же» или «иначе если»). Опять же, Python исполнит последующий исходный код с отступом, если условие истинно, или пропустит исходный код с отступом, если условие ложно. В этом примере исходного кода показана одна инструкция `elif`, но таких инструкций может быть несколько.

В последней инструкции `else` условное выражение отсутствует. Это похоже на «план Б» инструкции `if`: он исполняется, если все предыдущие условные выражения ложны. Если хотя бы одно из условий истинно, то исходный код `else` не вычисляется.

Результат этого примера исходного кода:

```
var <5
За пределами if-elif-else
```

Несколько условий

Условные выражения можно комбинировать с помощью операторов `and` и `or`. Это кодовый аналог фразы «Если пойдет дождь *и* мне нужно прогуляться, то я возьму с собой зонт». Вот несколько примеров:

```
if 4==4 and 4<10:
  print('Пример исходного кода 1.')

if 4==5 and 4<10:
  print('Пример исходного кода 2.')

if 4==5 or 4<10:
  print('Пример исходного кода 3.')
```

```
>> Пример исходного кода 1.
Пример исходного кода 3.
```

Текст `Пример исходного кода 2` не напечатался, потому что 4 не равно 5. Однако при использовании `or` по меньшей мере одно из условных выражений истинно, поэтому был исполнен последующий исходный код.

Циклы for

Теперь ваших навыков Python достаточно, чтобы распечатать числа от 1 до 10. Для этого можно применить следующий ниже фрагмент исходного кода:

```
print(1)
print(2)
print(3)
```

И так далее. Но эта стратегия не масштабируется – что, если я попрошу вас напечатать числа до миллиона?

Повторение исходного кода в Python осуществляется с помощью *циклов*. Самый важный вид цикла называется *циклом* `for`. Для того чтобы создать цикл `for`, нужно указать итерируемый объект (*итерируемый объект* – это переменная, используемая для прокручивания всех элементов в этой переменной; в качестве итерируемого объекта может использоваться список) и затем любое число строк исходного кода, которое должно быть исполнено внутри цикла `for`. Я начну с очень простого примера, а затем мы его разовьем:

```
for i in range(0,10):
  print(i+1)
```

Выполнение этого фрагмента исходного кода выведет числа от 0 до 10. Функция `range()` создает итерируемый объект с собственным типом данных, именуемым *диапазоном*, который часто используется в циклах `for`. Переменная диапазона содержит целые числа от 0 до 9. (Внимание! Исключительная верхняя граница. Кроме того, если вы начинаете считать от 0, то первый входной аргумент не нужен, поэтому `range(10)` совпадает с `range(0,10)`) . Но мои инструкции заключались в том, чтобы печатать числа от 1 до 10, поэтому внутри функции `print` нужно добавить 1. В этом примере также показано, что переменную итерации можно использовать как обычную числовую переменную.

Циклы `for` могут прокручивать другие типы данных. Рассмотрим следующий ниже пример:

```
theList = [ 2, 'hello', np.linspace(0,1,14) ]
for item in theList:
  print(item)
```

Теперь мы прокручиваем список в цикле, и на каждой итерации цикла значение переменной `item` устанавливается равным каждому элементу в списке.

Вложенные инструкции управления

Вложение инструкций управления потоком в другие инструкции управления потоком придает исходному коду дополнительный уровень гибкости. Попытайтесь выяснить, что именно исходный код делает, и предскажите его результат. Затем наберите его на Python и проверьте свою гипотезу:

```
powers = [0]*10

for i in range(len(powers)):
  if i%2==0 and i>0:
    print(f'{i} - это четное число')

  if i>4:
    powers[i] = i**2

print(powers)
```

Я еще не рассказал вам об операторе `%`. Он называется *оператором деления по модулю*, и он возвращает остаток после деления. Таким образом, 7%3 = 1, потому что 3 входит в 7 дважды с остатком 1. Аналогичным образом 6%2 = 0, потому что 2 входит в 6 три раза с остатком 0. Фактически k%2 = 0 для всех четных чисел и k%2 = 1 для всех нечетных чисел. Таким образом, выражение типа `i%2==0` позволяет проверять четность/нечетность числовой переменной `i`.

Измерение времени вычислений

При написании и оценивании исходного кода вам нередко нужно знать продолжительность времени, которое требуется компьютеру для выполнения определенных фрагментов исходного кода. В Python есть несколько способов измерить затрачиваемое время; здесь показан один простой метод с использованием библиотеки time:

```
import time

clockStart = time.time()
# некий исходный код...
compTime = time.time() - clockStart
```

Идея состоит в запрашивании локального времени операционной системы дважды (это результат работы функции `time.time()`): один раз перед запуском некоторого фрагмента исходного кода или функций и один раз после выполнения этого фрагмента кода. Разница во времени системных часов и есть время вычислений. Результатом является прошедшее время в секундах. Часто бывает удобно умножать результат на 1000, чтобы распечатывать результаты в миллисекундах (мс).

Получение помощи и приобретение новых знаний

Уверен, вы слышали фразу «Математика – это не зрелищный вид спорта». То же самое и с программированием: единственный способ научиться програм-

мировать – это программировать. Вы будете делать много ошибок, разочаровываться, потому что не сможете понять, как сообщить Python делать то, что вам нужно, будете встречать массу ошибок и предупреждающих сообщений, которые вы не сможете расшифровать, и просто будете сильно злиться на вселенную и все, что в ней есть (да, вы знаете, о чем я говорю).

Что делать, когда дела идут наперекосяк

Давайте я вам напоследок расскажу анекдот, чтобы вас поддержать: четыре инженера садятся в машину, а машина не заводится. Инженер-механик говорит: «Вероятно, проблема с зубчатым ремнем распредвала». Инженер-химик говорит: «Нет, я думаю, проблема в газовоздушной смеси». Инженер-электрик говорит: «Мне кажется, что неисправны свечи зажигания». Наконец, инженер-программист говорит: «Давайте просто выйдем из машины и заново войдем».

Мораль этой истории такова: когда вы сталкиваетесь с какими-то необъяснимыми проблемами в своем исходном коде, вы можете попробовать перезапустить *ядро*, то есть движок, на котором работает Python. Это не исправит ошибки программирования, но может устранить ошибки, связанные с перезаписью или переименованием переменных, перегрузкой памяти или системными сбоями. В блокнотах Jupyter можно перезапускать ядро через опции меню. Имейте в виду, что перезапуск ядра очищает все переменные и настройки среды. Возможно, вам придется перезапустить исходный код с самого начала.

Если ошибка сохраняется, то поищите в интернете сообщение об ошибке, имя используемой функции или краткое описание проблемы, которую вы пытаетесь решить. Python имеет огромное международное сообщество, и существует множество онлайновых форумов, в которых люди обсуждают и решают проблемы и недоразумения, связанные с программированием на Python.

Резюме

Овладение таким языком программирования, как Python, требует многих лет упорного обучения и практики. Даже достижение хорошего начального уровня занимает от недель до месяцев. Я надеюсь, что данная глава дала вам достаточно навыков, чтобы закончить эту книгу. Но, как я писал в главе 1, если вы обнаружите, что понимаете математику, но испытываете затруднения с исходным кодом, то, возможно, вы захотите отложить эту книгу, еще немного потренироваться в Python, а затем вернуться.

С другой стороны, вы также должны рассматривать эту книгу как способ улучшить свои навыки программирования на Python. Так что если вы не понимаете какой-то фрагмент исходного кода в книге, то изучение линейной алгебры – отличный повод изучить Python побольше!

Дополнение A

Теорема о ранге и нульности

Доказательство теоремы о ранге и нульности[1] относится к изложению матричных нуль-пространств в разделе «Нуль-пространства» главы 6 данной книги. Вкратце: нуль-пространство матрицы – это множество всех векторов $\mathbf{y}$, таких что $\mathbf{Ay} = \mathbf{0}$ (за исключением тривиального случая $\mathbf{y} = \mathbf{0}$). Доказательство включает в себя демонстрацию того, что $\mathbf{A}^T\mathbf{A}$ и $\mathbf{A}$ имеют одинаковую размерность нуль-пространства, а это означает, что они должны иметь одинаковый ранг. Начнем с доказательства того, что $\mathbf{A}$ и $\mathbf{A}^T\mathbf{A}$ имеют одинаковое нуль-пространство.

$$\mathbf{Ay} = \mathbf{0}; \tag{A.1}$$

$$\mathbf{A}^T\mathbf{Ay} = \mathbf{A}^T\mathbf{0}; \tag{A.2}$$

$$\mathbf{A}^T\mathbf{Ay} = \mathbf{0}. \tag{A.3}$$

Уравнения A.1 и A.3 показывают, что любой вектор в нуль-пространстве $\mathbf{A}$ также находится в нуль-пространстве $\mathbf{A}^T\mathbf{A}$. Это доказывает, что нуль-пространство $\mathbf{A}^T\mathbf{A}$ является подмножеством нуль-пространства $\mathbf{A}$. Это половина доказательства, потому что нам также нужно продемонстрировать, что любой вектор в нуль-пространстве $\mathbf{A}^T\mathbf{A}$ также находится в нуль-пространстве $\mathbf{A}$.

$$\mathbf{A}^T\mathbf{Ay} = \mathbf{0}; \tag{A.4}$$

$$\mathbf{y}^T\mathbf{A}^T\mathbf{Ay} = \mathbf{y}^T\mathbf{0}; \tag{A.5}$$

$$(\mathbf{Ay})^T\mathbf{Ay} = \mathbf{0}; \tag{A.6}$$

$$||\mathbf{Ay}|| = \mathbf{0}. \tag{A.7}$$

Уравнения A.4 и A.7 вместе показывают, что любой вектор в нуль-пространстве $\mathbf{A}^T\mathbf{A}$ также находится в нуль-пространстве $\mathbf{A}$.

---

[1] Материал взят из учебника автора «Линейная алгебра: теория, интуиция, исходный код» (Linear Algebra. Theory, Intuition, Code, Cohen, Sincxpress BV, 2021, стр. 191). – *Прим. перев.*

Теперь мы доказали, что $\mathbf{A}^T\mathbf{A}$ и $\mathbf{A}$ имеют одинаковые нуль-пространства. Почему это имеет значение? Строчное пространство (множество всех возможных взвешенных комбинаций строк) и нуль-пространство вместе охватывают все $\mathbb{R}^N$, и поэтому если нуль-пространства одинаковы, то строчные пространства должны иметь ту же размерность (это называется теоремой ранга и нульности). А ранг матрицы – это размерность строчного пространства, следовательно, ранги $\mathbf{A}^T\mathbf{A}$ и $\mathbf{A}$ одинаковы. Доказательство этого утверждения для $\mathbf{A}\mathbf{A}^T$ подчиняется тому же доказательству, что и выше, за исключением того, что вы начинаете с $\mathbf{y}^T\mathbf{A} = \mathbf{0}$. Приглашаю вас воспроизвести это доказательство с ручкой и бумагой.

Тематический указатель

A

Anaconda, установщик, 292

C

CGI-фильмы и видеоигры, графика в них, 134
Colab, среда, 292
  работа с исходным кодом, 293

D

def, ключевое слово (Python), 299

E

elif и else, инструкции (Python), 310

F

f-строка, 308
float, тип, 296
for, цикл (Python), 312

G

Google Colab, среда, 292
  работа с исходным кодом, 293

I

IDE (интегрированная среда разработки), 22, 292
  графическое отображение на Python, 303
  отступы в функциях, 300
if, инструкция (Python), 310
  инструкции elif и else, 310
  несколько условий, 311
int, тип, 296

L

LAPACK, библиотека, 203
LIVE EVIL, мнемоника для порядка следования операций, 89
LU-разложение, 174, 183
  взаимообмен строк посредством матриц перестановок, 185
  обеспечение уникальности, 184

M

matplotlib (Python), 77, 303

N

np.argmin, функция, 70
np.diag, функция, 79
np.dot, функция, 37, 42
np.eye, функция, 79
np.min, функция, 70
np.multiply, функция, 82
np.outer, функция, 42
np.random, функция, 78
np.sum, функция, 70
np.tril, функция, 79
np.triu, функция, 79
NumPy
  диагонализация матрицы, 237
  импорт в Python, 301
  индексация и нарезка, 302
  функции полиномиальной регрессии, 218
  функция svd, 256

P

Python, 20
  векторы, 26
  вычисление RREF-формы в библиотеке sympy, 182
  интегрированные среды разработки, 22
  краткое руководство, 291
    варианты использования и альтернативы, 291
    визуализации, 303
    измерение времени вычисления, 313

интегрированные среды разработки (IDE), 292
переложение формул в исходный код, 305
переменные, 294
получение помощи и приобретение новых знаний, 313
поток управления, 309
применение Python локально и онлайн, 292
форматирование печати и f-строки, 308
функции, 297
расположенный в облаке, 293
решение задачи о наименьших квадратах
переложение левообратной матрицы в исходный код, 192
сингулярное разложение, 256
создание анимаций данных, 131
терминологические разночтения с линейной алгеброй меловой доски, 35
функция для LU-разложения в библиотеке SciPy, 184

Q

QR-разложение
полное, 166
размеры матриц Q и R, 166
решение задачи о наименьших квадратах, 201
экономное или сокращенное, 166
QR и обратные матрицы, 169
QR-разложение , 165

R

RREF (строчно приведенная ступенчатая форма), 181

S

SciPy, библиотека (Python), 184
вычисление обобщенного собственного разложения, 247
statsmodels, библиотека
применение для создания регрессионной таблицы, 212
str, тип, 296
sympy, библиотека (Python), 182

T

type, функция, 296

А

Алгебра линейная, 17
приложения в исходном коде, 20
Анализ главных компонент (PCA), 59, 229
доказательство, что собственное разложение решает цель оптимизации PCA, 271
использование собственного и сингулярного разложений, 268
математика PCA-анализа, 269
шаги выполнения PCA, 271
PCA посредством сингулярного разложения, 272
Анализ линейный дискриминантный (LDA), 273
Анализ независимых компонент, 59
Анимация данных, создание, 131
Аппроксимация низкоранговая, 259
посредством сингулярного разложения, 275

Б

Базис, 57, 58, 62
важность в науке о данных и машинном обучении, 59
Библиотека функций Python, 301
Блокнот Jupyter, 22, 292
Буквы в линейной алгебре, 41

В

Вектор, 17
базисный, 235
нуль-пространство, 235
базисный ортогональный, 255
левый сингулярный, 254
линейная независимость, 51
линейно-взвешенная комбинация, 50
методы помимо точечного произведения векторов, 40
методы умножения помимо точечного произведения
адамарово умножение, 40
внешнее произведение, 41
перекрестное и тройное произведение, 42
модуль и единичные векторы, 35

нуль-пространственный (собственные векторы), 235
операции на векторах, 28
геометрия сложения и вычитания векторов, 30
геометрия умножения вектора на скаляр, 32
операция транспонирования, 33
сложение и вычитание двух векторов, 28
транслирование векторов на Python, 34
умножение вектора на скаляр, 31
усреднение векторов, 33
ортогональный собственный
доказательство ортогональности, 239
подпространство и охват, 54
правый сингулярный, 254
применения, 64
кластеризация методом *k*-средних, 68
корреляция и косинусное сходство, 64
фильтрация временных рядов и обнаружение признаков, 67
сингулярный, ортогональная природа, 259
собственный
в нуль-пространстве матрицы, сдвинутой на ее собственное число, 232
действительно-значный
в симметричных матрицах, 240
из собственного разложения сингулярной матрицы, 241
ортогональный, 238
отыскание, 234
неопределенность собственных векторов по знаку и шкале, 235
хранение в столбцах матрицы, а не в ее строках, 234
создание и визуализация в NumPy, 24
точечное произведение, 36
геометрия, 39
дистрибутивная природа, 38
умножение левого сингулярного вектора на правый сингулярный вектор, 258
умножение матрицы на вектор, 85
Вектор единичный, 35
создание из неединичных векторов, 35
Вектор нулей, 31
линейная независимость, 53, 54
приравнивание собственного числа к нулю, 233
Вектор
ортогональный, нулевое точечное произведение, 40
сингулярный, 272, 277
собственный, 228
Взаимообмен строк, 185
Взаимообмен строками посредством матриц перестановок, 185
Визуализации на Python, 303
Визуализация матриц, 76
Время вычисления, измерение на Python, 313
Вычитание векторов, 29
в линейно-взвешенной комбинации, 50
геометрия вычитания векторов, 30

Г

Геометрия вектора, собственные векторы, 228
Геометрия векторов, 27
сложение и вычитание векторов, 30
точечное произведение, 39
умножение вектора на скаляр, 32
График крутого склона, 230

Д

Диагонализация одновременная двух матриц, 247
Диапазон, тип, 312
Дисперсия, 129
комбинируемая с помощью линейно-взвешенной комбинации в PCA, 269
конвертация сингулярных чисел, 260
Длина (векторы), 25
терминологические разночтения между математикой и Python, 35
Длина геометрическая (векторы), 35
Доказательство
математическое в противовес пониманию на уровне интуиции на основе исходного кода, 20
мягкое, 21
путем отрицания, 153

З

Зависимость линейная, 213

И

Изображение
визуализация крупных матриц в виде изображений, 76
сжатие и сингулярное разложение, 275
Индексация
в NumPy, 302
математические соглашения по сравнению с соглашениями в программировании, 37
матриц, 77
списков и связанных типов данных в Python, 297
Индекс в формате строка, столбец (элементы матрицы), 33
Инструкция управления вложенная, 312
Интерпретация геометрическая обратной матрицы, 156

К

Кадр данных pandas, 211
Квадрат ошибки между предсказанными и наблюдаемыми данными, 195
Квадраты наименьшие
обычные (OLS), 212
применения, поиск в параметрической решетке для отыскания модельных параметров, 218
Кластеризация методом *k*-средних, 68
Ковариация, 128
Комбинация линейно-взвешенная
в умножении матрицы на вектор, 86
комбинированная с дисперсией в PCA, 269
Комбинация линейно-взвешенная, 50, 62
Комментарий, 298
Компаратор (Python), 309
Компонента, 270
Компонента
параллельная (ортогональное разложение векторов), 44
перпендикулярная (ортогональное разложение векторов), 44
Константа, 175, 191
Корреляция по сравнению с косинусным сходством, 66
Коэффициент, 50, 175
коэффициент регрессии для регрессоров, 212
отыскание множества коффициентов, которое минимизирует квадраты ошибок, 196
Коэффициент корреляции, 64
нормализации, 64
Коэффициент корреляции Пирсона, 38
выраженный в терминах линейной алгебры, 65
по сравнению с косинусным сходством, 66
формула, 65
Коэффициент регрессии, 212

М

Массив
векторы в виде массивов NumPy, 26
матрица в виде двумерного массива, 88
NumPy, 301
Массив неориентированный, 26
Математика
на меловой доске в противовес имплементированной в рабочем коде, 25
отношение к обучению, 19
усвоение, математические доказательства в противовес пониманию на уровне интуиции на основе программирования, 20
Матрица, 17, 76, 91, 97
адъюгатов, 150
верхнетреугольная
преобразование плотной матрицы, с использованием приведения строк, 178
верхнетреугольные матрицы, 168, 202
верхние треугольные матрицы, 79
высокая, 79
вычисление левообратной матрицы, 151
экономное QR-разложение по сравнению с полным, 166
вычисление определителя, 117
Гилберта, обратная ей матрица и их произведение, 155
диагональная
вычисление обратной матрицы, 148
матрица сингулярных чисел, 255
хранение собственного числа, 236
диагональные матрицы, 79, 234
единичная, 79
вектор констант в первом столбце, 183
замена в обобщенном собственном разложении, 247

обратная матрица, содержащая преобразование матрицы в единичную матрицу, 144
получение посредством строчно приведенной ступенчатой формы, 181
квадратичная форма, 243
квадратная
вычисление обратной матрицы для полноранговой матрицы, 149
диагонализация, 236
для собственного разложения, 231
определенность, 245
уникальное LU-разложение полноранговой матрицы, 183
квадратные матрицы, 90
по сравнению с неквадратными, 78
ковариаций
использование собственного разложения в PCA, 271
многопараметрических данных, 128
обобщенное собственное разложение в линейном дискриминантном анализе, 273
ковариаций и корреляций, 128
матрица корреляций в примере предсказания велопрокатов, 209
конвертирование уравнений в матрицы, 175
кофакторов, 150
левообратная, 145, 151, 192
математика, 80
сдвиг матрицы, 81
сложение и вычитание матриц, 80
умножение на скаляр и адамарово умножение, 82
миноров, 149
неквадратная
наличие односторонней обратной матрицы, 145
применение обратной матрицы Мура–Пенроуза, 264
неквадратные матрицы, 90
по сравнению с квадратными матрицами, 78
необратимая, 146
нормы матриц, 97
нулей, 79
нуль-пространство, 100, 104
обратная, 144, 202
вычисление, 146
вычисление для диагональной матрицы, 148
вычисление обратной матрицы для матрицы 2×2, 146
вычисление односторонней обратной матрицы, 151
геометрическая интерпретация, 156
обратная матрица ортогональной матрицы, 163
односторонняя, 145
полная, 145
посредством метода устранения по Гауссу–Жордану, 182
псевдообратная матрица Мура–Пенроуза, 154
типы обратных матриц и условия обратимости, 145
уникальность, 153
численная стабильность обратной матрицы, 155
QR-разложение, 169
обратная Мура–Пенроуза, сингулярное разложение, 262
одноранговая, 111
операции
транспонирование, 88
LIVE EVIL, мнемоника для порядка следования операций, 89, 92
определитель, 117
ортогональная, 132, 162
матрицы перестановок, 185
матрицы сингулярных векторов, 255
ортогональные столбцы и столбцы с единичной нормой, 162
преобразование неортогональных матриц процедурой Грама–Шмидта, 164
преобразование неортогональных матриц QR-разложением, 165
особые свойства, относящиеся к собственному разложению, 238
перестановок, 164, 202
взаимообмен строк, 185
плохо обусловленная, 261
обработка, 262
поворота, 131
полноранговая
вычисление обратной матрицы для квадратной матрицы, 149
отсутствие нуль-значных собственных чисел, 242
правообратная, 145
применения, 128

геометрические трансформанты посредством умножения матриц на векторы, 131
матрицы ковариаций многопараметрических данных, 128
обнаружение признаков изображения, 135
применения ранга, 114
прямоугольные матрицы, 78
псевдообратная, 145
псевдообратная Мура–Пенроуза, 154, 213
ранг, 108
рангово-дефицитная, 146
рангово-дефицитная необратимая, 146
рангово-пониженная, 146
собственное разложение, 242
расчетная
в примере предсказания велопрокатов, 210
полиномиальной регрессии, 215
рангово-пониженная, 214
сдвиг матрицы для преобразования рангово-пониженной матрицы в полноранговую матрицу, 214
решетка, 150
свойства ранга, 108
симметричная
особые свойства, относящиеся к собственному разложению
действительно-значные собственные числа, 240
ортогональные собственные векторы, 238
положительно (полу) определенная, 246
симметричные матрицы, 89, 92
создание из несимметричных матриц, 89
сингулярная, 146
собственное разложение, 241
скалярная, 232
след матрицы, 99
случайных чисел, 78
применение LU-разложения, 185
создание и визуализация в NumPy, 76
визуализация, индексация и нарезка матриц, 76
специальные матрицы, 78
специальные матрицы, 91
стандартное умножение, 82
правила допустимости умножения матриц, 83
умножение матриц, 84
стобцовое пространство, 100
строчное пространство, 100, 104
тождественного преобразования, 79
транспонирование, 33
треугольная, нижнетреугольное – верхнетреугольное (LU) разложение, 183
треугольные матрицы, 79
чистого поворота, 131, 164
широкая, правообратная матрица для широкой полноранговой матрицы, 145
эрмитова, 257
Метод, 299
Метод аддитивный (создание симметричных матриц), 91
Метод левообратный, 201
по сравнению с решателем методом наименьших квадратов NumPy, 193
Метод мультипликативный (создание симметричных матриц), 91
Метод наименьших квадратов, 192
геометрическая перспектива, 194
почему он работает, 195
применения, 207
поиск в параметрической решетке для отыскания модельных параметров, 218
полиномиальная регрессия, 215
решение посредством QR-разложения, 201
Метод наименьших квадратов посредством QR-разложения, 201
Метод устранения по Гауссу–Жордану, 181
Многочлен характеристический, 119
Многочлен характеристический матрицы, 233
Множество базисное, 58
потребность в линейной независимости, 61
Множество векторов, 49, 62
линейная независимость, 52
Множество декартово базисное, 58
Множество стандартное базисное, 58
Модель линейная общая, 84, 190
на простом примере, 197
настройка, 190
решение, 192
геометрическая перспектива наименьших квадратов, 194

почему работает метод наименьших квадратов, 195
точность решения, 193
решение с мультиколлинеарностью, 213
терминология, 190
Модель статистическая, 190
регуляризация, 213
Модуль (векторов), 27
единичные векторы и модуль, 35
собственные векторы, 238, 244
Мультиколлинеарность, 213

Н

Наименьшие квадраты, применения, 207
предсказание количеств велопрокатов на основе погоды, 207
Направление (или угол) вектора, собственные векторы, 238
Направление (или угол) векторов, 27
Нарезка
в NumPy, 302
матрицы, 77
Независимость линейная, 51, 62
гарантия уникальности в базисных множествах, 61
ее определение на практике, 52
математика, 53
независимость и вектор нулей, 54
Независимость нелинейная, 54
Неопределенность собственных векторов по знаку и шкале, 235
Норма (вектора), 35
соотношение двух норм, 270
Норма матрицы, 97
индуцированная, 98
поэлементная, 98
уравнение Фробениуса, 98
Норма Фробениуса, 214
Нормализация для коэффициента корреляции, 64
Нуль-пространство, 104

О

Обнаружение признаков, а также фильтрация временных рядов, 67
Обнаружение признаков изображения, 135
Объект итерируемый, 312
Оператор умножения в Python (*), 40
умножение матриц, 82
Операция транспонирования, 26, 33
на матрицах, 88
порядок следования операций с матрицами, правило LIVE EVIL, 89
симметричная матрица, эквивалентная результату своего транспонирования, 90
транспонирование левой матрицы в матрицу ковариаций, 130
Определенность, 245
положительная (полу) определенность, 245
Определенность положительная, 245
Определитель, 117
вычисление, 117
приравнивание к нулю определителя матрицы, сдвинутой на ее собственное число, 232
свойства, 117
с линейными зависимостями, 119
Ориентация
вдоль столбца (векторы), 24
вдоль строки (векторы), 24
Ориентация (векторов)
в сложении векторов, 30
Ориентация (векторы), 24
Освоение языка Python (Лутц), 291
Отражение Хаусхолдера, 164
Очертание (векторы в Python), 25
Ошибка
в Python, 295
квадраты ошибок между предсказанными и наблюдаемыми данными, 195

П

Переменная зависимая, 190, 198
в примере предсказания велопрокатов, 208
Переменная независимая, 190, 198
в примере предсказания велопрокатов, 208
Переменная разряженная, 210
Переменная (Python), 294
именование, 297
индексация, 297
типы данных, 296
Пересечение или сдвиг, 175, 191, 200
добавление члена пересечения в наблюдаемые и предсказанные данные, 200

Подавление шума, 230
Подматрица, 77
Подпространство и охват (вектор), 54, 62
  базис, 60
Подстановка обратная, 181, 182, 202
Поиск в параметрической решетке для отыскания модельных параметров, 218
Положение стандартное (векторы), 27
(Полу)определенность
положительная, 245
Поток управления в Python, 309
  вложенные инструкции управления, 312
  инструкции if, 310
    инструкции elif и else, 310
    несколько условий, 311
  компараторы, 309
Предсказание количеств велопрокатов на основе погоды (пример), 207
  мультиколлинеарность, 213
  предсказанные данные по сравнению с наблюдаемыми данными, 211
  регуляризация матрицы, 213
  создание регрессионной таблицы с помощью библиотеки statsmodels, 212
Преобразование
  Хаусхолдера, 164
  откат геометрического преобразования с помощью обратной матрицы, 156
Преподаватель, как эту книгу использовать, 23
Приведение строк, 178
  метод устранения по Гауссу–Жордану, 181
  обратная матрица посредством метода устранения по Гауссу–Жордану, 182
  цель, 178
Примеры исходного кода из книги, 22
Программирование
  жесткое, 307
  линейно-алгебраические приложения в рабочем коде, 20
  мягкое, 307
  понимание на уровне интуиции на основе исходного кода в противовес математическим доказательствам, 20
Проекция
  ортогональная, 44
  ортогональная вектора, 194, 195
Проецирование размерности данных, 230
Произведение
  внешнее, 41
    в умножении матрицы на вектор, 88
    по сравнению с транслированием, 41
  перекрестное, 42
  точечное, 36
    в коэффициентах корреляции, 65
    в умножении матрицы на вектор, 88
    в фильтрации временных рядов, 67
    геометрия, 39
    деление на произведение векторных норм, 65
    дистрибутивная природа, 38
    для ортогональных матриц, 163
    корреляция Пирсона и косинусное сходство на основе точечного произведения, 66
    нулевое точечное произведение ортогональных векторов, 40
    обозначение по сравнению с внешним произведением, 41
  тройное, 42
Пространство
  столбцовое, 100
  строчное, 104
Процедура Грама–Шмидта, алгоритм, 164

P

Разложение, 162
  векторов ортогональное, 31, 42
  на простые множители, 42
  сингулярное, 254
    вычисление из собственного разложения, 259
      сингулярное разложение матрицы AtA, 260
    ключевые моменты, 263
    на Python, 256
    низкоранговые аппроксимации, 275
    обратная матрица Мура–Пенроуза, 262
    общая картина, 254
      важные признаки, 255
      сингулярные числа и ранг матрицы, 256
    одноранговые слои матрицы, 257
    применение для шумоподавления, 276
    применения собственного и сингулярного разложений, 268
      анализ главных компонент посредством сингулярного разложения, 272
  собственное, 227

бесчисленное число тонкостей, 247
вычисление для матрицы, помноженной на ее транспонированную версию, 259
диагонализация квадратной матрицы, 236
интерпретации собственных чисел и собственных векторов, 228
анализ главных компонент в статистике, 229
подавление шума, 230
уменьшение размерности (сжатие данных), 231
квадратичная форма, определенность и собственные числа, 245
обобщенное, 246
на матрицах ковариаций в линейном дискриминантном анализе, 273
особые свойства симметричных матриц, 238
отыскание собственных векторов, 234
отыскание собственных чисел, 231
применения собственного и сингулярного разложений, 268
анализ главных компонент, 268
линейный дискриминантный анализ, 273
разные интерпретации, 228
сингулярное разложение, 254
сингулярных матриц, 241
Холецкого, 246
Размерность (векторы), 24
в сложении векторов, 29
в точечном произведении, 37
геометрические измерения в Python, 25
Ранг
матрицы, 108
применения, 114
свойства ранга матрицы, 108
сдвинутых матриц, 113
сложенных и умноженных матриц, 112
специальных матриц, 110
Ранг (матриц)
из собственного разложения сингулярной матрицы, 242
Ранг (матрицы)
сингулярное разложение и одноранговые слои матрицы, 257
сингулярные числа, 256
Расстояние, 69
Расстояние евклидово, 68, 69
Растягивание собственного вектора матрицы, 228
Регрессия
полиномиальная, 215
доля регуляризации, 214
создание регрессионной таблицы с помощью библиотеки statsmodels, 212
Регрессор, 210
Регуляризация, 213
Регуляризация L2, 98
Результат геометрического преобразования, 86
Результат преобразования
геометрические результаты преобразования в умножении матрицы на вектор, 86
Решение с минимальной нормой (min-norm), 213
Решение тривиальное, 31

С

Самообучение, как эту книгу использовать, 23
Свертка, 68
Свертка (изображение), 137
Свойство ранга матрицы, 108
Сдвиг, 175
Сдвиг матрицы, 81, 91, 214
на ее собственное число, 232
Сжатие данных, 231, 275
Сигмоида, функция, 307
Симуляция методом Монте-Карло, 246
Система уравнений
конвертирование уравнений в матрицы, 175
решение посредством обратной матрицы, 182
решение с помощью метода устранения по Гауссу–Жордану, 182
Скаляр
обратное значение, 146
отрицательный, изменение направления векторов, 32
сложение скаляра с вектором, 32
сложение скаляра с вектором и умножение скаляра на вектор, 50
умножение вектора на скаляр, 31
След матрицы, 99
Слово ключевое в Python, 299
Сложение векторов, 28

в точечном произведении векторов, 37
геометрия сложения векторов, 30
сложение скаляра с вектором, 32
сложение скаляра с вектором и умножение скаляра на вектор, 50
Соотнесенность между векторами, 38
Список, 296
индексация, 297
представление векторов на Python, 26
умножение списка на скаляр, 31
Среда разработки интегрированная (IDE), 292
Среда Colab, использование в ней блокнотов Jupyter, 22
Среда Google Colab, 22
Стабильность численная обратной матрицы, 155
Статистика
анализ главных компонент, 229
дисперсия, 129
корреляция, 64
общая линейная модель, 84
решение обратных задач, 43
сдвиг матрицы, 81
Столбец с единичной нормой (ортогональные матрицы), 162
Сходство косинусное, 66
Сходство между векторами, 38

Т

Тензор, 137
Тип булев, 309
Тип данных
в Python, 296
переменных, в которых хранятся векторы, 31
представление векторов на Python, 26
Транслирование, 69
по сравнению с внешним произведением, 41
транслирование векторов на Python, 34
Трансформанта геометрическая, применение умножения матрицы на вектор, 131
Требования предварительные, 19

У

Угол (или направление) векторов, 27
Уменьшение размерности, 231
Умножение
адамарово, 40
матриц, 82
вектора на скаляр, 31
геометрия умножения вектора на скаляр, 32
матриц, 91
матрицы на вектор, 85
геометрические трансформанты, 131
линейно-взвешенные комбинации, 86
результаты геометрических преобразований, 86
матрицы на скаляр, 82
методы помимо точечного
произведения векторов, 40
адамарово умножение, 40
внешнее произведение, 41
перекрестное и тройное произведения, 42
сложение скаляра с вектором и умножение скаляра на вектор, 50
точечное произведение векторов, 36
умножение матриц и точечное произведение векторов, 37
умножение матрицы на скаляр и адамарово умножение матрицы, 82
Уникальность
гарантирование со стороны линейной независимости, 61
обратной матрицы, 153
QR-разложения, 168
Упражнения по программированию, 22, 46, 62, 71, 92
Уравнение
матричное, 144
для матрицы ковариаций, 129
работа с матричными уравнениями, 176
по сравнению со скалярным уравнением, 176
переложение формул в исходный код, 305
системы уравнений, 174, 181
конвертирование уравнений в матрицы, 175
работа с матричными уравнениями, 176
скалярное по сравнению с матричным уравнением, 176
Усреднение векторов, 33

Устранение по Гауссу–Жордану, 181, 204
обратная матрица, 182

Ф

Фильтрация, 67
изображение, 135
Фильтрация временных рядов, 67
Форма квадратичная матрицы
нормализованная квадратичная форма матрицы ковариаций в данных, 270
Форма квадратичная нормализованная, 270
Форма матрицы квадратичная, 243
Форма матрицы ступенчатая, 178
предпочтительные формы, 179
Форма ступенчатая строчно приведенная (RREF), 181
Форматирование печати и *f*-строки, 308
Формула, переложение формул в исходный код, 305
Функция (Python), 297
библиотеки, 301
импорт NumPy, 301
индексация и нарезка в NumPy, 302
методы в качестве функций, 299
написание своих собственных функций, 299

Ц

Центрирование по среднему значению, 64
Центроид, 68

Ч

Число кондиционное матрицы, 155, 201, 261
Число сингулярное, 254
важные свойства, 259
конвертация в дисперсию, 260
ранг матрицы, 256
Число собственное, 228
график матрицы ковариаций в данных, 230
действительно-значное, 240
из собственного разложения сингулярной матрицы, 241
комплексно-значное, 240
определение знака квадратичной формы, 245
отыскание, 231
подавление шума, 230
уравнение собственного числа, 228
векторно-скалярная версия, 257
для диагонализации матрицы, 236

Ш

Шум
плохо обусловленная матрица в качестве фактора усиления, 264
применение сингулярного разложения для шумоподавления, 276

Э

Элемент опорный, 178
в устранении по Гауссу–Жордану, 181

Я

Ядро
в фильтрации временных рядов, 67
в фильтрации изображения, 135
гауссово (двумерное), 135

Книги издательства «ДМК ПРЕСС»
можно купить оптом и в розницу
в книготорговой компании «Галактика»
(представляет интересы издательств
«ДМК ПРЕСС», «СОЛОН ПРЕСС», «КТК Галактика»).
Адрес: г. Москва, пр. Андропова, 38, оф. 10;
тел.: **(499) 782-38-89**, электронная почта: **books@alians-kniga.ru**.
При оформлении заказа следует указать адрес (полностью),
по которому должны быть высланы книги;
фамилию, имя и отчество получателя.
Желательно также указать свой телефон и электронный адрес.
Эти книги вы можете заказать и в интернет-магазине: **http://www.galaktika-dmk.com/**.

Майк Икс Коэн

Прикладная линейная алгебра для исследователей данных

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com
Зам. главного редактора *Сенченкова Е. А.*
Перевод *Логунов А. В.*
Корректор *Синяева Г. И.*
Верстка *Чаннова А. А.*
Дизайн обложки *Мовчан А. Г.*

Гарнитура PT Serif. Печать цифровая.
Усл. печ. л. 26,65. Тираж 200 экз.

Веб-сайт издательства: **www.dmkpress.com**